Тестирование, контроль и оптимизация кода

JAVA

- Настройка и применение JVM, JMH, сборщиков мусора
- Кэширование и анализ байт-кода, модулярность и виртуальные потоки
- Многопоточность, асинхронность и реактивный код
- JIT-компиляция и построение эффективных CI/CD пайплайнов
- Observability, работа в контейнерах и с большими данными (Spark, Flink)
- Оптимизация Java-приложений как на сервере, так и на десктопе или Android

Тестирование, контроль и оптимизация кода Java

Ещё больше свежей и актуальной литературы по Java ты найдёшь в наших каналах:
https://max.ru/bookofgeek
https://t.me/bookofgeek

Это практическое руководство охватывает весь цикл разработки высокопроизводительного кода: от понимания внутренней работы JVM до настройки сборщиков мусора, анализа JIT-компиляции, применения JMH, работы с многопоточностью и построения эффективных CI/CD пайплайнов и многое другое.

Вы узнаете: какие инструменты и подходы позволяют выявлять узкие места; как замерять реальные метрики; оптимизировать многопоточные и реактивные приложения; правильно кешировать и анализировать байт-код; использовать современные сборщики мусора и эффективно оптимизировать мобильные и десктопные Java-приложения. Отдельные главы посвящены виртуальным потокам, модулярности, Observability и работе в контейнерах и с большими данными (Spark, Flink).

Прочитав книгу, Вы получите:

- целостное понимание производительности Java-приложений;
- знания о том, как проводить тестирование под нагрузкой и отслеживать деградации в CI/CD;
- десятки примеров, шаблонов и советов для ускорения приложений как на сервере, так и на десктопе или Android;
- рекомендации по выбору GC, построению observability-стека и оптимизации всего пути.

Книга будет полезна как опытным Java-разработчикам, так и тем, кто только начинает осваивать продвинутые аспекты производительности.

https://max.ru/bookofgeek

Содержание

Введение.....21

B.1. Цель и задачи книги **22**

B.2. Как работать с книгой..... **23**

B.3. Почему важна оптимизация кода?..... **24**

B.4. Обзор современных инструментов и подходов **26**

B.4.1. Java Virtual Machine (JVM) и ее внутреннее устройство 26
B.4.2. Java Microbenchmark Harness (JMH).....27
B.4.3. Оптимизация алгоритмов и структур данных 27
B.4.4. Многопоточное программирование и параллельные вычисления 27
B.4.5. Работа с базами данных, вводом-выводом и памятью.....28

ГЛАВА 1. **Основы производительности Java**.....29

1.1. Внутреннее устройство JVM..... **31**

Архитектура JVM: из чего состоит виртуальная машина 32
1. Класс-лоадер (Class Loader) 32
2. Управление памятью (Memory Management) 33
3. JIT-компилятор (Just-In-Time Compiler).....34
4. Сборщик мусора (Garbage Collector, GC) 35

1.2. Основные метрики производительности **36**

1. Время отклика (Response Time) 37
2. Пропускная способность (Throughput).....38
3. Использование процессора и памяти (CPU/Memory Usage).....38
4. Активность сборщика мусора (GC Activity).....39
5. Количество потоков и их активность.....40

1.3. Профилирование и анализ **41**

Что такое профилирование? 41

https://max.ru/bookofgeek

Инструменты профилирования 42

1.4. Практические рекомендации **45**

Выводы **45**

ГЛАВА 2. **HotSpot и JIT-оптимизации** 47

2.1. Введение в HotSpot и JIT-компиляцию **48**

2.2. Архитектура HotSpot **49**

2.3. Принципы работы JIT-компилятора **50**

2.4. Алгоритмы и техники JIT-оптимизации **51**

2.5. Роль HotSpot в управлении памятью **52**

2.6. Визуализация и мониторинг JIT-компиляции **52**

2.7. Тюнинг JIT-компиляции **53**

2.8. JIT и особенности многопоточных приложений **54**

2.9. Инструменты для визуализации и анализа JIT-компиляции **55**

2.10. Проблемы и ограничения JIT-компиляции **56**

2.11. Применение JIT в реальных приложениях **57**

2.12. Будущее JIT-оптимизаций **57**

Выводы **58**

ГЛАВА 3. **Введение в JMH** (Java Microbenchmark Harness) 61

3.1. Что такое JMH и зачем он нужен **62**

3.2. Установка и настройка JMH **63**

Maven 63

Gradle 65

3.3. Основные концепции и аннотации JMH **65**

https://max.ru/bookofgeek

@Benchmark 65
@BenchmarkMode 65
@State 66
@Setup и @TearDown 66
Пример бенчмарка 67
Анализ результатов 67

Выводы **68**

ГЛАВА 4. **Создание и выполнение бенчмарков с JMH** 71

4.1. Структура простого бенчмарка **73**
4.1.1. Основные элементы 73
4.1.2. Пояснения к коду 74

4.2. Настройка параметров и режимов работы **75**
4.2.1. Параметры бенчмарков 75
4.2.2. Режимы работы 76

4.3. Анализ результатов бенчмарков **77**

Выводы **77**

ГЛАВА 5. **Оптимизация алгоритмов и структур данных** 79

5.1. Краткий обзор структур данных **81**
5.1.1. Массивы 81
5.1.2. Связанные списки (Linked List) 82
5.1.3. Стек (Stack) 83
5.1.4. Очередь (Queue) 84
5.1.5. Хеш-таблица (Hash Table) 85
5.1.6. Дерево (Tree) 85
5.1.7. Графы (Graphs) 86

5.2. Выбор правильных структур данных **87**

5.3. Оптимизация алгоритмов сортировки и поиска **88**

https://max.ru/bookofgeek

5.3.1. Оптимизация алгоритмов сортировки 88
5.3.2. Оптимизация алгоритмов поиска 88
Сортировка слиянием 88
Оптимизация сортировки слиянием....89
Быстрая сортировка 91
Оптимизация быстрой сортировки....91

5.4. ПРАКТИЧЕСКИЕ ПРИМЕРЫ С ИСПОЛЬЗОВАНИЕМ JMH....93

Бенчмарки для различных структур данных 93
Пример 1. Сравнение производительности поиска в массиве и в хеш-таблице 93
Пример 2. Сравнение производительности сортировки в различных структурах данных 94

5.5. Детальный разбор деревьев и графов 95

5.5.1. Основные типы деревьев 96
1. Бинарное дерево поиска (BST) 96
2. Сбалансированные деревья....98
3. AVL-дерево....98
4. Красно-черные деревья 101
5.5.2. Алгоритмы и операции с деревьями....101

5.6. Применение деревьев в реальных задачах 102

5.6.1. Бенчмарки с использованием JMH для деревьев 102
Пример бенчмарка для поиска в различных деревьях .102
5.6.2. Введение в графы 103
Определение....104
5.6.3. Представление графов в памяти 104
1. Матрица смежности....104
2. Списки смежности 105
5.6.4. Алгоритмы работы с графами 105
Поиск в глубину (DFS)....105
Поиск в ширину (BFS) 106
Алгоритм Дейкстры 106
Алгоритм Флойда-Уоршелла 107
5.6.5. Практическое применение графов....108
5.6.6. Бенчмарки JMH: производительность алгоритмов на графах. 108
Пример: сравнение BFS и Dijkstra....108
5.6.7. Оптимизация работы с графами 109

https://max.ru/bookofgeek

5.7. Примеры использования деревьев и графов для оптимизации ... 109

Оптимизация в навигации и логистике ... 110
Оптимизация сетевых инфраструктур ... 110
Принятие решений и компиляция ... 110
Работа с зависимостями и сборкой проектов ... 111
Оптимизация хранения и доступа к данным ... 111
Машинное обучение и анализ больших данных ... 111
Разработка игр и моделирование ... 112

Выводы ... 112

ГЛАВА 6. **Параллельные и многопоточные приложения** ... 113

6.1. Основы многопоточности в Java ... 114

Создание потоков ... 114
Основы синхронизации ... 115

6.2. Пул потоков и управление потоками ... 118

Создание пула потоков ... 118
Параллельное выполнение задач ... 119
Типы пулов потоков ... 120

6.3. Оптимизация многопоточных приложений ... 120

6.3.1. Избегание избыточной синхронизации ... 120
6.3.2. Использование конкурентных коллекций ... 121
6.3.3. Минимизация блокировки ... 122
6.3.4. Использование атомарных операций ... 122
6.3.5. Использование ***ExecutorService*** для управления потоками ... 123

Выводы ... 124

ГЛАВА 7. **Использование функций языка для оптимизации** ... 125

7.1. Лямбда-выражения и потоки (Streams API) ... 126

7.2. Введение в модули (Java Modules) ... 128

Основы работы с модулями ... 128

https://max.ru/bookofgeek

Основные преимущества модулей 129
Пример использования модулей 129

7.3. Новые коллекции и утилиты **129**

Выводы **131**

ГЛАВА 8. **Виртуальные потоки** 133

8.1. Подробное введение в проблему традиционной многопоточности .. **134**

8.2. История появления виртуальных потоков **135**

8.3. Архитектура и внутренняя реализация **136**

8.4. Примеры использования **136**

8.5. Производительность и ограничения **137**

8.6. Интеграция с существующими библиотеками **138**

Практические советы и best practices 139

8.7. Тестирование кода с виртуальными потоками **139**

Выводы **140**

ГЛАВА 9. **Модулизация и производительность Java-приложений** .. 141

9.1. Введение в модулизацию (Java Platform Module System) **142**

9.2. Почему модули важны для производительности **143**

9.3. Уменьшение времени запуска за счет модулярности **144**

Механизмы оптимизации загрузки 145
Практические рекомендации для ускорения запуска через модули . 145
Пример влияния модуляризации на время запуска 146
Особенности при контейнеризации 146

9.4. Минимизация используемых зависимостей **147**

9.5. Снижение использования памяти **149**

https://max.ru/bookofgeek

SKLADCHIK.ORG

Экономия памяти за счет уменьшения объема классов 149
Влияние на скорость загрузки классов 150
Ленивая загрузка (Lazy Loading) 150

9.6. Модулизация в больших проектах 151

Управление зависимостями и предотвращение "загрязнения" пространства классов 152
Оптимизация времени старта и потребления памяти 152
Улучшение масштабируемости и параллельной разработки 152
Облегчение тестирования и повышения качества 153
Разграничение ответственности и повышение читаемости архитектуры 153
Поддержка эволюции проекта 153

9.7. Практика: разбиение монолита на модули 154

9.7.1. Анализ текущей архитектуры 154
9.7.2. Разделение по слоям и функциональности 155
9.7.3. Определение и контроль зависимостей между модулями 155
9.7.4. Минимизация публичного API 156
9.7.5. Постепенный рефакторинг и миграция 156
9.7.6. Тестируемость каждого модуля 157
9.7.7. Технические детали: использование системы модулей Java 157

9.8. Инструменты для модуляризации в Java 158

9.9. Рефакторинг монолита в модульную структуру 159

9.10. Влияние модулярности на тестирование и деплой 160

9.11. Пример модулярного подхода 161

9.12. Модули и контейнеризация 162

9.13. Best practices при модуляризации 163

Выводы 165

Глава 10. **Сборщики мусора в Java** 167

10.1. Обзор типов GC 169

Serial GC 169

https://max.ru/bookofgeek

Parallel GC (Throughput Collector) 169
G1 GC (Garbage-First) 169
ZGC (Z Garbage Collector) 170
Shenandoah GC 170

10.2. Выбор GC под тип приложения 170

Ключевые факторы, влияющие на выбор GC 171
Примеры выбора GC для типовых приложений 172
Важность эмпирической валидации 173

10.3. Тюнинг и мониторинг GC 174

Метрики GC 174
Инструменты мониторинга 174
Влияние GC на производительность 175

Выводы 175

ГЛАВА 11. **Работа с памятью и управление ресурсами** 177

11.1. Профилирование использования памяти 178

Основные подходы к профилированию памяти 178
Пример профилирования памяти с помощью *jvisualvm* 179

11.2. Оптимизация работы с кучей и стеком 179

11.2.1. Оптимизация работы с кучей 180
11.2.2. Оптимизация работы со стеком 180

11.3. Управление и минимизация утечек памяти 181

11.3.1. Причины утечек памяти 181
11.3.2. Методы для предотвращения утечек памяти: 182

Выводы 183

ГЛАВА 12. **Оптимизация ввода-вывода** 185

12.1. Эффективные методы работы с файлами и сетью 186

12.1.1. Работа с файлами 186

https://max.ru/bookofgeek

12.1.2. Работа с сетью 188

12.2. Использование неблокирующего ввода-вывода (NIO) 190

12.3. Примеры оптимизации ввода-вывода 191

12.3.1. Использование буферизации и кеширования 191
12.3.2. Неблокирующий I/O 192
12.3.3. Использование асинхронного ввода-вывода 192

Выводы 193

ГЛАВА 13. **Безопасность и производительность** 195

13.1. Влияние безопасного программирования на производительность ... 196

Защитное программирование и его цена 197
Пример. Валидация входных данных 197
Пример. Логирование 198

13.2. Оптимизация без ущерба для безопасности 198

13.2.1. Уровни проверки и инварианты 198
13.2.2. Использование ***assert*** 198
13.2.3. Изоляция критичных проверок 199
13.2.4. Кеширование и повторное использование 199
13.2.5. Адаптивная безопасность 200
13.2.6. Безопасные, но быстрые алгоритмы 200
13.2.7. Инструментальный анализ влияния безопасности 200

13.3. Советы по написанию безопасного и быстрого кода 201

Выводы 202

ГЛАВА 14. **Декомпилирование и анализ байт-кода** 203

14.1. Основы декомпиляции Java кода 204

14.2. Инструменты для декомпиляции 206

14.2.1. JD-GUI (Java Decompiler GUI) 206
14.2.2. Procyon 207

https://max.ru/bookofgeek

14.2.3. CFR (Class File Reader) 207
14.2.4. FernFlower 207

14.3. Анализ байт-кода для оптимизации 208

14.3.1. Инструменты для анализа байт-кода 208
14.3.2. Оптимизация на уровне байт-кода 208
14.3.3. **Пример: анализ байт-кода с использованием ASM** 209

Выводы 210

ГЛАВА 15. **Оптимизация запросов к базе данных** 213

15.1. Эффективное использование JDBC 215

15.1.1. Использование Connection Pooling (пула соединений) 215
15.1.2. Использование PreparedStatement 217
15.1.3. Использование Batch Processing для массовых операций 217

15.2. Оптимизация запросов ORM (Hibernate, JPA) 218

15.2.1. Использование кеширования 218
15.2.2. Ленивая загрузка (Lazy Loading) 219
15.2.3. Использование FetchType.LAZY и FetchType.EAGER 219

15.3. Профилирование и оптимизация SQL-запросов 220

15.3.1. Использование EXPLAIN PLAN 220
15.3.2. Индексация 220
15.3.3. Профилирование с использованием инструментов 220
15.3.4. Разделение запросов 221

Выводы 221

ГЛАВА 16. **Тестирование и контроль качества кода** 223

16.1. Введение в тестирование Java-приложений 225

16.2. Юнит-тестирование и инструменты 226

16.3. Интеграционное тестирование 228

https://max.ru/bookofgeek

16.4. Нагрузочное тестирование **229**

16.5. Контроль качества кода (Code Quality) **230**

Инструменты для контроля качества кода 230
Пример настройки SonarQube для проекта 230
Применение практик рефакторинга 231

Выводы **231**

ГЛАВА 17. **Виды тестов при нагрузочном тестировании**....233

17.1. Тестирование под нагрузкой (Load Testing) **235**

Цель тестирования под нагрузкой 235
Особенности тестирования под нагрузкой 236
Пример: тестирование веб-приложения 237
Инструменты для тестирования под нагрузкой 237

17.2. Стресс-тестирование (Stress Testing) **238**

Цель стресс-тестирования 239
Особенности стресс-тестирования 239
Пример стресс-тестирования 240
Инструменты для стресс-тестирования 241

17.3. Тестирование скачков нагрузки (Spike Testing) **242**

Цель тестирования скачков нагрузки 242
Особенности тестирования скачков нагрузки 243
Пример стресс-тестирования скачков нагрузки 243
Инструменты для тестирования скачков нагрузки 244

17.4. Тестирование на продолжительное время (Soak Testing) **245**

Цель тестирования на продолжительное время 245
Особенности тестирования на продолжительное время 246
Пример тестирования на продолжительное время 246
Инструменты для тестирования на продолжительное время 247

17.5. Тестирование масштабируемости (Scalability Testing) **248**

Цель тестирования масштабируемости 248
Особенности тестирования масштабируемости 249
Пример тестирования масштабируемости 249

https://max.ru/bookofgeek

Инструменты для тестирования масштабируемости 250

17.6. Тестирование конфигураций (Configuration Testing) 251

Особенности тестирования конфигураций....251
Пример тестирования конфигураций 252
Инструменты для тестирования конфигураций 253

17.7. Создание эталона производительности (Baseline Testing) 254

Особенности создания эталона производительности....254
Пример. Переход с Java 11 на Java 17 255
Инструменты для создания эталона производительности 255

17.8. Сравнительное тестирование (Comparison Testing)....256

Особенности сравнительного тестирования....257
Пример. Сравнение JDBC и R2DBC для доступа к базе данных . 257
Инструменты для сравнительного тестирования 258

Выводы **259**

ГЛАВА 18. **Методы и техники оптимизации кода**.....261

18.1. Основы рефакторинга кода.... **262**

18.2. Паттерны и антипаттерны оптимизации.... **264**

18.3. Инструменты для анализа и улучшения кода **266**

18.4. Best Practices оптимизации кода **266**

Выводы **267**

ГЛАВА 19. **Продвинутые техники и подходы**.....271

19.1. JIT-компиляция и ее влияние на производительность **273**

19.2. Inline-оптимизация и ее применение **274**

19.3. Использование специализированных библиотек и инструментов.. **276**

Выводы **277**

https://max.ru/bookofgeek

ГЛАВА 20. **Метрики и Observability: как контролировать и понимать поведение Java-приложений**.....279

20.1. Основы Observability **281**

Что входит в Observability 282
Метрики как важнейший элемент Observability 283

20.2. Prometheus — стандарт де-факто для сбора метрик **283**

20.3. Micrometer — фасад для метрик в Java **284**

20.4. Grafana — визуализация и дашборды **285**

20.5. Подключаем все вместе: Prometheus + Micrometer + Grafana **286**

20.6. Какие метрики собирать? **287**

Системные метрики 287
JVM метрики 288
Приложенческие метрики 289

20.7. Алерты и автоматическая реакция **290**

Алерты в Prometheus 290
Алерты в Grafana 292
Пример настройки алерта в Grafana 292
Автоматическая реакция на алерты 293

20.8. Observability в Kubernetes **294**

Расширение: OpenTelemetry и трассировки 294
Что такое OpenTelemetry? 295
Трассировки в OpenTelemetry 295
Интеграция OpenTelemetry с Micrometer 296
Преимущества интеграции OpenTelemetry 297

20.9. Best practices **297**

Выводы **298**

ГЛАВА 21. **Кеширование и его влияние**.....299

https://max.ru/bookofgeek

21.1. Когда и как кешировать **300**

21.2. Виды кешей **301**

Локальные кеши 301
Распределенные кеши 301

21.3. Использование Ehcache, Caffeine, Redis **302**

Ehcache 302
Caffeine 302
Redis 303

21.4. Оптимизация hit/miss rate **303**

Выводы **304**

ГЛАВА 22. **Работа с большими данными**.....305

22.1. Java и Big Data: Введение в Spark и Flink **306**

Apache Spark 306
Apache Flink 307

22.2. Потоковая обработка данных **308**

Хранение и обработка данных в HDFS и S3 308
Использование Kafka вместе со Spark/Flink 309
Практики мониторинга и отладки Big Data-приложений 309
Интеграция с системами машинного обучения 310

22.3. Производительность и ресурсоемкость **310**

Выводы **311**

ГЛАВА 23. **Асинхронное и реактивное программирование**....313

23.1. Асинхронность и реактивность: основные понятия **314**

Асинхронность 315
Реактивность 315

23.2. Reactor, RxJava и Project Loom **316**

https://max.ru/bookofgeek

RxJava 316
Project Reactor 316
Project Loom 317

23.3. Производительность реактивных приложений 317

Мифы и реальность 318

23.4. JMH и реактивный код 318

Общие принципы 319
Пример: бенчмарк RxJava против Project Loom 319
Практические советы 320

Выводы 320

ГЛАВА 24. **Контейнеризация и производительность** 323

24.1. Оптимизация Java в контейнерах 324

Лучшие практики оптимизации Java в контейнерах 325

24.2. Настройки JVM при ограниченных ресурсах 326

24.3. Мониторинг в Kubernetes 328

Основные компоненты мониторинга Java в Kubernetes 328

Выводы 329

ГЛАВА 25. **CI/CD и автоматизированный контроль производительности** 331

25.1. JMH и нагрузочные тесты в CI 332

Почему стоит запускать JMH в CI? 332
Пример настройки JMH в Maven-проекте 333
Запуск JMH в GitHub Actions 333
Парсинг и анализ результатов 334

25.2. Метрики и алерты в CI/CD-пайплайнах 334

Какие метрики отслеживать? 334

https://max.ru/bookofgeek

Сравнение с эталонными значениями 334
Визуализация метрик 335

25.3. GitHub Actions, Jenkins, GitLab CI для контроля деградации 335

GitHub Actions 335
Jenkins 336
GitLab CI 336

25.4. ИНТЕГРАЦИЯ НАГРУЗОЧНОГО ТЕСТИРОВАНИЯ 337

Использование Gatling, k6, Apache JMeter 337
Пример Gatling в Jenkins 337

Выводы 338

ГЛАВА 26. **Front-to-Back производительность** 341

26.1. Эффективные форматы передачи данных 343

JSON 343
Protobuf 344
Avro 344

26.2. Оптимизация сериализации/десериализации 345

Общие рекомендации 345
Jackson (JSON) 346
Protobuf 346
JMH и сериализация 346

26.3. Латентность REST и gRPC 347

REST 347
gRPC 348

26.4. Практические рекомендации для снижения задержек и увеличения пропускной способности коммуникационного слоя 349

Выводы 350

ГЛАВА 27. **Мобильные и десктопные Java-приложения** 351

https://max.ru/bookofgeek

SKLADCHIK.ORG

27.1. Особенности JVM на Android **352**
Dalvik и ART 352
Ограничения и особенности 353
Практики оптимизации Android-приложений на Java 353

27.2. Оптимизация JavaFX/Swing-приложений **353**
Производительность JavaFX 354
Практики оптимизации JavaFX 354
Swing: советы по производительности 354
Инструменты для профилирования десктопных Java-приложений 354
Общие подходы 355

Выводы **355**

Заключение **357**
Итоговые рекомендации 358
Ресурсы для дальнейшего изучения 359
Будущее оптимизации Java 360

Полезные утилиты и инструменты **363**

Ссылки на дополнительные материалы **367**

Глоссарий терминов **371**

Список использованных источников информации **377**

https://max.ru/bookofgeek

Благодарности

Во время работы над этой книгой меня очень поддерживали и мотивировали моя семья и друзья. Хочу выразить им большую благодарность. Их вера в меня, добрые слова и участие стали мощной мотивацией на всём пути.

Также спасибо за поддержку коллегам, которые принимали участие в обсуждение тех или иных кодовых решений.

https://max.ru/bookofgeek

Ещё больше свежей и актуальной литературы по Java ты найдёшь в наших каналах:
https://max.ru/bookofgeek
https://t.me/bookofgeek

Введение

https://max.ru/bookofgeek

B.1. Цель и задачи книги

Цель этой книги — предоставить читателю обширное понимание методов оптимизации кода на языке Java, а также поделиться практическим опытом по тестированию и контролю кода, который поможет создавать высокопроизводительные приложения.

Я не претендую на полноту описания всех возможных подходов в оптимизации, но постарался охватить ключевые техники и методы, которые использую в своей практике. Каждый разработчик может найти полезные идеи, будь то новичок или опытный специалист.

Процесс оптимизации кода затрагивает несколько аспектов и этапов, таких как нагрузочное тестирование (НТ), процесс покрытия кода тестами (юнит-тестирование, интеграционные тесты, смоки и другие), упрощение кода, улучшение структуры и переработка алгоритмов. Эти процессы взаимосвязаны, и оптимизация часто требует комплексного подхода, включающего их все. Важно понимать, что невозможно достичь идеальной оптимизации сразу — это процесс, который требует внимательности и тщательного подхода.

В этой книге я постарался затронуть все основные аспекты оптимизации и дать представление о большинстве известных подходов и технологий, ко-

https://max.ru/bookofgeek

торые применяются в реальных проектах. Каждая глава книги может быть прочитана как независимый модуль, что дает возможность углубиться в ту тему, которая наиболее актуальна для текущих задач читателя. Это позволяет использовать книгу как справочник и сосредоточиться только на нужных аспектах.

Сегодня время и производительность играют ключевую роль в успехе многих приложений, и знание инструментов оптимизации, а также понимание работы внутренних механизмов языка Java становятся необходимыми для любого разработчика. Миллисекунды могут решать, будет ли ваше приложение конкурентоспособным на рынке. Эта книга шаг за шагом проведет вас через важнейшие этапы анализа производительности, выявления узких мест и улучшения работы программы с помощью различных методов и инструментов.

Книга предназначена как для опытных разработчиков, работающих над масштабными и сложными проектами, так и для начинающих специалистов, которые хотят понять, как создавать код, способный работать быстро и эффективно. Моя цель — помочь вам овладеть инструментами и методами, которые позволят значительно улучшить производительность ваших Java-приложений.

B.2. Как работать с книгой

Эта книга написана с учетом **различных уровней подготовки читателей**, и каждый сможет найти здесь полезные материалы. Важно отметить, что каждая глава является независимым модулем, который можно читать отдельно, в зависимости от того, какой аспект оптимизации или тестирования вас интересует в данный момент. Вы можете сразу перейти к нужной теме, не обязательно следовать строгому порядку изложения.

Книга разделена на две основные части: ознакомление с базовыми принципами и углубленный анализ более сложных тем. В первых главах мы рассмотрим основные методы оптимизации, такие как выбор структур данных,

https://max.ru/bookofgeek

параллельные вычисления и базовые техники тестирования. Эти главы дают представление о ключевых подходах, которые можно сразу применить на практике.

В более продвинутых разделах мы углубляемся в анализ производительности, включая методы профилирования, использование специализированных инструментов и оптимизацию работы с памятью, базами данных и многопоточными приложениями. Эти главы ориентированы на тех, кто уже имеет опыт работы с Java и хочет повысить эффективность своего кода.

Каждая глава построена так, чтобы быть полезной независимо от других, что дает вам возможность выбрать только те темы, которые вам актуальны на данный момент. В книге вы найдете как теоретические основы, так и практические рекомендации, сопровождающиеся примерами кода и реальными кейсами.

B.3. Почему важна оптимизация кода?

Оптимизация кода — это не просто стремление к улучшению скорости работы программы, это многогранный процесс, который охватывает различные аспекты разработки и имеет прямое влияние на всю жизнь продукта, от его разработки до эксплуатации. Важно понимать, что оптимизация касается не только быстродействия, но и множества других факторов, которые влияют на эффективность и жизнеспособность приложений в долгосрочной перспективе. Это процесс, который затрагивает как инфраструктурные ресурсы, так и пользовательский опыт.

Одним из первых и самых очевидных аспектов оптимизации является **снижение затрат на вычислительные ресурсы**. Когда код неэффективен, он использует больше процессорного времени, памяти и других ресурсов, что может привести к увеличению операционных расходов на поддержку серверной инфраструктуры и других систем. Каждое приложение, будь то веб-сервис или мобильная платформа, нуждается в рациональном распреде-

https://max.ru/bookofgeek

лении ресурсов для эффективной работы, и оптимизация кода играет здесь ключевую роль.

Следующий важный момент — **повышение стабильности и масштабируемости приложений**. Когда приложение написано неэффективно, оно может работать хорошо на малых объемах данных или с небольшой нагрузкой, но при увеличении нагрузки начинает «тормозить» или даже выходить из строя. Это становится особенно критичным, когда приложение используется большим числом пользователей или работает в условиях повышенной нагрузки. Оптимизация позволяет не только улучшить текущие показатели работы, но и обеспечить возможности для масштабирования, что в свою очередь позволяет системе справляться с большими объемами данных и запросов, не теряя в стабильности.

Также важным результатом оптимизации является **улучшение пользовательского опыта**. Время отклика системы или приложения напрямую влияет на восприятие пользователя. Чем быстрее приложение реагирует на действия пользователя, тем выше его удовлетворенность. Оптимизированный код помогает минимизировать задержки, улучшая скорость загрузки страниц, обработку запросов и выполнение операций, что, в свою очередь, приводит к положительному пользовательскому опыту и повышению лояльности клиентов.

Если код не оптимизирован, это может привести к целому ряду проблем, таких как **избыточная нагрузка на серверы**, **увеличение затрат на инфраструктуру** и даже **ухудшение конкурентоспособности продукта**. В условиях жесткой конкуренции на рынке технологий, где каждая секунда отклика имеет значение, недостаток внимания к оптимизации может стать причиной потери пользователей и уменьшения доли на рынке. Знание современных подходов и методов оптимизации позволяет разработчикам не только создать эффективный код, но и предвидеть, какие решения могут стать узким местом в будущем, а какие помогут устранить проблемы и повысить производительность.

В конечном счете, **оптимизация — это не просто техника улучшения кода, это важная часть стратегии разработки**, которая помогает обеспечить успех продукта в условиях быстро меняющегося мира технологий. Согласование между производительностью, стабильностью и затратами на ресурсы помогает не только улучшить существующие решения, но и в будущем адаптироваться к новым вызовам.

https://max.ru/bookofgeek

B.4. Обзор современных инструментов и подходов

Эффективная оптимизация кода на языке Java невозможна без глубокого понимания как теоретических основ, так и практических инструментов, которые помогают разработчикам достигать значительных улучшений в производительности.

В этой книге мы подробно рассмотрим самые современные и важные инструменты и подходы, которые должны быть в арсенале каждого разработчика, стремящегося оптимизировать свои приложения. Без знаний о таких инструментах и методах невозможно добиться реальных и устойчивых улучшений в производительности приложений, а также эффективно управлять ресурсами и масштабировать системы. Давайте рассмотрим, что будет подробно освещено в этой книге.

B.4.1. Java Virtual Machine (JVM) и ее внутреннее устройство

Один из ключевых аспектов оптимизации Java-приложений — это понимание работы **Java Virtual Machine (JVM)**.

JVM является центральным элементом всей экосистемы Java, и для эффективной оптимизации необходимо понимать, как она работает на низком уровне. Мы разберем, как JVM управляет памятью, что такое **garbage collection (GC)** и как выбрать подходящий алгоритм сборщика мусора для различных сценариев.

Особое внимание будет уделено **JIT-компиляции (Just-In-Time)**, которая играет важную роль в оптимизации производительности Java-программ. JIT-компиляция позволяет JVM компилировать байт-код в машинный код непосредственно во время выполнения, что значительно ускоряет выполнение программы.

Мы рассмотрим, как настроить JVM для улучшения производительности, как работает **HotSpot** и как можно влиять на поведение JIT-компилятора. Также в книге будет рассмотрена **tiered compilation** и другие механизмы оптимизации, встроенные в JVM.

https://max.ru/bookofgeek

B.4.2. Java Microbenchmark Harness (JMH)

Одним из самых важных инструментов для точного измерения производительности кода является **Java Microbenchmark Harness (JMH)**.

JMH предоставляет высокоточные средства для написания бенчмарков, которые позволяют измерять время выполнения отдельных фрагментов кода с высокой степенью точности. В книге мы подробно разберем, как правильно использовать JMH для создания и анализа бенчмарков.

Мы научимся писать бенчмарки с учетом особенностей многозадачности, правильно измерять производительность кода, учитывая различные параметры, такие как warm-up, thrashing и cold starts. Также будет рассмотрено, как интерпретировать результаты и каких ошибок нужно избегать при использовании JMH. Реальные примеры бенчмарков помогут вам понять, как оптимизировать критические части приложения с помощью точных измерений.

B.4.3. Оптимизация алгоритмов и структур данных

Каждый опытный разработчик понимает, что **правильный выбор алгоритмов и структур данных** — это основа для оптимизации производительности программы. В книге мы рассмотрим наиболее эффективные алгоритмы и структуры данных для различных задач, а также разберем, как выбирать между ними в зависимости от контекста. В разделе будет рассказано о таких алгоритмах, как **быстрая сортировка**, **поиск в глубину и ширину**, **деревья поиска**, и других. Мы также обсудим, как уменьшить время работы алгоритмов и сократить потребление памяти.

Мы подробно изучим методы оптимизации работы с памятью, включая улучшение **работы с кучей** и **управление стеками**. Понимание внутренней структуры данных и эффективное использование памяти позволяет снизить нагрузку на систему и улучшить отклик программы. Также будут рассмотрены принципы работы с **собственными структурами данных**, их оптимизация и использование в реальных приложениях.

B.4.4. Многопоточное программирование и параллельные вычисления

Многопоточность — это неотъемлемая часть современных приложений, и ее эффективное использование критически важно для повышения произ-

https://max.ru/bookofgeek

водительности. Мы разберем основные подходы к **многопоточному программированию**, включая **пул потоков**, **управление состоянием потоков** и методы синхронизации. Для этого будут рассмотрены такие механизмы, как **synchronized** блоки, **ReentrantLock**, **ReadWriteLock** и другие. Также в книге будет описано, как использовать **Java Concurrency Utilities** и библиотеки для параллельных вычислений, такие как **ForkJoinPool**.

Важным моментом является правильное использование **параллельных потоков** для ускорения выполнения программ и обработки больших объемов данных. Будет показано, как можно ускорить выполнение задач, делая вычисления параллельными, а также как избежать типичных ошибок, таких как **условия гонки** и **deadlock**. Также будет рассмотрено, как эффективно работать с **параллельными стримами** в Java.

B.4.5. Работа с базами данных, вводом-выводом и памятью

В книге будет особое внимание уделено **оптимизации работы с базами данных**. Мы рассмотрим, как правильно оптимизировать **SQL-запросы**, как использовать индексы для ускорения поиска данных и как избежать типичных ошибок при работе с базами данных. Мы также изучим, как **JPA** и **Hibernate** могут быть использованы для улучшения производительности работы с объектами, а также как правильно настраивать их для обработки больших объемов данных.

Работа с **вводом-выводом** также будет рассмотрена в контексте оптимизации. Мы изучим методы эффективной работы с файлами, сетевыми соединениями, а также использование **NIO** (Non-blocking I/O), что позволяет значительно повысить производительность в приложениях с большим количеством операций ввода-вывода. Важной темой будет **оптимизация работы с памятью**: мы рассмотрим принципы работы с **heap**, **stack**, а также способы минимизации утечек памяти и эффективного использования кэширования.

В этой книге мы постараемся предоставить читателям всестороннее понимание современных инструментов и подходов, которые позволят им значительно улучшить производительность своих Java-программ, обеспечив тем самым лучшие результаты в разработке и поддержке приложений.

https://max.ru/bookofgeek

Глава 1.

Основы производительности Java

https://max.ru/bookofgeek

О чём глава:

- Внутреннее устройство JVM.
- Основные метрики производительности.
- Профилирование и анализ.

В современном мире, где цифровые технологии развиваются с невероятной скоростью, приложения сталкиваются с возрастающими требованиями к производительности. Это связано с увеличением объемов данных, которые необходимо обрабатывать, ростом сложности вычислений, а также с повышением ожиданий пользователей. Они хотят, чтобы программы работали быстро, стабильно и без сбоев.

Java, будучи одним из самых популярных языков программирования, широко используется для создания как небольших приложений, так и сложных корпоративных систем. Ее популярность обусловлена несколькими факторами: переносимостью благодаря платформе Java Virtual Machine (JVM), богатой экосистемой библиотек и инструментов, а также активным сообществом разработчиков. Однако создание высокопроизводительных приложений на Java требует не только знания синтаксиса языка, но и понимания того, как работает платформа и какие факторы влияют на быстродействие.

Одним из ключевых компонентов платформы является JVM. Именно она отвечает за выполнение Java-программ, управляя памятью, оптимизируя выполнение кода с помощью JIT-компиляции (Just-In-Time) и обеспечивая сборку мусора. Знание устройства JVM помогает разработчикам понимать, как их код превращается из текстового представления в байт-код, а затем в машинный код, и как эти процессы могут быть оптимизированы. Например, понимание того, как JVM управляет памятью, позволяет эффективнее работать с кучей (heap) и избегать проблем с утечками памяти или чрезмерной загрузкой процессора.

https://max.ru/bookofgeek

Но само по себе понимание устройства JVM недостаточно. Важной частью работы с производительностью является использование основных метрик, которые помогают оценивать, насколько эффективно работает приложение. Среди таких метрик можно выделить время отклика (response time), пропускную способность (throughput), использование памяти и процессора, а также частоту и длительность сборок мусора. Эти показатели позволяют разработчикам оценить, где приложение может быть улучшено, и понять, влияет ли конкретное изменение в коде на производительность в лучшую сторону.

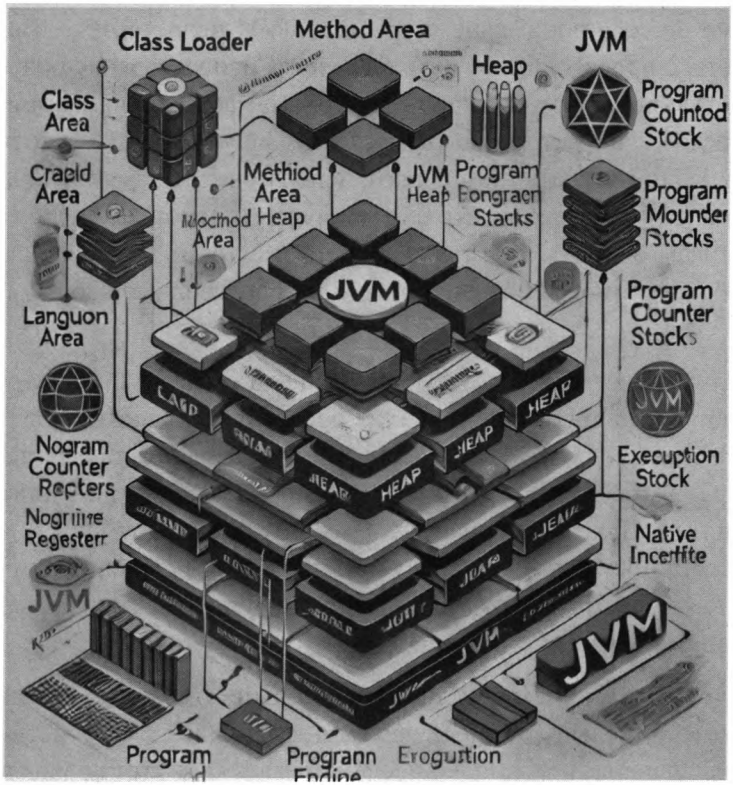
Для выявления узких мест и последующей оптимизации разработчики используют профилирование и анализ производительности. Профилирование — это процесс сбора данных о работе приложения, таких как загрузка процессора, потребление памяти, время выполнения методов или запросов к базе данных. Для этого существует множество инструментов, например VisualVM, JProfiler или YourKit. Эти инструменты помогают разработчику выявить, какие части приложения являются «бутылочными горлышками» и требуют внимания.

Таким образом, понимание производительности Java — это не просто знание языка, а целый комплекс навыков и знаний: от устройства JVM до умения анализировать метрики и профилировать код. Этот подход позволяет разработчикам не только создавать приложения, которые соответствуют современным требованиям, но и строить архитектуру, способную масштабироваться и адаптироваться к новым вызовам.

1.1. Внутреннее устройство JVM

Java Virtual Machine (JVM) — это фундамент всей Java-платформы, обеспечивающий независимость кода от операционной системы и архитектуры оборудования. Благодаря JVM написанный один раз Java-код можно запускать на любом устройстве, где установлена соответствующая виртуальная машина. Но, чтобы использовать все преимущества Java по максимуму, особенно в части производительности и оптимизации, необходимо понимать, как JVM устроена изнутри.

https://max.ru/bookofgeek



Изображение 1.1.

Архитектура JVM: из чего состоит виртуальная машина

Современная JVM — это многоуровневая, высокооптимизированная система, включающая в себя несколько ключевых компонентов, каждый из которых играет важную роль в процессе выполнения программ:

1. Класс-лоадер (Class Loader)

Class **Loader** отвечает за загрузку *class*-файлов (байт-кода) в память во время выполнения программы. Он организован в виде иерархии и работает по принципу делегирования — сначала родитель пытается загрузить класс, и только если не получилось, дело переходит к дочернему загрузчику.

https://max.ru/bookofgeek

Основные загрузчики:

- **Bootstrap ClassLoader** — загружает базовые классы Java (java.lang, java.util и т. д.) из rt.jar (или модулей в Java 9+).
- **Extension ClassLoader** — отвечает за загрузку расширений из lib/ext.
- **Application ClassLoader** — загружает пользовательские классы из classpath.

Особенности:

- Загрузка классов происходит лениво — класс загружается только тогда, когда впервые требуется.
- Можно создавать собственные загрузчики (например, для плагинов), обеспечивая изоляцию и горячую замену компонентов.

2. Управление памятью (Memory Management)

Управление памятью — одна из сильных сторон JVM. Платформа берет на себя всю работу по выделению и освобождению памяти, освобождая разработчика от необходимости управлять ею вручную, как в C/C++.

Основные области памяти:

➢ Куча (Heap)

Куча — это основная область памяти для хранения всех объектов, созданных с помощью *new*.

Она делится на поколения:

- **Young Generation (Eden + Survivor)** — здесь появляются новые объекты. Молодое поколение часто подвергается сборке мусора (Minor GC).

https://max.ru/bookofgeek

- **Old Generation (Tenured)** — объекты, которые «пережили» несколько сборок в молодом поколении, перемещаются сюда. Сборки в старом поколении происходят реже, но они более ресурсоемкие (Major GC).
- **Metaspace** (вместо PermGen с Java 8) — используется для хранения метаинформации о классах (структура, методы, поля).

Настройки, влияющие на поведение кучи:

```
-Xms512m -Xmx1024m    # Начальный и максимальный размер кучи
-XX:NewRatio=3        # Соотношение между Old и Young Generation
-XX:+UseG1GC          # Выбор сборщика мусора
```

➢ Стек (Stack)

Для каждого потока создается собственный стек, где хранятся:

- Фреймы вызова методов (Call Frames).
- Локальные переменные.
- Операнды и временные значения.

Проблемы:

- Переполнение стека (*StackOverflowError*) может возникнуть при слишком глубокой рекурсии.
- Поскольку стек поток-локальный, к нему требуется быстрый доступ, что критично для производительности.

➢ Дополнительные области:

- **Program Counter** (PC) Register — указывает на текущую исполняемую инструкцию байт-кода.
- **Native Method Stack** — используется для вызовов нативных методов (на C/C++ через JNI).

3. JIT-компилятор (Just-In-Time Compiler)

https://max.ru/bookofgeek

JIT-компиляция — это механизм, при котором наиболее часто исполняемые участки байт-кода компилируются в машинный код "на лету". Это позволяет JVM достигать скорости, близкой к нативным приложениям.

Как работает:

- JVM отслеживает горячие участки кода (hotspots) с помощью интерпретатора.
- При достижении определенного порога вызовов (например, 10 000 раз) фрагмент компилируется JIT-компилятором.
- Скомпилированный машинный код кешируется и используется повторно.

Типы JIT-компиляторов:

- **C1** (Client) — быстрее запускается, ориентирован на приложения с коротким жизненным циклом (например, GUI).
- **C2** (Server) — производит более глубокую оптимизацию, подходит для серверных нагрузок.
- **Tiered Compilation** — комбинирует C1 и C2, сначала выполняя быструю компиляцию, а затем — глубокую.

Техники оптимизации:

- **Inlining** — вставка тела метода прямо в вызывающий код.
- **Loop Unrolling** — разворачивание циклов для уменьшения количества итераций.
- **Escape Analysis** — позволяет объектам жить в стеке, а не в куче.
- **Dead Code Elimination** — удаление кода, который никогда не будет выполнен.

4. Сборщик мусора (Garbage Collector, GC)

https://max.ru/bookofgeek

Garbage Collector отвечает за автоматическое освобождение памяти от неиспользуемых объектов. Это избавляет разработчиков от ошибок управления памятью и утечек.

Таблица 1.1. Типы GC в JVM

Название	**Особенности**
Serial GC	Однопоточный, подходит для систем с небольшими объемами памяти
Parallel GC	Использует многопоточную обработку, оптимален для серверов с мощными CPU
CMS	Concurrent Mark-Sweep: минимизирует паузы, но менее эффективен по памяти
G1 (Garbage First)	Делит кучу на регионы, обеспечивает предсказуемые паузы. Подходит для приложений со строгими SLA
ZGC / Shenandoah	Низкие паузы, масштабируемые на большие объемы памяти. Подходят для latency-critical систем.

Настройка и профилирование:

- Использование флагов *-XX:+PrintGCDetails, -Xlog:gc* позволяет отслеживать работу GC.
- Использование инструментов (VisualVM, JFR, JMC) помогает выявить проблемы в распределении и утечке памяти.

1.2. Основные метрики производительности

Для эффективной диагностики и последующей оптимизации Java-приложений важно не только измерять производительность, но и понимать, *что именно* следует измерять и *почему это важно*.

https://max.ru/bookofgeek

Производительность — понятие многогранное, и поэтому анализируется по множеству параметров.

Ниже приведены ключевые метрики, которые необходимо отслеживать в ходе разработки, тестирования и эксплуатации Java-программ, особенно в высоконагруженных и критичных по времени отклика системах.

1. Время отклика (Response Time)

Время отклика — это интервал между поступлением запроса от пользователя или системы и получением полного ответа.

Данная метрика особенно важна для:

- веб-приложений и REST API;
- пользовательских интерфейсов, где задержка в несколько сотен миллисекунд уже воспринимается как «тормоза»;
- микросервисной архитектуры, где задержка на одном участке может повлиять на всю цепочку сервисов.

Виды времени отклика:

- *Среднее время отклика (average)* — общее представление о работе приложения.
- *95-й/99-й перцентиль (P95, P99)* — важен для оценки худших сценариев и поведения под нагрузкой.
- *Максимальное время (max)* — позволяет обнаружить пиковые аномалии.

Инструменты для измерения:

- JMeter, Gatling, k6 — для синтетической нагрузки;
- APM-инструменты (например, New Relic, AppDynamics, Datadog) — для мониторинга в реальной среде.

https://max.ru/bookofgeek

2. Пропускная способность (Throughput)

Пропускная способность определяет количество успешно обработанных операций или запросов за определенный интервал времени (например, *запросов в секунду — RPS*, *транзакций в минуту — TPM*).

Эта метрика напрямую связана с масштабируемостью и эффективностью системы при высокой нагрузке.

Типичные сценарии использования **throughput**:

- для оценки производительности веб-сервера;
- при нагрузочном тестировании баз данных;
- в системах обработки сообщений (например, Kafka, RabbitMQ), где важна обработка тысяч событий в секунду.

На что влияет **throughput**:

- число активных потоков/соединений;
- задержки при взаимодействии с внешними ресурсами (БД, API);
- время блокировок при синхронизации или работе с I/O.

3. Использование процессора и памяти (CPU/Memory Usage)

Процессорные и **памятьевые ресурсы** — это базовая основа любой вычислительной среды. Перегрузка CPU или чрезмерное потребление памяти указывают на неэффективные алгоритмы, утечки памяти, избыточные потоки или ошибки в конфигурации виртуальной машины.

CPU Usage:

https://max.ru/bookofgeek

- Высокая загрузка CPU может быть как признаком продуктивной активности (например, интенсивных вычислений), так и свидетельством утечек, бесконечных циклов, неэффективных алгоритмов.
- Часто проблемы проявляются в виде CPU spikes — кратковременных, но резких скачков нагрузки.

Memory Usage:

- Постоянный рост потребления памяти может указывать на утечки (memory leaks) — объекты, которые не освобождаются сборщиком мусора.
- Важно отслеживать, как объекты перемещаются между поколениями кучи, особенно в Old Generation.

Инструменты мониторинга:

- jconsole, jvisualvm, JMC (Java Mission Control);
- APM-системы, а также top, htop, vmstat, jstat для анализа ресурсов на уровне ОС.

4. Активность сборщика мусора (GC Activity)

Java-программы работают с **автоматическим управлением памятью**, поэтому активность Garbage Collector — одна из главных метрик, особенно в системах с высоким временем отклика или требованиями к устойчивости.

Что важно отслеживать:

- Количество и частоту GC-событий (Minor и Major GC).
- Продолжительность пауз (Stop-The-World).
- Объем собираемой и неосвобождаемой памяти.
- Фрагментацию кучи.

https://max.ru/bookofgeek

Почему важно:

- Частые и долгие сборки мусора приводят к фризам, просадкам RPS и нарушению SLA.
- Правильная настройка GC (тип, параметры, размер поколений) позволяет стабилизировать поведение приложения под нагрузкой.

Полезные инструменты:

- *-Xlog:gc, -XX:+PrintGCDetails, -XX:+PrintGCTimeStamps* — флаги для логирования.
- VisualVM, JMC, GCeasy.io — для визуализации и анализа логов GC.

5. Количество потоков и их активность

Java-программы часто используют многопоточность, и неправильное управление потоками может привести к:

- избыточному переключению контекста (context switching);
- блокировкам (*synchronized*, *ReentrantLock*);
- утечкам ресурсов (незакрытые потоки).

Метрики по потокам:

- Общее количество активных потоков.
- Состояние потоков (RUNNABLE, BLOCKED, WAITING, TIMED_WAITING).
- Доля времени, которую потоки проводят в ожидании или блокировке.
- Эффективность использования пула потоков (в *ExecutorService* или *ForkJoinPool*).

Инструменты анализа:

- jstack, jcmd Thread.print — для создания thread dump.

https://max.ru/bookofgeek

- VisualVM, YourKit, IntelliJ Profiler — позволяют отследить блокировки и «мертвые» потоки.

1.3. Профилирование и анализ

Профилирование — один из важнейших этапов работы над производительностью Java-приложения. Оно позволяет не просто *измерить* работу кода, но и *понять*, где возникают узкие места, насколько эффективно расходуются ресурсы и какие участки требуют оптимизации.

Без регулярного профилирования любая оптимизация становится «стрельбой в темноту».

Что такое профилирование?

Профилирование — это процесс динамического анализа, направленный на измерение различных характеристик выполнения приложения: времени исполнения методов, количества вызовов, использования памяти, количества объектов, работы сборщика мусора, активности потоков и многого другого.

Главная цель — получить полную и точную картину поведения приложения в процессе его работы, найти «горячие точки» (hot spots) и устранить их.

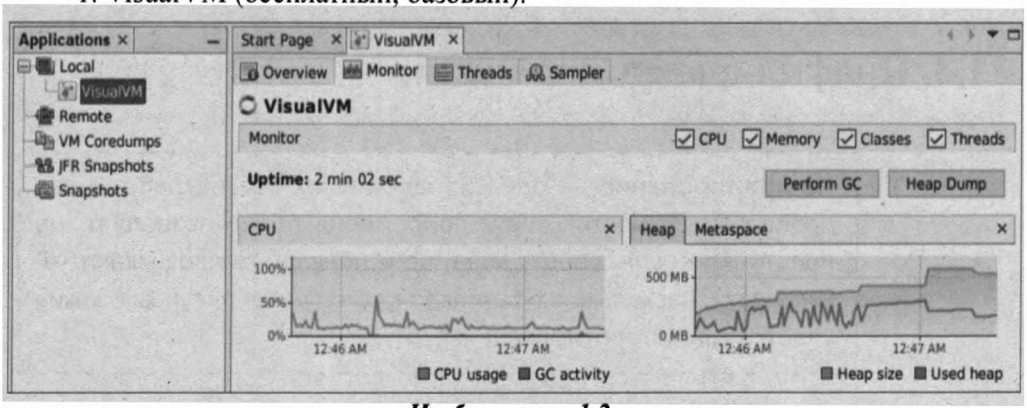
Ключевые задачи профилирования:

1. Оценка производительности кода на уровне методов, классов и пакетов.
2. Выявление утечек памяти (memory leaks).
3. Отслеживание длительных или избыточных операций ввода-вывода.
4. Анализ параллельной активности и блокировок между потоками.
5. Мониторинг и диагностика работы GC и его влияния на приложение.

https://max.ru/bookofgeek

Инструменты профилирования

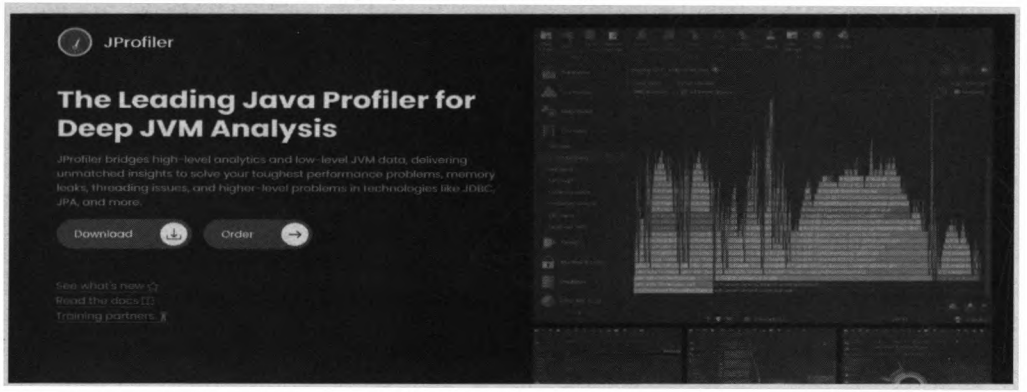
1. VisualVM (бесплатный, базовый).



Изображение 1.2.

- Поставляется вместе с JDK.
- Позволяет наблюдать в реальном времени:
  - » использование памяти (heap, permgen/metaspace);
  - » активность сборщика мусора;
  - » CPU-нагрузку по методам;
  - » потоки и их состояния;
  - » легкий анализ heap dump.
- Поддерживает плагины, например, для анализа GC или мониторинга MBeans.

2. JProfiler (платный, профессиональный).

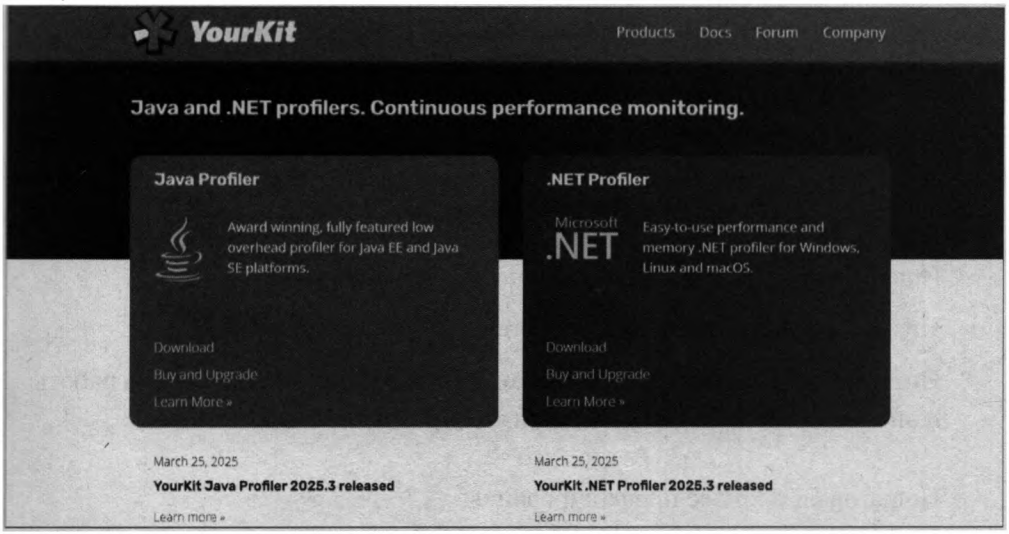


Изображение 1.3.

https://max.ru/bookofgeek

- Один из самых мощных инструментов профилирования.
- Возможности:
  - » Высокоточная CPU и memory-профилизация.
  - » Анализ SQL-запросов и JDBC-драйверов.
  - » Визуализация утечек памяти, контроль аллокаций объектов.
  - » Мгновенный захват snapshot'ов производительности.

3. YourKit (платный, флагманский инструмент).

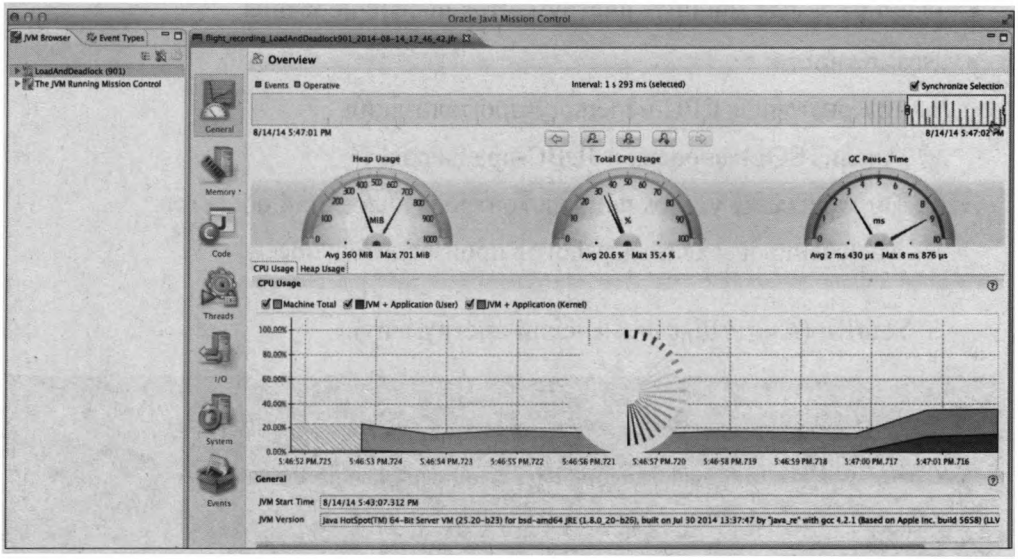


Изображение 1.4.

- Удобный, легкий в использовании интерфейс.
- Основные функции:
  - » Диагностика утечек.
  - » Анализ deadlock-ситуаций.
  - » Сравнение snapshot'ов до и после изменений в коде.
  - » Работа с большими dump-файлами без потери производительности.

4. Java Flight Recorder + Java Mission Control (входит в JDK, enterprise-уровень).

https://max.ru/bookofgeek



Изображение 1.5.

- Используются для детального анализа в продакшн-среде.
- Низкий overhead (нагрузка на систему минимальна).
- Инструменты позволяют записывать все поведение JVM во время работы и анализировать позже без влияния на производительность.

Пошаговый процесс профилирования:

1. **Определение цели анализа.**

   Что мы ищем: медленные методы? утечки? тормоза в GC? блокировки потоков?

2. **Выбор инструмента.**
   - На ранней стадии — VisualVM.
   - Для глубокой диагностики — JProfiler или YourKit.
   - В продакшене — Flight Recorder.

3. **Сбор данных:**
   - CPU-семплирование (sampling) или трассировка (tracing).
   - Метрики памяти (heap usage, retained size, allocation rate).
   - Thread dump и heap dump.

https://max.ru/bookofgeek

4. **Анализ «горячих точек».**
    - Определите функции, занимающие наибольшее процессорное время.
    - Оцените частоту аллокаций и количество живых объектов.
    - Отследите задержки на уровне ввода-вывода или синхронизации.
5. **Оптимизация.**
    - Упростите или перепишите неэффективные участки.
    - Используйте кеширование, асинхронность, пул потоков, более легкие структуры данных.
    - Настройте GC или измените параметры JVM, если проблема в сборке мусора.
6. **Повторное тестирование.**
    - Проведите сравнение метрик «до» и «после».
    - Оцените реальный прирост производительности.

1.4. Практические рекомендации

- Изолируйте и воспроизводите. Профилируйте отдельные модули или сценарии в тестовой среде, максимально приближенной к боевой.
- Измеряйте под нагрузкой. Профилирование без имитации реальных пользовательских сценариев часто не покажет проблем.
- Избегайте преждевременной оптимизации. Профилируйте только после того, как код стабилен и понятен.
- Сравнивайте snapshot'ы. Это помогает понять, действительно ли изменения в коде повлияли на производительность.
- Визуализируйте данные. Используйте flame graphs, диаграммы памяти и деревья вызовов (call trees) — они наглядны и показывают приоритеты.

Выводы

https://max.ru/bookofgeek

Производительность Java-приложений — это результат совокупности множества факторов: архитектурных решений, грамотной работы с памятью, правильного распределения нагрузки между потоками, а также умения выявлять и устранять узкие места в коде. Глубокое понимание внутреннего устройства JVM, ее жизненного цикла, особенностей сборки мусора и работы JIT-компилятора позволяет не только писать корректный код, но и добиваться от него максимальной эффективности.

> **Ключевые метрики** — такие как **время отклика**, **пропускная способность**, **использование ресурсов** и **активность GC** — дают нам основу для анализа поведения системы. Они становятся отправной точкой для принятия решений об оптимизации и доработке приложения. Без четкого контроля этих показателей невозможно построить масштабируемую, отказоустойчивую и отзывчивую систему.

Регулярное профилирование и мониторинг должны стать неотъемлемой частью процесса разработки, а не только экстренной мерой при возникновении проблем. Использование современных инструментов анализа, таких как VisualVM, YourKit, JProfiler или Java Flight Recorder, позволяет проводить не просто поверхностную диагностику, а глубокий разбор до уровня конкретных методов, потоков и объектов в памяти.

> **Комплексный подход** к **производительности** — это не только реакция на проблемы, но и упреждающая стратегия. Чем раньше в жизненном цикле проекта вы начнете применять практики анализа и оптимизации, тем меньше рисков и затрат возникнет в будущем.

Знания, изложенные в этой главе, закладывают прочный фундамент для дальнейшего изучения. Они помогут вам уверенно двигаться в сторону более тонкой настройки, продвинутых техник оптимизации, работы с большими нагрузками и построения по-настоящему высокопроизводительных Java-приложений.

https://max.ru/bookofgeek

Ещё больше свежей и актуальной литературы по Java
ты найдёшь в наших каналах:
https://max.ru/bookofgeek
https://t.me/bookofgeek

Глава 2.

HotSpot и JIT-оптимизации

https://max.ru/bookofgeek

О чём глава:

- Введение в HotSpot и JIT-компиляцию.
- Архитектура HotSpot.
- Принципы работы JIT-компилятора.
- Алгоритмы и техники JIT-оптимизации.
- Роль HotSpot в управлении памятью.
- Визуализация и мониторинг JIT-компиляции.
- Тюнинг JIT-компиляции.
- JIT и особенности многопоточных приложений.
- Инструменты для визуализации и анализа JIT-компиляции.
- Проблемы и ограничения JIT-компиляции.
- Применение JIT в реальных приложениях.
- Будущее JIT-оптимизаций.

2.1. Введение в HotSpot и JIT-компиляцию

Java — это язык программирования, который изначально задумывался с целью достижения переносимости между различными платформами. Это возможно благодаря концепции виртуальной машины Java (JVM), которая компилирует байт-код в нативный код во время выполнения.

Именно тут в дело вступает HotSpot — одна из самых распространенных реализаций JVM.

https://max.ru/bookofgeek

HotSpot является виртуальной машиной, используемой для выполнения байт-кода Java и других языков, работающих на JVM. Основной особенностью HotSpot является использование JIT-компиляции (Just-In-Time compilation), которая позволяет компилировать байт-код в нативный код уже на стадии выполнения программы. Это значительно ускоряет выполнение, так как компиляция происходит на лету, что позволяет использовать более эффективные оптимизации, основанные на реальном поведении приложения.

JIT-компиляция — это ключевая технология в HotSpot, которая изменяет динамическую природу исполнения Java-программ, позволяя улучшить производительность за счет оптимизации кода в процессе выполнения.

2.2. Архитектура HotSpot

Архитектура HotSpot включает несколько важных компонентов, обеспечивающих высокую производительность и гибкость работы JVM. Это:

- **Интерпретатор**. При первом запуске приложения байт-код исполняется интерпретатором, который выполняет каждую байт-кодовую инструкцию поочередно. Хотя интерпретация является медленной, этот процесс позволяет быстро запустить приложение и начать выполнение.
- **JIT-компилятор**. По мере выполнения программы JIT-компилятор анализирует байт-код и компилирует наиболее горячие участки (горячие методы или циклы) в нативный код. Это позволяет значительно ускорить выполнение тех частей приложения, которые часто вызываются.
- **Сборщик мусора (GC)**. HotSpot также включает в себя различные механизмы управления памятью, включая сборку мусора, что важным образом влияет на производительность.

https://max.ru/bookofgeek

- **C1 и C2 компиляторы**. Существует два уровня JIT-компиляции: C1 (простой компилятор) и C2 (более сложный компилятор), каждый из которых использует различные методы оптимизации. C2 используется для долгосрочной оптимизации и улучшения производительности на высоком уровне.
- **Загружаемые классы и динамическая компиляция**. HotSpot может динамически загружать классы и выполнять компиляцию только тех методов, которые необходимы на данный момент.

Таким образом, HotSpot представляет собой многоуровневую архитектуру, которая начинается с интерпретации байт-кода и постепенно переходит к компиляции наиболее используемых частей программы, улучшая тем самым общую производительность.

2.3. Принципы работы JIT-компилятора

JIT-компилятор в **JVM** выполняет компиляцию на лету — он анализирует код, который часто выполняется, и заменяет его на оптимизированную версию, которая будет работать быстрее. В отличие от традиционной компиляции, где весь код компилируется заранее, JIT-компиляция позволяет программам оптимизировать код непосредственно в процессе выполнения.

Принципы работы JIT-компилятора можно разделить на несколько этапов:

1. **Интерпретация**. Когда приложение запускается, JVM использует интерпретатор для выполнения байт-кода. Это позволяет быстро начать выполнение программы, но производительность низкая.

2. **Профилирование**. Во время работы программы JVM собирает информацию о часто вызываемых методах и участках кода (горячие участки). Эти данные используются для принятия решения о том, какие части кода нуждаются в компиляции.

https://max.ru/bookofgeek

3. **Компиляция**. На основе собранной информации JIT-компилятор компилирует наиболее часто используемые участки кода в нативный код. Этот процесс происходит на лету, что дает прирост производительности.

4. **Оптимизация**. JIT-компилятор использует различные алгоритмы оптимизации, такие как inlining (встраивание вызова метода в код), dead code elimination (удаление неиспользуемых частей кода) и loop unrolling (распаковка циклов), чтобы ускорить выполнение программы.

5. **Долгосрочная оптимизация**. В случае с C2 компилятором, JVM проводит более сложную оптимизацию, используя данные о поведении программы, чтобы еще больше улучшить производительность.

2.4. Алгоритмы и техники JIT-оптимизации

JIT-компиляция включает в себя множество алгоритмов и техник, направленных на улучшение производительности. Среди них:

- **Inlining (встраивание)**. Это процесс замены вызова метода непосредственно его кодом. Он уменьшает накладные расходы на вызов метода и может существенно ускорить выполнение программы, особенно в случае небольших методов, вызываемых часто.
- **Dead Code Elimination (удаление мертвого кода)**. Если часть кода никогда не будет исполнена (например, из-за условий ветвлений), то JIT-компилятор удаляет эти участки, освобождая ресурсы.
- **Loop Unrolling (распаковка циклов)**. Этот процесс заключается в замене циклов на несколько повторяющихся блоков кода. Это снижает накладные расходы на проверку условий цикла и увеличивает производительность.
- **Constant Folding (сведение констант)**. Это оптимизация, при которой вычисления с постоянными значениями выполняются на этапе компиляции, а не во время выполнения программы.

https://max.ru/bookofgeek

- **Escape Analysis (анализ выхода)**. Этот метод позволяет определить, могут ли объекты быть локальными для метода, и избежать их выделения в куче, что может снизить нагрузку на сборщик мусора.

2.5. Роль HotSpot в управлении памятью

HotSpot, помимо JIT-компиляции, также играет важную роль в управлении памятью. Он использует систему управления памятью, которая делит кучу на несколько поколений (Young Generation, Old Generation), что помогает эффективно организовать сборку мусора.

- **Young Generation** — это область памяти, где создаются новые объекты. Объекты, не пережившие несколько сборок мусора, часто удаляются здесь. Основной сборщик мусора, называемый **Minor GC**, работает в этом поколении.
- **Old Generation** — это область памяти, где живут долгоживущие объекты. Если объект пережил несколько сборок мусора, он перемещается сюда. Сборка мусора в этом поколении называется **Major GC**.
- **Сборщик мусора** — HotSpot использует различные алгоритмы для очистки памяти, в том числе **Serial GC**, **Parallel GC**, **G1 GC**, и **ZGC**, которые различаются по производительности и поведению.

Правильное управление памятью в HotSpot напрямую влияет на производительность. Например, частая работа с большим количеством объектов, которые долго живут, может привести к задержкам при сборке мусора, что в свою очередь влияет на время отклика приложения.

2.6. Визуализация и мониторинг JIT-компиляции

Для оценки эффективности JIT-компиляции и мониторинга работы JVM существуют различные инструменты. Одним из самых популярных является **JVisualVM** — визуальный инструмент для мониторинга и профилирования приложений Java.

https://max.ru/bookofgeek

JVisualVM позволяет отслеживать следующие параметры:

- **Использование процессора** — как часто используются методы, какие из них горячие.
- **Память** — сколько памяти использует приложение, и как часто происходят сборки мусора.
- **Профилирование** — позволяет увидеть, какие участки кода занимают больше всего времени.

Также можно использовать параметр *-XX:+PrintCompilation*, который позволяет выводить информацию о каждом методе, который был скомпилирован JIT. Эта информация полезна для понимания того, как работает JVM и какие методы оптимизируются.

2.7. Тюнинг JIT-компиляции

Тюнинг JIT-компиляции позволяет значительно улучшить производительность приложений Java.

Некоторые из опций, которые можно использовать для настройки JVM:

- **-XX:+TieredCompilation** — включение или выключение многоуровневой компиляции. Это полезно для баланса между временем старта приложения и производительностью.
- **-XX:CompileThreshold** — порог, после которого метод считается горячим и подлежит компиляции. Настройка этого значения позволяет контролировать, какие методы будут компилироваться JIT.
- **-XX:+AggressiveOpts** — включение более агрессивных оптимизаций для JIT-компиляции.

Тюнинг параметров JIT может помочь вам оптимизировать производительность вашего приложения, однако важно понимать, что чрезмерная настройка может привести к непредсказуемым результатам.

https://max.ru/bookofgeek

2.8. JIT и особенности многопоточных приложений

Многопоточность — это неотъемлемая часть современных Java-приложений.

JIT-компиляция должна учитывать следующие ее особенности для оптимальной работы:

1. **Синхронизация и блокировки**. При наличии блокировок и синхронизации JIT-компилятор должен учитывать, какие участки кода могут быть исполнены параллельно, а какие требуют синхронизации. Неоптимизированное использование блокировок может привести к значительным накладным расходам, даже если JIT-компиляция улучшает производительность других частей программы.

2. **Доступ к памяти**. Многопоточные приложения могут сталкиваться с проблемами гонок и многократного доступа к одной и той же области памяти. JIT-компилятор учитывает эти моменты при оптимизации кода, чтобы избежать чрезмерных накладных расходов, связанных с чтением и записью в память.

3. **Тема оптимизации с учетом параллелизма**. JIT-компилятор в Java пытается оптимизировать выполнение параллельных потоков, встраивая их в более эффективные кодовые фрагменты, что важно для улучшения производительности. Однако JIT не всегда может точно предсказать, как поведение одного потока повлияет на другие, особенно в сложных многопоточных приложениях.

Важным аспектом является использование **конкурентных структур данных** и **многопоточных фреймворков** (например, Fork/Join, Parallel Streams API), что позволяет улучшить параллелизм и более эффективно использовать JIT-компиляцию.

https://max.ru/bookofgeek

2.9. Инструменты для визуализации и анализа JIT-компиляции

Для того чтобы понимать, как работает JIT-компиляция и какие оптимизации кода применяются во время выполнения, существует ряд инструментов, которые помогают визуализировать и анализировать работу JIT-компилятора:

1. **JvisualVM** — один из самых популярных инструментов для мониторинга и профилирования приложений Java. Он позволяет отслеживать работу JVM, включая анализ работы JIT-компилятора. В JVisualVM можно увидеть статистику по скомпилированным методам, а также информацию о времени выполнения.

2. **JConsole** — еще один инструмент, который позволяет мониторить и анализировать поведение JVM. Он помогает отслеживать статистику сборщика мусора, работу JIT-компилятора, а также производительность памяти.

3. **Flight Recorder** — этот инструмент записывает все события, происходящие в JVM, и предоставляет информацию о том, как работает JIT-компилятор. Это позволяет получать данные о том, какие методы были скомпилированы и сколько времени потребовалось на выполнение этих операций.

4. **-XX:+PrintCompilation** — этот параметр JVM позволяет выводить информацию о каждом методе, который компилируется JIT. Вывод информации о скомпилированных методах помогает понять, какие части приложения требуют наибольшего внимания.

5. **Async-profiler** — это профилировщик, который позволяет визуализировать производительность многопоточных приложений и анализировать использование процессора, памяти и времени выполнения для каждого метода.

https://max.ru/bookofgeek

С помощью этих инструментов можно получить подробную информацию о работе JIT-компилятора и помочь в поиске «горячих» точек, которые могут быть улучшены.

2.10. Проблемы и ограничения JIT-компиляции

Несмотря на все преимущества JIT-компиляции, существуют несколько проблем и ограничений, которые необходимо учитывать:

1. **Задержки при старте**. Одной из проблем является то, что при первом запуске приложения JIT-компилятор не имеет данных о том, какие участки кода будут использоваться наиболее часто, и поэтому приходится полагаться на интерпретатор. Это может привести к увеличенной задержке при старте приложения.

2. **Память и нагрузка на сборщик мусора**. JIT-компилятор сохраняет скомпилированный код в памяти, что может привести к увеличению использования памяти. В случае с большими приложениями это может привести к проблемам с управлением памятью, особенно при высоких нагрузках.

3. **Неоптимизированные методы**. Некоторые методы или участки кода сложно оптимизировать с помощью JIT. Например, методы с большим количеством ветвлений или сложными вычислениями могут не получить значительного ускорения от JIT-компиляции.

4. **Трудности с многопоточностью**. Многопоточные приложения требуют особого внимания при использовании JIT, так как конкурентный доступ к памяти и синхронизация потоков могут создавать дополнительные накладные расходы, которые JIT не всегда может эффективно оптимизировать.

5. **Неопределенность**. Поскольку JIT-компиляция зависит от поведения приложения в реальном времени, трудно точно предсказать, какие оптимизации будут применены. Это может привести к нестабильному поведению приложения в зависимости от изменения нагрузки.

https://max.ru/bookofgeek

2.11. Применение JIT в реальных приложениях

JIT-компиляция в реальных приложениях может существенно ускорить выполнение задач, но важно учитывать, как именно она применяется в различных типах программ. Например:

1. **Веб-приложения** — для веб-приложений JIT-компиляция помогает ускорить выполнение часто используемых методов и циклов, что снижает время отклика. Важно также учитывать нагрузку на сервер и влияние JIT на обработку параллельных запросов.
2. **Многопоточные приложения** — JIT-компиляция помогает ускорить выполнение многопоточных приложений, но также важно учитывать влияние блокировок и синхронизации на производительность.
3. **Обработка больших данных** — в приложениях для обработки больших данных JIT-компиляция может ускорить выполнение алгоритмов обработки и анализа данных, однако из-за большого объема данных стоит внимательно следить за потреблением памяти.
4. **Приложения с низкими требованиями к времени отклика** — в приложениях с низкими требованиями ко времени отклика (например, для долгосрочных вычислений или обработки данных в фоновом режиме) JIT-компиляция может оказать меньшую пользу, так как компиляция происходит в реальном времени и может повлиять на начальную задержку.

2.12. Будущее JIT-оптимизаций

JIT-компиляция продолжает развиваться, и будущее этого подхода связано с улучшением гибкости и адаптивности. Возможности для дальнейшего совершенствования следующие:

1. **Адаптивная оптимизация**. Современные подходы в JIT-компиляции включают адаптивные алгоритмы, которые могут менять стратегию оптимизации в зависимости от данных и нагрузки. Это позволяет добиться лучших результатов на всех этапах работы приложения.

https://max.ru/bookofgeek

2. **Интеграция с другими технологиями**. JIT-компиляция продолжает интегрироваться с другими технологиями, такими как контейнеризация и облачные вычисления, что позволяет приложению работать более эффективно на различных инфраструктурах.

3. **Использование искусственного интеллекта и машинного обучения**. В будущем можно ожидать использование машинного обучения для прогнозирования, какие части кода следует оптимизировать в первую очередь, исходя из поведения приложения. Это может ускорить процесс оптимизации и улучшить результаты.

4. **Инновации в многопоточности**. Современные подходы к многопоточности и параллелизму, такие как Project Loom, могут значительно изменить подходы к JIT-оптимизации, улучшая параллельное выполнение кода и снижая накладные расходы на синхронизацию.

Таким образом, будущее JIT-компиляции обещает значительные улучшения в производительности и возможностях оптимизации приложений Java, что откроет новые горизонты для разработки эффективных и высокопроизводительных решений.

Выводы

JIT-компиляция в **Java** является одним из наиболее мощных инструментов для оптимизации производительности приложений. Важность HotSpot и JIT-компилятора нельзя недооценивать, поскольку они обеспечивают динамическую адаптацию кода в реальном времени, что значительно улучшает работу приложений по мере их эксплуатации.

Благодаря таким технологиям, как **JIT** и **HotSpot**, Java-программы могут выполнять вычисления быстрее, снижая накладные расходы на интерпретацию и улучшая использование процессорных ресурсов.

https://max.ru/bookofgeek

Однако, как мы видели в этой главе, JIT-компиляция не лишена своих ограничений и сложностей. Она не всегда идеальна для всех типов приложений, и оптимизация с её помощью требует внимательного подхода и тщательного мониторинга. Проблемы с памятью, многопоточностью и трудности предсказуемости оптимизаций остаются актуальными при использовании JIT, и именно поэтому важно понимать, как эффективно интегрировать и настраивать эти технологии в реальных приложениях.

Сложность, которая возникает при работе с многопоточными и распределёнными системами, не должна стать препятствием для использования JIT-оптимизаций. Напротив, именно эти системы могут извлечь максимальную выгоду от правильно настроенного JIT, значительно увеличив свою производительность и масштабируемость. Тем не менее, для того чтобы JIT-компиляция работала максимально эффективно, нужно правильно анализировать поведение приложения в реальных условиях, использовать инструменты мониторинга и визуализации, а также проводить тщательное тестирование.

Будущее JIT-компиляции, как и всей экосистемы Java, заключается в непрерывной адаптации и совершенствовании. Прогресс в области адаптивных оптимизаций, интеграции с другими технологиями, такими как машинное обучение и улучшенная многопоточность, обещает новые возможности для повышения производительности.

Технологии, такие как **Project Loom**, могут значительно изменить способы работы JIT-компиляции, открывая новые горизонты в управлении параллелизмом и улучшении эффективности работы многозадачных приложений.

Таким образом, успешная работа с JIT-компиляцией требует от разработчика глубокого понимания внутренней структуры JVM, способности анализировать результаты и настраивать компиляцию в зависимости от специфики приложения.

Важно помнить, что, хотя JIT и является мощным инструментом, без соответствующей настройки и понимания его работы он может привести к неоптимальным результатам. Поэтому для достижения наилучших результатов в производительности необходимо сочетать использование JIT с комплексным мониторингом, анализом и настройкой, а также учитывать особенности многопоточных и распределённых систем.

https://max.ru/bookofgeek

С учётом всех этих факторов, JIT-компиляция продолжит оставаться неотъемлемой частью высокопроизводительных Java-приложений, предоставляя разработчикам уникальные возможности для улучшения работы их кода.

https://max.ru/bookofgeek

Глава 3.

Введение в JMH (Java Microbenchmark Harness)

https://max.ru/bookofgeek

О чём глава:

- Что такое JMH и зачем он нужен.
- Установка и настройка JMH.
- Основные концепции и аннотации JMH.

3.1. Что такое JMH и зачем он нужен

Одной из наиболее сложных задач при оптимизации Java-приложений является точное и воспроизводимое измерение производительности.

Простой вызов *System.nanoTime()* или *System.currentTimeMillis()* не позволяет учитывать все нюансы выполнения кода на JVM, такие как прогрев JIT-компилятора, влияние сборки мусора, конкуренция потоков и флуктуации нагрузки на систему. В этом контексте Java Microbenchmark Harness (JMH) становится незаменимым инструментом.

JMH — это специализированная библиотека, разработанная командой OpenJDK, предназначенная для создания микробенчмарков, то есть миниатюрных тестов, направленных на измерение производительности отдельных операций или алгоритмов. Она позволяет устранять типичные ошибки измерения и обеспечивает высокую точность и повторяемость результатов.

JMH учитывает такие аспекты, как:

- Прогрев JVM и JIT-компиляцию.

https://max.ru/bookofgeek

- Управление состоянием между итерациями тестов.
- Контроль количества итераций, прогонов и уровня изоляции потоков.
- Минимизация шума от фоновых процессов и мусоросборщика.

Применение JMH особенно важно, когда необходимо:

- Сравнить два варианта реализации метода.
- Оценить влияние изменений в алгоритме.
- Выявить регрессии производительности между версиями.

3.2. Установка и настройка JMH

Maven

Для интеграции JMH в проект, использующий Maven, необходимо добавить зависимости и настроить плагины.

Пример конфигурации *pom.xml:*

```
<dependencies>
    <dependency>
        <groupId>org.openjdk.jmh</groupId>
        <artifactId>jmh-core</artifactId>
        <version>1.36</version>
    </dependency>
    <dependency>
        <groupId>org.openjdk.jmh</groupId>
        <artifactId>jmh-generator-annprocess</artifactId>
        <version>1.36</version>
        <scope>provided</scope>
    </dependency>
</dependencies>

<build>
```


https://max.ru/bookofgeek

```
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <version>3.8.1</version>
            <configuration>
                <source>1.8</source>
                <target>1.8</target>
                <annotationProcessorPaths>
                    <path>
                        <groupId>org.openjdk.jmh</groupId>
                        <artifactId>jmh-generator-annprocess</
artifactId>
                        <version>1.36</version>
                    </path>
                </annotationProcessorPaths>
            </configuration>
        </plugin>
        <plugin>
            <groupId>org.openjdk.jmh</groupId>
            <artifactId>jmh-maven-plugin</artifactId>
            <version>0.5.0</version>
            <executions>
                <execution>
                    <goals>
                        <goal>generate</goal>
                        <goal>run</goal>
                    </goals>
                </execution>
            </executions>
        </plugin>
    </plugins>
</build>
```

Для запуска бенчмарков используйте:

```
mvn clean install
mvn jmh:run
```


https://max.ru/bookofgeek

Gradle

Для пользователей Gradle настройка осуществляется следующим образом:

```
plugins {
    id 'java'
    id 'org.openjdk.jmh' version '0.5.0'
}
dependencies {
    implementation 'org.openjdk.jmh:jmh-core:1.36'
    annotationProcessor 'org.openjdk.jmh:jmh-generator-
annprocess:1.36'
}
repositories {
    mavenCentral()
}
```

Запуск бенчмарков осуществляется командой:

```
./gradlew jmh
```

3.3. Основные концепции и аннотации JMH

@Benchmark

Помечает метод как тестируемый. Метод должен быть *public*, не принимать аргументов и обычно ничего не возвращать:

```
@Benchmark
public void measureSort() {
    // Тестируемый код
}
```

@BenchmarkMode

Указывает, какая метрика будет использоваться.

https://max.ru/bookofgeek

Возможные режимы:

- **Throughput** — количество операций в секунду.
- **AverageTime** — среднее время выполнения одной операции.
- **SampleTime** — выборочные замеры.
- **SingleShotTime** — одиночный вызов, без прогрева.
- **AllModes** — все вышеперечисленные.

```
@Benchmark
@BenchmarkMode(Mode.AverageTime)
public void measureAverageTime() {
    // Код
}
```

@State

Позволяет создавать и использовать объекты состояния между запусками:

```
@State(Scope.Thread)
public class MyState {
    List<Integer> list;
    @Setup(Level.Trial)
    public void prepare() {
        list = IntStream.range(0, 1000).boxed().
collect(Collectors.toList());
    }
}
```

@Setup и @TearDown

Обеспечивают подготовку и очистку окружения:

```
@Setup(Level.Iteration)
public void setup() {
    // Инициализация
}
```


https://max.ru/bookofgeek

```
@TearDown(Level.Iteration)
public void cleanup() {
    // Очистка
}
```

Пример бенчмарка

```
public class ExampleBenchmark {

    @State(Scope.Thread)
    public static class BenchmarkState {
        List<Integer> list;

        @Setup(Level.Invocation)
        public void setup() {
            list = new ArrayList<>(1000);
            for (int i = 0; i < 1000; i++) {
                list.add(ThreadLocalRandom.current().nextInt());
            }
        }
    }

    @Benchmark
    @BenchmarkMode(Mode.AverageTime)
    @OutputTimeUnit(TimeUnit.MILLISECONDS)
    public List<Integer> testSort(BenchmarkState state) {
        List<Integer> sorted = new ArrayList<>(state.list);
        Collections.sort(sorted);
        return sorted;
    }
}
```

Анализ результатов

JMH по завершении тестов выводит таблицу с результатами.

https://max.ru/bookofgeek

Пример:

```
Benchmark                        Mode  Cnt   Score   Error  Units
ExampleBenchmark.testSort        avgt   25   2.543 ± 0.021  ms/op
```

Изображение 3.1.

Где:

- *Mode* — режим измерения (например, avgt — AverageTime).
- *Cnt* — количество прогонов.
- *Score* — средний результат.
- *Error* — статистическая погрешность.
- *Units* — единицы измерения.

Для детального анализа можно указать опции, такие как:

- *-wi* — количество прогревочных итераций;
- *-i* — количество измерительных итераций;
- *-f* — число форков.

Выводы

JMH — это мощный и специализированный инструмент, позволяющий Java-разработчикам проводить точные и надёжные измерения производительности.

В отличие от наивных подходов, таких как использование *System.nanoTime()*, JMH устраняет целый ряд проблем: от влияния JIT-компилятора и сборки мусора до многопоточности и прогрева JVM.

https://max.ru/bookofgeek

В рамках этой главы мы последовательно рассмотрели:

- что такое JMH, как он работает и какие задачи он решает;
- как подключить JMH к проекту через Maven и Gradle;
- основные аннотации JMH и их назначение: *@Benchmark, @BenchmarkMode, @State, @Setup, @TearDown;*
- пример полноценного бенчмарка, включая генерацию данных, замер производительности и возврат результата;
- интерпретацию результатов тестирования и настройку параметров, влияющих на точность измерений.

Важно понимать, что микробенчмарки — это не просто средства измерения, а полноценные инструменты анализа архитектуры и качества реализации. С их помощью можно:

- сравнивать разные алгоритмы и структуры данных;
- находить узкие места и потенциальные проблемы производительности;
- доказывать или опровергать гипотезы о том, как изменения в коде влияют на скорость выполнения;
- обеспечивать контроль регрессий производительности при рефакторинге и добавлении новых фичей.

JMH позволяет не только получить численные метрики, но и мыслить системно: понимать, почему один подход работает быстрее другого, как влияют настройки JVM и какие части кода требуют внимания в первую очередь. Освоение JMH — важный шаг для любого Java-инженера, стремящегося к высокой эффективности приложений.

В последующих главах мы углубимся в более сложные сценарии использования JMH, включая настройку форков, взаимодействие с потоками, профилирование с внешними инструментами и интеграцию с CI/CD. Это поможет расширить представление о производительности и научиться строить устойчивые, быстрые и масштабируемые решения на платформе JVM.

https://max.ru/bookofgeek

https://max.ru/bookofgeek

Глава 4.

Создание и выполнение бенчмарков с JMH

https://max.ru/bookofgeek

О чём глава:

- Структура простого бенчмарка.
- Настройка параметров и режимов работы.
- Анализ результатов бенчмарков.

Один из самых важных аспектов разработки высокопроизводительных Java-приложений — это возможность точно измерять и анализировать производительность различных компонентов кода. Однако, как бы вы ни старались оптимизировать свой код, без точных и воспроизводимых измерений вы не сможете быть уверены, что изменения действительно улучшили производительность.

> В этом контексте Java Microbenchmark Harness (JMH) становится незаменимым инструментом. Этот фреймворк позволяет проводить высокоточные и воспроизводимые тесты производительности для Java-программ и JVM-операций. JMH специально создан для устранения множества типичных ошибок, которые могут возникать при попытке измерить производительность на JVM.

JMH предназначен для разработки микробенчмарков — тестов, которые измеряют производительность кода с минимальными затратами ресурсов. В отличие от стандартных инструментов для измерения времени работы кода, JMH берет на себя управление многими аспектами работы JVM, такими как оптимизация JIT-компилятора, влияние сборки мусора и многозадачность, что делает его незаменимым для точных измерений. Это позволяет разработчику точно выявлять узкие места в коде, корректно измерять время выполнения и выполнять стресс-тесты в различных условиях.

https://max.ru/bookofgeek

В этой главе мы рассмотрим процесс создания и выполнения бенчмарков с использованием JMH. Мы научимся правильно структурировать тесты, настраивать параметры тестов и анализировать полученные результаты. Подробно разберем каждый этап, от написания простого бенчмарка до детальной интерпретации результатов.

4.1. Структура простого бенчмарка

Простой бенчмарк с использованием JMH состоит из нескольких основных элементов, включая аннотации, методы для измерения времени выполнения, а также настройки, которые обеспечивают точность и повторяемость тестов. Давайте начнем с базовой структуры, которая включает в себя только сам бенчмарк, класс состояния и необходимые аннотации.

4.1.1. Основные элементы

Вот пример самой базовой структуры JMH-бенчмарка:

```
import org.openjdk.jmh.annotations.*;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

@State(Scope.Thread) // Указываем, что каждый поток будет
использовать свой экземпляр состояния
public class SortingBenchmark {

    // Создаем список данных, которые будем сортировать
    private List<Integer> data;

    // Аннотация Setup позволяет выполнить инициализацию
данных перед запуском бенчмарков
    @Setup(Level.Trial)
    public void setup() {
```


https://max.ru/bookofgeek

```
        data = new ArrayList<>(1000);  // Инициализируем
список с 1000 элементов
        for (int i = 0; i < 1000; i++) {
            data.add(i);  // Заполняем список числами от 0 до
999
        }
    }

    // Основной метод, который будет измеряться
    @Benchmark
    @BenchmarkMode(Mode.AverageTime) // Режим измерения
среднего времени выполнения
    @OutputTimeUnit(TimeUnit.MILLISECONDS) // Единица
измерения - миллисекунды
    public List<Integer> testSort() {
        List<Integer> sorted = new ArrayList<>(data);
        Collections.sort(sorted);  // Сортируем данные с
использованием стандартной коллекции
        return sorted;
    }
}
```

4.1.2. Пояснения к коду

1. **@State(Scope.Thread)** — эта аннотация указывает, что каждый поток будет иметь свой собственный экземпляр состояния (в данном случае список данных). Важно, чтобы аннотация *@State* использовалась для создания независимого состояния, которое будет использоваться в тестах.
2. **@Setup(Level.Trial)** — метод, помеченный этой аннотацией, будет выполняться перед запуском тестов. В данном примере он инициализирует список данных, который будет использоваться в тестах. *Level.Trial* означает, что метод будет вызван один раз перед всеми тестами, выполняемыми на данном наборе данных.
3. **@Benchmark** — это основная аннотация, которая помечает метод как бенчмарк. JMH будет измерять время выполнения этого метода. Важно, чтобы метод был публичным и не принимал аргументов (если не используется *@Param*).

https://max.ru/bookofgeek

4. **@BenchmarkMode(Mode.AverageTime)** — определяет режим, в котором будет измеряться производительность. В данном случае используется режим *AverageTime*, который измеряет среднее время выполнения операции за итерацию.
5. **@OutputTimeUnit(TimeUnit.MILLISECONDS)** — указывает единицу измерения для результатов бенчмарка. В этом примере время будет измеряться в миллисекундах.
6. **Collections.sort(sorted)** — стандартный метод для сортировки списка. В рамках бенчмарка мы создаем копию списка, чтобы сохранить оригинальный неизменным.

4.2. Настройка параметров и режимов работы

4.2.1. Параметры бенчмарков

JMH позволяет гибко настраивать параметры тестов, такие как количество прогонов, количество итераций, режимы работы и т. д. Это позволяет точно настроить тесты в зависимости от требуемых условий.

Пример настройки параметров:

```
@Benchmark
@BenchmarkMode(Mode.Throughput) // Измерение пропускной
способности (количество операций в секунду)
@Warmup(iterations = 3) // Прогревка бенчмарков для JIT-
компилятора
@Measurement(iterations = 5) // Количество измерительных
итераций
@Fork(2) // Количество форков JVM для выполнения теста
@OutputTimeUnit(TimeUnit.MILLISECONDS) // Единицы времени для
вывода
public void testSort() {
    List<Integer> sorted = new ArrayList<>(data);
    Collections.sort(sorted);
}
```


https://max.ru/bookofgeek

В этом примере добавлены дополнительные параметры:

- **@Warmup(iterations = 3)** — параметр прогрева, определяющий количество итераций, которые будут выполняться для прогрева JVM и JIT-компилятора. Прогрев позволяет избежать влияния некомпилированного кода на результаты теста.
- **@Measurement(iterations = 5)** — указывает, сколько итераций будет выполняться в рамках теста для измерения результатов. После прогрева будут выполнены 5 измерительных итераций.
- **@Fork(2)** — указывает, сколько раз нужно "форкать" (запускать) JVM для выполнения теста. Это позволяет избежать влияния состояния одного прогрева на другие.

4.2.2. Режимы работы

JMH предоставляет несколько режимов работы для измерения производительности:

- **Mode.Throughput** — измеряет количество операций, выполненных за единицу времени (например, операции в секунду).
- **Mode.AverageTime** — измеряет среднее время выполнения операции.
- **Mode.SampleTime** — собирает выборочные данные о времени выполнения операций.
- **Mode.SingleShotTime** — измеряет время выполнения одной операции без прогрева.

Пример использования различных режимов:

```
@Benchmark
@BenchmarkMode({Mode.AverageTime, Mode.Throughput})
@OutputTimeUnit(TimeUnit.MILLISECONDS)
public List<Integer> testSort() {
    List<Integer> sorted = new ArrayList<>(data);
    Collections.sort(sorted);
    return sorted;
}
```


https://max.ru/bookofgeek

4.3. Анализ результатов бенчмарков

После выполнения тестов JMH выводит результаты в консоль. Результаты показывают, сколько времени заняло выполнение тестов, количество выполненных операций и погрешность.

Рассмотрим пример вывода:

```
Benchmark                    Mode   Cnt     Score     Error     Units
SortingBenchmark.testSort    avgt   25      1.234     ± 0.023   ms/op
```

Здесь:

- **Benchmark** — имя теста.
- **Mode** — режим измерения (например, *avgt* означает измерение среднего времени).
- **Cnt** — количество прогонов.
- **Score** — средний результат (в данном случае среднее время выполнения).
- **Error** — погрешность измерений.
- **Units** — единицы измерения (например, миллисекунды на операцию).

Выводы

JMH предоставляет разработчикам мощный инструмент для создания и выполнения точных бенчмарков в Java. Этот инструмент не только устраняет типичные ошибки, которые могут возникать при использовании стандартных методов измерения времени, но и позволяет детально настраивать параметры тестов для наиболее точных и повторяемых результатов.

В этой главе мы рассмотрели, как правильно создавать бенчмарки с использованием JMH, а также как настроить параметры для достижения точных и надежных результатов. Точное измерение производительности критически важно для оптимизации приложений и выявления узких мест в коде.

https://max.ru/bookofgeek

В дальнейшем использование JMH поможет вам проводить более качественную работу по оптимизации Java-программ и повышению их производительности.

https://max.ru/bookofgeek

Ещё больше свежей и актуальной литературы по Java
ты найдёшь в наших каналах:
https://max.ru/bookofgeek
https://t.me/bookofgeek

Глава 5.

Оптимизация алгоритмов и структур данных

https://max.ru/bookofgeek

О чём глава:

- Краткий обзор структур данных.
- Выбор правильных структур данных.
- Оптимизация алгоритмов сортировки и поиска.
- Практические примеры с использованием JMH.
- Детальный разбор деревьев и графов.
- Примеры использования деревьев и графов для оптимизации.

Следующая глава будет носить преимущественно **теоретический характер**, и я сразу хочу предупредить: если у вас нет базовых знаний в области алгоритмов и структур данных — чтение этого материала может оказаться непростым. В таком случае вы можете либо отложить изучение главы на потом, либо пробежаться по ней «по диагонали», обращая внимание лишь на ключевые идеи и общие принципы.

Тем не менее, я настоятельно рекомендую не пропускать ее полностью. Почему? Потому что именно теоретические основы — это фундамент, на котором строится вся дальнейшая работа по **оптимизации производительности**. Без понимания того, как устроены алгоритмы, как они взаимодействуют с памятью, какие существуют структуры данных и чем они отличаются друг от друга, невозможно по-настоящему эффективно решать задачи, связанные с повышением скорости, экономией ресурсов и улучшением масштабируемости приложений.

Эта глава — не просто академическое отступление. Это **основа**, без которой последующая практика может показаться набором бессистемных советов и решений.

https://max.ru/bookofgeek

Мы постараемся объяснить, **почему одни подходы работают лучше других, когда стоит использовать конкретные структуры данных** и **какие ошибки могут приводить к деградации производительности**. Все это поможет не только избежать критических узких мест в коде, но и научиться думать о производительности как о системной, стратегической задаче — а не просто о попытке «ускорить метод в два раза».

Поэтому даже если вы чувствуете, что теоретическая часть — не ваше, дайте этой главе шанс. Постарайтесь уловить основную идею, логику, которая стоит за алгоритмами и структурами. Возможно, вам не нужно будет запоминать все детали, но понимание общих принципов даст вам ту самую уверенность, с которой вы сможете принимать правильные архитектурные решения в будущем.

5.1. Краткий обзор структур данных

Структуры данных представляют собой фундаментальные строительные блоки для организации, хранения и обработки данных в программных системах. Они обеспечивают эффективный доступ, манипуляцию и анализ данных, и их правильный выбор влияет на производительность всего приложения.

Структуры данных можно **классифицировать** на **различные типы**, каждый из которых подходит для решения специфичных задач. Давайте более подробно рассмотрим каждую структуру.

5.1.1. Массивы

Массивы — это коллекции однотипных элементов, которые размещаются в памяти последовательно. Доступ к элементам осуществляется с помощью индексов, что позволяет получать элементы за время O(1).

https://max.ru/bookofgeek

Массивы являются одной из самых старых и простых структур данных, однако, несмотря на свою простоту, они имеют определенные ограничения. В частности, массивы имеют фиксированный размер, что может стать проблемой, если необходимо работать с динамическим набором данных.

Пример использования массива:

- Хранение списка студентов в классе, где размер не изменяется часто.
- Приложения с ограниченными данными, где память и скорость доступа критичны.

Плюсы:

- Быстрый доступ к данным.
- Малая нагрузка на систему при создании и использовании.

Минусы:

- Ограниченность в размере, что делает работу с изменяющимися данными неэффективной.
- Плохая производительность при вставке или удалении элементов в середину массива.

5.1.2. Связанные списки (Linked List)

Связанные списки — это динамические структуры данных, в которых элементы связаны между собой с помощью ссылок. Каждый элемент (узел) хранит данные и указатель на следующий узел. Это позволяет гибко управлять памятью и добавлять или удалять элементы без значительных затрат.

Типы связанных списков:

- **Односвязные списки** — каждый узел содержит ссылку только на следующий.

https://max.ru/bookofgeek

- **Двусвязные списки** — узлы содержат ссылки как на следующий, так и на предыдущий элемент.
- **Циклические списки** — последний узел ссылается на первый.

Пример использования связанного списка:

- Реализация стеков и очередей, где нужно часто добавлять и удалять элементы.

Плюсы:

- Эффективная вставка и удаление элементов в начале или середине списка.
- Динамическое распределение памяти.

Минусы:

- Сложность в доступе к элементам по индексу (требуется пройти по всему списку).
- Дополнительная память для хранения указателей.

5.1.3. Стек (Stack)

Стек — это структура данных, работающая по принципу LIFO (Last In, First Out), то есть последний добавленный элемент будет первым удален.

Стек идеально подходит для задач, где требуется обработка данных в порядке их поступления.

Пример использования стека:

- Реализация отмены действий в графическом редакторе.
- Поддержка рекурсивных вызовов в программах.

Плюсы:

https://max.ru/bookofgeek

- Эффективная работа с добавлением и удалением элементов.
- Используется для решения задач, связанных с хранением состояния.

Минусы:

- Ограниченный доступ к элементам — только с вершины стека.
- Невозможность произвольного доступа к элементам.

5.1.4. Очередь (Queue)

Очередь работает по принципу FIFO (First In, First Out). Это значит, что первый добавленный элемент будет первым удален.

Очереди широко применяются в системах, где требуется обработка элементов в порядке их поступления, например в задачах, связанных с распределением ресурсов.

Типы очередей:

- **Очереди с приоритетом** — очереди, где элементы имеют приоритет и обрабатываются в зависимости от этого приоритета.
- **Циклические очереди** — очереди с кольцевой структурой, где последний элемент ссылается на первый.

Пример использования очереди:

- Ожидание в очереди на обработку (например, задачи в многозадачном приложении).
- Системы распределенных вычислений.

Плюсы:

- Простота реализации.
- Подходит для задач, где важен порядок обработки данных.

https://max.ru/bookofgeek

Минусы:

- Неэффективно при необходимости произвольного доступа к элементам.

5.1.5. Хеш-таблица (Hash Table)

Хеш-таблица — это структура данных, которая позволяет хранить ассоциированные ключи и значения. С помощью хеш-функции ключ преобразуется в индекс, по которому сохраняется соответствующее значение.

Хеш-таблицы обеспечивают быстрый доступ к данным с амортизированным временем O(1).

Пример использования хеш-таблицы:

- Реализация словарей или индексов в базах данных.
- Использование кэширования для ускорения работы приложений.

Плюсы:

- Быстрый доступ по ключу.
- Хорошо работает с большими объемами данных.

Минусы:

- Возможные коллизии (если несколько ключей имеют одинаковые хеши).
- Память может быть неэффективно использована при малой нагрузке.

5.1.6. Дерево (Tree)

Дерево — это иерархическая структура данных, в которой каждый узел может иметь несколько дочерних узлов.

https://max.ru/bookofgeek

Деревья используются для представления иерархий и позволяют эффективно выполнять поиск, вставку и удаление элементов.

Типы деревьев:

- **Бинарное дерево поиска** (BST) — каждый узел в дереве имеет два потомка, и значения в левом поддереве меньше значения узла, а в правом — больше.
- **Красно-черное дерево** — самобалансирующееся дерево поиска, которое обеспечивает сбалансированное выполнение операций.

Пример использования дерева:

- Системы поиска (например, базы данных).
- Реализация алгоритмов минимизации и оптимизации.

Плюсы:

- Быстрый поиск, вставка и удаление элементов.
- Хорошо подходит для работы с упорядоченными данными.

Минусы:

- Требуют дополнительных ресурсов для поддержания баланса.
- Сложность в реализации и анализе.

5.1.7. Графы (Graphs)

Графы представляют собой коллекцию узлов (вершин) и ребер, которые соединяют эти узлы. Графы могут быть ***ориентированными*** или ***неориентированными***, взвешенными или невзвешенными, и применяются для решения широкого спектра задач, включая поиск кратчайших путей, планирование и маршрутизацию.

Пример использования графа:

https://max.ru/bookofgeek

- Карты и маршруты (например, поиск кратчайшего пути между двумя городами).
- Моделирование сетевых соединений или зависимостей.

Плюсы:

- Универсальность и гибкость.
- Подходят для моделирования сложных взаимоотношений.

Минусы:

- Высокие требования к памяти и вычислительным ресурсам.
- Неэффективность при использовании матриц смежности для хранения больших графов.

5.2. Выбор правильных структур данных

Выбор подходящей структуры данных зависит от множества факторов: от типов операций, которые необходимо выполнять, до ограничений по времени и памяти. Например, если в вашей задаче важен быстрый доступ к данным, хеш-таблицы могут быть оптимальным выбором.

Если же нужно работать с упорядоченными данными, стоит обратить внимание на деревья.

Параметры, которые следует учитывать при выборе структуры данных:

- **Тип операции**: частый ли доступ к элементам или добавление/удаление элементов?
- **Требования к памяти**: сколько памяти необходимо для хранения данных и как она будет использоваться?
- **Наличие ограничений по времени**: насколько важно время отклика системы?

https://max.ru/bookofgeek

5.3. Оптимизация алгоритмов сортировки и поиска

Алгоритмы сортировки и поиска играют ключевую роль в большинстве вычислительных задач. Рассмотрим, как их можно оптимизировать.

5.3.1. Оптимизация алгоритмов сортировки

- **Быстрая сортировка.** Несмотря на свою эффективность в среднем случае, быстрые сортировки могут иметь худшую производительность, если опорный элемент выбран неудачно. Для улучшения работы алгоритма можно использовать медиану элементов для выбора опорного элемента.
- **Сортировка слиянием**. Алгоритм, который разделяет данные на части и затем сливает их в отсортированном порядке. Оптимизации включают параллельное выполнение на многозадачных системах.
- **Тимсорт**. Комбинированный алгоритм, который использует преимущества сортировки вставками и слиянием для достижения максимальной производительности.

5.3.2. Оптимизация алгоритмов поиска

- **Бинарный поиск**. Один из самых эффективных алгоритмов для поиска в отсортированных данных. Его можно улучшить с помощью поиска с использованием кучи или более сложных структур данных, таких как B-деревья.
- **Поиск в графах**. Использование методов поиска в глубину и ширину для нахождения путей и минимизации затрат.

Сортировка слиянием

Сортировка слиянием — это алгоритм сортировки, основанный на принципе «разделяй и властвуй». Он разделяет входной массив на две половины, рекурсивно сортирует каждую половину, а затем сливает отсортированные половины в один отсортированный массив.

https://max.ru/bookofgeek

Это стабильный алгоритм сортировки с временной сложностью O(n log n), что делает его весьма эффективным для сортировки больших данных.

Оптимизация сортировки слиянием

- **Параллельная сортировка**. При работе с большими объемами данных полезно использовать параллельную сортировку. Например, можно распараллелить слияние подмассивов в многозадачных системах, чтобы ускорить выполнение. Такой подход значительно улучшает производительность при наличии нескольких процессоров.
- **Итеративная сортировка слиянием**. Вместо рекурсивного подхода можно использовать итеративную сортировку слиянием, что может снизить расходы на стек вызовов, особенно для больших данных.

Пример с параллельной сортировкой слиянием:

```
public class ParallelMergeSort {

    public static <T extends Comparable<? super T>> void
parallelMergeSort(T[] array) {
        ForkJoinPool pool = new ForkJoinPool();
        pool.invoke(new MergeSortTask<>(array, 0, array.length));
    }

    static class MergeSortTask<T extends Comparable<? super T>>
extends RecursiveTask<Void> {
        private T[] array;
        private int left;
        private int right;

        public MergeSortTask(T[] array, int left, int right) {
            this.array = array;
            this.left = left;
            this.right = right;
        }

        @Override
```


https://max.ru/bookofgeek

```
        protected Void compute() {
            if (right - left <= 1) {
                return null;              }

            int middle = (left + right) / 2;
            MergeSortTask<T> leftTask = new MergeSortTask<>(array,
left, middle);
            MergeSortTask<T> rightTask = new
MergeSortTask<>(array, middle, right);

            leftTask.fork();
            rightTask.fork();

            leftTask.join();
            rightTask.join();

            merge(array, left, middle, right);
            return null;
        }

        private void merge(T[] array, int left, int middle, int
right) {
            T[] temp = Arrays.copyOfRange(array, left, right);
            int i = left, j = middle, k = left;
            while (i < middle && j < right) {
                if (temp[i - left].compareTo(temp[j - left]) <= 0) {
                    array[k++] = temp[i++ - left];
                } else {
                    array[k++] = temp[j++ - left];
                }
            }
            while (i < middle) {
                array[k++] = temp[i++ - left];
            }
            while (j < right) {
                array[k++] = temp[j++ - left];
```


https://max.ru/bookofgeek

```
            }
        }
    }
}
```

В этом примере мы используем `ForkJoinPool` для распараллеливания рекурсивных задач сортировки слиянием. Это значительно ускоряет процесс сортировки при работе с большими массивами.

Быстрая сортировка

Быстрая сортировка — это один из наиболее популярных и быстрых алгоритмов сортировки с временной сложностью O(n log n) в среднем случае. В худшем случае ее сложность составляет O(n^2), что может случиться, если выбран плохой опорный элемент. Чтобы минимизировать риски, часто выбирают опорный элемент как медиану из трех элементов (первый, последний и средний).

Оптимизация быстрой сортировки

- **Рандомизация опорного элемента**. Выбор случайного элемента в качестве опорного позволяет избежать худших случаев при сортировке уже отсортированных данных.
- **Троичный выбор опорного элемента**. Чтобы минимизировать количество сравнений и потенциальных ошибок в выборе опорного элемента, можно выбрать медиану из трех случайных элементов.

Пример оптимизированной быстрой сортировки:

```
public class OptimizedQuickSort {

    public static <T extends Comparable<? super T>> void
quickSort(T[] array, int low, int high) {
        if (low < high) {
```


https://max.ru/bookofgeek

```
            int pivotIndex = partition(array, low, high);
            quickSort(array, low, pivotIndex - 1);
            quickSort(array, pivotIndex + 1, high);
        }
    }

    private static <T extends Comparable<? super T>> int
partition(T[] array, int low, int high) {
        int pivotIndex = medianOfThree(array, low, high);
        T pivot = array[pivotIndex];
        swap(array, pivotIndex, high);
        int i = low - 1;
        for (int j = low; j < high; j++) {
            if (array[j].compareTo(pivot) <= 0) {
                i++;
                swap(array, i, j);
            }
        }
        swap(array, i + 1, high);
        return i + 1;
    }

    private static <T extends Comparable<? super T>> int
medianOfThree(T[] array, int low, int high) {
        int mid = (low + high) / 2;
        if (array[low].compareTo(array[mid]) > 0) {
            swap(array, low, mid);
        }
        if (array[low].compareTo(array[high]) > 0) {
            swap(array, low, high);
        }
        if (array[mid].compareTo(array[high]) > 0) {
            swap(array, mid, high);
        }
        return mid;
    }

    private static <T> void swap(T[] array, int i, int j) {
```


https://max.ru/bookofgeek

```
            T temp = array[i];
            array[i] = array[j];
            array[j] = temp;
        }
    }
```

В этом примере добавлен метод *medianOfThree*, который помогает выбирать оптимальный опорный элемент для улучшения производительности быстрой сортировки.

5.4. Практические примеры с использованием JMH

JMH (Java Microbenchmarking Harness) — это инструмент для точного измерения производительности Java-программ. Включение JMH в процесс оптимизации алгоритмов и структур данных позволяет детально анализировать эффективность различных решений.

Бенчмарки для различных структур данных

Давайте создадим несколько примеров бенчмарков для различных структур данных, чтобы понять, как выбор структуры данных влияет на производительность.

Пример 1. Сравнение производительности поиска в массиве и в хеш-таблице

```
@Benchmark
@BenchmarkMode(Mode.AverageTime)
@OutputTimeUnit(TimeUnit.MILLISECONDS)
public void benchmarkArraySearch(Blackhole blackhole) {
    int[] array = new int[100000];
    for (int i = 0; i < array.length; i++) {
        array[i] = i;
    }
```


https://max.ru/bookofgeek

```
    for (int i = 0; i < 100000; i++) {
        blackhole.consume(array[i]);
    }
}

@Benchmark
@BenchmarkMode(Mode.AverageTime)
@OutputTimeUnit(TimeUnit.MILLISECONDS)
public void benchmarkHashMapSearch(Blackhole blackhole) {
    Map<Integer, Integer> map = new HashMap<>();
    for (int i = 0; i < 100000; i++) {
        map.put(i, i);
    }
    for (int i = 0; i < 100000; i++) {
        blackhole.consume(map.get(i));
    }
}
```

В этом примере мы создаем бенчмарки для поиска значений в массиве и хеш-таблице. Мы используем `Blackhole` для гарантии того, что результаты поиска не будут оптимизированы (убраны сборщиком мусора). Результаты дадут нам представление о различии в производительности при поиске в разных структурах данных.

Пример 2. Сравнение производительности сортировки в различных структурах данных

```
@Benchmark
@BenchmarkMode(Mode.AverageTime)
@OutputTimeUnit(TimeUnit.MILLISECONDS)
public void benchmarkArraySort(Blackhole blackhole) {
    int[] array = new int[100000];
    for (int i = 0; i < array.length; i++) {
        array[i] = ThreadLocalRandom.current().nextInt();
    }
    Arrays.sort(array);
    blackhole.consume(array);
```


https://max.ru/bookofgeek

```
}

@Benchmark
@BenchmarkMode(Mode.AverageTime)
@OutputTimeUnit(TimeUnit.MILLISECONDS)
public void benchmarkListSort(Blackhole blackhole) {
    List<Integer> list = new ArrayList<>(100000);
    for (int i = 0; i < 100000; i++) {
        list.add(ThreadLocalRandom.current().nextInt());
    }
    Collections.sort(list);
    blackhole.consume(list);
}
```

Этот бенчмарк позволяет сравнить скорость сортировки в массиве и списке. Анализ производительности этих операций на больших объемах данных покажет, какие структуры данных лучше подходят для разных типов операций.

5.5. Детальный разбор деревьев и графов

Дерево — это абстрактная структура данных, состоящая из узлов и ребер, где каждый узел может иметь несколько потомков, а каждый узел (кроме корня) имеет ровно одного родителя.

Как мы уже знаем, деревья являются мощным инструментом для представления и работы с иерархическими данными. Их применяют в различных областях, таких как базы данных, графы, поисковые алгоритмы и многие другие.

Основные компоненты дерева:

- **Корень** — верхний узел дерева, от которого исходят все остальные узлы.
- **Листья** — узлы, которые не имеют дочерних узлов.

https://max.ru/bookofgeek

- **Ребра** — соединения между узлами.
- **Глубина узла** — количество ребер на пути от корня до данного узла.
- **Высота дерева** — максимальная глубина среди всех узлов.

Деревья могут быть **сбалансированными** и **несбалансированными**.

Сбалансированные деревья оптимизированы для работы с данными, где операции вставки, удаления и поиска происходят за логарифмическое время, а **несбалансированные** деревья могут иметь худшую производительность в случае, если дерево сильно вытянуто (например, в виде списка).

5.5.1. Основные типы деревьев

Существует множество типов деревьев, каждый из которых подходит для различных задач в зависимости от особенностей и требований. Ранее мы уже упоминали деревья, рассмотрим сейчас их типы чуть подробнее.

1. Бинарное дерево поиска (BST)

Бинарное дерево поиска — это дерево, в котором каждый узел имеет не более двух детей, и для каждого узла значение всех узлов в левом поддереве меньше его значения, а в правом поддереве — больше.

Преимущества:

- Операции поиска, вставки и удаления выполняются за время O(log n) в среднем случае.
- Легко реализуемо.

Недостатки:

- При плохом распределении данных (например, если вставлять элементы по порядку) дерево может деградировать в список, что приведет к временной сложности O(n).

https://max.ru/bookofgeek

Пример кода для бинарного дерева поиска:

```
class BinarySearchTree {
    static class Node {
        int value;
        Node left, right;

        Node(int value) {
            this.value = value;
            left = right = null;
        }
    }

    private Node root;

    public BinarySearchTree() {
        root = null;
    }

    public void insert(int value) {
        root = insertRec(root, value);
    }

    private Node insertRec(Node root, int value) {
        if (root == null) {
            root = new Node(value);
            return root;
        }
        if (value < root.value) {
            root.left = insertRec(root.left, value);
        } else if (value > root.value) {
            root.right = insertRec(root.right, value);
        }
        return root;
    }

    public boolean search(int value) {
        return searchRec(root, value) != null;
```


https://max.ru/bookofgeek

```
    }

    private Node searchRec(Node root, int value) {
        if (root == null || root.value == value) {
            return root;
        }
        if (value < root.value) {
            return searchRec(root.left, value);
        }
        return searchRec(root.right, value);
    }

    public void inorderTraversal() {
        inorderRec(root);
    }

    private void inorderRec(Node root) {
        if (root != null) {
            inorderRec(root.left);
            System.out.print(root.value + " ");
            inorderRec(root.right);
        }
    }
}
```

2. Сбалансированные деревья

Чтобы избежать деградации бинарного дерева поиска в список, можно использовать сбалансированные деревья. Наиболее распространенные типы таких деревьев — это **AVL-деревья** и **деревья красно-черного типа**.

3. AVL-дерево

AVL-дерево — это бинарное дерево поиска, в котором для каждого узла разница высот левого и правого поддеревьев не превышает 1. Это дерево автоматически сбалансировано, и операции вставки и удаления всегда выполняются за O(log n).

https://max.ru/bookofgeek

Пример с использованием AVL-дерева:

```
class AVLTree {
    private class Node {
        int value, height;
        Node left, right;

        Node(int value) {
            this.value = value;
            height = 1;
        }
    }

    private Node root;

    public AVLTree() {
        root = null;
    }

    public void insert(int value) {
        root = insertRec(root, value);
    }

    private Node insertRec(Node node, int value) {
        if (node == null) {
            return new Node(value);
        }

        if (value < node.value) {
            node.left = insertRec(node.left, value);
        } else if (value > node.value) {
            node.right = insertRec(node.right, value);
        } else {
            return node;
        }

        node.height = Math.max(getHeight(node.left), getHeight(node.
right)) + 1;
```


https://max.ru/bookofgeek

```
        int balance = getBalance(node);

        if (balance > 1 && value < node.left.value) {
            return rotateRight(node);
        }

        if (balance < -1 && value > node.right.value) {
            return rotateLeft(node);
        }

        if (balance > 1 && value > node.left.value) {
            node.left = rotateLeft(node.left);
            return rotateRight(node);
        }

        if (balance < -1 && value < node.right.value) {
            node.right = rotateRight(node.right);
            return rotateLeft(node);
        }

        return node;
    }

    private Node rotateLeft(Node x) {
        Node y = x.right;
        x.right = y.left;
        y.left = x;
        x.height = Math.max(getHeight(x.left), getHeight(x.
right)) + 1;
        y.height = Math.max(getHeight(y.left), getHeight(y.
right)) + 1;
        return y;
    }

    private Node rotateRight(Node y) {
        Node x = y.left;
```


https://max.ru/bookofgeek

```
        y.left = x.right;
        x.right = y;
        y.height = Math.max(getHeight(y.left), getHeight(y.
right)) + 1;
        x.height = Math.max(getHeight(x.left), getHeight(x.
right)) + 1;
        return x;
    }

    private int getHeight(Node node) {
        return node == null ? 0 : node.height;
    }

    private int getBalance(Node node) {
        return node == null ? 0 : getHeight(node.left) -
getHeight(node.right);
    }
}
```

4. Красно-черные деревья

Красно-черное дерево — это еще один тип сбалансированного бинарного дерева поиска. В отличие от AVL-деревьев, где балансировка может требовать нескольких вращений, красно-черные деревья имеют более мягкие требования, что снижает сложность реализации, но они все равно обеспечивают балансировку дерева.

5.5.2. Алгоритмы и операции с деревьями

Помимо базовых операций вставки и поиска, деревья поддерживают и другие важные операции:

- **Удаление узла**. Процесс удаления узла из дерева может быть сложным, поскольку необходимо учитывать множество случаев — когда узел является листом, имеет одного потомка или двух.

https://max.ru/bookofgeek

- **Поиск минимального и максимального значения**. Для бинарного дерева поиска поиск минимального или максимального значения осуществляется за O(log n).
- **Преобразование дерева в список**. Можно осуществить преобразование дерева в список, что может быть полезно для дальнейших операций с данными.

5.6. Применение деревьев в реальных задачах

Деревья широко используются в различных областях:

- **Базы данных**. Для хранения индексов данных часто применяют структуры, основанные на деревьях (например, B-деревья и B+ деревья).
- **Обработки запросов**. Деревья широко используются в поисковых алгоритмах и для индексирования данных в системах управления базами данных.
- **Реализация графов**. Деревья могут быть использованы для представления связей в графах.
- **Операционные системы**. Деревья часто используются для организации процессов и управления памятью.

5.6.1. Бенчмарки с использованием JMH для деревьев

Теперь давайте рассмотрим пример бенчмарков для оценки производительности операций с деревьями.

Пример бенчмарка для поиска в различных деревьях

```
@Benchmark
@BenchmarkMode(Mode.AverageTime)
@OutputTimeUnit(TimeUnit.MILLISECONDS)
public void benchmarkBSTSearch(Blackhole blackhole) {
    BinarySearchTree tree = new BinarySearchTree();
    for (int i = 0; i < 100000; i++) {
        tree.insert(i);
```


https://max.ru/bookofgeek

```
    }
    for (int i = 0; i < 100000; i++) {
        blackhole.consume(tree.search(i));
    }
}

@Benchmark
@BenchmarkMode(Mode.AverageTime)
@OutputTimeUnit(TimeUnit.MILLISECONDS)
public void benchmarkAVLSearch(Blackhole blackhole) {
    AVLTree tree = new AVLTree();
    for (int i = 0; i < 100000; i++) {
        tree.insert(i);
    }
    for (int i = 0; i < 100000; i++) {
        blackhole.consume(tree.search(i));
    }
}
```

В этом примере мы сравниваем производительность поиска в бинарном дереве поиска и в AVL-дереве. JMH позволяет точно замерить время, затраченное на операции поиска в этих структурах данных, и выявить их различия в производительности.

5.6.2. Введение в графы

Граф — одна из самых мощных и универсальных структур данных в информатике. Он широко используется для моделирования отношений между объектами: от социальных сетей до маршрутов на картах, от систем рекомендаций до анализа связей в больших данных.

Графы применяются как в научных исследованиях, так и в промышленных решениях, включая машинное обучение, построение деревьев зависимостей и оптимизацию маршрутов.

https://max.ru/bookofgeek

Определение

Граф состоит из:

- **Вершин (nodes/vertices)** — основных элементов, представляющих объекты.
- **Ребер (edges)** — связей между вершинами, которые могут быть направленными (в ориентированных графах) или ненаправленными.

Граф можно описывать как G=(V,E)G = (V, E)G=(V,E), где:

- V — множество вершин, E — множество ребер.

Виды графов:

- Ориентированные и неориентированные.
- Взвешенные и невзвешенные.
- Связные и несвязные.
- Цикличные и ацикличные.
- Деревья — частный случай ацикличных графов.

5.6.3. Представление графов в памяти

Существует два основных способа представления графов в программах, описанных ниже.

1. Матрица смежности

Подходит для плотных графов. Используется двумерный массив, где значение *[i][j]* указывает на наличие ребра между вершинами *i* и *j*.

```
int[][] adjacencyMatrix = new int[n][n];
// Добавление ребра
adjacencyMatrix[0][1] = 1;
```

Плюсы:

- Быстрый доступ: O(1) к любой связи.

https://max.ru/bookofgeek

Минусы:

- Большой объем памяти: $O(n^2)$.

2. Списки смежности

Подходит для разреженных графов. Представляется как массив списков, где каждый список содержит соседей вершины.

```
List<List<Integer>> adjacencyList = new ArrayList<>();
for (int i = 0; i < n; i++) {
    adjacencyList.add(new ArrayList<>());
}
// Добавление ребра
adjacencyList.get(0).add(1);
```

Плюсы:

- Эффективность по памяти: $O(n + e)$.

Минусы:

- Более медленный доступ к конкретной связи: $O(k)$, где k — количество соседей.

5.6.4. Алгоритмы работы с графами

Поиск в глубину (DFS)

Используется для обхода графа и часто применяется в задачах, связанных с компонентами связности, цикличностью и топологической сортировкой.

```
public void dfs(int v, boolean[] visited, List<List<Integer>>
graph) {
    visited[v] = true;
    for (int neighbor : graph.get(v)) {
        if (!visited[neighbor]) {
            dfs(neighbor, visited, graph);
        }
    }
}
```


https://max.ru/bookofgeek

Сложность: **O(V + E)**.

Поиск в ширину (BFS)

Полезен для нахождения кратчайшего пути в невзвешенных графах и используется во многих алгоритмах маршрутизации.

```
public void bfs(int start, List<List<Integer>> graph) {
    boolean[] visited = new boolean[graph.size()];
    Queue<Integer> queue = new LinkedList<>();
    visited[start] = true;
    queue.add(start);

    while (!queue.isEmpty()) {
        int vertex = queue.poll();
        for (int neighbor : graph.get(vertex)) {
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                queue.add(neighbor);
            }
        }
    }
}
```

Сложность: **O(V + E)**.

Алгоритм Дейкстры

Позволяет находить кратчайшие пути во взвешенных графах без отрицательных ребер.

```
public int[] dijkstra(int start, List<List<int[]>> graph) {
    int[] dist = new int[graph.size()];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[start] = 0;

    PriorityQueue<int[]> pq = new PriorityQueue<>(Comparator.
comparingInt(a -> a[1]));
```


https://max.ru/bookofgeek

```
    pq.add(new int[]{start, 0});

    while (!pq.isEmpty()) {
        int[] node = pq.poll();
        int u = node[0];
        int d = node[1];

        if (d > dist[u]) continue;

        for (int[] edge : graph.get(u)) {
            int v = edge[0], weight = edge[1];
            if (dist[u] + weight < dist[v]) {
                dist[v] = dist[u] + weight;
                pq.add(new int[]{v, dist[v]});
            }
        }
    }

    return dist;
}
```

Сложность: **O((V + E) log V)** при использовании приоритетной очереди.

Алгоритм Флойда-Уоршелла

Нахождение кратчайших путей между всеми парами вершин.

```
public void floydWarshall(int[][] graph) {
    int n = graph.length;
    int[][] dist = new int[n][n];

    for (int i = 0; i < n; i++)
        dist[i] = Arrays.copyOf(graph[i], n);

    for (int k = 0; k < n; k++)
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
```


https://max.ru/bookofgeek

```
                if (dist[i][k] + dist[k][j] < dist[i][j])
                    dist[i][j] = dist[i][k] + dist[k][j];
}
```

Сложность: **O(V³)** — применим только к небольшим графам.

5.6.5. Практическое применение графов

Графы находят применение во множестве прикладных задач:

- Поисковики — построение графов ссылок между сайтами.
- Карты и навигация — маршруты между географическими точками.
- Социальные сети — представление связей между пользователями.
- Рекомендательные системы — графы предпочтений и товаров.
- Компиляторы — графы зависимостей между модулями.
- Системы контроля версий — графы коммитов и веток.

5.6.6. Бенчмарки JMH: производительность алгоритмов на графах

Мы можем использовать JMH для замера скорости различных алгоритмов работы с графами, таких как BFS и Dijkstra.

Пример: сравнение BFS и Dijkstra

```
@Benchmark
@BenchmarkMode(Mode.AverageTime)
@OutputTimeUnit(TimeUnit.MILLISECONDS)
public void benchmarkBFS() {
    List<List<Integer>> graph = GraphUtils.
generateUnweightedGraph(1000, 5000);
    GraphAlgorithms.bfs(0, graph);
}

@Benchmark
```


https://max.ru/bookofgeek

```
@BenchmarkMode(Mode.AverageTime)
@OutputTimeUnit(TimeUnit.MILLISECONDS)
public void benchmarkDijkstra() {
    List<List<int[]>> graph = GraphUtils.generateWeightedGraph(1000,
5000);
    GraphAlgorithms.dijkstra(0, graph);
}
```

Методы `generateUnweightedGraph` и `generateWeightedGraph` создают случайные графы с заданным числом вершин и ребер. Это позволяет имитировать реальную нагрузку и проводить корректные замеры времени выполнения.

5.6.7. Оптимизация работы с графами

При работе с графами важно учитывать следующие моменты:

- **Использовать правильное представление графа**: список смежности для разреженных, матрицу — для плотных.
- **Профилировать использование памяти**: графы могут потреблять много памяти при неэффективной реализации.
- **Избегать повторных расчетов**: кэширование путей, предрасчет расстояний и т.д.
- **Параллельные алгоритмы**: библиотеки типа Apache Flink, GraphX в Spark позволяют распараллеливать обработку больших графов.

5.7. Примеры использования деревьев и графов для оптимизации

Деревья и графы как структуры данных не просто являются академическими концепциями или частью курсов по алгоритмам. В реальных проектах они становятся основой для эффективного решения множества задач, где важны производительность, масштабируемость и экономия ресурсов.

https://max.ru/bookofgeek

Разработчики, инженеры данных, DevOps-специалисты и системные архитекторы применяют их для оптимизации систем различного масштаба — от мобильных приложений до распределенных облачных решений.

Оптимизация в навигации и логистике

Графы незаменимы в логистике и транспортных системах. Применение алгоритма Дейкстры или A* позволяет находить кратчайшие маршруты между точками на карте, учитывая весовые коэффициенты (например, расстояние, загруженность дорог, платные участки и т.д.). Это дает значительную оптимизацию маршрутов доставки, снижая расходы на топливо, время в пути и общее число задействованных транспортных единиц.

Яркий пример — использование графов в приложениях типа Google Maps, Яндекс.Карты, Uber или системах управления парками грузовиков.

Оптимизация сетевых инфраструктур

Деревья часто используются для построения маршрутов в компьютерных сетях. Минимальные остовные деревья (MST) — основа для оптимизации сетей при построении кабельной инфраструктуры, Wi-Fi-маршрутизации или оптимизации межсерверных соединений.

Применение алгоритмов Прима и Краскала позволяет минимизировать суммарную стоимость подключения всех узлов сети, избегая при этом циклов и избыточных соединений.

Принятие решений и компиляция

Деревья решений активно применяются в аналитике и машинном обучении, особенно в задачах классификации и регрессии. Такие деревья автоматически строятся на основе входных данных и позволяют эффективно разделять сложные данные на логически связанные подгруппы.

Алгоритмы на основе деревьев, такие как Random Forest и Gradient Boosting Trees, уже много лет являются базовым инструментом для решения бизнес-задач — от кредитного скоринга до прогнозирования оттока клиентов.

https://max.ru/bookofgeek

В компиляторах деревья выражений и синтаксические деревья (AST) позволяют эффективно обрабатывать исходный код, проверять его корректность, выполнять оптимизации и трансформации. Благодаря этому ускоряется сборка проектов, упрощается статический анализ и повышается надежность ПО.

Работа с зависимостями и сборкой проектов

В больших проектах со множеством модулей (например, на Java или C++) графы зависимостей позволяют строить корректные планы сборки. Такие графы учитывают, какие модули зависят от других, и позволяют оптимизировать процессы компиляции, исключая повторную сборку неизмененных компонентов. Это особенно актуально в системах CI/CD, где важно минимизировать время отклика между коммитом и деплоем.

Оптимизация хранения и доступа к данным

Деревья лежат в основе многих структур хранения: B-деревья и B+-деревья используются в индексах баз данных (например, в PostgreSQL, Oracle), обеспечивая логарифмическое время доступа к данным.

Красно-черные деревья используются в *TreeMap* и *TreeSet* в Java для обеспечения упорядоченного хранения и эффективного поиска. Это позволяет ускорить операции выборки, фильтрации и агрегирования, особенно на больших объемах информации.

Машинное обучение и анализ больших данных

Графовые базы данных (например, Neo4j, ArangoDB) позволяют эффективно анализировать взаимосвязи между объектами в системах рекомендаций, социальном графе, антиплагиате или распознавании мошеннической активности.

В отличие от традиционных реляционных СУБД, такие графовые хранилища позволяют выполнять сложные многосвязные запросы за миллисекунды, благодаря специально заточенной архитектуре и алгоритмам обхода.

https://max.ru/bookofgeek

Разработка игр и моделирование

В игровом ИИ графы используются для построения логики поведения персонажей, навигации по игровому миру (pathfinding), генерации сценариев и построения диалогов. Структуры вроде деревьев поведения (behavior trees) и графов состояний помогают оптимизировать поведение NPC, делая их реалистичнее и обеспечивая при этом эффективность работы даже на ограниченных ресурсах.

Выводы

Выбор правильной структуры данных и оптимизация алгоритмов сортировки и поиска являются ключевыми факторами для создания высокопроизводительных приложений. Структуры данных, такие как массивы, связанные списки, деревья, графы и хеш-таблицы, имеют свои сильные и слабые стороны в зависимости от контекста применения. Алгоритмы сортировки и поиска также требуют тщательной оптимизации для уменьшения временной и пространственной сложности.

> Весьма полезным инструментом для анализа производительности является **JMH**, который позволяет не только точно измерять время выполнения различных операций, но и обеспечивать высокую точность и надежность таких замеров. При правильном применении этих техник можно добиться значительного улучшения производительности и оптимизации приложений, что особенно важно при работе с большими данными и в системах реального времени.

В дальнейшем стоит исследовать более сложные и специализированные структуры данных, такие как B-деревья, деревья поиска, различные типы очередей с приоритетом и другие, чтобы дальше улучшать производительность при решении специфических задач.

https://max.ru/bookofgeek

Глава 6.

Параллельные и многопоточные приложения

https://max.ru/bookofgeek

О чём глава:

- Основы многопоточности в Java.
- Пул потоков и управление потоками.
- Оптимизация многопоточных приложений.

6.1. Основы многопоточности в Java

Многопоточность в **Java** позволяет разделить выполнение программы на несколько потоков, каждый из которых выполняется параллельно с другими.

Потоки в **Java** — это минимальные единицы выполнения, которые могут быть использованы для параллельной обработки задач, что, в свою очередь, значительно ускоряет выполнение программ на многоядерных процессорах.

Создание потоков

В Java существует два основных способа создания потока:

1. **Расширение класса Thread**. Для этого необходимо создать подкласс от класса Thread и переопределить его метод `run()`. Этот метод будет содержать логику, которую должен выполнять поток.

```
class MyThread extends Thread {
    @Override
    public void run() {
```


https://max.ru/bookofgeek

```
        // Логика выполнения потока
        System.out.println("Поток выполняется");
    }
}

public class Main {
    public static void main(String[] args) {
        MyThread thread = new MyThread();
        thread.start(); // Запуск потока
    }
}
```

2. **Реализация интерфейса Runnable**. Это более универсальный способ, который позволяет использовать потоки в различных местах кода, например в пуле потоков.

```
class MyRunnable implements Runnable {
    @Override
    public void run() {
        // Логика выполнения потока
        System.out.println("Поток выполняется");
    }
}

public class Main {
    public static void main(String[] args) {
        MyRunnable runnable = new MyRunnable();
        Thread thread = new Thread(runnable);
        thread.start(); // Запуск потока
    }
}
```

Основы синхронизации

Когда несколько потоков обращаются к общим данным, могут возникать проблемы, такие как **гонки потоков** (race conditions). Для защиты данных от одновременного доступа используется синхронизация.

https://max.ru/bookofgeek

В Java для этого существует несколько методов:

- **Ключевое слово *synchronized***. Оно позволяет гарантировать, что только один поток одновременно будет выполнять метод или блок кода.

```
public synchronized void increment() {
    count++;
}
```

- **Использование блокировок (*Lock*)**. В Java также существует интерфейс Lock, который предоставляет более гибкую синхронизацию.

```
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

public class SharedResource {
    private int count = 0;
    private final Lock lock = new ReentrantLock();  // Создаем
объект блокировки

    // Метод для безопасного инкремента
    public void increment() {
        lock.lock();  // Получаем блокировку
        try {
            count++;  // Операция с общими данными
        } finally {
            lock.unlock();  // Освобождаем блокировку
        }
    }

    public int getCount() {
        return count;
    }

    public static void main(String[] args) throws
InterruptedException {
        SharedResource sharedResource = new SharedResource();
```


https://max.ru/bookofgeek

```
        // Создаем несколько потоков
        Thread thread1 = new Thread(() -> {
            for (int i = 0; i < 1000; i++) {
                sharedResource.increment();
            }
        });

        Thread thread2 = new Thread(() -> {
            for (int i = 0; i < 1000; i++) {
                sharedResource.increment();
            }
        });

        // Запускаем потоки
        thread1.start();
        thread2.start();

        // Ожидаем завершения потоков
        thread1.join();
        thread2.join();

        // Выводим результат
        System.out.println("Final count: " + sharedResource.
getCount());
    }
}
```

- `ReentrantLock lock = new ReentrantLock();` — создаем объект блокировки.
- `lock.lock();` — поток захватывает блокировку перед выполнением критической секции.
- `lock.unlock();` — после завершения работы поток освобождает блокировку, чтобы другие потоки могли получить доступ к ресурсу.

Важный момент: блокировки в Java могут быть более гибкими, чем *synchronized*, поскольку они предоставляют дополнительные возможности,

https://max.ru/bookofgeek

такие как возможность попытки захвата блокировки с тайм-аутом и возможность прерывания потока, который ожидает блокировку.

6.2. Пул потоков и управление потоками

Пул потоков (Thread Pool) — это коллекция потоков, которые могут быть повторно использованы для выполнения задач.

Использование пула потоков позволяет значительно улучшить производительность многопоточных приложений, так как создание и уничтожение потоков — это ресурсоемкие операции. Пул потоков помогает избежать излишнего накладного времени на создание новых потоков, а также упрощает управление ими.

В Java для работы с пулом потоков используется класс **ExecutorService**, который является частью пакета java.util.concurrent.

Создание пула потоков

Для создания пула потоков можно использовать статический метод `newFixedThreadPool()` класса **Executors**, который создает пул с фиксированным количеством потоков.

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class ThreadPoolExample {
    public static void main(String[] args) {
        // Создаем пул потоков с 3 потоками
        ExecutorService executorService = Executors.
newFixedThreadPool(3);

        // Отправляем 5 задач в пул
        for (int i = 0; i < 5; i++) {
```


https://max.ru/bookofgeek

```
            executorService.submit(new Task(i));
        }

        // Завершаем работу пула потоков
        executorService.shutdown();
    }
}

class Task implements Runnable {
    private final int taskId;

    public Task(int taskId) {
        this.taskId = taskId;
    }

    @Override
    public void run() {
        System.out.println("Задача " + taskId + " выполняется
потоком " + Thread.currentThread().getName());
    }
}
```

- `ExecutorService executorService = Executors.newFixedThreadPool(3);` — создаем пул потоков с 3 потоками. Когда мы отправляем задачи, они будут выполнены этими потоками.
- `executorService.submit(new Task(i));` — отправляем задачи в пул для выполнения.
- `executorService.shutdown();` — после завершения задач мы закрываем пул потоков, чтобы освободить ресурсы.

Параллельное выполнение задач

С помощью пула потоков можно эффективно распараллелить задачи, обеспечивая более быструю обработку, особенно когда задача требует продолжительного времени на выполнение (например, обработка большого объема данных или сетевых запросов).

https://max.ru/bookofgeek

Типы пулов потоков

Java предоставляет несколько типов пулов:

- `newFixedThreadPool(int nThreads)` — пул с фиксированным числом потоков.
- `newCachedThreadPool()` — пул, который создает новые потоки по мере необходимости и переиспользует неактивные потоки, если они освобождаются.
- `newSingleThreadExecutor()` — пул, содержащий только один поток. Все задачи будут выполняться последовательно.

6.3. Оптимизация многопоточных приложений

Оптимизация многопоточных приложений — это ключевой аспект, который позволяет существенно повысить производительность, снижая время отклика и улучшая использование ресурсов, таких как процессоры и память.

Многопоточные приложения могут быть сложными для разработки и отладки, но, правильно применяя техники оптимизации, можно избежать множества проблем, связанных с производительностью.

6.3.1. Избегание избыточной синхронизации

Одна из частых ошибок в многопоточных приложениях — это избыточная синхронизация. Когда блокировка применяется к большому куску кода, это может привести к снижению производительности из-за того, что потоки вынуждены ожидать своей очереди на выполнение.

Пример:

```
public class SynchronizedExample {
    private int counter = 0;
```


https://max.ru/bookofgeek

```
    public synchronized void increment() {
        // Избыточная синхронизация
        counter++;
    }
}
```

Чтобы избежать избыточной синхронизации, необходимо минимизировать область критической секции, синхронизируя только те операции, которые действительно требуют защиты.

Оптимизированный код:

```
public class OptimizedExample {
    private int counter = 0;

    public void increment() {
        // Синхронизируем только операцию инкремента
        synchronized (this) {
            counter++;
        }
    }
}
```

6.3.2. Использование конкурентных коллекций

Вместо синхронизации обычных коллекций (например, *ArrayList* или *HashMap*), Java предоставляет классы из пакета *java.util.concurrent*, которые обеспечивают безопасность при многопоточном доступе. Эти коллекции значительно облегчают разработку многопоточных приложений, так как они уже реализуют необходимую синхронизацию.

Пример использования:

```
import java.util.concurrent.ConcurrentHashMap;

public class ConcurrentMapExample {
    private ConcurrentHashMap<String, Integer> map = new
ConcurrentHashMap<>();
```


https://max.ru/bookofgeek

```
    public void addElement(String key, Integer value) {
        map.put(key, value); // Безопасно для многопоточной среды
    }

    public Integer getElement(String key) {
        return map.get(key); // Безопасно для многопоточной среды
    }
}
```

6.3.3. Минимизация блокировки

Избегайте блокировки критических секций, которые занимают длительное время. Например, если вам нужно выполнить несколько операций в рамках одного метода, разделите их на меньшие части, чтобы другие потоки могли работать параллельно.

Пример плохой практики:

```
public synchronized void processLargeTask() {
    longTask1();
    longTask2();
    longTask3();
}
Оптимизированный код:
public void processLargeTask() {
    synchronized (this) {
        longTask1();
    }
    longTask2(); // Этот метод можно выполнить без
синхронизации
    longTask3(); // Этот метод можно выполнить без
синхронизации
}
```

6.3.4. Использование атомарных операций

https://max.ru/bookofgeek

Если вам нужно работать с простыми типами данных (например, *int, long, boolean*), используйте атомарные классы из пакета *java.util.concurrent.atomic*. Эти классы обеспечивают атомарные операции без необходимости явной синхронизации.

Пример использования *AtomicInteger*:

```
import java.util.concurrent.atomic.AtomicInteger;

public class AtomicExample {
    private AtomicInteger counter = new AtomicInteger(0);

    public void increment() {
        counter.incrementAndGet(); // Атомарный инкремент
    }

    public int getCount() {
        return counter.get();
    }
}
```

6.3.5. Использование *ExecutorService* для управления потоками

Пул потоков помогает избежать проблемы с созданием и уничтожением потоков, что снижает накладные расходы на управление потоками.

Также важно грамотно управлять жизненным циклом пула — например, закрывать пул потоков с помощью метода shutdown() после того, как все задачи выполнены.

Пример с использованием *ExecutorService*:

```
import java.util.concurrent.*;

public class ExecutorExample {
    private static final ExecutorService executor = Executors.
newFixedThreadPool(5);
```


https://max.ru/bookofgeek

```
    public static void main(String[] args) throws
InterruptedException {
        // Отправляем задачи на выполнение
        for (int i = 0; i < 10; i++) {
            executor.submit(() -> {
                System.out.println("Задача выполняется в
потоке: " + Thread.currentThread().getName());
            });
        }

        executor.shutdown();  // Завершаем работу с пулом
        executor.awaitTermination(1, TimeUnit.MINUTES);  //
Ждем завершения задач
    }
}
```

Выводы

Оптимизация многопоточных приложений — это важный процесс, который включает в себя как правильный выбор инструментов для синхронизации, так и грамотное распределение нагрузки между потоками.

Эффективное использование пулов потоков, конкурентных коллекций, атомарных операций и минимизация блокировок — это ключевые моменты, которые помогут улучшить производительность многопоточного приложения.

https://max.ru/bookofgeek

Глава 7.

Использование функций языка для оптимизации

https://max.ru/bookofgeek

О чём глава:

- Лямбда-выражения и потоки (Streams API).
- Введение в модули (Java Modules).
- Новые коллекции и утилиты.

7.1. Лямбда-выражения и потоки (Streams API)

Лямбда-выражения в Java были введены в версии 8 и представляют собой краткий способ представления анонимных функций.

Это мощный инструмент для упрощения кода, особенно при работе с коллекциями и потоками данных.

Лямбда-выражения позволяют выразить функциональные интерфейсы в компактной и читаемой форме.

Пример лямбда-выражения:

```
// Лямбда-выражение, которое реализует интерфейс Runnable
Runnable task = () -> System.out.println("Задача выполняется");
```

Лямбда-выражения полезны при работе с потоками данных (*Streams*). Streams API предоставляет функциональный стиль обработки коллекций, что позволяет легко работать с большими объемами данных без явного использования циклов и условий.

https://max.ru/bookofgeek

Пример использования Streams API:

```
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class StreamsExample {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);

        // Фильтрация четных чисел и вычисление их квадрата
        List<Integer> squaredEvens = numbers.stream()
                                .filter(n -> n % 2 == 0) // Фильтрация
                                .map(n -> n * n)          // Преобразование
                                .collect(Collectors.toList()); // Собираем
результат

        System.out.println(squaredEvens);  // Вывод: [4, 16, 36, 64, 100]
    }
}
```

Особенности использования Streams API:

- **Параллельные потоки**. В Java 8 был добавлен функционал для работы с параллельными потоками. Потоки данных могут быть обрабатываемы в несколько потоков, что ускоряет выполнение на многозадачных системах.

  Пример с параллельными потоками:

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8,
9, 10);

List<Integer> result = numbers.parallelStream()
                    .filter(n -> n % 2 == 0)
                    .collect(Collectors.toList());

System.out.println(result);  // Параллельная обработка четных
чисел
```


https://max.ru/bookofgeek

- **Функциональный стиль**. Потоки позволяют писать код в функциональном стиле, избавляя от явного написания циклов и условий, что делает код чище и понятнее.

Оптимизация с помощью Streams:

- **Lazy Evaluation**. Потоки данных в Java используют ленивую оценку, что означает, что операции выполняются только при необходимости. Это позволяет избежать избыточных вычислений.
- **Уменьшение использования промежуточных коллекций**. Использование потоков позволяет избежать создания временных коллекций в памяти, так как данные могут быть обработаны сразу.

7.2. Введение в модули (Java Modules)

С введением Java 9 был представлен модульный механизм, который позволяет разделить программу на отдельные части (модули). Каждый модуль может содержать пакеты и классы, а также определять зависимости от других модулей. Это помогает улучшить организацию кода и повысить безопасность и производительность.

Основы работы с модулями

Модуль описывается в специальном файле module-info.java, который указывает зависимости модуля и экспортируемые пакеты.

Пример создания модуля:

```
// Файл module-info.java
module com.example.myapp {
    requires java.base;  // Зависимость от другого модуля
    exports com.example.myapp.util;  // Экспорт пакета
}
```


https://max.ru/bookofgeek

Основные преимущества модулей

- **Инкапсуляция**. Модули позволяют скрывать внутреннюю реализацию, экспортируя только необходимые части приложения. Это улучшает безопасность, предотвращая доступ к чувствительным частям кода.
- **Управление зависимостями**. Явное указание зависимостей между модулями помогает избежать проблем с совместимостью версий и конфликта библиотек.
- **Производительность**. Модульная система позволяет Java-джит-компилятору оптимизировать выполнение программы, поскольку он может анализировать зависимости и исключать неиспользуемые части.

Пример использования модулей

- Модуль 1 (com.example.module1):

```
module com.example.module1 {
    exports com.example.module1;  // Экспортируем пакет
}
```

- Модуль 2 (com.example.module2):

```
module com.example.module2 {
    requires com.example.module1;  // Зависимость от модуля 1
}
```

Использование модулей помогает организовать большие проекты и легко управлять зависимостями между компонентами системы.

7.3. Новые коллекции и утилиты

В последние версии Java (начиная с Java 8) были добавлены новые коллекции и утилиты, которые значительно упрощают работу с данными.

https://max.ru/bookofgeek

1. **Коллекции в *java.util.concurrent*.**

   В пакет *java.util.concurrent* были добавлены новые коллекции, такие как *ConcurrentHashMap, CopyOnWriteArrayList*, которые обеспечивают безопасный доступ из нескольких потоков.

2. ***Optional*.**

   Класс *Optional* был введен для работы с потенциально отсутствующими значениями, что помогает избежать *NullPointerException* и улучшает читаемость кода.

   Пример использования ***Optional***:

```
import java.util.Optional;

public class OptionalExample {
    public static void main(String[] args) {
        Optional<String> name = Optional.ofNullable(getName());

        // Выводим имя, если оно есть, или значение по
умолчанию
        System.out.println(name.orElse("Неизвестно"));
    }

    public static String getName() {
        return null;  // Имя отсутствует
    }
}
```

3. ***ImmutableList* и другие неизменяемые коллекции**. С помощью библиотеки *Guava* и других вспомогательных библиотек можно создавать неизменяемые коллекции, которые являются безопасными для многопоточного доступа.

4. ***Collectors* для потоков**. Класс Collectors предоставляет различные способы сбора результатов из потока в коллекции. Примеры включают сбор в список, карту, объединение строк и т. д.

   Пример использования ***Collectors*:**

https://max.ru/bookofgeek

```
import java.util.*;
import java.util.stream.*;

public class CollectorsExample {
    public static void main(String[] args) {
        List<String> names = Arrays.asList("Alice", "Bob", "Charlie",
"David");

        // Сбор в коллекцию
        Set<String> uniqueNames = names.stream()
                                .collect(Collectors.toSet());

        System.out.println(uniqueNames);
    }
}
```

Выводы

Использование новых функций языка Java, таких как лямбда-выражения, Streams API, модули и новые коллекции, дает значительные преимущества в плане производительности, читаемости и безопасности кода.

Лямбда-выражения и **потоки** позволяют писать более компактный и эффективный код, модули помогают организовать проект, а новые коллекции и утилиты значительно упрощают работу с данными, особенно в многопоточных приложениях.

https://max.ru/bookofgeek

https://max.ru/bookofgeek

Ещё больше свежей и актуальной литературы по Java ты найдёшь в наших каналах:
https://max.ru/bookofgeek
https://t.me/bookofgeek

Глава 8.

Виртуальные потоки

https://max.ru/bookofgeek

О чём глава:

- Подробное введение в проблему традиционной многопоточности.
- История появления виртуальных потоков.
- Архитектура и внутренняя реализация.
- Примеры использования.
- Производительность и ограничения.
- Интеграция с существующими библиотеками.
- Сравнение с платформенными потоками.
- Практические советы и best practices.
- Тестирование кода с виртуальными потоками.

8.1. Подробное введение в проблему традиционной многопоточности

Как мы уже знаем, **многопоточность** в **Java** — мощный инструмент, позволяющий повысить производительность приложений за счет параллельного выполнения задач.

Однако классическая реализация потоков на платформенном уровне имеет ряд серьезных ограничений.

https://max.ru/bookofgeek

- Во-первых, каждый *Thread* в Java соответствует нативному потоку операционной системы, что означает высокие накладные расходы на создание, планирование и переключение контекста. Операционная система вынуждена держать в памяти всю инфраструктуру потока, даже если он простаивает, ожидая ресурса или I/O. Это ведет к быстрому исчерпанию лимитов, особенно при работе с большим количеством одновременных задач.
- Во-вторых, при блокировке потока, например, на чтение из сокета или работу с БД, операционная система блокирует весь поток, в результате чего ресурсы, такие как ядра CPU, не используются эффективно. Это ограничивает масштабируемость приложений.
- В-третьих, отладка и профилирование многопоточного кода зачастую оказываются сложными задачами, особенно если речь идет о высоконагруженных системах, где необходимо управлять тысячами параллельных операций.

Все это привело к необходимости переосмыслить модель многопоточности в Java, что и стало отправной точкой для проекта **Project Loom** и введения **виртуальных потоков**.

8.2. История появления виртуальных потоков

> Проект **Loom** был анонсирован в рамках OpenJDK с целью предоставить легкие и масштабируемые конструкции параллелизма, способные работать с миллионами параллельных задач без значительных накладных расходов.

Разработка началась около 2018 года. Идея была вдохновлена успешными реализациями легких потоков в других языках, таких как **goroutines** в Go и **green threads** в Erlang. Java стремилась сохранить совместимость с текущей моделью, при этом значительно расширив ее возможности.

После нескольких лет активной разработки, экспериментов и обратной связи от сообщества виртуальные потоки были официально включены в состав JDK, начиная с версии 21 в виде предварительной функции (preview).

https://max.ru/bookofgeek

Они получили широкое распространение благодаря простоте интеграции и высокой эффективности.

8.3. Архитектура и внутренняя реализация

> Главной особенностью виртуальных потоков является то, что они **не привязаны напрямую к потокам операционной системы**. Они управляются **планировщиком на уровне JVM**, что позволяет эффективно мультиплексить (перемешивать) множество виртуальных потоков поверх небольшого пула нативных потоков.

Каждый виртуальный поток реализуется как **стековое представление** и может приостанавливаться и возобновляться при блокирующих вызовах. Например, если виртуальный поток ожидает завершения операции ввода-вывода, он снимается с выполнения и его стек сохраняется. Когда операция завершена, выполнение продолжается с того же места, но, возможно, уже на другом потоке.

> Это достигается с помощью **continuations** — механизма, позволяющего сохранить и восстановить состояние выполнения.

JVM использует **ForkJoinPool**, адаптированный под виртуальные потоки, где каждый нативный поток способен обрабатывать множество виртуальных потоков. Это минимизирует накладные расходы и делает возможным запуск миллионов потоков.

8.4. Примеры использования

Создание виртуального потока стало таким же простым, как вызов:

```
Thread.startVirtualThread(() -> {
    System.out.println("Hello from virtual thread");
});
```


https://max.ru/bookofgeek

Или при использовании *ExecutorService*:

```
ExecutorService executor = Executors.newVirtualThreadPerTask
Executor();
executor.submit(() -> {
    // Выполняемый код
});
```

Параллельная обработка запросов:

```
var server = ServerSocket.open();
server.bind(new InetSocketAddress(8080));

while (true) {
    var socket = server.accept();
    Thread.startVirtualThread(() -> handleConnection(socket));
}
```

Каждое соединение обрабатывается в своем виртуальном потоке, без риска перегрузки ОС.

8.5. Производительность и ограничения

Преимущества:

- Низкие накладные расходы на создание и выполнение.
- Масштабируемость до миллионов задач.
- Совместимость с существующим API *java.lang.Thread* и *ExecutorService*.
- Эффективное использование CPU благодаря неблокирующей модели.

Ограничения:

- Не всякий код совместим. Например, если используется нативная блокировка (*synchronized*), виртуальный поток будет "застревать".

https://max.ru/bookofgeek

- Некоторые библиотеки могут не поддерживать корректное взаимодействие с виртуальными потоками, особенно те, которые полагаются на поток-локальные переменные (*ThreadLocal*).
- Диагностика и профилирование требуют обновления инструментов.

8.6. Интеграция с существующими библиотеками

Для большинства приложений переход на виртуальные потоки будет прозрачен. Современные фреймворки (Spring, Vert.x, Quarkus) уже начали интеграцию с Loom, предоставляя поддержку виртуальных потоков на уровне обработки запросов и асинхронных задач.

Пример с JDBC:

```
ExecutorService executor = Executors.
newVirtualThreadPerTaskExecutor();
executor.submit(() -> {
    try (Connection conn = dataSource.getConnection()) {
        var stmt = conn.prepareStatement("SELECT * FROM users");
        var rs = stmt.executeQuery();
        while (rs.next()) {
            System.out.println(rs.getString("name"));
        }
    }
});
```

Виртуальные потоки позволяют писать синхронный код с асинхронной производительностью.

Таблица 8.1. Сравнение с платформенными потоками

Характеристика	Платформенные потоки	Виртуальные потоки
Связь с ОС	Прямое соответствие	Уровень JVM
Время создания	Высокое	Очень низкое

https://max.ru/bookofgeek

Количество одновременно	Ограничено (тысячи)	Миллионы
Энергопотребление	Выше	Ниже
Поддержка отладчиков	Хорошая	Ограниченная (в процессе)
Поддержка ThreadLocal	Полная	Ограниченная
Масштабируемость	Средняя	Отличная

Практические советы и best practices

- Используйте *Executors.newVirtualThreadPerTaskExecutor()* для управления задачами.
- Избегайте блокирующих операций, которые не совместимы с Loom.
- Не используйте *ThreadLocal* без необходимости — переходите на *ScopedValue*.
- Профилируйте и тестируйте код на новых JDK.
- При работе с I/O-потоками и JDBC выбирайте библиотеки с поддержкой Loom.

8.7. Тестирование кода с виртуальными потоками

Для тестирования кода с виртуальными потоками можно использовать JUnit и его поддержку параллельного выполнения:

```
@Test
void testVirtualThreadBehavior() throws Exception {
    var result = new AtomicReference<String>();
    var thread = Thread.startVirtualThread(() -> result.
set("done"));
    thread.join();
    assertEquals("done", result.get());
}
```


https://max.ru/bookofgeek

Также важно:

- Проверять корректность параллельной логики.
- Использовать инструменты трассировки с поддержкой Loom (например, JFR, async-profiler).
- Следить за поведением при масштабной нагрузке.

Выводы

Виртуальные потоки стали революцией в мире Java-многопоточности. Они упрощают разработку высокопроизводительных и масштабируемых приложений, сохраняя при этом привычные парадигмы программирования. Благодаря их внедрению, Java теперь способна конкурировать с языками, изначально заточенными под асинхронность и параллелизм.

> Понимание **принципов работы виртуальных потоков**, их интеграции и ограничений — ключевой навык для современных разработчиков.

В следующих главах мы рассмотрим более подробно сценарии использования виртуальных потоков в продакшене, включая примеры из реальных проектов.

https://max.ru/bookofgeek

Глава 9.

Модулизация и производительность Java-приложений

https://max.ru/bookofgeek

О чём глава:

- Введение в модулизацию (Java Platform Module System).
- Почему модули важны для производительности.
- Уменьшение времени запуска за счет модулярности.
- Минимизация используемых зависимостей.
- Снижение использования памяти.
- Улучшение безопасности через строгие зависимости.
- Модулизация в больших проектах.
- Практика: разбиение монолита на модули.
- Инструменты для анализа модульных зависимостей.
- Рефакторинг монолита в модульную структуру.
- Влияние модулярности на тестирование и деплой.
- Пример модулярного подхода.
- Модули и контейнеризация.
- Best practices при модуляризации.

9.1. Введение в модулизацию (Java Platform Module System)

Модулизация — это процесс разделения приложения на независимые, легко управляемые и повторно используемые компоненты.

https://max.ru/bookofgeek

В Java эта концепция была официально введена в JDK 9 с внедрением **Java Platform Module System (JPMS)**. Модули позволяют разработчикам разделять большие программы на более мелкие, управляемые блоки, что упрощает их поддержание, тестирование и улучшает безопасность.

> Важнейшие элементы модулей — это **модульные дескрипторы**, которые определяют зависимости между модулями и их видимость для других частей приложения.

Модулизация представляет собой один из важных шагов в повышении производительности, так как она позволяет уменьшить размеры приложений, упростить их загрузку и управление зависимостями. Помимо этого, она дает возможности для оптимизации работы JVM, поскольку модули предоставляют JVM точные сведения о том, какие классы и пакеты могут быть использованы.

Модулизация помогает организовать код, уменьшая количество ненужных зависимостей и исключая излишнюю нагрузку на систему. Важно отметить, что в Java-модулях нет перекрытия с концепцией пакетов: модули могут содержать несколько пакетов, но помимо этого они могут также определять, какие пакеты доступны из других модулей.

9.2. Почему модули важны для производительности

При построении крупных Java-приложений одно из самых сложных препятствий — это управление зависимостями. Когда приложения становятся больше, сложнее и включают в себя множество библиотек, неуправляемые зависимости могут стать причиной множества проблем, таких как:

1. **Неэффективное использование памяти.** Без четкого разграничения модулей в приложении может быть много неиспользуемых классов и библиотек.
2. **Медленный запуск.** При старте программы загрузка ненужных зависимостей замедляет этот процесс.

https://max.ru/bookofgeek

3. **Трудности в обслуживании.** Большие приложения с большим числом зависимостей сложны в поддержке, так как сложно отследить, какие компоненты реально используются.

Модулизация помогает минимизировать эти проблемы:

- **Чистая и четкая структура зависимостей.** Модули позволяют точно указать, какие части приложения могут быть использованы другими частями.
- **Управление памятью.** Модули позволяют JVM исключать неиспользуемые классы из загрузки, что снижает потребление памяти.
- **Оптимизация времени старта.** Модули позволяют загружать только те компоненты приложения, которые действительно нужны в данный момент.

Кроме того, JPMS дает возможность строить системы с ограниченной видимостью, что делает приложения более безопасными. Например, определив модуль с конкретной областью функционала и скрыв его детали, мы уменьшаем риски несанкционированного доступа.

9.3. Уменьшение времени запуска за счет модулярности

Одним из наиболее ощутимых преимуществ перехода к модульной архитектуре в Java-приложениях является значительное сокращение времени запуска. В традиционных монолитных приложениях без явной модульной структуры виртуальная машина Java (JVM) вынуждена загружать обширные наборы классов, часто включая те, которые фактически не требуются для выполнения текущей задачи. Это приводит к увеличению времени старта, росту потребления памяти и ухудшению предсказуемости поведения приложения на этапе инициализации.

Модульная система Java (введенная в Java 9 через проект Jigsaw) решает эту проблему на архитектурном уровне. Благодаря явному определению зависимостей между модулями JVM получает точную информацию о том,

https://max.ru/bookofgeek

какие классы и библиотеки действительно нужны при старте приложения. В результате загрузка классов становится более целенаправленной и эффективной.

Механизмы оптимизации загрузки

Одним из ключевых факторов ускорения запуска является упрощение процесса загрузки классов (class loading). При правильно спроектированной модульной архитектуре:

- **Исключаются неиспользуемые классы**. Модули инкапсулируют свою реализацию и экспортируют наружу только те пакеты, которые явно объявлены в *module-info.java*. Таким образом, сторонние компоненты не имеют прямого доступа ко всем внутренностям модуля, что уменьшает количество загружаемых классов.
- **Уменьшается количество операций рефлексии**. Строгая граница между модулями снижает необходимость в использовании рефлексии для доступа к приватным API, что положительно сказывается на времени запуска.
- **Оптимизируется порядок загрузки**. JVM может заранее строить эффективные графы зависимостей и параллелить процесс инициализации модулей.

Практические рекомендации для ускорения запуска через модули

Чтобы добиться реального прироста скорости старта, важно учитывать несколько практических аспектов:

- **Минимизируйте размер модулей**. Чем меньше каждый отдельный модуль, тем быстрее он может быть загружен и проинициализирован. Старайтесь выделять четко ограниченные функциональные области в отдельные модули.
- **Избегайте чрезмерных зависимостей**. Если модуль зависит от большого количества других модулей, он теряет преимущество легкости и быстроты загрузки. Оптимальная архитектура предполагает минимально необходимые связи.

https://max.ru/bookofgeek

- **Разделяйте модули на этапы загрузки**. Можно проектировать модули так, чтобы основные критичные компоненты загружались первыми, а менее важные — откладывались до момента, когда они действительно понадобятся (ленивая загрузка).
- **Удаляйте ненужные зависимости**. Часто в процессе разработки в модули добавляются библиотеки "на всякий случай". Регулярный аудит зависимостей позволяет очистить проект и ускорить старт.
- **Используйте Ahead-Of-Time (AOT) компиляцию для критичных модулей**. Если это допустимо архитектурой проекта, можно предварительно скомпилировать тяжелые части приложения, что значительно сократит время запуска.

Пример влияния модуляризации на время запуска

Рассмотрим типичную ситуацию. Приложение на Java содержит около 2000 классов, из которых в реальности активно используется только 600–700. В старой монолитной архитектуре JVM при старте была вынуждена инициализировать всю эту массу, что приводило к стартовому времени в районе 15–20 секунд на стандартной машине.

После реорганизации кода в модульную структуру:

- Количество загружаемых классов сократилось на 40–50%.
- За счет уменьшения стартовой нагрузки время запуска уменьшилось до 8–10 секунд.
- Дополнительно модульная структура позволила быстрее инициализировать только необходимые сервисы, оставляя остальные компоненты в "спящем" состоянии до запроса.

Это показывает, что даже без глубоких изменений в бизнес-логике переход к модулям способен ощутимо повлиять на перформанс старта.

Особенности при контейнеризации

Если Java-приложение работает в контейнерах (например, в Docker/Kubernetes среде), время запуска становится еще более критичным, особен-

https://max.ru/bookofgeek

но при масштабировании или автоматическом восстановлении инстансов. Модульная архитектура помогает ускорить "прогрев" новых контейнеров, сокращая общее время запуска системы и повышая ее устойчивость.

Кроме того, уменьшение размера классов, загружаемых в память, напрямую влияет на скорость работы GC (сборщика мусора) и на общее потребление ресурсов контейнером.

9.4. Минимизация используемых зависимостей

Одним из ключевых преимуществ применения модульной архитектуры в Java-приложениях является возможность строгого контроля используемых зависимостей.

> В традиционной модели разработки через *classpath* все библиотеки и классы становятся глобально доступными приложению, что часто приводит к избыточной нагрузке: используются лишь отдельные части библиотек, а остальной код просто занимает память и замедляет выполнение.

Модулизация решает эту проблему на корневом уровне:

- **Явное объявление зависимостей**. В модульной системе Java каждый модуль через файл *module-info.java* четко указывает, от каких других модулей он зависит. Это позволяет избежать "скрытых" зависимостей, когда библиотека А использует библиотеку В, а разработчик об этом даже не подозревает.
- **Изоляция модулей**. Код одного модуля видит только те классы других модулей, которые явно экспортированы через *exports* и *requires*. Это предотвращает несанкционированное или случайное использование внутренних классов библиотек, что способствует более чистой архитектуре.
- **Оптимизация сборки**. При сборке приложения можно включить только те модули и зависимости, которые действительно необходимы для работы конечного решения. Это особенно важно для создания легких и эффективных приложений, где минимизация размера артефакта и набора зависимостей критична.

https://max.ru/bookofgeek

- **Снижение объема памяти**. Меньше зависимостей — меньше классов загружается в память JVM, что приводит к снижению потребления памяти как в Metaspace (где хранятся метаданные классов), так и в Heap. Кроме того, это снижает нагрузку на сборщик мусора, так как в куче оказывается меньше объектов.
- **Повышение безопасности**. При явном контроле зависимостей снижается вероятность того, что в приложение будет случайно включена уязвимая или ненужная библиотека. Вы точно знаете, какие модули и библиотеки используются, и можете своевременно обновлять или изолировать потенциально опасные зависимости.
- **Упрощение обновлений и миграций**. При наличии минимального набора зависимостей проще проводить обновления отдельных библиотек или замену их на альтернативные решения, поскольку зависимости модулей четко задокументированы и ограничены.

Практические советы:

- Регулярно проводите ревизию зависимостей модулей.
- Стремитесь к минимальной необходимой поверхности API (*exports* только то, что нужно).
- Используйте инструменты анализа зависимостей, такие как *jdeps*, для выявления скрытых или неиспользуемых связей.
- Для крупных проектов стройте модульную карту (dependency graph), чтобы визуально контролировать структуру приложения.

Пример:

Представим приложение, которому нужна только одна функция для сериализации JSON. В старой модели часто подключали бы целую библиотеку Jackson с множеством дополнительных модулей (например, поддержку YAML, XML и даже DatatypeConverters). При правильной модуляризации можно подключить только основной модуль Jackson Core, избежав загрузки ненужных вспомогательных модулей.

Таким образом, минимизация используемых зависимостей через модуляризацию не только делает приложение компактнее и быстрее, но и улучшает его сопровождаемость, безопасность и качество.

https://max.ru/bookofgeek

9.5. Снижение использования памяти

Модулизация играет важную роль в эффективной оптимизации использования памяти в современных Java-приложениях. Одним из главных источников избыточного потребления памяти в крупных системах является загрузка ненужных классов и библиотек. В традиционных монолитных приложениях вся логика и все сторонние зависимости загружаются в память независимо от их актуальной необходимости в данный момент времени. Это приводит к чрезмерному росту размера Metaspace и увеличению общего потребления оперативной памяти.

Модульная архитектура позволяет решать эту проблему системно. Благодаря явному описанию зависимостей между модулями и строгому контролю над составом каждой функциональной единицы приложения, JVM может загружать только те классы и библиотеки, которые действительно требуются для работы в конкретной ситуации. В результате достигается значительное сокращение количества загруженных классов и ресурсов, задействованных в процессе исполнения программы.

Экономия памяти за счет уменьшения объема классов

При модулизации каждый модуль четко определяет, какие пакеты он экспортирует наружу и какие пакеты он импортирует для своей работы. Это позволяет компилятору и JVM более агрессивно оптимизировать загрузку кода, игнорируя неиспользуемые классы и даже целые библиотеки.

Таким образом, в памяти оказывается только минимально необходимый набор компонентов, что особенно критично для приложений, работающих в ограниченных средах, таких как контейнеры или серверы с небольшим объемом оперативной памяти.

Более того, за счет уменьшения количества загружаемых классов снижается нагрузка на внутренние структуры JVM, связанные с управлением метаданными классов и разрешением ссылок между ними. Это приводит к уменьшению накладных расходов на управление памятью и ускорению работы системы в целом.

https://max.ru/bookofgeek

Влияние на скорость загрузки классов

Когда приложение построено как монолит, JVM при запуске вынуждена проанализировать и подготовить огромное количество классов, независимо от их реальной необходимости. Это не только занимает больше памяти, но и существенно увеличивает время старта приложения. В модульной архитектуре JVM может "рассуждать" о классах более эффективно: модульная модель явно описывает, какие компоненты являются обязательными, а какие могут быть загружены позднее.

Уменьшение числа классов, подлежащих анализу и подготовке в момент запуска, напрямую ускоряет процесс загрузки приложения и снижает время, необходимое для его выхода в рабочее состояние. Это критически важно для систем, где время старта играет большую роль, например, в serverless-архитектурах или высоконагруженных микросервисах, которые могут часто перезапускаться.

Ленивая загрузка (Lazy Loading)

Дополнительным преимуществом модулизации является возможность организовать ленивую загрузку модулей и классов.

> **Lazy loading** предполагает, что определенные части приложения загружаются в память только в момент их фактического обращения, а не заранее.

В модульной архитектуре это можно реализовать естественным образом, делая зависимости между модулями динамическими и загружая вспомогательные модули только тогда, когда пользователь или процесс действительно их потребует.

Применение ленивой загрузки дает сразу несколько плюсов:

- **Сокращение памяти**. Пока функциональность не требуется, соответствующие классы и ресурсы не занимают место в памяти.
- **Ускорение старта**. Приложение начинает работу быстрее, так как не тратит время на инициализацию всего доступного функционала.

https://max.ru/bookofgeek

- **Гибкость и масштабируемость**. Позволяет более гибко управлять ресурсами в зависимости от сценариев использования.

> **Пример**: в приложении для аналитики данные модули для построения графиков и сложной визуализации могут подгружаться только в момент, когда пользователь запрашивает построение графика, а не сразу при старте системы.

Практические рекомендации по снижению потребления памяти через модулизацию:

- **Разделение функциональности на мелкие модули**. Чем более атомарными будут модули, тем проще JVM оптимизировать их загрузку.
- **Минимизация экспортов**. Экспортировать наружу только те пакеты, которые действительно нужны другим модулям.
- **Изоляция редко используемого функционала**. Вынести вспомогательные функции в отдельные модули и загружать их лениво.
- **Использование динамической загрузки сервисов через *ServiceLoader***. Позволяет подключать реализации интерфейсов только по запросу.
- **Анализ метрик Metaspace и heap-памяти**. Регулярный мониторинг использования памяти позволяет вовремя выявить модули, которые чрезмерно потребляют ресурсы.

9.6. Модулизация в больших проектах

В больших программных проектах модулизация становится не просто удобной практикой, а практически необходимым условием для поддержания качества, производительности и управляемости кода.

Когда приложение состоит из десятков или сотен компонентов, интеграция множества библиотек и функциональных блоков без четкой структуры может привести к лавинообразному росту сложности, проблемам с производительностью и трудностям в сопровождении системы.

https://max.ru/bookofgeek

Модулизация позволяет логически и технически разделить проект на независимые или слабо связанные части, каждая из которых отвечает за конкретную область функциональности. Такое разделение улучшает масштабируемость, облегчает параллельную работу разных команд, упрощает поддержку и ускоряет внедрение новых возможностей.

Управление зависимостями и предотвращение "загрязнения" пространства классов

В масштабных проектах без модульной архитектуры очень быстро появляется проблема избыточных зависимостей. Компоненты приложения начинают неявно использовать классы друг друга, библиотеки втягиваются в проект через транзитивные зависимости, и в итоге любое изменение становится рискованным. Модулизация помогает контролировать, какие именно классы и пакеты доступны другим частям системы через явное объявление *exports* и *requires* в описании модулей.

Такой подход значительно снижает риск появления конфликтов версий библиотек ("dependency hell") и делает поведение системы более предсказуемым. Важно также, что при строгой модуляризации исключается несанкционированное использование внутренних реализаций модулей, так как только явно экспортируемые API становятся доступными внешним потребителям.

Оптимизация времени старта и потребления памяти

Одним из побочных, но крайне важных эффектов модулярности в больших приложениях является оптимизация времени запуска и сокращение используемой памяти. JVM получает возможность загружать только необходимые модули и классы, игнорируя весь ненужный код, который в старой монолитной архитектуре также попадал бы в память.

Для действительно больших систем это может дать прирост в старте на десятки процентов и значительно сократить Metaspace, особенно если проект использует множество вспомогательных библиотек, активируемых по условию.

Улучшение масштабируемости и параллельной разработки

https://max.ru/bookofgeek

Когда приложение разбито на модули, разные команды могут работать над разными частями системы независимо друг от друга. Четкие контрактные границы между модулями позволяют определить зоны ответственности, а строгие зависимости упрощают интеграцию. Появляется возможность разрабатывать, тестировать и деплоить отдельные части системы с минимальным влиянием на остальное приложение.

Это критично в условиях Agile-подходов и DevOps-практик, когда скорость вывода новых функций напрямую влияет на успех бизнеса.

Облегчение тестирования и повышения качества

Модулизация упрощает процесс тестирования. Вместо того чтобы запускать все тесты всего приложения, можно ограничиться тестированием конкретного модуля и его окружения. Изоляция функциональности позволяет писать более компактные и эффективные юнит- и интеграционные тесты, покрывающие только необходимую логику.

Также модульная структура облегчает статический анализ кода, контроль зависимостей, аудит безопасности и соблюдение архитектурных правил в большом проекте.

Разграничение ответственности и повышение читаемости архитектуры

Чем крупнее приложение, тем сложнее разработчикам ориентироваться в кодовой базе. Модулизация помогает на уровне кода отражать бизнес-логику: каждый модуль отвечает за свою функциональную область — например, модуль аутентификации, модуль обработки заказов, модуль уведомлений и т.д.

Такая структуризация делает архитектуру системы более очевидной как для новичков в проекте, так и для опытных разработчиков. Это значительно упрощает onboarding, ревью кода и выявление архитектурных проблем.

Поддержка эволюции проекта

https://max.ru/bookofgeek

С ростом проекта часто возникает необходимость заменять или масштабировать отдельные его части. Модулизация позволяет это делать без необходимости переписывать все приложение целиком. Достаточно заменить конкретный модуль или расширить его функциональность, не затрагивая остальную систему.

Кроме того, правильная модульная архитектура готовит проект к переходу на микросервисную архитектуру в будущем: изолированные модули проще превратить в самостоятельные сервисы при необходимости.

9.7. Практика: разбиение монолита на модули

Перевод монолитного приложения в модульную архитектуру — это сложная, но крайне важная задача для долгосрочной устойчивости проекта.

Несмотря на потенциальную сложность, правильный подход к модуляризации может значительно повысить управляемость кода, ускорить его развитие, а также улучшить производительность за счет более эффективного использования ресурсов.

Процесс разбиения монолита можно разбить на несколько ключевых этапов, каждый из которых требует отдельного внимания.

9.7.1. Анализ текущей архитектуры

Первым шагом необходимо провести тщательный анализ существующего монолита. Нужно ответить на вопросы:

- Как устроено приложение сейчас?
- Какие основные слои или функциональные области существуют?
- Где проходят главные границы ответственности между частями кода?
- Какие части приложения наиболее тесно связаны друг с другом?
- Какие зависимости могут вызвать проблемы при разделении?

На этом этапе полезно строить диаграммы зависимостей между пакетами и классами, например с помощью инструментов типа JDepend или Structure101.

https://max.ru/bookofgeek

Также важно выявить "горячие точки" — области кода, которые вызывают наибольшее количество изменений и которые стоит модулизировать в первую очередь.

9.7.2. Разделение по слоям и функциональности

Один из эффективных подходов к модуляризации — это разбиение приложения по архитектурным слоям:

- **Слой представления (UI)** — все, что связано с взаимодействием с пользователем.
- **Слой бизнес-логики** — правила обработки данных и бизнес-процессы.
- **Слой доступа к данным (DAO/Repository)** — работа с базами данных, внешними API.

Каждый слой может быть представлен в виде одного или нескольких модулей.

Также стоит выделять модули по бизнес-функциям. Например:

- модуль управления пользователями,
- модуль обработки заказов,
- модуль уведомлений,
- модуль авторизации и безопасности.

Такое функциональное разбиение облегчает поддержку и эволюцию кода.

9.7.3. Определение и контроль зависимостей между модулями

После логического разбиения важно правильно задекларировать зависимости:

- Каждый модуль должен явно указывать, какие другие модули ему нужны (*requires*).
- Нужно избегать циклических зависимостей (модуль A зависит от модуля B, а B от A).

https://max.ru/bookofgeek

- Общие интерфейсы и контракты следует вынести в отдельные "API-модули", чтобы минимизировать прямые связи.

> **Идеальная структура** — это дерево зависимостей, где зависимости идут в одну сторону — от высокоуровневых модулей к низкоуровневым.

Пример:

```
application (UI) → service-api → service-impl → persistence
```

9.7.4. Минимизация публичного API

При переходе на модули следует четко разграничить, какие классы являются частью публичного API модуля, а какие — его внутренней реализацией:

- Только те классы, которые действительно нужны другим модулям, должны быть помечены *exports*.
- Внутренние классы должны оставаться скрытыми.

Такой подход повышает инкапсуляцию и защищает систему от случайного нарушения контрактов.

9.7.5. Постепенный рефакторинг и миграция

Полный переход на модульную архитектуру редко происходит "за один коммит".

Рекомендуемый подход — **пошаговая миграция**:

- Выделить небольшую, но важную часть приложения в отдельный модуль.
- Настроить сборку и запуск с учетом нового модуля.
- Проверить корректность работы и тестов.
- Постепенно переносить другие части приложения.

https://max.ru/bookofgeek

Иногда имеет смысл сначала внедрить систему модулей только логически, сохранив технически монолитный проект, а затем уже физически разбивать его на модули.

9.7.6. Тестируемость каждого модуля

Очень важно обеспечить возможность отдельного тестирования каждого модуля:

- **Юнит-тесты** должны охватывать внутреннюю бизнес-логику модуля.
- **Интеграционные тесты** должны проверять взаимодействие модуля с внешними сервисами.
- Каждый модуль должен уметь собираться и тестироваться отдельно.

Это не только повысит качество кода, но и значительно ускорит цикл разработки и доставки.

9.7.7. Технические детали: использование системы модулей Java

Java начиная с версии 9 предлагает стандартный механизм модулей (*module-info.java*), который можно активно использовать для физического разбиения:

- *module-info.java* объявляет зависимости (*requires*) и экспортируемые пакеты (*exports*).
- Компилятор проверяет корректность модульной структуры на этапе сборки.
- JVM оптимизирует загрузку классов и уменьшает память за счет knowledge о границах модулей.

Поддержка модулей также реализована в популярных системах сборки:

- **Maven** с поддержкой мультипроектных сборок и профилями для модульных JAR'ов.

https://max.ru/bookofgeek

- **Gradle** с поддержкой проекта на уровне нескольких модулей через *settings.gradle.*

Пример пошаговой миграции монолита:

- Создать новый модуль *user-management.*
- Перенести в него все классы и интерфейсы, относящиеся к пользователям.
- Создать *module-info.java:*

```
module com.example.usermanagement {
    exports com.example.usermanagement.api;
    requires com.example.persistence;
}
```

- Обновить сборку и зависимости проекта.
- Перенастроить тесты, чтобы проверять новый модуль отдельно.
- Повторить процесс для следующей функциональной области.

9.8. Инструменты для модуляризации в Java

С момента внедрения **Java Platform Module System (JPMS)** в JDK 9 модулизация стала неотъемлемой частью экосистемы Java. Однако, чтобы эффективно использовать модульную систему, необходимо познакомиться с рядом инструментов и подходов, которые помогут при проектировании, разработке и деплое модульных приложений.

1. **JMOD** — это файл, представляющий модуль в Java. Он позволяет упаковывать модули вместе с их зависимостями, что упрощает переносимость и распределение приложений. Инструмент *jmod* помогает создавать, а также управлять этими модулями. JMOD-файлы полезны для создания и распространения модулей в масштабных системах.
2. **Maven и Gradle** — для работы с модулями можно использовать инструменты сборки, такие как Maven и Gradle. В обоих случаях требуется до-

https://max.ru/bookofgeek

бавить поддержку JPMS, которая позволяет управлять зависимостями между модулями, контролировать их версионность и процесс упаковки. Например, Maven предоставляет плагин *maven-jmod-plugin*, а Gradle — интеграцию с системой модулей через свои таски и зависимости.

3. **IDE и инструменты для рефакторинга** — многие современные IDE, такие как IntelliJ IDEA и Eclipse, имеют встроенную поддержку для работы с модулями. В IntelliJ IDEA можно использовать встроенные механизмы для анализа модульных зависимостей, а также для автоматического рефакторинга существующих приложений в модульный стиль. Эти инструменты могут значительно ускорить процесс модуляризации, предоставляя средства для обнаружения и устранения проблем с зависимостями.
4. **JLink** — инструмент для создания собственных исполняемых Java-образов, которые содержат только те модули, которые необходимы для работы приложения. JLink помогает уменьшить размер приложения, исключив ненужные модули и зависимости. Это также способствует повышению производительности, так как минимизируется объем данных, которые необходимо загружать в память.

9.9. Рефакторинг монолита в модульную структуру

Процесс перевода существующего монолитного приложения на модульную структуру может быть сложным и длительным. Важно правильно спланировать переход, чтобы избежать потери функциональности и не ухудшить производительность.

1. **Анализ существующего кода**. Перед тем как начать процесс модуляризации, необходимо провести анализ текущей структуры приложения. Это включает в себя:

   - Определение **глобальных зависимостей**: какие библиотеки и компоненты используются по всему приложению.
   - Оценку **связности компонентов**: как различные части приложения взаимодействуют между собой.
   - Выявление **потенциальных проблем**: избыточных зависимостей, дублирующего кода и сложных взаимодействий.

https://max.ru/bookofgeek

2. **Поэтапное разделение на модули**. Преобразование монолита в модульную структуру часто выполняется поэтапно. Рекомендуется начать с выделения наиболее независимых частей приложения, таких как слои данных или бизнес-логики, и постепенно двигаться к более сложным частям, которые могут требовать дальнейшего рефакторинга.

3. **Модульные тесты**. Важно обеспечить, чтобы каждый модуль был протестирован независимо. Это означает, что необходимо наладить систему модульных тестов для проверки функциональности каждого компонента. Это не только улучшит качество кода, но и поможет быстрее обнаруживать проблемы на ранних этапах разработки.

4. **Внесение изменений в процесс деплоя**. Когда приложение переходит на модульную структуру, изменяется и процесс его деплоя. Для распределенных систем важно обновлять модули по мере необходимости, чтобы избежать простоя. Использование **JLink** и инструментов для сборки и упаковки помогает ускорить этот процесс, минимизируя время, необходимое для деплоя и загрузки.

9.10. Влияние модулярности на тестирование и деплой

Модуляризация оказывает непосредственное влияние на процесс тестирования и деплоя Java-приложений. Важно понимать, как разделение на модули может улучшить производительность и упростить поддержание кода.

1. **Упрощение тестирования**. Каждый модуль может быть протестирован независимо, что снижает количество ошибок и ускоряет процесс обнаружения багов.

   Модулярность помогает избежать перегрузки тестов, так как тестируются только те части системы, которые требуют внимания.

   » **Модульные тесты**. Каждому модулю можно писать отдельные тесты, которые проверяют его внутреннюю логику и взаимодействие с другими модулями.

https://max.ru/bookofgeek

» **Интеграционные тесты**. Тестирование взаимодействия между модулями важно для обеспечения их правильной работы в составе системы.

2. **Обновления и деплой**. Модульная структура упрощает процесс деплоя, так как можно обновлять или заменять отдельные модули без необходимости перезагружать все приложение. Это позволяет улучшить **время отклика** и уменьшить **время простоя**, особенно в распределенных системах.

3. **Обновления библиотек**. Обновление зависимостей в модульной системе становится проще. В отличие от монолитных приложений, где обновление одной библиотеки может повлиять на всю систему, в модульных приложениях можно обновлять только те части, которые зависят от измененных библиотек.

9.11. Пример модулярного подхода

Рассмотрим пример, когда приложение разделяется на несколько модулей: модуль для работы с базой данных, модуль для бизнес-логики и модуль для пользовательского интерфейса.

1. **Модуль базы данных** — этот модуль включает все классы, которые взаимодействуют с базой данных. Он инкапсулирует логику работы с базой данных, включая доступ к данным, их модификацию и обновление.

2. **Модуль бизнес-логики** — этот модуль включает все основные операции и бизнес-логику. Он может зависеть от модуля базы данных, но не содержит деталей реализации работы с БД.

3. **Модуль пользовательского интерфейса** — данный модуль отвечает за представление данных, полученных от бизнес-логики. Он не должен знать о том, как именно данные получаются из базы данных, а только оперировать с результатами, которые предоставляет бизнес-логика.

Такой подход позволяет уменьшить связанные между собой части системы и облегчить тестирование и поддержку. Если нам нужно изменить логику работы с базой данных, мы можем обновить только модуль базы данных, не затрагивая другие части системы.

https://max.ru/bookofgeek

9.12. Модули и контейнеризация

Контейнеризация — это один из самых популярных способов развертывания и масштабирования приложений в современных распределенных системах. Она тесно связана с концепцией модульности, поскольку контейнеры обеспечивают изоляцию компонентов приложения, а модули могут быть использованы для обеспечения такой изоляции на уровне кода.

Взаимодействие между модульной системой Java и контейнерами может дать значительные преимущества в плане оптимизации производительности и управления приложением.

1. **Изоляция модулей в контейнерах**. Модульность позволяет разделить приложение на независимые компоненты, которые могут быть развернуты и масштабированы отдельно. В контейнерах, таких как **Docker**, можно создать образы для каждого отдельного модуля, что позволяет изолировать их друг от друга. Это уменьшает взаимозависимости и способствует более легкому управлению версиями и обновлениями модулей. Применение принципа **Single Responsibility** в контексте контейнеризации позволяет добиться оптимальной изоляции и надежности компонентов.

2. **Управление зависимостями между модулями**. Использование Docker-контейнеров в сочетании с модульной системой Java помогает в управлении зависимостями между различными частями приложения. Когда приложение состоит из нескольких контейнеров, каждый контейнер может отвечать за определенный модуль, что облегчает их обновление и масштабирование. Однако необходимо внимательно следить за зависимостями между модулями, чтобы избежать проблем с несовместимыми версиями. Это особенно важно для сложных распределенных систем, где могут возникать проблемы с производительностью из-за неправильного взаимодействия между контейнерами.

3. **Оптимизация контейнерных образов**. Применение модульности в контейнерных приложениях помогает значительно уменьшить размер образов, что снижает время их загрузки и ускоряет масштабирование. Это

https://max.ru/bookofgeek

особенно важно для микросервисных архитектур, где большое количество маленьких контейнеров может быть использовано для быстрого масштабирования. Благодаря разделению на модули можно исключить ненужные зависимости и снизить общий размер контейнера.

4. **Ресурсоемкость и контейнеризация**. Сочетание модульности и контейнеризации помогает эффективнее распределять ресурсы между различными компонентами приложения. Например, каждый контейнер может иметь собственные настройки JVM, что позволит более гибко управлять потреблением памяти и процессорных ресурсов. Это особенно полезно в условиях ограниченных ресурсов, например при запуске приложения на виртуальных машинах или в облачных средах.

9.13. Best practices при модуляризации

Модуляризация — это мощный инструмент для повышения производительности и поддерживаемости приложения. Однако, чтобы добиться максимальных результатов, необходимо следовать лучшим практикам при проектировании и реализации модульных систем. Рассмотрим несколько ключевых рекомендаций для эффективной модуляризации Java-приложений.

1. **Четкое разделение на модули**. Разделение приложения на модули должно быть логичным и легко понимаемым. Каждый модуль должен быть независимым и отвечать за определенную часть функционала, при этом его внутренности должны быть скрыты от других частей приложения. Хорошая практика — использование принципа **Single Responsibility Principle (SRP)**, который говорит о том, что каждый модуль должен решать одну задачу. Разбиение приложения на маленькие, легко поддерживаемые модули делает код более удобным для тестирования и масштабирования.

2. **Минимизация взаимозависимостей**. Одна из основных целей модуляризации — это снижение связности между различными частями приложения. Чем меньше зависимости между модулями, тем проще будет поддерживать и обновлять приложение. Необходимо тщательно продумывать, какие модули должны взаимодействовать друг с другом, а какие

https://max.ru/bookofgeek

можно оставить независимыми. Для этого можно использовать механизмы инкапсуляции, а также интерфейсы и абстракции для взаимодействия между модулями. Это помогает избежать ненужных зависимостей и делает код более гибким.

3. **Использование версии модулей**. В процессе разработки модульных приложений важно следить за версиями каждого модуля. Это особенно актуально для крупных проектов, где обновление одного модуля может повлиять на другие части системы. Важно использовать систему контроля версий для отслеживания изменений в модулях и для обеспечения совместимости между различными версиями. Применение версионности позволяет упростить обновления и тестирование новых функциональностей.

4. **Использование библиотек и фреймворков для управления зависимостями**. Для упрощения работы с зависимостями можно использовать такие инструменты, как **Maven** или **Gradle**, которые позволяют автоматически загружать и управлять зависимыми модулями. Эти инструменты помогают значительно упростить процесс сборки и развертывания приложения. Использование таких инструментов улучшает управляемость и совместимость между модулями.

5. **Учет требований к производительности**. При проектировании модулярной системы важно помнить о производительности. Каждый модуль должен быть оптимизирован с точки зрения времени работы и использования памяти. Избыточные зависимости и ненужные вычисления могут сильно снизить производительность системы. Также необходимо следить за временем старта приложения, так как большое количество модулей может увеличить время инициализации, если не оптимизировать загрузку.

6. **Тестирование и автоматизация**. Модуляризация облегчает тестирование приложения, так как каждый модуль можно тестировать отдельно. Это помогает обнаружить ошибки на ранних стадиях разработки и улучшить качество кода. Автоматизация тестирования через инструменты вроде **JUnit** или **TestNG** помогает быстро проверять функциональность отдельных модулей и их взаимодействие. Важно обеспечить полный цикл тестирования для каждого модуля, включая юнит-тесты, интеграционные тесты и тесты производительности.

https://max.ru/bookofgeek

7. **Документирование модулей**. Хорошая документация помогает разработчикам понять структуру приложения и быстро ориентироваться в коде. Каждый модуль должен быть хорошо документирован, особенно если он предоставляет API для других частей системы. Хорошая документация включает в себя описание функционала, интерфейсов, методов и типов данных, используемых в модуле. Это особенно важно для командной работы, когда несколько разработчиков взаимодействуют с одним и тем же проектом.

Выводы

Модуляризация — это ключ к созданию эффективных, масштабируемых и поддерживаемых Java-приложений. Процесс модуляризации помогает улучшить производительность, облегчить поддержку и обновление, а также облегчить тестирование и развертывание. Однако для того чтобы извлечь максимальную пользу из модульности, важно придерживаться лучших практик и учитывать особенности архитектуры, производительности и совместимости между модулями.

Правильный выбор структуры модулей, минимизация взаимозависимостей и грамотное использование инструментов и фреймворков для управления зависимостями помогут вам создать приложение, которое будет не только работать эффективно, но и легко масштабироваться и развиваться. Модульность также идеально сочетается с контейнеризацией, позволяя создавать легкие, изолированные контейнеры для каждого модуля, что способствует более гибкому и быстрому масштабированию.

В конечном итоге, модуляризация и контейнеризация помогают повысить не только производительность, но и качество кода, что является ключевым фактором в успешном развитии и эксплуатации сложных приложений.

https://max.ru/bookofgeek

https://max.ru/bookofgeek

Глава 10.

Сборщики мусора в Java

https://max.ru/bookofgeek

О чём глава:

- Обзор типов GC (Serial, G1, ZGC, Shenandoah).
- Выбор GC под тип приложения.
- Тюнинг и мониторинг GC.

Сборка мусора (Garbage Collection, GC) — это одна из ключевых функций виртуальной машины Java (JVM), освобождающая разработчика от необходимости вручную управлять памятью.

Однако автоматическое управление памятью не делает задачу оптимизации производительности тривиальной. Напротив, выбор подходящего сборщика мусора и правильная его настройка могут существенно повлиять на производительность приложения, особенно в условиях высоких нагрузок или при работе с большим объемом данных.

В этой главе мы рассмотрим основные типы сборщиков мусора, доступные в JVM, их архитектурные особенности, сценарии применения, подходы к выбору GC под конкретное приложение, а также методы мониторинга и тюнинга GC.

Мы также поговорим о том, какое влияние выбор GC оказывает на оптимизацию производительности Java-приложений.

https://max.ru/bookofgeek

10.1. Обзор типов GC

Serial GC

Serial Garbage Collector — это самый простой и наименее масштабируемый сборщик мусора. Он использует один поток как для основного приложения, так и для операций сборки мусора. Он подходит для приложений с небольшим объемом памяти и нестрогими требованиями ко времени отклика, таких как настольные приложения или утилиты командной строки.

- Использует остановку всего мира (*Stop-The-World*) на время сборки.
- Хорошо работает при малом объеме хипа (≤100MB).
- Поддерживается с ранних версий Java.
- Управляется с помощью флага: *-XX:+UseSerialGC*.

Parallel GC (Throughput Collector)

Parallel GC, также известный как *Throughput Collector*, использует несколько потоков для выполнения сборки мусора, что увеличивает пропускную способность системы.

- Идеален для многопроцессорных систем.
- Основной фокус — максимизация пропускной способности, а не минимизация пауз.
- Подходит для batch-приложений, offline-процессов.
- Включается с помощью: *-XX:+UseParallelGC*.

G1 GC (Garbage-First)

G1 GC — это современный сборщик, представленный в Java 7 и ставший сборщиком по умолчанию начиная с Java 9. Он делит хип на регионы и использует инкрементную, предсказуемую стратегию сборки.

https://max.ru/bookofgeek

- Уменьшает длительность пауз за счет параллельной и инкрементальной работы.
- Эффективен при больших объемах хипа (≥4GB).
- Подходит для серверных приложений и систем с критичными требованиями к задержкам.
- Конфигурируется с помощью: *-XX:+UseG1GC.*

ZGC (Z Garbage Collector)

ZGC — это масштабируемый, низкопаузный сборщик мусора, представленный в Java 11 как экспериментальный и стабилизированный в более поздних версиях. Он разработан для минимизации пауз до менее 10 мс, независимо от размера хипа.

- Работает с хипами до 16 ТБ.
- Паузы не зависят от размера хипа.
- Использует цветовую маркировку объектов (colored pointers).
- Включается с помощью: *-XX:+UseZGC.*

Shenandoah GC

Shenandoah — низкопаузный GC от Red Hat, включенный в OpenJDK. Он минимизирует время пауз, производя большую часть работы параллельно с приложением.

- Оптимален для интерактивных, откликчувствительных приложений.
- Имеет предсказуемые паузы, даже при большом хипе.
- Активируется с помощью: *-XX:+UseShenandoahGC*

10.2. Выбор GC под тип приложения

Выбор подходящего сборщика мусора (GC — Garbage Collector) в Java-приложении играет критическую роль в обеспечении стабильной произво-

https://max.ru/bookofgeek

дительности и предсказуемого поведения системы. Неправильный выбор GC или отсутствие его настройки может привести к долгим паузам, неоптимальному использованию памяти, снижению пропускной способности и, в худшем случае, к OutOfMemoryError.

Ключевые факторы, влияющие на выбор GC

Выбор сборщика мусора не универсален — он напрямую зависит от особенностей приложения, требований к латентности, архитектуры сервера и бизнес-логики. Рассмотрим основные параметры, которые следует учитывать:

1. **Объем хипа (Heap Size):**
   - Маленький хип (до 1-2 ГБ): допустимы простые сборщики вроде Serial или Parallel GC.
   - Средний и большой хип (от 4 ГБ до сотен ГБ): лучше подходят G1 GC, ZGC или Shenandoah, способные работать эффективно в условиях большого объема памяти.
2. **Характер нагрузки:**
   - **Интерактивные приложения и веб-серверы:** важна минимизация пауз и предсказуемость отклика. В этом случае приоритет у низкопаузных сборщиков — **G1 GC**, **Shenandoah** или **ZGC**.
   - **Batch-обработка и оффлайн-вычисления:** допустимы длительные паузы, но важна общая пропускная способность. Здесь подходят **Parallel GC** или даже **Serial GC**.
   - **Real-time системы:** требуют максимально детерминированного поведения и равномерного распределения работы сборщика. Подобные задачи могут частично решаться через **Shenandoah** или использование специализированных JVM.
3. **Допустимая длина паузы (Pause Time):**
   - Для некоторых бизнес-критичных сервисов (например, торговых платформ) важны **миллисекундные паузы**, а то и **sub-millisecond**. В таких случаях стоит выбирать **ZGC** или **Shenandoah**, которые обеспечивают **низкую и предсказуемую латентность**, даже при большом объеме хипа.

https://max.ru/bookofgeek

- Если задержки не критичны, можно использовать сборщики с ориентацией на throughput — например, **Parallel GC**.

4. **Требования к пропускной способности (Throughput)**:
   - **Parallel GC** показывает высокую общую пропускную способность за счет параллельной работы, но может давать длинные стоп-паузы.
   - **G1 GC** балансирует между throughput и латентностью.
   - **ZGC** и **Shenandoah** ориентированы больше на минимизацию пауз, чем на максимальную пропускную способность.
5. **Количество доступных ядер процессора:**
   - **Serial GC** подходит для систем с одним ядром.
   - **Parallel**, **G1**, **ZGC**, **Shenandoah** требуют многопоточности и масштабируются по числу доступных ядер. Для их эффективной работы желательно иметь от 4 и более CPU-ядер.

Примеры выбора GC для типовых приложений

Чтобы понять на практике, как выбирать сборщик мусора, рассмотрим несколько типовых сценариев.

Таблица 10.1.

Тип приложения	Рекомендуемые GC	Причины выбора
Веб-приложение с требованиями к отклику	G1 GC, Shenandoah	Предсказуемые и короткие паузы, работа на средних и больших хипах
Обработка больших данных (Big Data, ETL, Spark и пр.)	G1 GC, ZGC	Работа с большими хипами, при этом важна стабильность и масштабируемость
Настольные утилиты и микросервисы с малым хипом	Serial GC	Простота и низкие накладные расходы на фоне малого объема памяти

https://max.ru/bookofgeek

Batch-обработка отчетов, периодические задачи	Parallel GC	Максимальная пропускная способность, отсутствие требований к паузам
Игровые движки и графические приложения (низкая задержка)	ZGC, Shenandoah	Предсказуемая латентность, минимальные паузы при высокой нагрузке
Сервисы на Java, развернутые в Kubernetes	G1 GC, Shenandoah	Хороший баланс между паузами и производительностью, адаптация под контейнерные среды

Важность эмпирической валидации

Даже если выбран «оптимальный» сборщик мусора, это не означает автоматического улучшения. Поведение GC может меняться в зависимости от версии JVM, конфигурации ОС, нагрузки на систему и прочих факторов. Поэтому **необходимо производить нагрузочное тестирование**, анализировать GC-логи и профилировать приложения в реальных или приближенных к боевым условиях.

Используйте следующие практики:

- Включайте GC-логирование (*-Xlog:gc** для Java 11+) и анализируйте его с помощью **GCViewer**, **GCEasy**, **JClarity Censum**.
- Применяйте **JMH-бенчмарки** для изолированной оценки времени пауз.
- Используйте профилировщики, поддерживающие анализ памяти, например **JFR (Java Flight Recorder)** или **VisualVM**.

Выбор сборщика мусора — это не вопрос «лучший/худший», а задача найти **оптимальное соответствие между архитектурой приложения и профилем его нагрузки**. Один и тот же GC может прекрасно работать в одном проекте и провалиться в другом. Поэтому важно подходить к выбору **итеративно**: сначала на основании архитектурных требований, затем — на основе эмпирических данных.

https://max.ru/bookofgeek

10.3. Тюнинг и мониторинг GC

Оптимизация поведения GC начинается с мониторинга. Существуют различные инструменты и метрики для анализа его работы.

Метрики GC

- **GC Time** — общее время, потраченное на сборку.
- **Pause Time** — длительность Stop-The-World пауз.
- **Frequency** — частота срабатывания GC.
- **Heap Usage** — текущее и максимальное использование хипа.

Инструменты мониторинга

- **jstat** — утилита командной строки для сбора статистики GC.
- **VisualVM** — визуальный инструмент для анализа памяти и GC.
- **JFR** (Java Flight Recorder) — профилировщик JVM.
- **GC Logs** — включаются флагами типа *-Xlog:gc** и анализируются вручную или через GCViewer.

Примеры флагов для настройки:

```
-XX:MaxGCPauseMillis=100
-XX:+PrintGCDetails
-XX:+PrintGCDateStamps
-Xlog:gc*:file=gc.log:time
```

Тюнинг G1 GC:

```
-XX:+UseG1GC
-XX:MaxGCPauseMillis=200
-XX:InitiatingHeapOccupancyPercent=45
```


https://max.ru/bookofgeek

Эти параметры управляют желаемой продолжительностью паузы и частотой запуска GC.

Влияние GC на производительность

Сборка мусора может как улучшить, так и ухудшить производительность приложения в зависимости от выбора сборщика и его конфигурации.

Вот несколько аспектов, через которые GC влияет на перформанс:

1. **Паузы** — долгие Stop-The-World паузы могут приводить к таймаутам и деградации пользовательского опыта.
2. **Частота GC** — слишком частый GC может «съедать» CPU.
3. **Фрагментация памяти** — влияет на эффективность аллокации новых объектов.
4. **Использование CPU** — более агрессивные GC (например, Parallel) потребляют больше процессорного времени.

Рекомендации:

- Не полагайтесь на GC по умолчанию — он не всегда оптимален.
- Измеряйте: профилируйте приложения с разными GC.
- Комбинируйте настройки GC с настройкой хипа: *-Xms, -Xmx*.
- Мониторьте GC в продакшене и реагируйте на аномалии.

Выводы

Правильный выбор и настройка сборщика мусора может значительно повлиять на эффективность Java-приложения. Вместо того чтобы подходить к GC как к «черному ящику», важно изучить его работу, понять его взаимодействие с особенностями вашего приложения и научиться управлять этим процессом.

https://max.ru/bookofgeek

В этой главе мы познакомились с основными сборщиками мусора, доступными в JVM, узнали об их особенностях и вариантах применения, а также рассмотрели способы мониторинга и настройки их поведения.

GC — это не просто внутренний механизм JVM, а инструмент, активно участвующий в производительности. И как любой мощный инструмент, он требует знания и уважения. В следующих главах мы продолжим рассматривать аспекты производительности, включая управление памятью, профилирование и настройку JVM.

https://max.ru/bookofgeek

Ещё больше свежей и актуальной литературы по Java ты найдёшь в наших каналах:
https://max.ru/bookofgeek
https://t.me/bookofgeek

Глава 11.

Работа с памятью и управление ресурсами

https://max.ru/bookofgeek

О чём глава:

- Профилирование использования памяти.
- Оптимизация работы с кучей и стеком.
- Управление и минимизация утечек памяти.

11.1. Профилирование использования памяти

Профилирование памяти — это процесс анализа использования памяти в приложении для выявления излишнего потребления, утечек памяти и других проблем.

В Java есть несколько инструментов для профилирования, таких как **JVM-профилировщики** и **инструменты для анализа кучи**.

Основные подходы к профилированию памяти

1. **JVM-средства профилирования.**

   Java предоставляет несколько стандартных инструментов для профилирования, таких как:

   » **Jvisualvm** — это инструмент для мониторинга и анализа работы JVM, который предоставляет графики, статистику по памяти, загрузке процессора и сбору мусора. Он позволяет загружать дампы памяти и анализировать использование кучи.

https://max.ru/bookofgeek

» **Jconsole** — это еще один инструмент для мониторинга работы JVM в реальном времени. Он предоставляет визуализацию статистики по памяти, производительности и состоянию потоков.

» **JProfiler**, **YourKit** — эти сторонние инструменты предоставляют более углубленный анализ, такой как анализ утечек памяти, профилирование работы с потоками и сборкой мусора.

2. **Модульное использование памяти.**

   Чтобы эффективно управлять памятью, важно понимать, как работает распределение памяти в JVM. JVM делит память на несколько областей:

   » **Куча (Heap)** — память, выделенная для динамических объектов (например, созданных с помощью *new*).

   » **Стек (Stack)** — используется для хранения локальных переменных и вызовов функций.

   » **Метаспейс (Metaspace)** — область для хранения метаданных классов.

   » **Память для сборщика мусора** — выделяется под управление сборщиком мусора для организации работы с памятью.

Пример профилирования памяти с помощью *jvisualvm*

1. Запустите приложение с опцией включения мониторинга, например:

```
java -jar MyApplication.jar
```

2. Откройте *jvisualvm*, подключитесь к процессу Java и начните мониторить использование памяти. Вы увидите графики с данными о куче, метаспейсе и активности сборщика мусора.

3. Анализируйте данные для нахождения возможных проблем.

11.2. Оптимизация работы с кучей и стеком

Оптимизация работы с памятью требует внимания к куче и стеку, поскольку именно в этих областях происходит основное управление данными.

https://max.ru/bookofgeek

11.2.1. Оптимизация работы с кучей

- **Управление размером кучи**. Для приложений с интенсивным использованием памяти стоит настроить параметры размера кучи. Например, можно использовать параметры командной строки *-Xms* и *-Xmx* для задания начального и максимального размера кучи.

  Пример:

```
java -Xms512m -Xmx2g -jar MyApplication.jar
```

  Это установит начальный размер кучи в 512 MB и максимальный — в 2 GB. Важно сбалансировать эти значения, чтобы избежать частых сборок мусора или недостатка памяти.

- **Типы сборщиков мусора**. JVM поддерживает несколько различных сборщиков мусора, каждый из которых имеет свои особенности. Например, G1GC (Garbage First) используется для приложений с большими объемами памяти и может быть оптимизирован для минимизации времени пауз.

  Пример настройки сборщика мусора G1:

```
java -XX:+UseG1GC -Xms2g -Xmx2g -jar MyApplication.jar
```

- **Объектная аллокация.** Важно минимизировать создание объектов в куче. Особенно это касается объектов, которые часто создаются и уничтожаются (например, строки). Вместо того чтобы создавать новые объекты в цикле, можно использовать пул объектов или кэшировать результаты.

11.2.2. Оптимизация работы со стеком

- **Ограничение глубины рекурсии**. Каждый вызов функции создает новый фрейм в стеке. Большие или глубоко рекурсивные алгоритмы могут привести к переполнению стека. Чтобы избежать этого, можно использовать итерационные решения или оптимизировать рекурсивные алгоритмы.

  Пример. Вместо рекурсивного поиска факториала, который может привести к переполнению стека:

https://max.ru/bookofgeek

```
public static long factorial(int n) {
    if (n == 0) return 1;
    return n * factorial(n - 1);
}
```

лучше использовать итеративный подход:

```
public static long factorial(int n) {
    long result = 1;
    for (int i = 2; i <= n; i++) {
        result *= i;
    }
    return result;
}
```

- **Минимизация стекового пространства.** Использование маленьких объектов и упрощение стека вызовов помогает избежать проблем с памятью.

11.3. Управление и минимизация утечек памяти

Утечка памяти — это ситуация, когда объекты, которые больше не используются, остаются в памяти, что приводит к ее переполнению и снижению производительности.

Утечки памяти — одна из самых опасных проблем, так как они часто не проявляются сразу, а могут накапливаться со временем, приводя к сбоям и падению приложения.

11.3.1. Причины утечек памяти

- **Неудаленные ссылки на объекты**. Когда объект продолжает ссылаться на себя через другие объекты, даже если он больше не нужен, это приводит к утечке памяти.

https://max.ru/bookofgeek

- **Неосвобожденные ресурсы**. Часто утечки происходят, когда ресурсы (например, сокеты, файлы или соединения с базой данных) не освобождаются должным образом.
- **Использование коллекций**. Например, если вы добавляете объекты в коллекции, но забываете их удалить, это также может привести к утечкам.

11.3.2. Методы для предотвращения утечек памяти:

- **Использование *WeakReference* и *SoftReference*.**

  Эти классы позволяют JVM убирать объекты из памяти, когда они больше не используются, что помогает избежать утечек.

  Пример использования *WeakReference*:

```
import java.lang.ref.WeakReference;

public class WeakReferenceExample {
    public static void main(String[] args) {
        Object object = new Object();
        WeakReference<Object> weakRef = new
WeakReference<>(object);
        object = null;  // Ссылка больше не существует

        // Объект будет уничтожен сборщиком мусора, если нет
сильных ссылок
        System.out.println(weakRef.get());  // Выводит null,
если объект был удален
    }
}
```

- **Автоматическое закрытие ресурсов**.

  В Java 7 был введен механизм **try-with-resources**, который автоматически закрывает ресурсы, такие как потоки и соединения.

  Пример:

https://max.ru/bookofgeek

```
try (BufferedReader reader = new BufferedReader(new FileReader
("file.txt"))) {
    // Чтение файла
} catch (IOException e) {
    e.printStackTrace();
}
// reader автоматически закрывается
```

- **Профилирование утечек памяти**. Используйте инструменты, такие как **VisualVM**, **YourKit** или **JProfiler**, для выявления объектов, которые занимают слишком много памяти. Они позволяют анализировать дампы памяти и отслеживать объекты, которые не могут быть собраны сборщиком мусора.

Выводы

Работа с памятью и управление ресурсами являются основными аспектами оптимизации Java-приложений. Профилирование использования памяти, оптимизация работы с кучей и стеком, а также правильное управление утечками памяти помогут улучшить производительность и надежность ваших приложений. Важно регулярно профилировать приложение, используя доступные инструменты, и своевременно устранять проблемы с памятью, чтобы избежать потери производительности и сбоев.

https://max.ru/bookofgeek

https://max.ru/bookofgeek

Глава 12.

Оптимизация ввода-вывода

https://max.ru/bookofgeek

О чём глава:

- Эффективные методы работы с файлами и сетью.
- Использование неблокирующего ввода-вывода (NIO).
- Примеры оптимизации ввода-вывода.

12.1. Эффективные методы работы с файлами и сетью

Ввод-вывод (I/O) является одной из ключевых областей, где можно добиться значительной оптимизации в приложениях. В Java существует несколько способов работы с файлами и сетевыми соединениями, которые помогают повысить производительность.

12.1.1. Работа с файлами

Для работы с файлами в Java чаще всего используется ***java.io*** и ***java.nio***. Важно выбирать подходящий инструмент в зависимости от ситуации.

- **Buffered I/O**. Один из самых эффективных способов работы с файлами — это использование ***BufferedInputStream*** и ***BufferedOutputStream***.

  Буферизация позволяет уменьшить количество операций с диском, поскольку данные считываются или записываются в блоках, что ускоряет работу с файлами.

https://max.ru/bookofgeek

- Пример использования **BufferedInputStream** и **BufferedOutputStream**:

```
import java.io.*;

public class FileCopyExample {
    public static void main(String[] args) throws IOException {
        try (BufferedInputStream bis = new
BufferedInputStream(new FileInputStream("source.txt"));
             BufferedOutputStream bos = new
BufferedOutputStream(new FileOutputStream("destination.txt"))) {

            byte[] buffer = new byte[1024];
            int bytesRead;
            while ((bytesRead = bis.read(buffer)) != -1) {
                bos.write(buffer, 0, bytesRead);
            }
        }
    }
}
```

В этом примере используется буфер для чтения и записи данных, что значительно ускоряет операцию копирования файла по сравнению с прямым использованием ***FileInputStream*** и ***FileOutputStream***.

- **Memory-Mapped Files**. Если необходимо работать с большими файлами, можно использовать Memory-Mapped I/O с помощью MappedByteBuffer, который позволяет работать с файлами как с массивами в памяти. Это значительно ускоряет операции чтения и записи.

Пример использования **Memory-Mapped File**:

```
import java.io.*;
import java.nio.*;
import java.nio.channels.*;

public class MemoryMappedFileExample {
    public static void main(String[] args) throws IOException {
```


https://max.ru/bookofgeek

```
        try (RandomAccessFile file = new
RandomAccessFile("largefile.txt", "rw");
             FileChannel channel = file.getChannel()) {

            MappedByteBuffer buffer = channel.map(FileChannel.
MapMode.READ_WRITE, 0, channel.size());

            // Изменяем данные в файле
            buffer.put(0, (byte) 65);  // Изменяем первый байт
в файле
        }
    }
}
```

В этом примере создается **Memory-Mapped** файл, который позволяет напрямую манипулировать содержимым файла, как если бы это был обычный массив байт.

12.1.2. Работа с сетью

Для сетевого ввода-вывода Java предоставляет ***java.net***, а также поддерживает работу с неблокирующим I/O через **NIO**. При сетевом взаимодействии важно учитывать производительность, особенно в многозадачных и высоконагруженных приложениях.

- **Сокеты.** Сокеты предоставляют базовый механизм для работы с сетевыми соединениями. Использование **BufferedReader** и **PrintWriter** с сокетами позволяет эффективно управлять потоками данных.

Пример простого клиент-сервера с сокетами:

```
// Сервер
import java.io.*;
import java.net.*;

public class ServerExample {
    public static void main(String[] args) throws IOException {
```


https://max.ru/bookofgeek

```
        ServerSocket serverSocket = new ServerSocket(8080);
        Socket socket = serverSocket.accept();

        BufferedReader reader = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
        String message = reader.readLine();
        System.out.println("Получено сообщение: " + message);

        PrintWriter writer = new PrintWriter(socket.
getOutputStream(), true);
        writer.println("Ответ от сервера");

        socket.close();
        serverSocket.close();
    }
}

// Клиент
import java.io.*;
import java.net.*;

public class ClientExample {
    public static void main(String[] args) throws IOException {
        Socket socket = new Socket("localhost", 8080);

        PrintWriter writer = new PrintWriter(socket.
getOutputStream(), true);
        writer.println("Привет, сервер!");

        BufferedReader reader = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
        System.out.println("Ответ от сервера: " + reader.readLine());

        socket.close();
    }
}
```


https://max.ru/bookofgeek

12.2. Использование неблокирующего ввода-вывода (NIO)

Java NIO (New I/O) предоставляет возможности для работы с неблокирующим вводом-выводом. Это позволяет эффективно обрабатывать множество соединений с низкими затратами на потоки. В Java 7 и более поздних версиях появились новые возможности для асинхронного ввода-вывода.

Основные компоненты NIO:

1. **Channel (Канал)**. Канал используется для работы с данными. Например, FileChannel для работы с файлами и SocketChannel для сетевых соединений.
2. **Buffer (Буфер)**. Буфер используется для хранения данных перед их записью в канал или после их считывания из канала.
3. **Selector (Селектор)**. Селектор позволяет регистрировать каналы для асинхронных операций ввода-вывода. Это позволяет обрабатывать несколько каналов в одном потоке, используя неблокирующий режим.

Пример использования неблокирующего сокета с NIO:

```
import java.io.IOException;
import java.net.InetSocketAddress;
import java.nio.ByteBuffer;
import java.nio.channels.SocketChannel;

public class NonBlockingIOExample {
    public static void main(String[] args) throws IOException {
        // Открываем неблокирующий канал
        SocketChannel socketChannel = SocketChannel.open();
        socketChannel.configureBlocking(false); // Устанавливаем
неблокирующий режим

        // Подключаемся к серверу
```


https://max.ru/bookofgeek

```
        socketChannel.connect(new InetSocketAddress("localhost",
8080));

        // Проверяем, подключен ли канал
        while (!socketChannel.finishConnect()) {
            System.out.println("Ожидание подключения...");
        }

        // Отправляем данные
        String message = "Привет, сервер!";
        ByteBuffer buffer = ByteBuffer.wrap(message.getBytes());
        socketChannel.write(buffer);

        // Закрываем канал
        socketChannel.close();
    }
}
```

В этом примере используется SocketChannel в неблокирующем режиме. Мы пытаемся подключиться к серверу и отправляем сообщение, при этом процесс не блокирует выполнение программы, позволяя работать с другими задачами.

12.3. Примеры оптимизации ввода-вывода

12.3.1. Использование буферизации и кеширования

- Когда вы работаете с большими объемами данных (например, при чтении/записи файлов), использование буферов значительно ускоряет операции.
- BufferedInputStream и BufferedOutputStream или аналогичные механизмы позволяют уменьшить количество системных вызовов, что значительно повышает производительность.

https://max.ru/bookofgeek

12.3.2. Неблокирующий I/O

Использование NIO помогает при работе с большим количеством сетевых соединений, так как он позволяет эффективно обрабатывать данные без необходимости создания нового потока для каждого соединения.

12.3.3. Использование асинхронного ввода-вывода

В Java 7 был введен новый API для асинхронного ввода-вывода (например, AsynchronousFileChannel и AsynchronousSocketChannel), который позволяет более эффективно работать с ресурсами, не блокируя выполнение программы.

Пример использования асинхронного канала для записи в файл:

```
import java.nio.file.*;
import java.nio.channels.*;
import java.nio.charset.*;
import java.nio.*;

public class AsyncFileWriteExample {
    public static void main(String[] args) throws Exception {
        AsynchronousFileChannel channel = AsynchronousFileChannel.
open(Paths.get("output.txt"), StandardOpenOption.WRITE);
        ByteBuffer buffer = ByteBuffer.wrap("Асинхронная запись!".
getBytes(StandardCharsets.UTF_8));

        channel.write(buffer, 0, null, new CompletionHandler
<Integer, Void>() {
            public void completed(Integer result, Void attachment) {
                System.out.println("Запись завершена: " + result +
" байт");
            }

            public void failed(Throwable exc, Void attachment) {
                exc.printStackTrace();
```


https://max.ru/bookofgeek

```
            }
        });

        // Ожидаем завершения операции
        Thread.sleep(1000);
    }
}
```

Выводы

Оптимизация операций ввода-вывода — это один из ключевых факторов, влияющих на производительность современных приложений, особенно тех, которые интенсивно работают с файлами, сетевыми соединениями или базами данных.

Низкая эффективность ввода-вывода может стать узким местом даже в самых хорошо спроектированных системах, замедляя обработку данных, увеличивая время отклика и потребляя лишние ресурсы.

Одним из первых шагов на пути оптимизации является использование буферизации. Буферизация позволяет значительно сократить количество обращений к физическим устройствам ввода-вывода, группируя небольшие операции в более крупные и тем самым снижая накладные расходы. Это особенно актуально при работе с файловыми системами, сетевыми потоками и потоками данных между сервисами.

Неблокирующий ввод-вывод (Non-blocking I/O) становится обязательным элементом в высоконагруженных системах и многопоточных приложениях, поскольку он позволяет эффективно использовать доступные ресурсы, не блокируя потоки выполнения при ожидании завершения операции. Вместо блокировки потока операция продолжается асинхронно, а уведомление о ее завершении приходит через события или обратные вызовы, что позволяет выполнять тысячи операций ввода-вывода параллельно.

https://max.ru/bookofgeek

Асинхронные операции ввода-вывода обеспечивают еще более высокий уровень масштабируемости, позволяя строить реактивные и событийно-ориентированные архитектуры, где время простоя потоков минимально, а производительность линейно растет с увеличением количества клиентов или задач.

Однако оптимизация ввода-вывода требует внимательного подхода: необходимо выбирать правильные инструменты и технологии в зависимости от конкретных условий задачи. При работе с большими файлами эффективнее использовать буферизированные потоки и прямой доступ к файлам (например, через *FileChannel* и *MappedByteBuffer*). В сетевых приложениях стоит обратить внимание на современные фреймворки, поддерживающие неблокирующий ввод-вывод, такие как Netty или использование NIO в Java.

Также важной практикой является профилирование и мониторинг операций ввода-вывода. Только измеряя фактические задержки, пропускную способность и частоту обращений, можно выявить узкие места и принять обоснованные решения о внедрении тех или иных техник оптимизации.

В заключение можно сказать: грамотная оптимизация ввода-вывода — это не разовая задача, а процесс, который требует комплексного подхода, постоянного анализа и адаптации под изменяющиеся условия эксплуатации. Чем внимательнее разработчик относится к организации ввода-вывода, тем более отзывчивым, масштабируемым и эффективным становится приложение.

https://max.ru/bookofgeek

Глава 13.

Безопасность и производительность

https://max.ru/bookofgeek

О чём глава:

- Влияние безопасного программирования на производительность.
- Оптимизация без ущерба для безопасности.
- Советы по написанию безопасного и быстрого кода.

13.1. Влияние безопасного программирования на производительность

При разработке приложений на Java мы часто оказываемся перед дилеммой: как обеспечить высокую производительность, не жертвуя безопасностью.

> **Безопасность** — это не только защита от внешних атак, но и соблюдение внутренних правил и инвариантов, гарантирующих корректную работу приложения. Однако все защитные меры, включая проверки, логирование, шифрование и валидацию, влекут за собой накладные расходы.

В этой главе мы рассмотрим, как безопасное программирование влияет на производительность и как минимизировать эти затраты.

https://max.ru/bookofgeek

Защитное программирование и его цена

Защитное программирование, или **defensive programming** — это практика написания кода, устойчивого к ошибкам, неожиданным входным данным и другим видам некорректного использования.

Она включает:

- Проверку входных параметров на *null* или некорректные значения.
- Защиту от выхода за пределы массива.
- Использование immutable-объектов.
- Валидацию данных.
- Шифрование и маскирование чувствительной информации.
- Логирование событий безопасности.

Каждое из этих действий требует времени CPU и/или памяти. Например, валидация входных параметров добавляет условия в методы, проверки *null* вызывают дополнительные ветвления, а логирование может повлиять на I/O-подсистему.

Пример. Валидация входных данных

```
public class UserService {
    public void updateEmail(User user, String newEmail) {
        if (user == null || newEmail == null || !newEmail.
contains("@")) {
            throw new IllegalArgumentException("Invalid
arguments");
        }
        user.setEmail(newEmail);
    }
}
```


https://max.ru/bookofgeek

Такой код устойчив к некорректным входам, но каждое условие добавляет лишние инструкции, особенно если метод вызывается очень часто (например, в высоконагруженном REST API).

Пример. Логирование

```
logger.debug("Received updateEmail request for user {}", user.
getId());
```

Если уровень логирования DEBUG включен, строка форматируется и записывается в лог-файл. Это не только нагружает CPU, но и может блокировать I/O, особенно если используется синхронная запись.

13.2. Оптимизация без ущерба для безопасности

Правильный подход к балансу между безопасностью и производительностью — не исключать защитные меры, а проектировать систему так, чтобы они были эффективны.

13.2.1. Уровни проверки и инварианты

Один из методов — разделение проверки на:

- **Пограничную** — выполняется на границе внешнего интерфейса (API, UI, база данных).
- **Внутреннюю** — на уровне бизнес-логики, проверяет корректность переходов между состояниями.

Пограничную проверку можно делать тщательно и с большими накладными расходами (ведь это внешнее воздействие), а внутреннюю — упрощать, полагаясь на корректность предыдущих уровней.

13.2.2. Использование *assert*

https://max.ru/bookofgeek

В местах, где важна проверка во время разработки, но не требуется в production, можно использовать *assert*:

```
public void setAge(int age) {
    assert age > 0 && age < 120 : "Invalid age: " + age;
    this.age = age;
}
```

При отключенной опции *-ea* (enable assertions) эти проверки не исполняются, не влияя на производительность в релизе.

13.2.3. Изоляция критичных проверок

Некоторые проверки могут быть вынесены в обертки:

```
public void processRequest(User user) {
    SecureContext.run(() -> internalProcess(user));
}
```

Где *SecureContext.run* отвечает за аутентификацию, журналирование, контроль прав и т.д. Такой подход позволяет легко включать/отключать защиту и профилировать ее отдельно.

13.2.4. Кеширование и повторное использование

Если валидация или дешифровка выполняется над одними и теми же данными многократно, можно использовать кеш:

```
private Map<String, Boolean> emailValidationCache = new
ConcurrentHashMap<>();

public boolean isValidEmail(String email) {
    return emailValidationCache.computeIfAbsent(email, this::
validateEmail);
}
```


https://max.ru/bookofgeek

13.2.5. Адаптивная безопасность

Пример — адаптивное логирование. В обычных условиях логирование ведется минимально. При возникновении подозрительной активности включается расширенное логирование только на конкретных участках:

```
if (anomalyDetector.isActive()) {
    logger.warn("Suspicious activity detected for user {}",
user.getId());
}
```

13.2.6. Безопасные, но быстрые алгоритмы

Некоторые криптографические алгоритмы сильно влияют на производительность. Например, использование RSA вместо ECC может быть неоправданно тяжелым.

Java предлагает более производительные реализации, если использовать новые API (*java.security, javax.crypto*) и нативные библиотеки.

13.2.7. Инструментальный анализ влияния безопасности

JMH, профилировщики, трассировщики и мониторинг помогают оценить, сколько ресурсов тратится на защиту. Это позволяет принимать решения, основанные на данных, а не интуиции.

Пример сравнения метода с защитой и без:

```
@Benchmark
public void withValidation(MyState state) {
    if (state.value == null) throw new IllegalStateException();
    process(state.value);
}

@Benchmark
public void withoutValidation(MyState state) {
    process(state.value);
}
```


https://max.ru/bookofgeek

Если *state.value* гарантированно не *null*, второй вариант может быть быстрее в 10–20%, особенно на горячих путях.

Таблица 13.1. Типичные компромиссы между безопасностью и скоростью

Безопасная мера	**Потенциальные накладные расходы**	**Оптимизация**
Валидация параметров	Время на сравнение и ветвление	Делать на границе API
Шифрование	CPU и задержки	Использовать аппаратное ускорение, современные алгоритмы
Логирование	Задержки на I/O, особенно sync	Асинхронное логирование, уровень TRACE отключен
Иммутабельность	Дополнительное копирование	Использовать builder-подход
Чрезмерное использование *synchronized*	Блокировка потоков, снижение параллелизма	Применять только в критических секциях

13.3. Советы по написанию безопасного и быстрого кода

1. **Всегда проверяйте входные данные, но делайте это эффективно**. Применяйте схемы валидации (например, Bean Validation), где это возможно.
2. **Не проверяйте то, что уже проверено**. Избегайте дублирующих проверок между слоями.
3. **Избегайте лишней синхронизации**. Используйте lock-free алгоритмы или коллекции из *java.util.concurrent.*

https://max.ru/bookofgeek

4. **Используйте современные сборщики мусора и профилируйте утечки**. Безопасность не должна становиться причиной OOM.
5. **Соблюдайте принцип наименьших привилегий,** но не мешайте производительности избыточными проверками прав.
6. **Минимизируйте зависимости от тяжелых библиотек безопасности**, если можно использовать встроенные средства Java.

Выводы

Безопасность и производительность — это не враги, а два аспекта одного качественного кода. Защитные меры обязательно должны присутствовать, особенно в публичных API, при работе с базами данных, пользовательским вводом, файловой системой и сетью. Однако они не должны становиться "узким горлышком" и причиной деградации скорости.

Разумная архитектура, многослойная модель безопасности, профилирование и грамотное применение современных средств Java позволяют достичь баланса: чтобы приложение оставалось и безопасным, и быстрым.

Безопасный код можно и нужно оптимизировать — и это одно из самых ценных умений инженера Java на современном этапе развития языка и платформы.

https://max.ru/bookofgeek

Глава 14.

Декомпилирование и анализ байт-кода

https://max.ru/bookofgeek

О чём глава:

- Основы декомпиляции Java-кода.
- Инструменты для декомпиляции.
- Анализ байт-кода для оптимизации.

14.1. Основы декомпиляции Java кода

Декомпиляция в мире Java — это процесс преобразования байт-кода обратно в форму, близкую к исходному коду, который может быть воспринят и проанализирован человеком.

Хотя результат декомпиляции редко бывает абсолютно идентичен оригинальному исходнику (например, комментарии, форматирование кода, а иногда и имена локальных переменных безвозвратно теряются на этапе компиляции), восстановленный код позволяет получить ценную информацию о структуре программы, ее логике и принципах работы.

Когда Java-программа компилируется, исходные .java-файлы преобразуются в .class-файлы — компактные бинарные представления программы в формате байт-кода.

https://max.ru/bookofgeek

Этот байт-код предназначен для исполнения виртуальной машиной Java (JVM) и сохраняет основную структуру программы: классы, методы, поля, сигнатуры методов, а также инструкции, которые JVM будет выполнять. Благодаря своей независимости от аппаратной архитектуры байт-код позволяет одной и той же Java-программе работать на множестве различных устройств и операционных систем.

Однако иногда возникает необходимость заглянуть "под капот" скомпилированной программы — например, чтобы понять поведение сторонней библиотеки без доступа к ее исходным текстам, найти причину ошибки в продакшн-окружении, где исходники недоступны, или проанализировать безопасность компонента, поступившего из ненадежного источника.

Процесс декомпиляции позволяет:

- **Исследовать чужие библиотеки и приложения.** Если исходный код утерян или изначально был недоступен, декомпиляция помогает восстановить представление о работе программы.
- **Анализировать уязвимости.** Без исходников можно изучить потенциальные проблемы безопасности в сторонних компонентах, например в старых версиях библиотек, где официальные патчи отсутствуют.
- **Проводить реверс-инжиниринг.** Иногда необходимо понять, как устроены сложные алгоритмы или механизмы лицензирования, чтобы интегрировать совместимую функциональность, соблюдать требования совместимости или построить безопасную альтернативу.
- **Восстанавливать утраченный код.** При отсутствии резервных копий проекта можно декомпилировать скомпилированные .class-файлы для частичного восстановления исходного кода.

Однако следует помнить о важных юридических и этических аспектах. Декомпиляция чужого кода может нарушать лицензионные соглашения, особенно если в контракте или лицензии явно запрещено проведение обратного инжиниринга. Кроме того, авторские права защищают не только исходный код, но и результат компиляции. Поэтому перед использованием декомпиляции важно убедиться, что действия соответствуют действующему законодательству, внутренним правилам организации и лицензионным соглашениям на используемые программные продукты.

https://max.ru/bookofgeek

С технической точки зрения для декомпиляции Java-кода существует множество инструментов, таких как **JD-GUI**, **CFR**, **Procyon**, **FernFlower** и встроенные декомпиляторы в IDE (например, IntelliJ IDEA). Они способны преобразовывать .class-файлы в *.java*-файлы, восстанавливая при этом структуру классов, методов, интерфейсов и полей с достаточно высокой степенью достоверности.

Качество результата декомпиляции зависит от многих факторов:

- Уровня оптимизаций компилятора.
- Наличия или отсутствия отладочной информации в *.class*-файлах.
- Применения обфускации — специальной техники запутывания кода для затруднения анализа.
- Используемой версии Java и особенностей байт-кода.

Таким образом, декомпиляция — это мощный инструмент анализа и восстановления информации о Java-программах, который требует аккуратного и ответственного применения.

14.2. Инструменты для декомпиляции

Для декомпиляции Java-байт-кода существует несколько популярных инструментов, которые позволяют преобразовывать *.class*-файлы обратно в исходный код Java.

14.2.1. JD-GUI (Java Decompiler GUI)

JD-GUI — это один из самых популярных графических декомпиляторов для Java. Он позволяет загружать *.class*-файлы и просматривать их содержимое в виде исходного кода. JD-GUI поддерживает многие современные версии Java и позволяет исследовать не только одиночные классы, но и целые архивы *.jar*.

Сайт: *https://java-decompiler.github.io/*

https://max.ru/bookofgeek

Пример использования:

- Откройте JD-GUI и загрузите *.class*- или *.jar*-файл.
- Декомпилированный код появится в правой панели, и вы сможете просматривать исходный код.

14.2.2. Procyon

Procyon — это еще один популярный декомпилятор Java, который поддерживает новые версии Java, включая лямбда-выражения и аннотации. Он не только позволяет декомпилировать *.class*-файлы, но и может работать с библиотеками, упакованными в *.jar*-файлы.

Сайт: *https://github.com/mstrobel/procyon*

Пример использования:

- Установите Procyon и запустите его с командной строки:

```
java -jar procyon-decompiler-0.5.30.jar myclass.class
```

14.2.3. CFR (Class File Reader)

CFR — это другой популярный декомпилятор для Java, который активно поддерживает последние версии Java. Он является более точным в декомпиляции сложных конструкций, таких как лямбда-выражения и методы с обобщениями.

Сайт: *https://www.benf.org/other/cfr/*

Пример использования:

```
java -jar cfr.jar myfile.class
```

14.2.4. FernFlower

FernFlower — это современный декомпилятор, который интегрирован в IDE IntelliJ IDEA. Он является высококачественным инструментом для декомпиляции Java и активно используется разработчиками для анализа скомпилированного кода.

https://max.ru/bookofgeek

Пример использования:

- Откройте *.class*-файл в IntelliJ IDEA, и декомпиляция произойдет автоматически.

14.3. Анализ байт-кода для оптимизации

После того как мы декомпилировали код, возникает необходимость анализировать его для дальнейшей оптимизации. Анализ байт-кода позволяет понять, какие участки кода могут быть улучшены с точки зрения производительности и использования ресурсов.

14.3.1. Инструменты для анализа байт-кода

- **JVM Bytecode Viewer (JVMBCV)** — это инструмент, который позволяет просматривать и анализировать байт-код программы. Он дает детализированное представление о структуре байт-кода, что может быть полезно для оценки работы виртуальной машины.
- **ASM** — это библиотека для анализа и манипуляции байт-кодом на уровне JVM. Она позволяет программно изменять байт-код, что может быть полезно для создания более оптимизированных версий классов.
- **Javap** — это инструмент, который поставляется вместе с JDK и позволяет разобрать *.class*-файл и вывести байт-код в удобочитаемом виде. Javap помогает оценить, какие методы были скомпилированы, какие использованы оптимизации и насколько эффективен код.

Пример использования *javap*:

```
javap -c MyClass.class
```

Вывод покажет байт-код для каждого метода в классе.

14.3.2. Оптимизация на уровне байт-кода

После анализа байт-кода можно внести некоторые изменения для улучшения производительности программы.

https://max.ru/bookofgeek

- **Удаление мертвого кода (Dead Code Elimination)**. Это процесс удаления частей программы, которые не влияют на ее поведение, например условных операторов, которые всегда ложны.
- **Инлайнинг методов**. Замена вызовов маленьких методов на их код прямо в месте вызова. Это помогает уменьшить накладные расходы на вызовы методов.
- **Параллельные и многопоточные оптимизации**. Некоторые оптимизации могут быть достигнуты путем изменения байт-кода для поддержки многозадачности или распараллеливания кода.
- **Удаление ненужных синхронизаций**. Синхронизация в многозадачных приложениях может добавить лишние накладные расходы. Удаление ненужных блокировок в байт-коде помогает уменьшить время выполнения.

14.3.3. Пример: анализ байт-кода с использованием ASM

Использование **ASM** для анализа байт-кода позволяет делать глубокий анализ и манипуляцию байт-кодом. В следующем примере создадим программу, которая добавляет новую инструкцию в метод:

```
import org.objectweb.asm.*;

public class BytecodeAnalyzer {
    public static void main(String[] args) throws Exception {
        // Открытие класса
        ClassReader reader = new ClassReader("com/example/
MyClass");
        ClassWriter writer = new ClassWriter(0);

        // Обработчик байт-кода
        ClassVisitor visitor = new MyClassVisitor(writer);
        reader.accept(visitor, 0);

        // Получаем модифицированный байт-код
        byte[] modifiedBytecode = writer.toByteArray();
    }
```


https://max.ru/bookofgeek

```
    // Класс для обработки и модификации байт-кода
    static class MyClassVisitor extends ClassVisitor {
        public MyClassVisitor(ClassVisitor cv) {
            super(Opcodes.ASM7, cv);
        }

        @Override
        public MethodVisitor visitMethod(int access, String name,
String descriptor, String signature, String[] exceptions) {
            MethodVisitor mv = super.visitMethod(access, name,
descriptor, signature, exceptions);
            // Изменение байт-кода метода
            return mv;
        }
    }
}
```

В этом примере используется библиотека **ASM**, чтобы создать инструмент для анализа и модификации байт-кода.

Выводы

Декомпиляция и **анализ байт-кода** занимают важное место в арсенале разработчика, стремящегося к глубокому пониманию работы Java-программ. Они позволяют увидеть внутренние процессы, скрытые за уровнем исходного кода, и помогают лучше понять, как JVM интерпретирует и выполняет инструкции.

Хотя процесс декомпиляции не способен восстановить оригинальный исходный код в точности (например, потеряны могут быть комментарии, имена локальных переменных и стиль форматирования), он все же предоставляет достаточно информации о структуре классов, методах, логике исполнения и взаимосвязях между компонентами.

https://max.ru/bookofgeek

Декомпиляция особенно полезна:

- При исследовании сторонних библиотек, исходный код которых недоступен.
- Для анализа скомпилированного кода на наличие ошибок или уязвимостей.
- При восстановлении кода в случае потери исходников.
- Для изучения оптимизаций, выполняемых компилятором и JIT-компилятором.
- В процессе аудита безопасности приложений.

Среди инструментов для декомпиляции наиболее популярными являются:

- **JD-GUI** — удобный графический интерфейс для просмотра декомпилированного Java-кода.
- **Procyon** — мощный декомпилятор, хорошо справляющийся с новыми языковыми конструкциями Java.
- **CFR** — современный декомпилятор, способный качественно обрабатывать сложный байт-код, включая лямбда-выражения и новые синтаксические элементы.

В дополнение к инструментам для декомпиляции существуют специализированные библиотеки для анализа и трансформации байт-кода:

- **ASM** — легкая и быстрая библиотека для низкоуровневого анализа, генерации и модификации Java-байт-кода.
- **ByteBuddy** — более высокоуровневая библиотека для динамического создания и изменения классов во время выполнения программы.
- **Javassist** — инструмент для редактирования байт-кода с удобным API, позволяющим работать почти на уровне исходного кода.

Работа с байт-кодом открывает возможности для глубокого анализа производительности приложений, реализации динамических прокси, разработки сложных механизмов сериализации/десериализации, оптимизации за-

https://max.ru/bookofgeek

грузки классов, создания AOP-решений и многих других задач, требующих детального контроля над структурой программы.

Важно помнить, что анализ и модификация байт-кода должны использоваться ответственно. Особенно в случаях, когда речь идет о сторонних приложениях, поскольку декомпиляция может затрагивать вопросы лицензирования и интеллектуальной собственности.

Таким образом, навыки работы с байт-кодом и знание инструментов декомпиляции позволяют разработчикам и инженерам по производительности глубже понимать поведение приложений, находить узкие места, обеспечивать безопасность кода и разрабатывать более эффективные и надежные решения.

https://max.ru/bookofgeek

Глава 15.

Оптимизация запросов к базе данных

https://max.ru/bookofgeek

О чём глава:

- Эффективное использование JDBC.
- Оптимизация запросов ORM (Hibernate, JPA).
- Профилирование и оптимизация SQL запросов.

Работа с **базой данных** является неотъемлемой частью большинства Java-приложений, и ее производительность напрямую влияет на общую эффективность системы. Неоптимизированные запросы, неправильное использование JDBC, а также недостатки в настройках ORM могут стать критическими узкими местами, существенно снижая скорость работы приложения.

Важно помнить, что даже если сама база данных настроена правильно, проблемы могут скрываться в самом приложении, например в плохо сформулированных SQL-запросах или в ненадежной работе с соединениями.

В этой главе мы сосредоточимся на том, как улучшить производительность приложений, взаимодействующих с базой данных. Мы рассмотрим основные подходы к оптимизации запросов, как эффективно работать с JDBC и популярными ORM-фреймворками, такими как Hibernate и JPA. Также будет уделено внимание методам профилирования SQL-запросов и инструментам для мониторинга, что поможет выявлять и устранять узкие места, повышая производительность и сокращая время отклика базы данных.

https://max.ru/bookofgeek

Благодаря правильной настройке и анализу взаимодействия с базой данных можно добиться значительных улучшений в производительности Java-приложений, обеспечив их стабильную и быструю работу в условиях реальных нагрузок.

15.1. Эффективное использование JDBC

JDBC (Java Database Connectivity) — это стандартный API для работы с реляционными базами данных в Java. С его помощью можно выполнять SQL-запросы и обновления данных.

Однако часто разработчики сталкиваются с проблемами производительности, если не используют лучшие практики при работе с JDBC.

15.1.1. Использование Connection Pooling (пула соединений)

Одним из самых простых и эффективных способов оптимизации работы с базой данных является использование **Connection Pooling**. Каждый раз при открытии нового соединения с базой данных могут возникать значительные накладные расходы, такие как установление соединения, аутентификация и настройка. Чтобы избежать этих затрат, часто используются пулы соединений.

Популярные библиотеки для пуллинга соединений в Java:

- **HikariCP** — современный и высокоэффективный пул соединений.
- **Apache DBCP** — библиотека для работы с пулом соединений.
- **C3P0** — еще одна популярная библиотека для пуллинга.

Пример использования **HikariCP**:

```
import com.zaxxer.hikari.HikariConfig;
import com.zaxxer.hikari.HikariDataSource;
```


https://max.ru/bookofgeek

```
import java.sql.Connection;
import java.sql.SQLException;

public class DatabaseUtil {
    private static HikariDataSource dataSource;

    static {
        HikariConfig config = new HikariConfig();
        config.setJdbcUrl("jdbc:mysql://localhost:3306/mydb");
        config.setUsername("user");
        config.setPassword("password");
        config.setMaximumPoolSize(10); // Максимальное количество
соединений в пуле
        dataSource = new HikariDataSource(config);
    }

    public static Connection getConnection() throws SQLException {
        return dataSource.getConnection();
    }

    public static void close(Connection connection) {
        if (connection != null) {
            try {
                connection.close();
            } catch (SQLException e) {
                e.printStackTrace();
            }
        }
    }
}
```

В этом примере **HikariCP** создает пул соединений и позволяет извлекать соединения по мере необходимости.

Это существенно ускоряет обработку запросов, так как соединения не нужно устанавливать заново для каждого запроса.

https://max.ru/bookofgeek

15.1.2. Использование PreparedStatement

PreparedStatement помогает предотвратить SQL инъекции и позволяет более эффективно выполнять повторяющиеся запросы. При использовании **PreparedStatement** запрос компилируется заранее, а его параметры передаются позднее. Это ускоряет выполнение, так как база данных может переиспользовать тот же план выполнения запроса.

Пример использования:

```
String sql = "SELECT * FROM users WHERE username = ?";
try (Connection conn = DatabaseUtil.getConnection();
     PreparedStatement stmt = conn.prepareStatement(sql)) {
     stmt.setString(1, "johndoe");
     ResultSet rs = stmt.executeQuery();
     while (rs.next()) {
        System.out.println(rs.getString("username"));
     }
} catch (SQLException e) {
    e.printStackTrace();
}
```

15.1.3. Использование Batch Processing для массовых операций

При вставке или обновлении большого объема данных можно значительно ускорить выполнение операций, используя **batch processing**. Это позволяет отправить несколько запросов на сервер базы данных за один раз, уменьшая накладные расходы на установление соединений и передачу данных.

Пример использования:

```
String sql = "INSERT INTO users (username, password) VALUES (?, ?)";
try (Connection conn = DatabaseUtil.getConnection();
     PreparedStatement stmt = conn.prepareStatement(sql)) {
     for (int i = 1; i <= 1000; i++) {
        stmt.setString(1, "user" + i);
        stmt.setString(2, "password" + i);
        stmt.addBatch();
```


https://max.ru/bookofgeek

```
            if (i % 100 == 0) {  // Каждый 100-й запрос отправляется в базу
                stmt.executeBatch();
            }
        }
        stmt.executeBatch();  // Отправка оставшихся запросов
    } catch (SQLException e) {
        e.printStackTrace();
    }
```

Этот подход значительно снижает нагрузку на базу данных при массовых вставках.

15.2. Оптимизация запросов ORM (Hibernate, JPA)

Использование **ORM (Object-Relational Mapping)**, например **Hibernate** или **JPA (Java Persistence API)**, значительно упрощает работу с базой данных, но также может привести к проблемам с производительностью, если не следить за тем, как формируются SQL-запросы.

15.2.1. Использование кеширования

Одним из методов оптимизации запросов при использовании Hibernate и JPA является кеширование. Кеширование позволяет избежать многократных обращений к базе данных для получения одинаковых данных.

Hibernate поддерживает два типа кеша:

1. **Первичный кеш** (для каждого сеанса).
2. **Вторичный кеш** (для всего приложения).

Пример включения второго уровня кеша в Hibernate:

```
<hibernate-configuration>
    <session-factory>
```


https://max.ru/bookofgeek

```
        <property name="hibernate.cache.use_second_level_cache"
>true</property>
        <property name="hibernate.cache.region.factory_
class">org.hibernate.cache.ehcache.EhCacheRegionFactory</
property>
    </session-factory>
</hibernate-configuration>
```

15.2.2. Ленивая загрузка (Lazy Loading)

Lazy Loading позволяет загружать связанные объекты только по мере необходимости, а не сразу при запросе основного объекта. Это может существенно уменьшить количество запросов к базе данных, если связанная информация не требуется для выполнения операции.

Пример использования ленивой загрузки в Hibernate:

```
@Entity
public class User {
    @OneToMany(fetch = FetchType.LAZY)
    private List<Order> orders;
}
```

При таком подходе связанные **orders** будут загружены только при обращении к ним, а не сразу при загрузке **User**.

15.2.3. Использование FetchType.LAZY и FetchType.EAGER

- **FetchType.LAZY** — данные загружаются по мере необходимости.
- **FetchType.EAGER** — все данные загружаются сразу.

Использование **EAGER** для всех связей может привести к большому количеству ненужных запросов и проблемам с производительностью, поэтому важно тщательно выбирать стратегии загрузки.

https://max.ru/bookofgeek

15.3. Профилирование и оптимизация SQL-запросов

15.3.1. Использование EXPLAIN PLAN

Программисты часто не знают, насколько эффективен тот или иной SQL-запрос. Один из способов узнать это — использовать **EXPLAIN PLAN**.

Эта команда в SQL показывает, как оптимизатор базы данных будет выполнять запрос, и помогает выявить проблемные участки.

Пример запроса:

```
EXPLAIN SELECT * FROM users WHERE username = 'johndoe';
```

Результат покажет, как именно база данных будет выполнять запрос, включая использование индексов и возможные полные сканирования таблиц.

15.3.2. Индексация

Необходимо внимательно подходить к выбору колонок, для которых следует создавать индексы. Индексы значительно ускоряют операции поиска, но могут замедлять операции вставки и обновления, так как нужно обновлять индекс при изменении данных.

Пример создания индекса:

```
CREATE INDEX idx_username ON users(username);
```

15.3.3. Профилирование с использованием инструментов

Для профилирования SQL-запросов можно использовать инструменты, такие как:

- **Hibernate Profiler** — для анализа запросов, генерируемых Hibernate.

https://max.ru/bookofgeek

- **JProfiler** — для глубокого анализа работы Java-приложений, включая работу с базой данных.
- **VisualVM** — инструмент для мониторинга и профилирования приложений Java.

15.3.4. Разделение запросов

Вместо выполнения одного сложного запроса можно разбить его на несколько более простых запросов, что может улучшить производительность, особенно если запросы выполняются параллельно.

Пример:

```
SELECT * FROM users WHERE id = 1;
SELECT * FROM orders WHERE user_id = 1;
```

Выводы

Оптимизация запросов к базе данных является одним из краеугольных камней общей стратегии повышения производительности Java-приложений. Даже при высокоэффективном коде приложения недостаточная оптимизация работы с базой данных может стать критическим узким местом, способным резко замедлить отклик системы и увеличить нагрузку на инфраструктуру.

Ключевыми техниками в этой области являются:

- **Пуллинг соединений** — использование пулов соединений позволяет избежать дорогостоящей операции создания новых соединений к базе данных для каждого запроса. Пул соединений, таких как HikariCP или Apache DBCP, обеспечивает быстрое переиспользование уже существующих соединений, значительно снижая среднее время обработки запроса.
- **PreparedStatement** — подготовленные выражения позволяют не только ускорить выполнение повторяющихся SQL-запросов за счет их предкомпиляции, но и повышают безопасность системы, предотвращая атаки типа SQL Injection.

https://max.ru/bookofgeek

- **Кеширование** — грамотное кеширование результатов часто запрашиваемых данных (с использованием, например, Redis, Ehcache или встроенных возможностей ORM-фреймворков) может существенно снизить нагрузку на базу данных и ускорить время ответа для пользователя.
- **Профилирование SQL-запросов** — регулярный анализ выполнения SQL-запросов с помощью инструментов вроде Hibernate Profiler, pg_stat_statements (для PostgreSQL) или EXPLAIN PLAN помогает выявлять неэффективные запросы, не оптимальные индексы и проблемы с выполнением транзакций.

Важно не полагаться слепо на удобные абстракции, такие как ORM-фреймворки (Hibernate, JPA), поскольку они могут скрывать под собой неочевидные проблемы с производительностью. Вместо этого разработчикам необходимо обладать базовыми знаниями SQL, понимать, как формируются запросы, какие индексы используются базой данных и как устроена внутренняя оптимизация выполнения запросов.

Работа с базой данных должна строиться на принципах осознанного проектирования: правильное моделирование схемы БД, минимизация количества обращений, тщательная настройка параметров соединений и использование только тех решений, которые действительно необходимы.

Таким образом, эффективная работа с базой данных — это результат комплексного подхода: грамотной архитектуры, правильной настройки инфраструктуры, регулярного мониторинга и оптимизации кода запросов. Только сочетание этих факторов позволяет создать по-настоящему производительные и масштабируемые Java-приложения, способные справляться с высокими нагрузками и обеспечивать стабильную работу даже в условиях быстрого роста данных и пользователей.

https://max.ru/bookofgeek

Глава 16.

Тестирование и контроль качества кода

https://max.ru/bookofgeek

О чём глава:

- Введение в тестирование Java-приложений.
- Юнит-тестирование и инструменты.
- Интеграционное тестирование.
- Нагрузочное тестирование.
- Контроль качества кода (Code Quality).

Тестирование и **контроль качества кода** — это неотъемлемые части процесса разработки надежного и эффективного программного обеспечения. В контексте Java-приложений тестирование помогает убедиться, что все компоненты системы работают корректно, соответствуют требованиям заказчика и устойчивы к ошибкам. Этот процесс охватывает разные уровни проверки — от тестирования отдельных методов и классов до интеграционных сценариев и нагрузочных испытаний.

В современном разработке тестирование уже не рассматривается как дополнительная активность — оно стало обязательной практикой, встроенной в жизненный цикл проекта. Правильная организация тестов позволяет не только обнаруживать баги на ранних этапах, но и служит гарантией качества при рефакторинге и масштабировании приложения.

Существует множество типов тестирования: модульное, интеграционное, функциональное, нагрузочное и другие. Каждый из них решает свои задачи и требует применения подходящих инструментов. Для экосистемы Java раз-

https://max.ru/bookofgeek

работано большое количество мощных фреймворков и библиотек для тестирования, таких как JUnit, TestNG, Mockito, AssertJ, а также платформы для автоматизации тестирования и интеграции с CI/CD.

Кроме самих тестов, важным аспектом является контроль качества кода: статический анализ, покрытие тестами, соблюдение код-стандартов и обеспечение читаемости и поддержки исходного кода в долгосрочной перспективе.

В этой главе мы подробно рассмотрим:

- основные принципы и подходы к тестированию Java-приложений;
- популярные инструменты, библиотеки и практики для написания и автоматизации тестов;
- методы повышения качества кода через тестирование и статический анализ.

Цель этой главы — дать практические рекомендации и сформировать понимание того, как тестирование помогает строить надежные, масштабируемые и качественные Java-системы.

16.1. Введение в тестирование Java-приложений

Тестирование Java-приложений — это процесс, который включает в себя проверку работы кода, выявление ошибок и недостатков, а также оценку производительности системы. Важной частью этого процесса является использование различных видов тестов, таких как **юнит-тесты**, **интеграционные тесты** и **нагрузочные тесты**.

Основные цели тестирования:

- **Проверка функциональности** — удостовериться, что приложение работает как ожидается.
- **Выявление ошибок** — находить баги и другие проблемы, прежде чем приложение попадет в эксплуатацию.

https://max.ru/bookofgeek

- **Улучшение производительности** — проверка, как приложение работает под нагрузкой.

Инструменты для тестирования Java-приложений:

- **JUnit** — популярный фреймворк для юнит-тестирования.
- **TestNG** — альтернатива JUnit с дополнительными возможностями.
- **Mockito** — фреймворк для создания мок-объектов в тестах.
- **Cucumber** — инструмент для поведения-тестирования с использованием Gherkin-формата.
- **Arquillian** — фреймворк для интеграционного тестирования в контейнерах.

16.2. Юнит-тестирование и инструменты

Юнит-тестирование — это метод тестирования, при котором проверяются отдельные компоненты системы (например, методы или классы). Основная цель — убедиться, что каждый отдельный элемент работает корректно.

JUnit является основным инструментом для юнит-тестирования в Java. Он предоставляет аннотации и средства для написания тестов, ассертов и организации тестовых случаев.

Пример юнит-теста с использованием JUnit:

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class CalculatorTest {
    @Test
    public void testAddition() {
```


https://max.ru/bookofgeek

```
        Calculator calculator = new Calculator();
        int result = calculator.add(2, 3);
        assertEquals(5, result, "2 + 3 должно быть равно 5");
    }

    @Test
    public void testSubtraction() {
        Calculator calculator = new Calculator();
        int result = calculator.subtract(5, 3);
        assertEquals(2, result, "5 - 3 должно быть равно 2");
    }
}
```

Этот код тестирует две операции: сложение и вычитание. Мы создаем объект калькулятора, вызываем метод и проверяем, что результат соответствует ожидаемому.

Mockito — это фреймворк для создания мок-объектов, который позволяет заменять зависимости в тестируемых объектах, тем самым изолируя тесты от внешних факторов.

Пример с использованием **Mockito**:

```
import static org.mockito.Mockito.*;

public class UserServiceTest {
    @Test
    public void testGetUser() {
        UserDAO mockUserDAO = mock(UserDAO.class);
        when(mockUserDAO.getUserById(1)).thenReturn(new User
(1, "John Doe"));

        UserService userService = new
UserService(mockUserDAO);
        User user = userService.getUser(1);
```


https://max.ru/bookofgeek

```
        assertEquals("John Doe", user.getName());
    }
}
```

В этом примере мы заменяем реальный объект **UserDAO** на его мок-версию и настраиваем ее так, чтобы она возвращала тестового пользователя. Это позволяет протестировать метод **getUser()** без взаимодействия с реальной базой данных.

16.3. Интеграционное тестирование

Интеграционное тестирование фокусируется на тестировании взаимодействий между различными компонентами системы. Оно проверяет, правильно ли работает интеграция между модулями, а также проверяет корректность работы системы в целом.

Интеграционные тесты могут включать:

- Взаимодействие с базой данных.
- Взаимодействие с внешними сервисами.
- Обработку ошибок и исключений.
- Тестирование API.

Пример интеграционного теста с использованием **Spring Boot** и **JUnit**:

```
import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.beans.factory.annotation.Autowired;

@SpringBootTest
public class UserServiceIntegrationTest {

    @Autowired
```


https://max.ru/bookofgeek

```
    private UserService userService;

    @Test
    public void testGetUserFromDatabase() {
        User user = userService.getUser(1);
        assertNotNull(user);
        assertEquals("John Doe", user.getName());
    }
}
```

Здесь мы тестируем работу сервиса, который взаимодействует с реальной базой данных. Используя аннотацию **@SpringBootTest**, мы интегрируем Spring-контекст и обеспечиваем полноту тестирования.

16.4. Нагрузочное тестирование

Нагрузочное тестирование — это процесс проверки того, как приложение будет вести себя при высоких нагрузках. Цель — проверить устойчивость приложения, определить его пределы и выявить потенциальные узкие места.

Для нагрузочного тестирования Java-приложений можно использовать различные инструменты:

- **JMeter** — популярный инструмент для тестирования производительности, который позволяет симулировать различные сценарии нагрузки на приложение.
- **Gatling** — инструмент для нагрузочного тестирования с акцентом на высокую производительность и гибкость.
- **Apache Bench** — инструмент для тестирования HTTP-сервисов.

Пример использования JMeter:

https://max.ru/bookofgeek

1. Запускаем JMeter и создаем тест-план.
2. Добавляем Thread Group, которая симулирует определенное количество пользователей.
3. Добавляем HTTP Request, чтобы отправить запрос на сервер.
4. Анализируем результаты, смотря на время отклика, количество успешных запросов и другие параметры.

16.5. Контроль качества кода (Code Quality)

Контроль качества кода включает в себя не только тестирование, но и использование инструментов и методик, которые помогают поддерживать и улучшать кодовую базу.

Инструменты для контроля качества кода

- **SonarQube** — инструмент для анализа качества кода, который обнаруживает баги, уязвимости и нарушения стилей кода.
- **Checkstyle** — инструмент для проверки соответствия кода определенным стандартам кодирования.
- **PMD** — анализатор кода, который обнаруживает потенциально опасные или неэффективные конструкции.
- **FindBugs** — инструмент для статического анализа кода, который помогает находить ошибки и потенциальные проблемы.

Пример настройки SonarQube для проекта

1. Устанавливаем и запускаем **SonarQube** на сервере.
2. Настраиваем **SonarScanner**, чтобы интегрировать анализ с нашим проектом.

https://max.ru/bookofgeek

3. Запускаем анализ с помощью SonarScanner:

```
sonar-scanner -Dsonar.projectKey=my-java-project -Dsonar.sources=src
```

SonarQube анализирует исходный код и предоставляет отчет с подробным описанием потенциальных проблем.

Применение практик рефакторинга

Рефакторинг — это процесс улучшения структуры кода без изменения его функциональности.

Он может включать:

- Удаление дублирования кода.
- Оптимизацию алгоритмов.
- Улучшение читаемости кода.
- Применение паттернов проектирования.

Выводы

Тестирование и **контроль качества кода** — это важнейшие аспекты разработки Java-приложений, обеспечивающие надежность, производительность и безопасность системы. Важно не только проводить юнит-тестирование, но и интеграционные и нагрузочные тесты, чтобы убедиться, что система работает корректно и устойчиво при различных условиях.

Использование инструментов для контроля качества кода помогает поддерживать высокие стандарты и обнаруживать проблемы до того, как они попадут в продуктивную среду.

https://max.ru/bookofgeek

https://max.ru/bookofgeek

Ещё больше свежей и актуальной литературы по Java
ты найдёшь в наших каналах:
https://max.ru/bookofgeek
https://t.me/bookofgeek

Глава 17.

Виды тестов при нагрузочном тестировании

https://max.ru/bookofgeek

О чём глава:

- Тестирование под нагрузкой.
- Стресс-тестирование.
- Тестирование скачков нагрузки.
- Тестирование на продолжительное время.
- Тестирование масштабируемости.
- Тестирование конфигураций.
- Создание эталона производительности.
- Сравнительное тестирование.

> **Нагрузочное тестирование** является неотъемлемой частью процесса разработки высококачественного программного обеспечения. Его цель — проверить, как система будет вести себя в условиях реальной эксплуатации, под воздействием различных уровней нагрузки.

Этот процесс позволяет выявить узкие места, оценить производительность, стабильность и устойчивость системы перед выпуском в продуктивную среду.

В данной главе мы рассмотрим различные виды тестов, используемых при нагрузочном тестировании. Каждый из них имеет свои цели и применяется в определенных сценариях. Мы подробно объясним, что такое тестирование под нагрузкой, стресс-тестирование, тестирование скачков нагрузки и другие виды испытаний, а также как и когда они могут быть полезны для разработки высокопроизводительных приложений.

https://max.ru/bookofgeek

Кроме того, мы обсудим важность создания эталона производительности для сравнения результатов, а также дадим рекомендации по проведению тестов на масштабируемость и проверку конфигураций системы. Также будет затронуто сравнительное тестирование, которое помогает выбрать оптимальную конфигурацию для достижения наилучших результатов в реальных условиях эксплуатации.

Каждый из этих видов тестов помогает разработчикам и операционным командам выявить потенциальные проблемы до того, как они станут критичными, и сделать систему более эффективной и устойчивой.

17.1. Тестирование под нагрузкой (Load Testing)

Тестирование под нагрузкой — это один из важнейших этапов нагрузочного тестирования, направленный на определение того, как система будет функционировать при увеличении количества пользователей или запросов, которые она должна обрабатывать в реальных условиях эксплуатации.

Цель этого тестирования — убедиться, что приложение способно справляться с ожидаемой нагрузкой без потери производительности и стабильности. Это позволяет оценить не только способность системы выдерживать нагрузку, но и обнаружить потенциальные слабые места, которые могут проявиться при увеличении объема данных или числа пользователей.

Цель тестирования под нагрузкой

Основная цель тестирования под нагрузкой заключается в том, чтобы понять, как система будет вести себя при постепенном увеличении нагрузки. Это дает возможность:

- Оценить, насколько эффективно система может обрабатывать увеличенное количество пользователей или запросов.
- Измерить время отклика и выявить возможные задержки в системе.

https://max.ru/bookofgeek

- Оценить использование ресурсов системы (таких как процессор (CPU), оперативная память (RAM), дисковое пространство, пропускная способность сети и т.д.).
- Проверить, как система справляется с предсказуемой и стабильной нагрузкой, прежде чем приступить к более сложным и агрессивным сценариям, таким как стресс-тестирование.

Тестирование под нагрузкой служит основой для более сложных тестов, таких как стресс-тестирование и тестирование на продолжительное время. Без предварительного понимания того, как система ведет себя при обычной нагрузке, невозможно правильно оценить, как она будет работать в экстремальных условиях.

Особенности тестирования под нагрузкой

1. **Постепенное увеличение нагрузки**. В отличие от стресс-тестирования, где нагрузка увеличивается до максимально возможных значений, в тестировании под нагрузкой нагрузка увеличивается постепенно и измеряется на каждом уровне. Это позволяет точно установить порог, при котором система начинает терять производительность или стабильность. Обычно это делается в виде шагов, например, от 1 000 пользователей до 10 000 с интервалами по 1 000 пользователей.

2. **Измерение ключевых показателей**. Во время тестирования под нагрузкой измеряются критически важные метрики:

   » **Время отклика** — время, необходимое для обработки запроса. Оно должно быть минимальным на всех этапах нагрузки.

   » **Использование ресурсов** — сколько процессора (CPU) и оперативной памяти (RAM) использует система на разных уровнях нагрузки. Избыточное потребление ресурсов может указывать на проблемы в производительности.

   » **Пропускная способность (throughput)** — это количество запросов, которые система может обработать за единицу времени. Высокая пропускная способность является важным показателем для серверных приложений.

https://max.ru/bookofgeek

3. **Проверка стабильности системы**. Основной задачей является определение того, как система справляется с ожидаемой нагрузкой. Тестирование под нагрузкой позволяет убедиться, что приложение сохраняет нормальную работу и не теряет функциональности при увеличении числа запросов.
4. **Использование как базовой проверки**. Тестирование под нагрузкой часто проводится до начала более сложных тестов (например, стресс-тестирования или тестирования на продолжительное время). Оно служит первичной проверкой стабильности и работоспособности системы в нормальных условиях эксплуатации.

Пример: тестирование веб-приложения

Представим, что у вас есть веб-приложение, рассчитанное на 10 000 пользователей. В ходе тестирования под нагрузкой система подвергается симулированной нагрузке от 1 000 до 10 000 виртуальных пользователей. На каждом уровне нагрузки фиксируются следующие метрики:

- Время отклика для разных типов запросов (например, GET, POST и т.д.).
- Нагрузка на сервер — сколько ресурсов (CPU и RAM) потребляется на каждом уровне нагрузки.
- Пропускная способность — сколько запросов система может обработать за единицу времени на разных уровнях нагрузки.
- Количество ошибок (например, 5xx ошибки), возникающих при повышении нагрузки.

Этот подход позволяет вам заранее понять, на каком уровне нагрузка начнет оказывать влияние на производительность, а также какие метрики нуждаются в оптимизации.

Инструменты для тестирования под нагрузкой

Для проведения тестирования под нагрузкой используется множество специализированных инструментов, которые позволяют симулировать виртуальных пользователей и отслеживать ключевые показатели работы системы.

https://max.ru/bookofgeek

Некоторые из самых популярных инструментов включают:

1. **Jmeter** — один из наиболее известных инструментов для нагрузочного тестирования, который поддерживает создание сложных сценариев со множеством виртуальных пользователей. Он позволяет мониторить такие метрики, как время отклика, пропускная способность и ошибки.
2. **Gatling** — инструмент с поддержкой асинхронного моделирования, что позволяет эффективно использовать ресурсы при тестировании с большим количеством виртуальных пользователей. Gatling обладает высокой производительностью и удобным интерфейсом для создания сценариев тестирования.
3. **Locust** — это инструмент для нагрузочного тестирования, который использует Python для описания сценариев тестов. Он позволяет создавать высоконагруженные сценарии с миллионами пользователей и мониторить их производительность в реальном времени.
4. **Apache Bench (ab)** — это простой инструмент для тестирования производительности веб-серверов. Он полезен для быстрого тестирования с несколькими пользователями и определения, как система работает при конкретной нагрузке.
5. **k6** — современный инструмент для нагрузочного тестирования, ориентированный на удобство и производительность. Он поддерживает создание тестов на JavaScript, что облегчает интеграцию с другими инструментами и процессами CI/CD.

17.2. Стресс-тестирование (Stress Testing)

Стресс-тестирование — это важный аспект тестирования производительности, направленный на определение поведения системы в условиях экстремальной нагрузки, которая выходит за пределы ее нормальной рабочей способности. Это позволяет не только оценить, как система справляется с чрезмерной нагрузкой, но и выявить возможные точки отказа, которые могут привести к сбоям.

https://max.ru/bookofgeek

Такой подход помогает удостовериться в том, что система способна выдержать критические условия эксплуатации, а также восстановиться после сбоев. В отличие от тестирования под нагрузкой, стресс-тестирование стремится испытать систему на пределе ее возможностей и даже за его пределами, чтобы выяснить, какие слабые места могут привести к отказам.

Цель стресс-тестирования

Основной целью стресс-тестирования является проверка того, как система работает в условиях экстремальных и нестандартных нагрузок. Это дает возможность:

- **Выявить точки отказа**. Стресс-тестирование помогает понять, какие компоненты системы становятся уязвимыми при превышении нормальной нагрузки. Это могут быть как ошибки в программной части (например, переполнение буфера или утечка памяти), так и проблемы с аппаратным обеспечением (например, нехватка ресурсов или сбои в сетевой инфраструктуре).
- **Оценить устойчивость**. Стресс-тестирование помогает проверить, насколько система устойчива в условиях аномальной нагрузки. Важно, чтобы она не только работала до того момента, пока нагрузка не превышает максимально возможный предел, но и чтобы в случае отказа система смогла восстановиться.
- **Проверить способность к восстановлению (Recovery Testing)**. Один из ключевых аспектов стресс-тестирования — это проверка того, насколько быстро система может восстановиться после сбоев. Важен не только момент отказа, но и то, как быстро система возвращается в рабочее состояние после того, как нагрузка снижается до нормального уровня.

Особенности стресс-тестирования

1. **Нагрузка значительно превышает номинальную**. В отличие от тестирования под нагрузкой, где нагрузка увеличивается постепенно до нормального предела, стресс-тестирование предполагает экстремальные условия, в которых нагрузка значительно превышает максимально допу-

https://max.ru/bookofgeek

стимый уровень. Например, если система рассчитана на обработку 1 000 запросов в секунду, стресс-тестирование может включать нагрузку в 2 000 или 5 000 запросов в секунду.

2. **Фокус на устойчивости, ошибках и исключениях**. В процессе стресс-тестирования особое внимание уделяется тому, как система реагирует на неожиданные и аномальные ситуации. Важно выявить, как система обрабатывает ошибки и исключения, не приводят ли они к срыву работы всей системы. Например, могут быть рассмотрены такие ошибки, как 500 — внутренние ошибки сервера, переполнение памяти или перегрузка соединений с базой данных.

3. **Проверка на способность к восстановлению**. При стресс-тестировании особое внимание уделяется тому, как быстро система восстанавливается после того, как нагрузка снижается. Это называется **recovery testing**. Проверяется не только время, которое требуется системе для возвращения в рабочее состояние, но и ее способность продолжать функционировать нормально после сбоя.

4. **Анализ поведения системы в условиях предельной нагрузки**. Во время стресс-тестирования важно отслеживать все метрики производительности, такие как время отклика, использование ресурсов (CPU, RAM), пропускная способность, количество сбоев и ошибок. Это помогает выявить, где система начинает проявлять слабые места, и позволяет принять меры для улучшения ее устойчивости.

Пример стресс-тестирования

Рассмотрим пример стресс-тестирования для интернет-магазина, который проводит распродажу и ожидает значительное увеличение трафика. Во время распродажи веб-приложение может столкнуться с перегрузкой из-за резкого увеличения количества пользователей.

1. **Симуляция нагрузки**. При стресс-тестировании мы можем подать на систему нагрузку, которая в два или более раза превышает ожидаемую. Например, если магазин рассчитывает на 100 000 пользователей в пиковые моменты, то можно провести стресс-тестирование с нагрузкой в 200 000 пользователей.

https://max.ru/bookofgeek

2. **Измерение ошибок**. Важно отслеживать, как система реагирует на такую нагрузку. Происходят ли ошибки 500 (ошибки сервера)? Какие части системы не выдерживают увеличенной нагрузки? Например, база данных может не справляться с количеством запросов или серверы могут начать терять соединения с клиентами.

3. **Проверка восстановления**. После того как нагрузка будет снижена до нормального уровня, необходимо проверить, насколько быстро система восстанавливается. Важно, чтобы приложение продолжало работать корректно, даже если на пике нагрузки произошел сбой. Например, как быстро серверы восстанавливают доступность или как быстро база данных восстанавливает нормальную работу после блокировки.

Инструменты для стресс-тестирования

Для стресс-тестирования существуют различные инструменты, которые могут симулировать экстремальные условия и помогать анализировать поведение системы в таких условиях.

Некоторые из популярных инструментов включают:

- **Chaos Monkey** — это инструмент, входящий в состав набора инструментов Netflix для тестирования устойчивости системы. Он создает случайные сбои в системе (например, выключает серверы или отключает сервисы), чтобы проверить, как система справляется с такими ситуациями.

- **k6** — это инструмент для тестирования производительности, который поддерживает создание нагрузочных тестов с помощью сценариев на JavaScript. Он подходит для проведения стресс-тестов, позволяя моделировать экстремальные нагрузки и отслеживать метрики в реальном времени.

- **Artillery** — современный инструмент для тестирования нагрузки и производительности. Он позволяет генерировать экстремальную нагрузку на систему, анализировать отклики и проверять устойчивость приложения в условиях повышенной нагрузки.

https://max.ru/bookofgeek

17.3. Тестирование скачков нагрузки (Spike Testing)

Тестирование скачков нагрузки (Spike Testing) — это процесс оценки реакции системы на внезапные и кратковременные изменения нагрузки.

Этот тип тестирования используется для проверки устойчивости приложения в случае резкого и краткосрочного увеличения или уменьшения количества пользователей, что может происходить, например, в результате появления вирусного контента, акций, новостей или других факторов, которые могут вызвать кратковременное изменение трафика. Spike testing является важной частью оценки производительности системы, так как помогает выявить, как она справляется с резкими и непредсказуемыми колебаниями нагрузки.

Цель тестирования скачков нагрузки

Основная цель **Spike Testing** — проверить, как система справляется с внезапными скачками нагрузки. В отличие от других типов нагрузочных тестов, таких как **load testing** или **stress testing**, где нагрузка увеличивается постепенно или до экстремальных значений, при скачках нагрузки изменение происходит резко и кратковременно. Этот вид тестирования помогает определить:

- **Реакцию на резкие изменения нагрузки**. Как быстро система реагирует на внезапный рост или спад числа пользователей? Будет ли она продолжать работать корректно, не вызвав сбоев, ошибок или замедлений в производительности?
- **Устойчивость и стабильность**. Способна ли система поддерживать свою функциональность и доступность при кратковременных и резких скачках нагрузки? Как быстро она восстанавливается после того, как нагрузка падает обратно до нормальных значений?

https://max.ru/bookofgeek

- **Время восстановления**. Как быстро система возвращается к нормальной работе после того, как скачок нагрузки прекращается? Это включает в себя восстановление от возможных сбоев и ошибок, связанных с увеличением числа пользователей.

Особенности тестирования скачков нагрузки

1. **Резкое изменение нагрузки**. В Spike Testing нагрузка увеличивается или уменьшается очень быстро, что делает тест более сложным для системы, чем другие типы нагрузочных тестов. К примеру, число пользователей может увеличиться в несколько раз за несколько секунд, а затем так же резко снизиться.
2. **Симуляция краткосрочных изменений**. В отличие от других видов нагрузочных тестов, где нагрузка увеличивается постепенно и стабильно, в spike-тестировании фокус делается на кратковременном скачке. Это может быть связано с внезапным увеличением числа пользователей в случае вирусной активности, акций, новостей или других внешних факторов.
3. **Проверка устойчивости и восстановления**. Основное внимание в Spike Testing уделяется тому, как система справляется с пиковой нагрузкой и как быстро она восстанавливается после ее завершения. Это проверяет не только саму систему на устойчивость, но и ее способность к быстрому восстановлению без потери данных или функциональности.
4. **Краткосрочное воздействие**. В отличие от стресс-тестирования, где нагрузка может оставаться на высоком уровне длительное время, при скачках нагрузки воздействие является краткосрочным. После резкого увеличения трафика нагрузка также быстро падает до нормального уровня, что делает важным для системы не только выдержать пик, но и вернуться к стабильному состоянию без длительных перерывов.

Пример стресс-тестирования скачков нагрузки

Предположим, что социальная сеть публикует вирусный пост, который вызывает мгновенный интерес у пользователей. В течение 30 секунд количество активных пользователей на платформе возрастает в 5 раз.

https://max.ru/bookofgeek

После этого скачок резко падает, возвращая количество пользователей к нормальному уровню. Система должна справиться с этим скачком нагрузки и вернуть систему в стабильное состояние сразу после его окончания.

Важно в этом контексте измерить, как система:

- Реагирует на резкое увеличение трафика, не приводя к ошибкам или замедлениям.
- Сохраняет свою функциональность и доступность на протяжении всего пика.
- Восстанавливает нормальное функционирование после того, как скачок нагрузки завершился, и не возникает ли сбоев или длительных задержек.

Инструменты для тестирования скачков нагрузки

Для проведения Spike Testing используются различные инструменты, которые позволяют симулировать резкие изменения нагрузки и отслеживать поведение системы при таких скачках.

Некоторые популярные инструменты включают:

1. **Jmeter** — один из самых популярных инструментов для проведения нагрузочных тестов, который также поддерживает симуляцию скачков нагрузки. JMeter позволяет создавать сценарии, где нагрузка резко увеличивается и затем быстро уменьшается, чтобы проверить, как система справляется с таким стрессом.

2. **BlazeMeter** — платформа для тестирования производительности, которая позволяет моделировать скачки нагрузки, включая резкие изменения в числе пользователей. BlazeMeter предлагает облачную инфраструктуру для выполнения тестов на высоких нагрузках.

3. **Artillery** — это инструмент с открытым исходным кодом, предназначенный для нагрузочного тестирования и моделирования скачков нагрузки. Artillery позволяет быстро создавать сценарии для тестирования экстремальных условий нагрузки и оценивает, как система справляется с кратковременными пиками.

https://max.ru/bookofgeek

17.4. Тестирование на продолжительное время (Soak Testing)

Soak Testing, или тестирование на продолжительное время, направлено на выявление проблем, которые могут проявиться только при длительной эксплуатации системы.

Этот тип тестирования используется для того, чтобы оценить, как система функционирует в условиях постоянной работы, и выявить возможные утечки ресурсов, проблемы с производительностью или другие дефекты, которые могут возникнуть после длительного времени работы.

Цель тестирования на продолжительное время

Основная цель **Soak Testing** — это проверка системы в условиях длительной эксплуатации, чтобы обнаружить проблемы, которые не видны при краткосрочных нагрузочных тестах. В отличие от других типов тестов, где нагрузка может быть краткосрочной или пиковой, в soak-тестировании нагрузка сохраняется на среднем или низком уровне, но длится долго. Это позволяет выявить такие проблемы, как:

- **Утечки памяти**. Это проблемы, которые возникают, когда приложение или система не освобождают память, которую больше не используют, что может привести к исчерпанию доступных ресурсов и сбоям в работе приложения.
- **Накопление ошибок**. Некоторые ошибки могут накапливаться в процессе длительной работы системы, особенно в сложных многозадачных или распределенных приложениях. Эти ошибки могут проявиться только после нескольких часов или дней работы.
- **Деградация производительности**. В процессе работы система может начать замедляться, особенно если ресурсы не используются эффективно. Soak Testing помогает выявить такие проблемы, как увеличение времени отклика или снижение пропускной способности системы.

https://max.ru/bookofgeek

- **Проблемы с ресурсами**. В процессе долгосрочной работы могут проявляться проблемы с ресурсами, такие как перегрузка процессора (CPU) или увеличение использования памяти.

Особенности тестирования на продолжительное время

1. **Низкая или средняя нагрузка**. В отличие от стресс-тестирования или нагрузочного тестирования, где нагрузка может быть высокой и кратковременной, в soak-тестировании нагрузка поддерживается на среднем или низком уровне, но она продолжается длительное время. Такая нагрузка позволяет выявить проблемы, которые могут быть не видны при кратковременных тестах.
2. **Длительное время тестирования**. Soak-тестирование длится от нескольких часов до нескольких суток, в зависимости от требований системы. Это позволяет смоделировать реальные условия эксплуатации, в которых приложение или система работает круглосуточно.
3. **Мониторинг ресурсов**. Постоянный мониторинг системы является важной частью soak-тестирования. Для выявления утечек памяти, перегрузок и деградации производительности необходимо внимательно отслеживать использование ресурсов в процессе тестирования. Это включает в себя мониторинг потребления CPU, RAM, сетевого трафика и других ключевых показателей.
4. **Обнаружение долгосрочных проблем**. Так как тестирование длится длительное время, оно помогает выявить такие проблемы, которые могут не проявляться сразу после запуска системы. Это важно для гарантии того, что приложение или система будет работать стабильно даже при длительном функционировании.

Пример тестирования на продолжительное время

Представим, что мы тестируем ERP-систему, которая работает под постоянной нагрузкой, составляющей 20% от пикового значения нагрузки, на протяжении 48 часов. Цель теста — наблюдать за поведением системы на протяжении длительного времени и проверить, насколько стабильными остаются ее ресурсы.

https://max.ru/bookofgeek

Во время тестирования могут быть зафиксированы следующие параметры:

- **Потребление памяти**. Важно отслеживать, не увеличивается ли потребление памяти со временем, что может указывать на утечку памяти.
- **Время отклика**. Важно измерить, как меняется среднее время отклика системы по мере ее эксплуатации. Это может помочь выявить проблемы с производительностью.
- **Использование CPU**. Наблюдение за процессором позволяет понять, не происходит ли его чрезмерная загрузка, которая может указывать на неэффективность в управлении ресурсами.
- **Ошибки и сбои**. В ходе тестирования могут появляться ошибки или сбои, которые не видны при краткосрочных нагрузочных тестах, но становятся заметными в процессе длительного использования.

Инструменты для тестирования на продолжительное время

Для проведения **Soak Testing** применяются специализированные инструменты для мониторинга и анализа производительности системы. Они позволяют отслеживать использование ресурсов в реальном времени и фиксировать отклонения от нормальных показателей.

Некоторые популярные инструменты для этого типа тестирования:

- **Dynatrace** — платформа для мониторинга приложений, которая предоставляет подробные отчеты о производительности и позволяет отслеживать ресурсы, такие как память, CPU и время отклика.
- **New Relic** — инструмент для мониторинга и аналитики производительности, который позволяет отслеживать ресурсы и выявлять потенциальные проблемы в работе системы.
- **VisualVM** — утилита для мониторинга виртуальной машины Java (JVM), которая позволяет отслеживать использование памяти, выполнение потоков и другие параметры в процессе работы приложения.

https://max.ru/bookofgeek

- **Jmeter** — хотя JMeter используется в основном для нагрузочного тестирования, его можно настроить для длительного тестирования с помощью сценариев, симулирующих среднюю нагрузку на протяжении продолжительного времени.

17.5. Тестирование масштабируемости (Scalability Testing)

Тестирование масштабируемости — это тип тестирования, который позволяет оценить, как система может адаптироваться и поддерживать высокую производительность при увеличении нагрузки или доступных ресурсов.

Это важный процесс для любого приложения, которое предполагает рост числа пользователей, объема данных или нагрузки, а также необходимость в динамическом увеличении ресурсов для поддержания стабильности и производительности.

Цель тестирования масштабируемости

Основная цель **Scalability Testing** — это проверка того, как система справляется с изменением условий, таких как увеличение числа пользователей, объема данных или других внешних факторов.

В ходе тестирования проводится анализ того, насколько эффективно система может:

- **Масштабироваться горизонтально** — при добавлении новых ресурсов (например, серверов, контейнеров или подов) для обработки большего числа запросов или пользователей.
- **Масштабироваться вертикально** — при увеличении ресурсов на уже существующих узлах, например добавлении CPU или памяти, для улучшения обработки данных.

https://max.ru/bookofgeek

Кроме того, тестирование масштабируемости позволяет выявить **пределы масштабируемости** системы, то есть когда она уже не сможет эффективно масштабироваться, даже при добавлении дополнительных ресурсов.

Это критически важно для понимания границ системы и возможности ее дальнейшего роста.

Особенности тестирования масштабируемости

1. **Горизонтальное и вертикальное масштабирование**. Важно не только проверить, как система работает при увеличении ресурсов на одном узле (вертикальное масштабирование), но и как она ведет себя при добавлении новых узлов в кластер (горизонтальное масштабирование). Разные типы масштабирования могут потребовать разных подходов и технологий.
2. **Определение пределов масштабируемости**. Одной из ключевых задач масштабируемости является определение, где система начинает терять производительность при добавлении дополнительных ресурсов. Это помогает разработчикам и архитекторам оценить, когда система уже не сможет эффективно масштабироваться и потребуется переработка архитектуры.
3. **Сравнение производительности**. После масштабирования важно измерить метрики производительности, такие как время отклика (latency), пропускная способность (throughput), использование ресурсов (CPU, память, диск) и другие параметры. Это позволяет оценить, как масштабирование влияет на производительность системы и насколько эффективно она справляется с увеличением нагрузки.
4. **Тестирование с изменяющейся нагрузкой**. В отличие от стресс-тестирования или нагрузочного тестирования, где нагрузка может быть фиксированной, в тестировании масштабируемости нагрузка изменяется по мере добавления ресурсов. Это позволяет имитировать реальное увеличение нагрузки на систему в процессе ее эксплуатации.

Пример тестирования масштабируемости

Предположим, что тестируется **микросервисное приложение**, развернутое в Kubernetes. Приложение изначально обслуживает 1 000 пользователей,

https://max.ru/bookofgeek

и в процессе тестирования нагрузка увеличивается до 10 000 пользователей. Для этого динамически добавляются новые **поды** в Kubernetes.

Задача заключается в следующем:

- **Наблюдение за изменением производительности**. Измеряются такие метрики, как **latency** (время отклика) и **throughput** (пропускная способность), чтобы понять, как изменяется эффективность работы приложения при увеличении нагрузки.
- **Анализ ресурсов**. При увеличении числа подов и нагрузки важно отслеживать, как изменяется использование ресурсов, таких как CPU, память и сеть, чтобы понять, насколько эффективно система масштабируется.
- **Оценка пределов масштабируемости**. Важно определить, когда система начинает испытывать проблемы с производительностью, несмотря на добавление новых ресурсов. Это может быть связано с такими проблемами, как синхронизация данных между подами, использование ресурсов на уровне базы данных или проблемы с сетевой инфраструктурой.

Инструменты для тестирования масштабируемости

Для эффективного проведения тестирования масштабируемости используются различные инструменты, которые позволяют не только имитировать рост нагрузки, но и анализировать влияние масштабирования на производительность:

- **Kubernetes HPA (Horizontal Pod Autoscaler)** — инструмент для автоматического масштабирования числа подов в Kubernetes в зависимости от загрузки ресурсов, таких как CPU или память. HPA позволяет динамически адаптировать систему под изменяющуюся нагрузку, что важно для масштабируемости в микросервисных приложениях.

- **Prometheus** — система мониторинга и сбора метрик, которая позволяет отслеживать использование ресурсов и производительность системы в реальном времени. Prometheus собирает и хранит данные о метриках, таких как CPU, память, латентность, и может быть использован для анализа поведения системы при масштабировании.

https://max.ru/bookofgeek

- **Grafana** — платформа для визуализации данных, которая позволяет строить графики и панели мониторинга для анализа метрик, собранных с помощью Prometheus. Это дает наглядное представление о том, как система ведет себя при изменении нагрузки и добавлении новых ресурсов.
- **k6** — инструмент для нагрузочного тестирования, который поддерживает скрипты на JavaScript и позволяет симулировать множество пользователей и нагрузку на систему. Он используется для создания тестов, которые проверяют, как система справляется с увеличением нагрузки и как изменяется производительность при масштабировании.

17.6. Тестирование конфигураций (Configuration Testing)

Цель тестирования конфигураций заключается в том, чтобы понять, как различные изменения в настройках системы, таких как параметры JVM, настройки памяти, пула потоков или кешей, влияют на производительность и поведение приложения.

Этот процесс позволяет оптимизировать настройки, находя оптимальные параметры для работы системы при различных условиях нагрузки.

Особенности тестирования конфигураций

1. **Изменение ключевых параметров системы**. В ходе тестирования конфигураций важно изменять такие параметры, как:

   » **Размер heap memory (кучи)**. Это определяет, сколько памяти будет выделено для хранения объектов в памяти. Недостаток памяти может вызвать частые сборки мусора, а излишки — неэффективное использование ресурсов.

https://max.ru/bookofgeek

- **Размер и настройки пулов потоков (thread pools)**. Важный параметр для многозадачных приложений, который влияет на параллелизм выполнения задач и управление их очередями.
- **Настройки кешей**. Параметры, определяющие, как данные кешируются для ускорения доступа, особенно в высоконагруженных приложениях. Например, кеширование запросов к базе данных или вебстраниц.
- **Сборщик мусора (Garbage Collector)**. Выбор подходящего сборщика мусора и его настройки могут существенно повлиять на время отклика приложения и потребление памяти.
- **Конфигурация базы данных**. Параметры подключения, количество соединений, буферизация и другие настройки, влияющие на производительность взаимодействия с базой данных.

2. **Сравнительный анализ разных конфигураций**. Тестирование конфигураций позволяет проводить сравнение различных настроек и их влияние на систему. Важно определить, какие комбинации параметров обеспечивают наилучшую производительность при конкретных условиях эксплуатации.

3. **Поиск оптимальных параметров для определенной нагрузки**. Одной из ключевых задач является нахождение настроек, которые максимально эффективно поддерживают заданную нагрузку. Например, для высоконагруженных систем с большим количеством параллельных запросов важно оптимизировать размер пулов потоков и параметры кеширования, чтобы обеспечить максимальную пропускную способность.

Пример тестирования конфигураций

Предположим, что необходимо провести серию тестов, чтобы определить, как изменение **размера heap memory** влияет на производительность системы.

Для этого можно использовать различные значения, такие как 512MB, 1024MB и 2048MB, и оценить влияние этих изменений на следующие метрики:

https://max.ru/bookofgeek

- **GC activity** — частота и продолжительность сборок мусора, которые могут существенно влиять на производительность, если память не оптимально распределена.
- **Latency (время отклика)** — время, которое требуется для выполнения операций, таких как обработка запросов от пользователей или выполнение вычислений.
- **Throughput (пропускная способность)** — количество операций или запросов, которые система может обработать за единицу времени.

Этот тест позволит оценить, как размер памяти влияет на работу сборщика мусора, и поможет определить оптимальный размер heap memory для минимизации задержек и оптимизации использования ресурсов.

Инструменты для тестирования конфигураций

Для эффективного проведения тестирования конфигураций и анализа изменений в производительности, можно использовать следующие инструменты:

- **VisualVM** — мощный инструмент для мониторинга работы Java-приложений в реальном времени. Он позволяет анализировать использование памяти, процессорного времени, а также отслеживать активность сборщика мусора и выявлять потенциальные проблемы в конфигурации.
- **JMH (Java Microbenchmarking Harness)** — инструмент для точного измерения производительности отдельных методов и операций, включая влияние изменений в конфигурации на производительность.
- **Jconsole** — инструмент для мониторинга и управления Java-приложениями в реальном времени. JConsole позволяет отслеживать использование ресурсов, таких как память и потоки, и оценивать поведение приложения при изменении конфигураций.
- **Prometheus** — система мониторинга и сбора метрик, которая позволяет собирать и хранить данные о различных параметрах работы системы, таких как использование CPU, памяти, время отклика и другие метрики. Prometheus может быть интегрирован с другими инструментами, такими как Grafana, для визуализации и анализа данных.

https://max.ru/bookofgeek

17.7. Создание эталона производительности (Baseline Testing)

Цель создания **эталона производительности** (baseline testing) — фиксация текущего состояния производительности системы до внесения изменений в код, архитектуру или инфраструктуру.

Этот эталон служит отправной точкой, с которой можно сравнивать результаты после изменений, чтобы отслеживать отклонения, улучшения или регрессии в производительности.

Особенности создания эталона производительности

1. **Проведение до изменений в коде или инфраструктуре**. Эталон производительности создается в начальный момент, до того как планируются изменения в системе. Это может быть до обновления версии Java, внедрения нового фреймворка, перехода на новую версию базы данных или изменения конфигурации серверов. Без четкого эталона невозможно объективно оценить влияние изменений на производительность.
2. **Использование как референс в CI/CD пайплайнах**. Эталон производительности часто используется как базовый референс в процессе непрерывной интеграции и доставки (CI/CD). В рамках CI/CD пайплайна важно проводить тестирование на производительность с постоянной проверкой against baseline. Это помогает быстро выявлять любые отклонения, которые могут указывать на регрессии в производительности, влияющие на стабильность приложения.
3. **Фиксация всех ключевых метрик**. Для создания полноценного эталона необходимо зафиксировать ряд важных метрик, которые будут служить базой для дальнейших сравнений. К таким меткам относятся:

   » **Throughput (Пропускная способность)** — количество обработанных запросов или операций за определенный период времени.

https://max.ru/bookofgeek

- **Latency (Задержка)** — время, которое проходит от начала запроса до получения ответа.
- **CPU Usage (Использование процессора)** — нагрузка на процессор, которая может указывать на оптимизацию или перегрузку системы.
- **Memory Usage (Использование памяти)** — объем памяти, который используется приложением, включая данные в heap и различные кеши.
- **GC Activity (Активность сборщика мусора)** — показатели частоты и продолжительности сборок мусора, которые могут существенно повлиять на производительность.
- **Error Rates (Частота ошибок)** — уровень ошибок или сбоев в системе, который также может быть индикатором потенциальных проблем.

4. Фиксация этих метрик в момент создания эталона позволяет затем легко отслеживать изменения в системе, особенно если возникают проблемы в будущем.

Пример. Переход с Java 11 на Java 17

Представьте себе проект, в котором используется версия Java 11 и планируется переход на Java 17. До обновления необходимо создать эталон производительности, зафиксировав ключевые метрики, такие как throughput, GC время, latency и использование памяти. Эти данные будут использоваться в дальнейшем для сравнения с результатами, полученными после миграции на Java 17. При проведении тестирования на новой версии Java важно, чтобы показатели throughput, latency и GC не ухудшились. Если же после обновления будут зафиксированы значительные отклонения, это может указывать на регрессию производительности, требующую внимания.

Инструменты для создания эталона производительности

Для фиксации и измерения ключевых метрик производительности можно использовать различные инструменты.

Вот несколько популярных решений:

1. **JMH (Java Microbenchmarking Harness)**. JMH — это специализированный инструмент для точного измерения производительности Java-программ. Он позволяет создавать микробенчмарки для тестирования

https://max.ru/bookofgeek

конкретных частей кода и фиксировать их производительность. Это идеальный инструмент для создания эталонов производительности, особенно когда речь идет о тонких настройках JVM или улучшениях алгоритмов.

2. **Jmeter**. JMeter — это инструмент для тестирования производительности, который используется для измерения производительности как веб-приложений, так и баз данных, API и других сервисов. В JMeter можно настроить нагрузки и проводить тесты на пропускную способность, время отклика и стабильность, что делает его полезным для создания эталонов для более крупных систем.

3. **k6**. k6 — это современный инструмент для нагрузочного тестирования, который позволяет измерять производительность приложений при различных уровнях нагрузки. Он позволяет создавать сценарии тестирования, а также собирать метрики, такие как время отклика, ошибки и пропускная способность.

4. **Prometheus** и **InfluxDB**. Prometheus и InfluxDB — это системы мониторинга и сбора метрик, которые помогают собирать данные о производительности системы в реальном времени. Prometheus собирает метрики с помощью экспортеров, а InfluxDB предоставляет временную базу данных для хранения этих метрик. Оба инструмента идеально подходят для долгосрочного мониторинга производительности и создания эталонов на уровне инфраструктуры.

17.8. Сравнительное тестирование (Comparison Testing)

Цель сравнительного тестирования (comparison testing) — оценка и сравнение производительности двух или более реализаций, библиотек, конфигураций или архитектурных решений. Оно позволяет выявить, какое из решений наиболее эффективно в контексте заданных требований и условий эксплуатации.

https://max.ru/bookofgeek

Особенности сравнительного тестирования

1. **Сочетание различных видов тестов**. Сравнительное тестирование может включать в себя несколько видов тестирования, таких как нагрузочные, стрессовые, soak-тесты и другие. Например, при сравнении двух решений для работы с базой данных можно провести тесты на нагрузку, оценить их устойчивость к стрессу или протестировать производительность при длительном времени работы. Это позволяет получить комплексную картину производительности и выявить сильные и слабые стороны каждого решения.
2. **Полезность при выборе технологий, библиотек или архитектурных решений**. Сравнительное тестирование очень полезно на этапе выбора между альтернативными технологиями. Например, если компания решает, использовать ли для работы с базой данных классический JDBC или реактивную модель через R2DBC, то сравнительное тестирование поможет выбрать наилучшее решение с точки зрения производительности. Также оно полезно при принятии решений о выборе фреймворков, архитектурных подходов или баз данных, чтобы убедиться в их оптимальности для конкретного проекта.
3. **Необходимость одинаковых условий тестирования**. Важно, чтобы все варианты тестируемых решений использовались в одинаковых условиях. Например, при сравнении двух библиотек для работы с базой данных нужно удостовериться, что каждая из них работает с одинаковым набором данных, в одинаковых средах (серверы, JVM, ОС) и при тех же нагрузках. Только при соблюдении этих условий можно гарантировать, что результаты сравнения будут объективными.

Пример. Сравнение JDBC и R2DBC для доступа к базе данных

Одним из примеров сравнительного тестирования может быть анализ производительности двух технологий для работы с базой данных: традиционного JDBC и более современной асинхронной библиотеки R2DBC. В рамках такого теста важно оценить ключевые метрики, такие как:

» **Latency (Задержка)** — сколько времени требуется для выполнения запроса к базе данных.

https://max.ru/bookofgeek

- **Throughput (Пропускная способность)** — сколько запросов система может обработать за единицу времени.
- **CPU Usage (Использование процессора)** — как нагрузка на процессор меняется при использовании различных подходов.

Также, к примеру, JDBC может показывать стабильную производительность в условиях синхронных запросов, но может страдать от большего времени отклика в условиях высокой нагрузки. В то время как R2DBC, использующий асинхронный подход, может продемонстрировать лучшие результаты по *throughput*, но потребовать больше ресурсов для обработки большего числа одновременных соединений.

Инструменты для сравнительного тестирования

Для проведения сравнительного тестирования можно использовать различные инструменты, которые позволяют эффективно собирать метрики и анализировать производительность.

Некоторые из них включают:

1. **JMH (Java Microbenchmarking Harness)**. JMH — это мощный инструмент для проведения микробенчмарков в Java, который предоставляет точные и стабильные результаты тестирования производительности. Он идеально подходит для сравнения производительности разных вариантов кода или библиотек на уровне методов и операций.
2. **Gatling**. Gatling — это инструмент для нагрузочного тестирования, который поддерживает сценарии с высокой нагрузкой и сложные профили тестов. Gatling может быть полезен для сравнительного тестирования при нагрузке, позволяя провести тестирование различных библиотек или решений для работы с веб-сервисами.
3. **Jmeter**. JMeter — популярный инструмент для тестирования производительности, поддерживающий широкий спектр тестов, включая стрессовые и нагрузочные. JMeter позволяет создавать сценарии для тестирования различных решений и получать результаты по меткам времени, пропускной способности и ошибкам.

https://max.ru/bookofgeek

4. **Custom Benchmark Wrappers**. Для специфических случаев может быть полезно использовать собственные обертки для бенчмарков, которые позволяют собирать необходимые метрики и автоматически проводить сравнительные тесты в заданных условиях. Например, можно создать собственные скрипты, которые автоматически изменяют параметры конфигурации и запускают сравнение между различными версиями кода или библиотеками.

Выводы

Нагрузочное тестирование — это не просто генерация запросов и наблюдение за метриками. Это комплексный и структурированный подход к анализу поведения системы в условиях, максимально приближенных к реальной эксплуатации.

> **Цель нагрузочного тестирования** заключается не только в том, чтобы проверить, как система справляется с определенным количеством запросов, но и в том, чтобы понять, как она реагирует на различные сценарии: от стабильной работы при постоянной нагрузке до экстремальных условий, выходящих за пределы нормальной эксплуатации.

Каждый тип нагрузочного тестирования раскрывает различные аспекты поведения приложения, и все эти аспекты важны для понимания надежности и производительности системы в реальных условиях. Например, тестирование под нагрузкой позволяет оценить, как система работает при обычных или предсказуемых условиях, в то время как стресс-тестирование выявляет уязвимости при резких скачках нагрузки. Тестирование на продолжительное время помогает понять, как система ведет себя при длительном функционировании, в то время как тестирование масштабируемости дает четкое представление о том, как система будет масштабироваться с увеличением числа пользователей или объемов данных. Все эти тесты направлены на то, чтобы выявить возможные узкие места и предотвратить проблемы до того, как система попадет в реальную эксплуатацию.

https://max.ru/bookofgeek

Правильный выбор типов тестов, а также их эффективное комбинирование позволяет не только выявить слабые места и проблемные участки, но и обосновать архитектурные решения, сделать обоснованные выводы об оптимизациях, проведенных на различных уровнях системы, а также повысить уверенность в готовности приложения к реальной эксплуатации. Знание того, как система ведет себя под нагрузкой, дает возможность заранее подготовиться к реальным условиям и сделать систему более устойчивой и производительной.

Помимо выявления уязвимостей, нагрузочное тестирование позволяет также протестировать различные решения, принятые в процессе проектирования системы, и удостовериться в их эффективности в условиях реальной эксплуатации. Например, можно проверить, насколько выбранная архитектура поддерживает нагрузку и насколько эффективно выполнены оптимизации на уровне кода или инфраструктуры. Важно помнить, что даже если система демонстрирует отличные результаты при нагрузке в лабораторных условиях, важно провести все виды тестирования, чтобы учесть все возможные сценарии в реальной эксплуатации.

В следующих главах мы подробнее рассмотрим, как строить эффективную стратегию нагрузочного тестирования, какие инструменты и методологии использовать для его автоматизации, а также как интегрировать нагрузочное тестирование в CI/CD процессы для регулярного мониторинга и оценки производительности системы на всех этапах разработки.

https://max.ru/bookofgeek

Глава 18.

Методы и техники оптимизации кода

https://max.ru/bookofgeek

О чём глава:

- Основы рефакторинга кода.
- Паттерны и анти-паттерны оптимизации.
- Инструменты для анализа и улучшения кода.
- Best Practices оптимизации кода.

Оптимизация кода — это процесс улучшения его производительности, читаемости и устойчивости. Включает в себя не только улучшение скорости работы приложения, но и упрощение его структуры и повышение поддерживаемости.

Основные методы оптимизации делятся на два направления: **рефакторинг** и использование **оптимизационных паттернов**.

18.1. Основы рефакторинга кода

Темы рефракторинга мы уже касались ранее. Освежим в памяти.

Рефакторинг — это процесс улучшения внутренней структуры существующего кода без изменения его функциональности. Рефакторинг помогает избавиться от дублирования, улучшить читаемость и упростить поддерживаемость кода. Рефакторинг должен проводиться регулярно, чтобы избежать накопления технического долга.

https://max.ru/bookofgeek

Основные цели рефакторинга:

- Упрощение кода.
- Повышение читаемости и понимания.
- Повышение производительности (в некоторых случаях).
- Устранение дублирования.

Пример рефакторинга: предположим, у нас есть класс с методом, который выполняет несколько задач, что затрудняет его тестирование и поддержку.

До рефакторинга:

```
public class OrderService {
    public void processOrder(Order order) {
        if (order.isValid()) {
            saveOrder(order);
            sendConfirmationEmail(order);
        }
    }

    private void saveOrder(Order order) {
        // Сохранение заказа в базе данных
    }

    private void sendConfirmationEmail(Order order) {
        // Отправка подтверждения на email
    }
}
```

Здесь метод *processOrder* выполняет несколько задач — сохранение заказа и отправку email. Мы можем провести рефакторинг, выделив эти операции в отдельные классы, чтобы улучшить тестируемость и поддержку.

После рефакторинга:

```
public class OrderService {
    private final OrderSaver orderSaver;
```


https://max.ru/bookofgeek

```
    private final EmailSender emailSender;

    public OrderService(OrderSaver orderSaver, EmailSender
emailSender) {
        this.orderSaver = orderSaver;
        this.emailSender = emailSender;
    }

    public void processOrder(Order order) {
        if (order.isValid()) {
            orderSaver.save(order);
            emailSender.sendConfirmation(order);
        }
    }
}
```

Теперь каждую операцию можно тестировать отдельно, а сам класс *OrderService* стал более чистым и фокусируется только на процессе обработки заказа.

18.2. Паттерны и антипаттерны оптимизации

Паттерны оптимизации — это проверенные решения для часто встречающихся проблем в разработке. Они помогают решать задачи более эффективно и минимизируют риск возникновения ошибок.

Примеры паттернов оптимизации:

- **Cache Pattern** — кеширование данных для ускорения доступа.
- **Flyweight Pattern** — уменьшение потребления памяти за счет использования общих объектов.
- **Lazy Loading** — загрузка данных по мере необходимости, а не заранее.

https://max.ru/bookofgeek

Пример паттерна **Cache**:

```
public class UserService {
    private final Map<Integer, User> userCache = new HashMap<>();

    public User getUserById(int id) {
        if (!userCache.containsKey(id)) {
            User user = loadUserFromDatabase(id);
            userCache.put(id, user);
        }
        return userCache.get(id);
    }

    private User loadUserFromDatabase(int id) {
        // Загрузка из базы данных
    }
}
```

Этот код использует паттерн **Cache** для кеширования пользователей, чтобы избежать повторных запросов в базу данных.

Антипаттерны — это типичные, но неэффективные или опасные подходы к решению проблем. Они могут привести к ухудшению производительности, усложнению кода или появлению багов.

Примеры антипаттернов:

- **God Object** — создание объектов, которые содержат слишком много обязанностей.
- **Spaghetti Code** — код, который сложен для понимания и изменения из-за отсутствия структуры и логики.
- **Premature Optimization** — попытка оптимизировать код до того, как это действительно необходимо, что может привести к сложному и трудно поддерживаемому решению.

https://max.ru/bookofgeek

18.3. Инструменты для анализа и улучшения кода

Для оптимизации кода важно использовать инструменты, которые помогают выявить потенциальные проблемы, избыточность, а также помогают следить за качеством кода.

Некоторые популярные инструменты для анализа и улучшения кода:

- **SonarQube** — инструмент для анализа качества кода, который предоставляет отчеты по багам, уязвимостям и нарушениям стандартов.
- **Checkstyle** — инструмент для проверки соответствия кода стандартам кодирования.
- **PMD** — анализатор кода, который ищет потенциально опасные участки и антипаттерны.
- **FindBugs** — инструмент для анализа байт-кода и поиска багов в Java-коде.
- **JProfiler** — инструмент для профилирования Java-приложений, который помогает найти узкие места в производительности.

Пример использования SonarQube:

1. Установите и настройте SonarQube на сервере.
2. Интегрируйте SonarQube с вашим проектом с помощью **SonarScanner**.
3. Запустите анализ:

```
sonar-scanner -Dsonar.projectKey=my-java-project -Dsonar.sources=src
```

4. После выполнения анализа откройте интерфейс SonarQube для просмотра отчетов и улучшений.

18.4. Best Practices оптимизации кода

https://max.ru/bookofgeek

Использование лучших практик (Best Practices) помогает избежать многих распространенных ошибок и создать более читаемый, поддерживаемый и быстрый код.

Основные лучшие практики:

1. **Использование подходящих структур данных**. Выбирайте правильные структуры данных для конкретных задач (например, **ArrayList** для поиска по индексу, **HashMap** для быстрого поиска по ключу).
2. **Минимизация операций с памятью**. Избегайте создания ненужных объектов и освобождайте ресурсы, как только они больше не нужны.
3. **Избегайте дублирования**. Всегда старайтесь избегать дублирования кода, вынося общие операции в отдельные методы или классы.
4. **Профилирование и тестирование производительности**. Всегда профилируйте ваш код, прежде чем приступать к его оптимизации. Оцените, где реально есть проблемы, и оптимизируйте только эти участки.
5. **Использование многозадачности разумно**. Применяйте многозадачность только тогда, когда она действительно может улучшить производительность.

Пример оптимизации с использованием коллекций:

```
// Вместо использования List для поиска по индексу
List<String> list = new ArrayList<>();
// Используем HashSet для быстрых поисков по ключу
Set<String> set = new HashSet<>();
```

В этом примере использование ***HashSet*** для быстрого поиска элементов может быть гораздо более эффективным, чем использование ***ArrayList***.

Выводы

Оптимизация кода — это многогранный и многослойный процесс, который требует комплексного подхода и тщательного внимания к деталям. Этот процесс не ограничивается простыми улучшениями производительности, он

https://max.ru/bookofgeek

охватывает весь спектр задач, связанных с улучшением структуры, читаемости и поддерживаемости кода, что в свою очередь способствует созданию более стабильных и быстрых приложений.

> Одним из ключевых аспектов оптимизации является **рефакторинг**. Этот процесс позволяет улучшить архитектуру и структуру кода без изменения его функциональности.

Рефакторинг — это не просто улучшение производительности, но и повышение удобства работы с кодом в будущем, облегчение его расширения и модификации. Он играет важную роль в обеспечении долгосрочной поддержки и развития программного обеспечения, особенно когда речь идет о больших и сложных проектах, где поддерживаемость кода становится критически важным фактором.

> Важную роль в оптимизации играют также **паттерны** и **антипаттерны**. Паттерны оптимизации предоставляют проверенные и надежные решения для распространенных проблем, связанных с производительностью, такими как уменьшение времени отклика, эффективное использование памяти или обработка больших объемов данных.

Следование таким паттернам помогает избежать распространенных ошибок и ускоряет процесс разработки. В то же время антипаттерны служат напоминанием о том, что нужно избегать, помогая не попадать в ловушки, которые могут привести к ухудшению производительности или усложнению поддержки системы.

Для эффективного контроля за качеством кода и его производительностью на всех этапах разработки крайне важно использовать инструменты для анализа. Такие инструменты, как статический анализ кода, профилировщики и тесты на производительность, позволяют не только выявлять узкие места, но и предотвращать их до того, как они станут реальной проблемой. Они помогают команде разработчиков поддерживать приложение на высо-

https://max.ru/bookofgeek

ком уровне, обеспечивая стабильность и производительность даже при изменении внешних условий или добавлении новых функций.

Наконец, следование лучшим практикам разработки является неотъемлемой частью оптимизации кода. Лучшие практики способствуют созданию чистого, понятного и поддерживаемого кода, что сокращает время на разработку и тестирование, а также упрощает процесс отладки и дальнейшей эволюции системы. Эти практики включают в себя правильное оформление кода, использование принципов SOLID, написание модульных и интеграционных тестов, а также строгое следование стандартам кодирования.

В итоге эффективная оптимизация кода — это не только улучшение производительности, но и создание системы, которая будет легко поддерживаться и развиваться в долгосрочной перспективе. Это комплексный процесс, в котором важно соблюдать баланс между производительностью, читаемостью, поддерживаемостью и стабильностью кода, что позволяет достичь высоких результатов в работе над проектом.

https://max.ru/bookofgeek

https://max.ru/bookofgeek

Глава 19.

Продвинутые техники и подходы

https://max.ru/bookofgeek

О чём глава:

- JIT-компиляция и ее влияние на производительность.
- Inline-оптимизация и ее применение.
- Использование специализированных библиотек и инструментов.

Для разработки высокоэффективных приложений, способных выдерживать интенсивные нагрузки и работать с большими объемами данных, стандартных методов оптимизации часто оказывается недостаточно. В таких случаях необходимо применять более углубленные и специализированные подходы, которые требуют детального понимания работы Java Virtual Machine (JVM) и особенностей самого языка Java. Это включает в себя использование продвинутых техник, таких как JIT-компиляция, inline-оптимизация и использование специализированных библиотек и инструментов, которые позволяют значительно повысить производительность и сократить время отклика приложений.

JIT-компиляция и inline-оптимизация, например, могут существенно ускорить выполнение кода, особенно в тех случаях, когда традиционные методы оптимизации, такие как улучшение структуры кода или кеширование, оказываются недостаточными. Эти подходы направлены на улучшение работы непосредственно на уровне байт-кода и позволяют JVM более эффективно управлять ресурсами.

Кроме того, для решения специфических задач, таких как работа с большими объемами данных или распределенные вычисления, важным элемен-

https://max.ru/bookofgeek

том оптимизации является использование специализированных библиотек и инструментов. Они позволяют эффективно решать узкие места производительности и адаптировать приложение под конкретные условия эксплуатации.

В этой главе мы подробно рассмотрим эти продвинутые техники и подходы, проанализируем их применение в реальных проектах и оценим, как они могут быть интегрированы в процесс разработки для достижения максимальной эффективности.

19.1. JIT-компиляция и ее влияние на производительность

JIT-компиляция (Just-In-Time Compilation) — это ключевая особенность JVM, которая существенно влияет на производительность Java-программ. В отличие от традиционной компиляции, когда код компилируется заранее, в JIT-компиляции исходный код компилируется в машинный код во время выполнения программы, что позволяет JVM адаптировать выполнение кода в зависимости от реальных условий работы.

Как работает JIT-компиляция:

1. Программа запускается интерпретатором, и байт-код выполняется в виртуальной машине.
2. Во время выполнения JVM определяет, какие участки кода часто выполняются (горячие участки).
3. Эти участки кода компилируются в машинный код с помощью JIT-компилятора, что позволяет улучшить производительность.

Влияние на производительность:

https://max.ru/bookofgeek

- **Ускорение работы часто используемых функций**. JIT-компиляция позволяет значительно ускорить выполнение тех частей кода, которые выполняются многократно, путем их компиляции в оптимизированный машинный код.
- **Динамическая оптимизация**. JVM может оптимизировать код в реальном времени, включая такие операции, как удаление мертвого кода или инлайнинг функций.
- **Задержки при первом запуске**. В первые моменты работы программы JIT-компиляция может вызвать небольшие задержки, так как байт-код компилируется в машинный код.

Пример:

Предположим, что у нас есть метод, который вызывает один и тот же фрагмент кода многократно. JVM распознает это как горячий участок и компилирует его в машинный код.

```
public class JitExample {
    public static void main(String[] args) {
        for (int i = 0; i < 1000000; i++) {
            compute(i);
        }
    }

    public static int compute(int i) {
        return i * i;
    }
}
```

В данном примере метод *compute* будет часто вызываться, и JVM может JIT-компилировать его для повышения производительности.

19.2. Inline-оптимизация и ее применение

https://max.ru/bookofgeek

Inline-оптимизация — это техника, при которой JVM или компилятор заменяет вызовы методов прямо в месте их вызова, интегрируя тело метода в код. Это позволяет избежать накладных расходов, связанных с вызовами методов, таких как передача параметров и возврат значений.

Как работает inline-оптимизация:

1. Когда метод вызывает другой метод, JVM может заменить вызов этого метода на непосредственно исполнимый код.
2. Inline-оптимизация часто используется для небольших методов, где накладные расходы на вызов метода становятся значительными.
3. JVM может автоматически выполнять эту оптимизацию, если метод маленький и не имеет побочных эффектов.

Пример:

Допустим, у нас есть метод, который просто возвращает значение, и этот метод вызывается часто:

```
public class InlineExample {
    public static int addFive(int x) {
        return x + 5;
    }

    public static void main(String[] args) {
        int result = 0;
        for (int i = 0; i < 1000000; i++) {
            result = addFive(i);
        }
    }
}
```

В этом примере JVM может применить inline-оптимизацию и просто вставить код *x + 5* в место вызова метода *addFive*, тем самым избавив от накладных расходов на вызов.

https://max.ru/bookofgeek

Влияние на производительность:

- **Снижение затрат на вызов метода**. Inline-оптимизация уменьшает накладные расходы на вызовы метода, что особенно важно для небольших методов, которые вызываются часто.
- **Увеличение размера кода**. Чрезмерное использование inline-оптимизации может привести к увеличению размера исполнимого кода, что может сказаться на его производительности (например, при недостаточной памяти кэширования).

19.3. Использование специализированных библиотек и инструментов

Для оптимизации кода часто используются **специализированные библиотеки и инструменты**, которые обеспечивают дополнительные возможности для улучшения производительности или упрощения работы с высокоэффективными алгоритмами.

Примеры таких библиотек и инструментов:

- **Guava** (Google's open-source Java library). Обеспечивает набор утилит для работы с коллекциями, кешированием, кешированием в памяти, а также поддержку функциональных операций. Guava помогает избежать написания собственного кода для часто встречающихся задач.

  Пример использования кеша с помощью Guava:

```
Cache<Integer, String> cache = CacheBuilder.newBuilder()
    .maximumSize(100)
    .expireAfterWrite(10, TimeUnit.MINUTES)
    .build();
```

- **Akka**. Библиотека для создания высокопроизводительных распределенных приложений. Akka предоставляет возможности для работы с акторной моделью, что позволяет масштабировать многопоточные приложения.

https://max.ru/bookofgeek

- **Netty**. Библиотека для создания высокоскоростных сетевых приложений с поддержкой неблокирующего ввода-вывода. Netty идеально подходит для приложений, которые требуют высокой пропускной способности и низкой задержки.
- **JMH** (Java Microbenchmarking Harness). Библиотека для микробенчмаркинга, которая помогает точно измерить производительность отдельных частей кода в реальных условиях работы JVM. JMH важен для оценки эффективности различных оптимизаций.

Пример кода с использованием JMH для тестирования производительности:

```
@Benchmark
public void testMethod() {
    // Код для тестирования
}
```

- **JVM инструменты**. JVisualVM, JConsole, YourKit — инструменты для профилирования и мониторинга Java-приложений, которые позволяют анализировать использование памяти, CPU и другие метрики производительности.

Выводы

Продвинутые техники оптимизации, такие как JIT-компиляция, inline-оптимизация и использование специализированных библиотек и инструментов, могут существенно повысить производительность Java-приложений, особенно в условиях интенсивных нагрузок. Эти методы позволяют глубже взаимодействовать с JVM и максимально эффективно использовать ее возможности, что требует от разработчика не только знаний языка, но и глубокого понимания внутренней работы системы.

Хотя многие из этих техник могут быть автоматически применены JVM, для достижения наилучших результатов важно понимать, когда и как их эффективно использовать. Тщательное управление процессом оптимизации, выбор подходящих техник и их корректная интеграция в архитектуру при-

https://max.ru/bookofgeek

ложения позволяют значительно улучшить производительность, масштабируемость и устойчивость системы.

Осознание их применения и оптимизация на этом уровне — это ключ к созданию высокоэффективных, стабильных и масштабируемых приложений.

https://max.ru/bookofgeek

Ещё больше свежей и актуальной литературы по Java ты найдёшь в наших каналах:

https://max.ru/bookofgeek
https://t.me/bookofgeek

Глава 20.

Метрики и Observability: как контролировать и понимать поведение Java-приложений

https://max.ru/bookofgeek

О чём глава:

- Основы Observability.
- Prometheus — стандарт де-факто для сбора метрик.
- Micrometer — фасад для метрик в Java.
- Grafana — визуализация и дашборды.
- Подключаем все вместе: Prometheus + Micrometer + Grafana.
- Какие метрики собирать?
- Алерты и автоматическая реакция.
- Observability в Kubernetes.
- Расширение: OpenTelemetry и трассировки.
- Best practices.

Современные распределенные приложения, особенно те, которые построены на базе Java, требуют непрерывного мониторинга и анализа их поведения в реальном времени. В условиях сложной архитектуры, состоящей из множества микросервисов и взаимодействий, без должной видимости (observability) становится чрезвычайно трудно диагностировать проблемы производительности, выявлять причины сбоев или даже точно понимать, как распределяется нагрузка между различными компонентами системы.

Ключевыми элементами системы наблюдаемости являются **метрики**, **логи** и **трассировки**. Эти данные позволяют отслеживать поведение приложения на различных уровнях, начиная от времени отклика и использования ресурсов, заканчивая более глубокими аспектами, такими как зависания или ошибки в процессе обработки запросов. Метрики, в частности, предоставляют ценные количественные данные о работе системы, которые позволяют не только отслеживать текущую производительность, но и прогнозировать будущие проблемы.

https://max.ru/bookofgeek

В этой главе мы подробно рассмотрим метрики как главный инструмент оценки производительности Java-приложений. Мы сосредоточим внимание на таких популярных инструментах, как Prometheus, Micrometer и Grafana, которые используются для сбора, агрегации, визуализации и создания алертов. Эти инструменты помогают обеспечить полный контроль над системой, а также позволяют оперативно выявлять и устранять узкие места в работе приложения.

20.1. Основы Observability

Observability (наблюдаемость) — это ключевая способность системы предоставлять необходимую информацию для диагностики ее состояния и поведения.

Система с хорошей наблюдаемостью помогает разработчикам и операционным командам не только поддерживать ее работоспособность, но и выявлять проблемы на ранних этапах, предсказывать потенциальные сбои и оптимизировать производительность.

Основные цели, которые можно достичь с помощью хорошей наблюдаемости:

- **Выявление узких мест и деградаций**. Возможность оперативно заметить, где приложение испытывает проблемы, будь то медленные запросы, перегрузка ресурсов или ошибки в логике работы.
- **Контроль за SLA и производительностью**. Метрики и логи помогают следить за тем, как приложение соответствует заранее установленным стандартам качества обслуживания (SLA) и производительности.
- **Оповещения о сбоях**. Система наблюдаемости предоставляет механизмы для настройки уведомлений и алертов, что помогает быстро реагировать на возникшие проблемы.
- **Анализ причин изменений в поведении системы**. Наблюдаемость позволяет не только фиксировать отклонения, но и исследовать их при-

https://max.ru/bookofgeek

чины, чтобы понять, что привело к изменениям в производительности или сбоям в работе.

Что входит в Observability

Observability делится на несколько ключевых компонентов, каждый из которых предоставляет свою ценную информацию о работе системы:

1. **Метрики** — это агрегированные числовые показатели, которые предоставляют информацию о состоянии системы. Метрики помогают отслеживать такие параметры, как:
   - **Нагрузка** на систему (например, количество запросов в секунду или количество пользователей).
   - **Использование ресурсов** (CPU, память, дисковое пространство).
   - **Производительность** (время отклика, throughput, ошибки).
   - **Ошибки и сбои** (частота и типы ошибок, количество неудачных запросов). Эти данные позволяют не только мониторить систему в реальном времени, но и делать прогнозы на основе исторической информации.

2. **Логи** — это структурированные записи о событиях в приложении. Логи дают точную информацию о том, что происходило внутри системы в момент обработки запроса или выполнения задачи. Логи могут содержать:
   - Детали запросов, их параметры и результаты.
   - Ошибки и исключения, произошедшие во время работы системы.
   - Статус операций (например, успешные и неуспешные транзакции). Логи особенно полезны для отладки и анализа инцидентов, так как дают точную информацию о событиях, которые привели к сбою или ухудшению производительности.

3. **Трассировки** — это цепочки вызовов между сервисами, которые помогают отслеживать путь запроса через систему. Трассировки предоставляют информацию о том, как данные проходят через различные компоненты системы, какие сервисы взаимодействуют друг с другом и сколько времени уходит на каждый этап обработки. Трассировки полезны для:

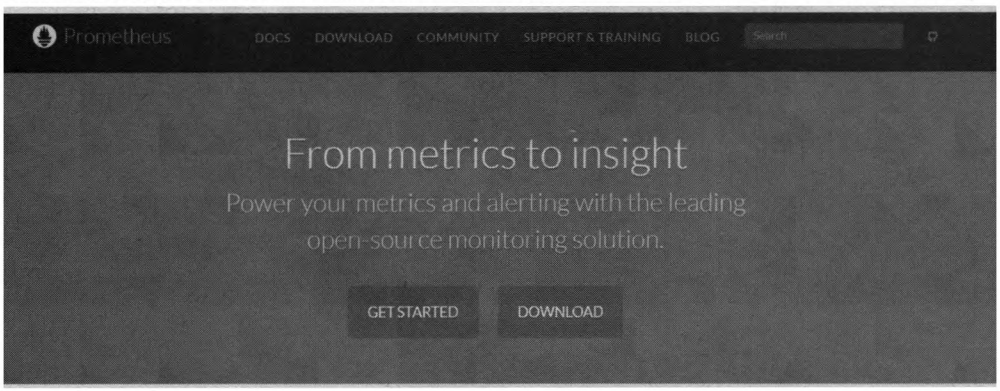
https://max.ru/bookofgeek

- Отслеживания производительности запросов в распределенных системах.
- Выявления узких мест во взаимодействиях между микросервисами.
- Оценки времени ответа на запросы, которые проходят через несколько сервисов.

Метрики как важнейший элемент Observability

В данном материале мы уделим особое внимание **метрикам**. Они являются основным источником количественной информации о работе системы, которая может использоваться для оценки ее производительности, обнаружения аномалий и предсказания возможных проблем. Мы рассмотрим ключевые типы метрик, методы их сбора, а также инструменты, которые помогают эффективно мониторить и анализировать их.

20.2. Prometheus — стандарт де-факто для сбора метрик



Изображение 20.1.

Prometheus — это система мониторинга с временными рядами, изначально созданная в SoundCloud и ныне активно поддерживаемая сообществом CNCF. Prometheus отлично подходит для Java-приложений благодаря поддержке экспортеров и тесной интеграции с экосистемой Spring Boot через Micrometer.

https://max.ru/bookofgeek

Основные возможности Prometheus:

- Хранение метрик в формате временных рядов.
- Гибкий язык запросов PromQL.
- Поддержка алертов.
- Pull-модель сбора метрик (в отличие от push).
- Интеграция с Grafana.

Prometheus по умолчанию ожидает, что приложение будет отдавать метрики по HTTP-интерфейсу в формате */metrics*.

20.3. Micrometer — фасад для метрик в Java

Изображение 20.2.

Micrometer — это библиотека, предназначенная для сбора и экспорта метрик из Java-приложений. Она предоставляет единый API для взаимодействия с различными системами мониторинга: Prometheus, Datadog, New Relic, Graphite и другими.

Micrometer входит в состав Spring Boot Actuator, что делает его отличным выбором для большинства Java-приложений.

https://max.ru/bookofgeek

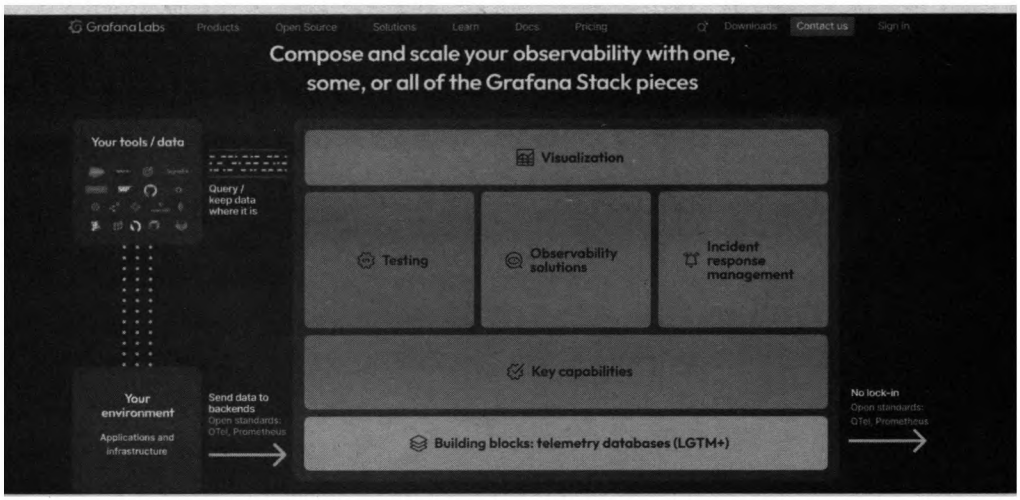
Преимущества Micrometer:

- Единый API, независимый от бэкенда метрик.
- Поддержка таймеров, счетчиков, гистограмм, распределений.
- Интеграция с Spring Boot Actuator.
- Поддержка тегов (labels) для агрегации метрик.

Пример простой метрики:

```
Counter requestCounter = meterRegistry.counter("http.requests",
"method", "GET");
requestCounter.increment();
```

20.4. Grafana — визуализация и дашборды



Изображение 20.3.

Grafana — это мощная система визуализации метрик, которая позволяет создавать дашборды, графики, панели и алерты на основе данных из Prometheus и других источников.

https://max.ru/bookofgeek

Возможности Grafana:

- Динамические дашборды.
- Интеграция с Prometheus, Loki, Jaeger, Elasticsearch и др.
- Уведомления (Slack, Email, Webhooks).
- Поддержка переменных и шаблонов.
- Удобный язык запросов и построения графиков.

20.5. Подключаем все вместе: Prometheus + Micrometer + Grafana

1. Подключаем зависимость Micrometer в *build.gradle:*

```
implementation 'io.micrometer:micrometer-core'
implementation 'io.micrometer:micrometer-registry-prometheus'
```

Для Spring Boot:

```
implementation 'org.springframework.boot:spring-boot-starter-
actuator'
implementation 'io.micrometer:micrometer-registry-prometheus'
```

2. Включаем эндпоинт в *application.yml:*

```
management:
  endpoints:
    web:
      exposure:
        include: prometheus
  metrics:
    export:
      prometheus:
        enabled: true
```

Теперь метрики будут доступны на:

https://max.ru/bookofgeek

http://localhost:8080/actuator/prometheus

3. Настраиваем Prometheus:

```
scrape_configs:
  - job_name: 'my-app'
    metrics_path: '/actuator/prometheus'
    static_configs:
      - targets: ['localhost:8080']
```

4. Создаем дашборд в Grafana и подключаем Prometheus как источник данных.

20.6. Какие метрики собирать?

Сбор метрик — ключевая часть мониторинга производительности Java-приложений. Правильно настроенные метрики позволяют не только отслеживать состояние системы, но и предсказывать возможные проблемы, выявлять узкие места и своевременно реагировать на сбои.

Важно понимать, какие метрики собирать, чтобы получить наиболее полную картину о работе приложения и его окружения. Разделим метрики на три категории: **системные**, **JVM** и **приложенческие**.

Системные метрики

Системные метрики предоставляют информацию о состоянии операционной системы, на которой работает приложение. Эти метрики необходимы для понимания того, как приложение использует ресурсы и как это влияет на его производительность.

https://max.ru/bookofgeek

- **Загрузка CPU**. Эта метрика показывает, какую часть от общего вычислительного ресурса использует приложение. Высокая загрузка CPU может свидетельствовать о проблемах с производительностью, например, из-за неэффективного алгоритма или недостаточной оптимизации. Важно отслеживать не только пиковые значения, но и их стабильность, чтобы понять, является ли использование процессора нормальным для текущей нагрузки.
- **Потребление памяти**. Метрика отображает, сколько памяти (RAM) используется приложением. Переполнение памяти может привести к значительным замедлениям и сбоям. Потребление памяти должно отслеживаться в реальном времени для своевременного обнаружения утечек памяти или недостаточности выделенных ресурсов.
- **GC pause time**. Время пауз, когда происходит сборка мусора (Garbage Collection). Частые или длительные паузы могут сильно замедлять работу приложения, особенно в реальном времени. Эта метрика помогает выявить проблемы с настройками JVM или чрезмерным потреблением памяти.
- **Количество потоков**. Эта метрика показывает, сколько потоков активно используется в процессе выполнения приложения. Важно отслеживать количество потоков, чтобы убедиться, что приложение не создает слишком много потоков, что может привести к проблемам с производительностью и увеличению нагрузки на систему.
- **Открытые дескрипторы**. Дескрипторы файлов и сетевых соединений, открытых приложением. Если приложение не закрывает дескрипторы после использования, это может привести к утечкам и исчерпанию системных ресурсов. Мониторинг количества открытых дескрипторов помогает избежать таких проблем.

JVM метрики

JVM метрики отражают состояние и поведение самого виртуального машины Java. Они дают более глубокое понимание того, как приложение использует ресурсы внутри JVM, а также позволяют выявить потенциальные проблемы с производительностью и управлением памятью.

https://max.ru/bookofgeek

- **jvm.memory.used** — эта метрика показывает общий объем памяти, используемой JVM. Она помогает отслеживать, сколько памяти потребляет приложение и насколько эффективно используются ресурсы. Это важно для понимания того, как JVM управляет памятью и в какой момент может произойти сборка мусора.
- **jvm.gc.pause** — время, затраченное на сборку мусора. Часто бывают проблемы с производительностью, если время на сборку мусора слишком велико. Эта метрика помогает отслеживать паузы в работе приложения, вызванные процессами GC, и позволяет анализировать, нужно ли оптимизировать работу с памятью или изменить настройки JVM.
- **jvm.threads.live** — количество активных потоков, работающих в JVM. Это дает представление о текущем использовании потоков и позволяет выявить проблемы, связанные с чрезмерным количеством потоков, которые могут привести к излишнему использованию системных ресурсов и снижению производительности.
- **jvm.classloader.loaded** — количество классов, загруженных в JVM. Эта метрика полезна для диагностики проблем с загрузкой классов, таких как избыточная загрузка или неправильная конфигурация загрузчика классов, что может влиять на производительность приложения.

Приложенческие метрики

Приложенческие метрики касаются работы самого приложения и его компонентов. Эти метрики помогают отслеживать производительность в контексте выполнения бизнес-логики и взаимодействия с пользователями или другими сервисами.

- **Время отклика API (http.server.requests)** — время, которое приложение тратит на обработку входящих HTTP-запросов. Это критическая метрика для оценки производительности веб-приложений. Время отклика позволяет понять, как быстро приложение реагирует на запросы, и может служить индикатором проблем с производительностью, таких как узкие места в базе данных или медленные внешние API.

https://max.ru/bookofgeek

- **Количество ошибок (http.server.errors)** — количество ошибок, возникающих в процессе обработки HTTP-запросов. Эта метрика помогает быстро выявлять проблемы с доступностью или функциональностью приложения. Например, большое количество ошибок 500 (Internal Server Error) может указывать на проблемы с сервером или внутренними компонентами приложения.

- **Количество входящих сообщений (Kafka, RabbitMQ)** — эта метрика отслеживает количество сообщений, поступающих в очередь сообщений (например, Kafka или RabbitMQ). Важно следить за количеством сообщений, чтобы понять, насколько эффективно обрабатывается очередь, и избежать переполнения или задержек в обработке сообщений.

- **Количество активных пользователей** — эта метрика показывает количество пользователей, которые активно взаимодействуют с приложением в определенный момент времени. Она помогает отслеживать нагрузку на приложение и дает представление о его текущей загрузке. Анализ активности пользователей помогает определять, когда приложение испытывает максимальные нагрузки и как эффективно оно справляется с этим.

20.7. Алерты и автоматическая реакция

Алерты и **автоматическая реакция** — это важнейшие инструменты для мониторинга и поддержания стабильности приложений в реальном времени. С помощью правильной настройки алертов можно оперативно реагировать на проблемы и предотвращать потенциальные сбои или деградацию производительности.

В этом контексте Prometheus и Grafana предоставляют мощные инструменты для настройки уведомлений и автоматических действий при возникновении определенных условий.

Алерты в Prometheus

https://max.ru/bookofgeek

Prometheus предоставляет гибкий механизм для создания алертов с использованием **Alertmanager**, который позволяет настроить уведомления в случае, если показатели системы выходят за пределы нормальных значений. Алерты могут быть настроены на основе различных метрик, таких как время отклика API, нагрузка на CPU, использование памяти и многие другие.

Пример настройки правила алерта в Prometheus:

```
groups:
  - name: application.rules
    rules:
      - alert: HighRequestLatency
        expr: http_server_requests_seconds_sum / http_server_
requests_seconds_count > 0.5
        for: 1m
        labels:
          severity: warning
        annotations:
          summary: "Высокая задержка запросов"
```

Давайте разберем, что происходит в этом примере:

- `alert: HighRequestLatency` — название алерта, которое будет отображаться при активации этого правила.
- `expr: http_server_requests_seconds_sum / http_server_requests_seconds_count > 0.5` — выражение, которое вычисляет среднюю задержку запросов, деля сумму времени выполнения запросов на количество запросов. Если это значение превышает 0.5 секунды, алерт будет срабатывать.
- `for: 1m` — это условие означает, что алерт сработает только в том случае, если превышение этого порога сохраняется в течение 1 минуты.
- `labels: severity: warning` — метка, указывающая на уровень серьезности алерта. В данном случае это предупреждение (warning).
- `annotations: summary: "Высокая задержка запросов"` — аннотация с кратким описанием проблемы, которая будет отображаться в уведомлении.

https://max.ru/bookofgeek

Алерты в Prometheus могут быть настроены не только на мониторинг времени отклика, но и на любую другую метрику, например количество ошибок, загрузку CPU, паузы GC и так далее. После того как алерт сработает, он может быть передан в **Alertmanager**, который, в свою очередь, отправит уведомление через выбранный канал, например в Slack, Email или же в Telegram.

Алерты в Grafana

Grafana предоставляет еще более гибкие возможности для настройки алертов, в том числе через интерфейс пользователя (UI). В Grafana можно настроить оповещения по любым панелям, меткам или графикам, которые отслеживаются в реальном времени. В отличие от Prometheus, где алерты настраиваются через конфигурационные файлы, в Grafana все настройки можно выполнить прямо через веб-интерфейс.

Пример настройки алерта в Grafana

1. **Создание панели**. Для начала нужно создать панель с необходимыми метриками, например с отображением количества ошибок API.
2. **Добавление алерта**.
   - » Перейдите в настройки панели, выберите вкладку **Alert** и нажмите **Create Alert**.
   - » Укажите условие для алерта, например, если количество ошибок API за последние 5 минут превышает 10.
   - » Настройте период времени, за который будет происходить проверка условия (например, 5 минут).
3. **Настройка уведомлений**.
   - » В Grafana можно настроить уведомления на различные каналы. Например, отправка уведомлений через **Telegram**:
     - Для этого потребуется настроить интеграцию с Telegram с помощью Webhook в Grafana.
     - Настроив канал Telegram в Графане, можно указать, чтобы уведомления отправлялись в определенный чат или группу

https://max.ru/bookofgeek

при срабатывании алерта.

4. **Конфигурация каналов оповещения**. В Grafana можно настроить несколько типов уведомлений:
   - **Email** — отправка уведомлений на электронную почту.
   - **Slack** — интеграция с Slack для отправки сообщений в каналы.
   - **Webhook** — возможность отправки данных в сторонние системы или мессенджеры, такие как Telegram или Discord.

5. Настройка оповещений через Webhook может быть использована для отправки уведомлений в любые системы, поддерживающие Webhook, что дает возможность интеграции с широким спектром внешних инструментов.

Автоматическая реакция на алерты

Важно понимать, что алерты не только информируют о проблемах, но и могут служить триггерами для автоматической реакции. Например, можно настроить автоматическое масштабирование сервисов в Kubernetes при срабатывании алерта на высокую нагрузку.

Пример автоматической реакции:

1. **Автоматическое масштабирование в Kubernetes**. Если нагрузка на сервис превышает установленный предел, Kubernetes может автоматически масштабировать количество реплик сервисов. Для этого необходимо настроить **Horizontal Pod Autoscaler (HPA)**.

2. **Автоматический перезапуск сервисов**. В случае обнаружения проблем с сервисом (например, его высокое потребление памяти) можно настроить автоматический перезапуск контейнера с помощью инструмента, такого как **Kubernetes Liveness и Readiness Probes**.

3. **Использование внешних инструментов**. При срабатывании алерта можно настроить автоматическое выполнение скриптов или запуск процессов, например очистку кеша, оптимизацию базы данных или перераспределение нагрузки между узлами.

https://max.ru/bookofgeek

20.8. Observability в Kubernetes

Kubernetes делает задачу мониторинга особенно актуальной — у нас много подов, сервисов, нодов, прокси, ingress-контроллеров.

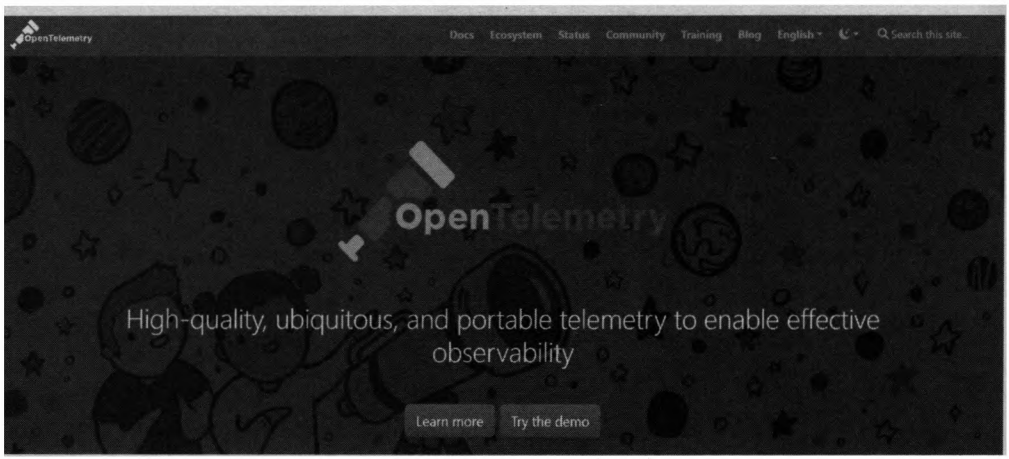
Что важно мониторить:

- Поды (ресурсы, рестарты).
- CPU/Memory Usage.
- Количество реплик.
- Latency между сервисами.

Подключение Prometheus и Grafana в кластер: используется Prometheus Operator или kube-prometheus-stack Helm-чарт.

```
helm install kube-prometheus-stack prometheus-community/kube-
prometheus-stack
```

Расширение: OpenTelemetry и трассировки



Изображение 20.4.

https://max.ru/bookofgeek

Для тех, кто хочет не только собирать метрики, но и интегрировать их с более глубоким анализом поведения системы, например с трассировками, **OpenTelemetry** представляет собой мощное решение, которое стоит рассмотреть.

Это открытый стандарт, который помогает собирать, обрабатывать и экспортировать данные наблюдаемости, такие как метрики, логи и трассировки, из различных сервисов и компонентов системы.

Что такое OpenTelemetry?

OpenTelemetry — это проект с открытым исходным кодом, предоставляющий набор API, библиотек, инструментов и интеграций для сбора и обработки данных наблюдаемости.

Он объединяет в себе функционал для сбора:

- **Метрик** — для мониторинга производительности и состояния системы.
- **Логов** — для анализа событий и ошибок.
- **Трассировок** — для отслеживания запросов через распределенные системы и анализа задержек.

Одной из основных целей OpenTelemetry является создание универсального подхода к сбору данных из различных источников и их унификация. Благодаря этому разработчики и операционные команды могут получать полную картину о состоянии системы, используя стандартизированные инструменты и форматы.

Трассировки в OpenTelemetry

Трассировка представляет собой метод отслеживания пути выполнения запроса через систему, от начальной точки до конечной, и получения информации о времени, затраченном на каждом этапе.

https://max.ru/bookofgeek

Это особенно важно в распределенных системах, таких как микросервисы, где запрос может проходить через несколько сервисов, баз данных и внешних API.

С помощью OpenTelemetry можно собирать трассировки, которые содержат информацию о каждом шаге обработки запроса, времени, затраченном на каждый этап, а также информацию о взаимодействиях между компонентами системы.

Эти данные позволяют:

- Выявлять узкие места и элементы, которые замедляют работу системы.
- Анализировать взаимодействие между сервисами, устраняя проблемы с производительностью.
- Понимать, как различные компоненты взаимодействуют друг с другом, и выявлять проблемы на стыке этих взаимодействий.

OpenTelemetry предоставляет стандартные API и инструменты для реализации трассировки в вашем приложении, а также интеграцию с множеством популярных инструментов для визуализации и анализа этих данных (например, **Jaeger**, **Zipkin** и **Grafana Tempo**).

Интеграция OpenTelemetry с Micrometer

Для пользователей **Micrometer** (широко используемой библиотеки для сбора метрик в экосистеме Java) открывается возможность расширить возможности мониторинга, интегрировав метрики с трассировками.

Micrometer работает над интеграцией с **OpenTelemetry Metrics API**, что позволяет объединить два ключевых аспекта наблюдаемости — метрики и трассировки — в единую систему.

Это интеграционное решение позволяет:

- Собирать метрики и трассировки с помощью единого API и отправлять их в различные системы хранения и визуализации данных (например, Prometheus, Grafana или OpenTelemetry Collector).

https://max.ru/bookofgeek

- Взаимодействовать с существующими инфраструктурами мониторинга и трассировки без необходимости переписывать код или использовать отдельные инструменты для каждого типа данных.
- Получать более полную информацию о работе приложения: например, вы можете анализировать производительность с точки зрения метрик (время отклика, нагрузка), а также детально отслеживать время, затраченное на каждый этап обработки запроса через трассировки.

Преимущества интеграции OpenTelemetry

Интеграция OpenTelemetry с Micrometer и другими инструментами имеет несколько важных преимуществ:

1. **Унифицированный подход**. Вместо того, чтобы использовать несколько разных инструментов для сбора метрик и трассировок, вы получаете единое решение для мониторинга системы.
2. **Гибкость**. OpenTelemetry поддерживает множество форматов данных и интеграций, что позволяет легко настроить решение под вашу инфраструктуру и требования.
3. **Совместимость с современными инструментами**. OpenTelemetry работает с большинством популярных инструментов для визуализации и анализа данных, таких как Jaeger, Zipkin, Prometheus и другие.
4. **Упрощение процессов анализа**. С использованием трассировок и метрик в едином решении можно быстрее находить и устранять проблемы с производительностью, а также выявлять узкие места в распределенных системах.

20.9. Best practices

- Старайтесь не создавать слишком много меток — это ведет к взрыву кардинальности.
- Используйте Gauge с осторожностью — особенно для нестабильных значений.

https://max.ru/bookofgeek

- Сохраняйте метрики стабильными — не переименовывайте без необходимости.
- Группируйте метрики по бизнес-смыслу: *order.service.requests, inventory.cache.hit* и т.д.
- Делайте автоматическое тестирование метрик в CI (наличие ключевых метрик, отсутствие сбоев экспорта).

Выводы

Метрики — это основа для принятия технических решений. Без измерений нет улучшений. Инструменты вроде Prometheus, Micrometer и Grafana позволяют вам не просто наблюдать за системой, а активно влиять на ее поведение, замечая отклонения на ранней стадии.

Независимо от того, создаете ли вы микросервис или монолит, запускаете ли приложение локально или в Kubernetes, — встроенный мониторинг и наблюдаемость должны быть обязательной частью вашей архитектуры.

https://max.ru/bookofgeek

Глава 21.

Кеширование и его влияние

https://max.ru/bookofgeek

О чём глава:

- Когда и как кешировать.
- Виды кешей (локальные, распределенные).
- Использование Ehcache, Caffeine, Redis.
- Оптимизация hit/miss rate.

21.1. Когда и как кешировать

Кеширование — это один из самых эффективных способов оптимизации производительности приложений.

Кеширование позволяет уменьшить нагрузку на ресурсоемкие операции, такие как запросы к базе данных, сетевые вызовы или дорогостоящие вычисления, за счет хранения результатов этих операций для последующего быстрого доступа.

Основные случаи, когда кеширование особенно полезно:

- **Часто запрашиваемые данные**. Например, конфигурационные параметры, пользовательские профили, справочники.
- **Медленные ресурсы**. Если источник данных работает медленно или имеет ограниченную пропускную способность (например, внешние API).

https://max.ru/bookofgeek

- **Дорогостоящие вычисления**. Результаты сложных расчетов, которые не меняются часто, лучше кешировать.

Важно понимать, что кеш — это компромисс между **свежестью данных** и **скоростью доступа**. Поэтому необходимо оценивать, какие данные действительно можно кешировать, а какие должны быть получены в режиме реального времени.

21.2. Виды кешей

Существует несколько подходов к кешированию в зависимости от архитектуры приложения и характера нагрузки.

Локальные кеши

Локальные кеши хранятся в памяти того же процесса, где работает приложение. Это обеспечивает крайне быстрый доступ к данным.

- **Плюсы**: высокая скорость, отсутствие сетевой задержки.
- **Минусы**: ограничены объемом памяти, недоступны другим инстансам приложения.

Примеры библиотек: **Caffeine**, **Ehcache (в embedded режиме)**.

Распределенные кеши

Распределенные кеши используются в кластерах или распределенных системах, где требуется единый источник кешированных данных для множества инстансов.

- **Плюсы**: общий доступ к данным, масштабируемость.

https://max.ru/bookofgeek

- **Минусы**: сетевые задержки, необходимость обеспечения согласованности.

Популярные решения: **Redis**, **Hazelcast**, **Apache Ignite**.

Выбор между локальным и распределенным кешем зависит от характера приложения. Если система запускается на одном узле и не требует общей памяти — подойдет локальный кеш. В противном случае стоит использовать распределенное решение.

21.3. Использование Ehcache, Caffeine, Redis

Ehcache

Ehcache — одна из старейших и наиболее зрелых Java-библиотек для кеширования. Позволяет настраивать TTL, максимальный размер, политику вытеснения и поддерживает как локальный, так и распределенный режим (в связке с Terracotta).

```
CacheManager cacheManager = CacheManagerBuilder.
newCacheManagerBuilder()
    .withCache("preConfigured",
        CacheConfigurationBuilder.newCacheConfigurationBuilder
(Long.class, String.class, ResourcePoolsBuilder.heap(100)))
    .build();
cacheManager.init();
```

Caffeine

Caffeine — высокопроизводительная библиотека локального кеширования от Google. Она предоставляет современные политики вытеснения (например, на основе частоты доступа), асинхронные загрузчики и поддержку JCache API.

https://max.ru/bookofgeek

```
Cache<String, Object> cache = Caffeine.newBuilder()
    .maximumSize(10_000)
    .expireAfterWrite(10, TimeUnit.MINUTES)
    .build();
```

Redis

Redis — in-memory база данных, часто используемая как распределенный кеш. Она подходит для масштабируемых систем и предоставляет широкие возможности: TTL, списки, множества, сериализация, pub/sub и т. д.

С помощью Spring Boot легко интегрировать Redis:

```
@Bean
public RedisTemplate<String, Object> redisTemplate(RedisConnection
Factory factory) {
    RedisTemplate<String, Object> template = new RedisTemplate<>();
    template.setConnectionFactory(factory);
    return template;
}
```

21.4. Оптимизация hit/miss rate

Hit rate — это доля запросов, при которых нужные данные уже были в кеше. Чем выше **hit rate**, тем эффективнее работает кеш. Противоположный показатель — **miss rate** — указывает, сколько раз данные пришлось получать из исходного источника.

Методы повышения **hit rate**:

- **Выбор подходящих ключей**. Ключ должен быть уникальным, однозначно идентифицировать запрашиваемый ресурс.
- **Предварительное прогревание кеша**. Загрузка популярных данных при старте системы.

https://max.ru/bookofgeek

- **Установка правильного TTL**. Слишком короткий TTL может привести к частым miss, слишком длинный — к устаревшим данным.
- **Анализ запросов**. Часто запрашиваемые ресурсы следует кешировать в первую очередь.

Не стоит стремиться к 100% hit rate — в реальных системах это практически недостижимо. Однако разумная стратегия может довести эффективность до 80–90%, что уже дает значительный прирост производительности.

Выводы

Кеширование — мощный инструмент для оптимизации Java-приложений. Грамотный выбор кеша (локальный или распределенный), продуманное TTL и внимание к hit rate способны многократно ускорить работу системы.

Однако необходимо помнить о потенциальных рисках: устаревшие данные, неочевидные зависимости, гонки и необходимость синхронизации при распределенном подходе. Кеш — это не серебряная пуля, а инструмент, которым нужно пользоваться с пониманием и осторожностью.

https://max.ru/bookofgeek

Глава 22.

Работа с большими данными

https://max.ru/bookofgeek

О чём глава:

- Java и big data: Spark, Flink.
- Потоковая обработка данных.
- Производительность и ресурсоемкость.

22.1. Java и Big Data: Введение в Spark и Flink

Обработка больших данных (Big Data) стала неотъемлемой частью современных приложений — от анализа пользовательского поведения до обработки событий в реальном времени.

Язык Java, благодаря своей зрелости, масштабируемости и богатому экосистемному окружению, широко используется в инструментах и фреймворках для Big Data. Среди наиболее популярных решений — Apache Spark и Apache Flink.

Apache Spark

Apache Spark — распределенный фреймворк для обработки больших объемов данных, ориентированный на скорость и удобство использования.

https://max.ru/bookofgeek

Он поддерживает как пакетную (batch), так и потоковую (streaming) обработку данных. Ядро Spark написано на Scala, но предоставляет высокоуровневые API для Java.

Преимущества Java при работе с Spark:

- Статическая типизация и ранняя проверка ошибок.
- Простота интеграции с существующими Java-приложениями.
- Доступ к JVM-инструментам для мониторинга и оптимизации.

Пример использования Spark с Java:

```
SparkConf conf = new SparkConf().setAppName("BigDataApp").
setMaster("local");
JavaSparkContext sc = new JavaSparkContext(conf);
JavaRDD<String> data = sc.textFile("/data/input.txt");
JavaRDD<String> filtered = data.filter(s -> s.contains("error"));
filtered.saveAsTextFile("/data/errors");
```

Apache Flink

Apache Flink — это платформа для распределенной потоковой обработки данных с минимальной задержкой. Она ориентирована на реальные бизнес-сценарии, где важна скорость реакции на события.

Преимущества Java при работе с Flink:

- Легкая интеграция с enterprise-решениями.
- Высокая производительность при правильной настройке JVM и сборщиков мусора.
- Наличие богатой документации и поддержки сообщества.

Пример простого Flink-приложения:

https://max.ru/bookofgeek

```
StreamExecutionEnvironment env = StreamExecutionEnvironment.
getExecutionEnvironment();
DataStream<String> stream = env.readTextFile("input.txt");
stream.filter(value -> value.contains("success"))
      .writeAsText("output.txt");
env.execute("Flink Success Filter");
```

22.2. Потоковая обработка данных

Потоковая обработка (stream processing) — это технология анализа данных в режиме реального времени. В отличие от пакетной обработки (batch processing), потоковая обработка позволяет работать с данными по мере их поступления. Это особенно важно для бизнес-приложений, требующих немедленного реагирования на события.

Системы, такие как Apache Kafka (в связке с Spark Streaming или Flink), позволяют эффективно обрабатывать данные, поступающие от IoT-устройств, логов, пользовательских действий и т.д. Java является основным языком для построения таких решений, особенно благодаря библиотекам, таким как Kafka Streams.

Ключевые принципы эффективной потоковой обработки:

- Минимизация задержки (latency) обработки.
- Гарантия доставки данных (exactly-once, at-least-once).
- Горизонтальное масштабирование через кластеризацию.

Хранение и обработка данных в HDFS и S3

Хранилища, такие как HDFS (Hadoop Distributed File System) и S3 (Simple Storage Service от AWS), играют ключевую роль в Big Data-сценариях. Java-приложения могут напрямую взаимодействовать с этими хранилищами через Hadoop API или SDK от облачных провайдеров.

https://max.ru/bookofgeek

Особенности оптимизации:

- Использование Parquet/ORC-форматов вместо CSV/JSON.
- Оптимизация партиционирования и bucket-стратегий.
- Использование комбинированного чтения (*CombineFileInputFormat*).

Использование Kafka вместе со Spark/Flink

Kafka — это распределенная система обмена сообщениями, идеально подходящая для обработки больших потоков данных. Java-программы могут использовать Kafka Producer/Consumer API для быстрой интеграции с системами аналитики.

Сценарии использования:

- Реализация ETL-пайплайнов.
- Интеграция с ClickHouse, Druid и другими OLAP-хранилищами.
- Многопоточность и backpressure control в обработчиках событий.

Практики мониторинга и отладки Big Data-приложений

Работа с большими данными требует постоянного мониторинга, так как даже незначительные ошибки могут привести к огромным затратам ресурсов или потере данных.

Инструменты:

- Prometheus + Grafana для сбора метрик Spark/Flink.
- Ganglia, JMX и Metrics API.
- Логирование с помощью Fluent Bit, Elasticsearch, Kibana.

Советы по отладке:

https://max.ru/bookofgeek

- Использовать sampling для логирования больших объемов.
- Разбивать pipeline на модули для поэтапной валидации.
- Профилировать GC и heap-состояние на ранней стадии.

Интеграция с системами машинного обучения

Big Data-приложения часто интегрируются с ML-пайплайнами. Java может использовать библиотеки, такие как:

- Apache Mahout.
- DeepLearning4J.
- TensorFlow Java API.

Интеграция происходит путем подготовки данных в Spark/Flink и передачи их в модель либо через REST-интерфейсы, либо через gRPC.

Примеры:

- Предсказание спроса в реальном времени.
- Аномалия-детекторы для логов.
- Обогащение потоков данных метками или вероятностными признаками.

22.3. Производительность и ресурсоемкость

При работе с большими данными особенно важно учитывать:

1. **Управление памятью**.

   Работа с большими объемами данных требует разумного управления памятью. Стоит использовать профилировщики памяти, чтобы выявлять утечки и лишние аллокации.

   Советы:

https://max.ru/bookofgeek

- Используйте *-Xmx, -Xms* и *-XX:+UseG1GC* для настройки JVM.
- Избегайте создания большого числа временных объектов.

2. **Выбор сборщика мусора.**

   Для больших данных особенно актуальны G1GC и ZGC, так как они минимизируют паузы и работают эффективно при больших хипах.

3. **Сетевые и дисковые I/O.**

   Большие данные часто хранятся в распределенных файловых системах (HDFS, S3), и важно оптимизировать работу с сетью и дисками:

   - Используйте буферизацию.
   - Параллелизируйте операции.

4. **Распараллеливание и масштабируемость.**

   Big Data-фреймворки позволяют эффективно масштабироваться, для чего используются:

   - Распределение данных по узлам кластера.
   - Обработка в несколько потоков или процессов.
   - Балансировка нагрузки.

Выводы

Java прекрасно подходит для работы с большими данными благодаря зрелой экосистеме, производительности и интеграции с современными инструментами.

Spark и **Flink** — мощные фреймворки, с которыми можно строить масштабируемые, устойчивые и производительные системы обработки данных.

https://max.ru/bookofgeek

Однако важно помнить о правильной настройке JVM, выборе подходящих алгоритмов и профилировании кода — это ключевые аспекты успешной работы в области Big Data.

https://max.ru/bookofgeek

Глава 23.

Асинхронное и реактивное программирование

https://max.ru/bookofgeek

О чём глава:

- Асинхронность и реактивность: основные понятия.
- Reactor, RxJava и Project Loom.
- Производительность реактивных приложений.
- JMH и реактивный код.

Современные Java-приложения все чаще работают в условиях высокой нагрузки, требуют обработки большого количества запросов, событий и потоков данных в реальном времени.

В таких условиях традиционные синхронные подходы начинают демонстрировать свои ограничения. Асинхронное и реактивное программирование предоставляет мощный инструментарий для создания производительных, масштабируемых и отзывчивых приложений.

23.1. Асинхронность и реактивность: основные понятия

Прежде чем углубляться в конкретные библиотеки и подходы, важно четко понимать разницу между асинхронным и реактивным программированием.

https://max.ru/bookofgeek

Асинхронность

Асинхронное программирование — это стиль, при котором выполнение операций не блокирует поток выполнения.

Это достигается за счет использования **обратных вызовов** (callback), Future/CompletableFuture и неблокирующих API. Асинхронность позволяет запускать задачи параллельно, используя меньше потоков, и в целом повышает эффективность использования ресурсов.

Пример на *CompletableFuture:*

```
CompletableFuture.supplyAsync(() -> loadData())
    .thenApply(data -> process(data))
    .thenAccept(result -> save(result));
```

Асинхронность особенно полезна в приложениях, где преобладают операции ввода-вывода: сетевые вызовы, обращения к базе данных и т.п.

Реактивность

Реактивное программирование базируется на асинхронности, но добавляет семантику потоков данных.

Оно строится вокруг идеи **реактивного потока** — модели, в которой подписчик подписывается на поток данных и получает уведомления по мере их появления. Все это сопровождается управлением обратным давлением (backpressure), что критично для контроля нагрузки.

В Java эта модель формализована в виде спецификации Reactive Streams (https://www.reactive-streams.org/), поддерживаемой библиотеками Reactor, RxJava, Akka Streams и другими.

https://max.ru/bookofgeek

23.2. Reactor, RxJava и Project Loom

RxJava

RxJava — это Java-порт ReactiveX. Она предоставляет API для работы с потоками событий. RxJava активно использовалась до появления Reactor и остается популярной в Android-разработке.

RxJava API строится на типах *Observable, Single, Flowable, Completable.*

Пример:

```
Observable.fromArray(1, 2, 3, 4)
    .map(i -> i * 2)
    .subscribe(System.out::println);
```

RxJava предоставляет обширный набор операторов (map, filter, merge, zip и т.д.), но ее API может быть сложным для начинающих, особенно в части backpressure и многопоточности.

Project Reactor

Reactor — библиотека от Spring, построенная на спецификации Reactive Streams. Она лежит в основе Spring WebFlux.

Основные типы:

- *Mono<T>* — 0 или 1 элемент.
- *Flux<T>* — 0 или N элементов.

Пример:

```
Flux.just("A", "B", "C")
    .map(String::toLowerCase)
    .subscribe(System.out::println);
```


https://max.ru/bookofgeek

Reactor имеет лаконичный и предсказуемый API, тесно интегрирован со Spring 5+, поддерживает backpressure "из коробки" и идеально подходит для серверной разработки.

Project Loom

Project Loom — это новая парадигма, которая меняет правила игры. Он предлагает **виртуальные потоки** — легкие потоки, которые управляются JVM, а не операционной системой.

Преимущества:

- Код остается синхронным по стилю, но не блокирует потоки.
- Нет необходимости в использовании callback'ов или *CompletableFuture.*
- Масштабируемость до миллионов параллельных задач.

Пример:

```
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    List<Future<String>> results = IntStream.range(0, 10_000)
        .mapToObj(i -> executor.submit(() -> getData(i)))
        .toList();
}
```

В отличие от реактивных библиотек, Loom позволяет писать читаемый и поддерживаемый код без потери производительности.

Однако, несмотря на простоту, Loom не отменяет необходимость грамотного проектирования и анализа.

23.3. Производительность реактивных приложений

Реактивные системы позволяют обрабатывать миллионы событий в секунду, не создавая тысячи потоков.

https://max.ru/bookofgeek

Это делает их особенно привлекательными для микросервисов, систем реального времени, web-приложений, работающих под высокой нагрузкой.

Мифы и реальность

- **Миф**: реактивный код всегда быстрее.

  Реальность: реактивность повышает масштабируемость, но не всегда повышает скорость.

- **Миф**: реактивность экономит ресурсы.

  Реальность: да, если вы много работаете с I/O. При CPU-интенсивных задачах выигрыш минимален.

- **Миф**: Project Loom убьет Reactor.

  Реальность: Loom и реактивные библиотеки решают разные задачи и могут сосуществовать.

Где реактивность дает выигрыш:

- Веб-серверы, обрабатывающие тысячи одновременных соединений.
- Proxy и API Gateway.
- Интеграционные слои между сервисами (например, GraphQL-шлюзы).
- Многопоточные обработчики событий, например Kafka consumers.

23.4. JMH и реактивный код

Бенчмаркинг реактивного кода — нетривиальная задача. Большинство реактивных API асинхронны, что нарушает синхронную природу JMH. Однако есть способы измерить производительность реактивного потока с помощью вспомогательных конструкций.

https://max.ru/bookofgeek

Общие принципы

- Используйте *CountDownLatch* или *StepVerifier* для ожидания завершения.
- Избегайте внутренних *Thread.sleep* — они портят измерения.
- Сравнивайте не только время выполнения, но и потребление ресурсов.

Пример: бенчмарк RxJava против Project Loom

```
@Benchmark
public void rxJavaBench() throws InterruptedException {
    CountDownLatch latch = new CountDownLatch(1);
    Observable.range(1, 1000)
        .map(i -> i * 2)
        .observeOn(Schedulers.computation())
        .doOnComplete(latch::countDown)
        .subscribe();
    latch.await();
}

@Benchmark
public void loomBench() throws Exception {
    try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
        List<Future<Integer>> list = IntStream.range(0, 1000)
            .mapToObj(i -> executor.submit(() -> i * 2))
            .toList();
        for (Future<Integer> future : list) {
            future.get();
        }
    }
}
```

Такие тесты дают понимание не только "что быстрее", но и насколько реалистичен подход для ваших условий.

https://max.ru/bookofgeek

Таблица 24.1. Сравнение подходов

Критерий	RxJava / Reactor	Project Loom
Стиль кода	Декларативный, потоковый	Императивный, читаемый
Обратное давление	Да	Нет (пока что)
Поддержка Spring	Отличная (WebFlux)	В процессе
Производительность	Отличная на I/O	Отличная на CPU + I/O
Время изучения	Среднее/высокое	Низкое
Отладка	Сложно	Просто

Практические советы

- Не переходите на реактивность только ради моды. Взвесьте плюсы и минусы.
- Используйте реактивность для обработки событий, I/O и API-шлюзов.
- Для простых сервисов может хватить *Loom* или *CompletableFuture*.
- Не забывайте о мониторинге потоков, пауз GC и локациях памяти.
- Измеряйте реальную нагрузку — симуляции редко дают точную картину.

Выводы

Асинхронное и **реактивное программирование** — это мощный инструмент, но не универсальное решение. Оно требует другого подхода к архитектуре, обработки ошибок и тестированию. Важно понимать, когда эти подходы уместны, и использовать их осознанно.

https://max.ru/bookofgeek

Project Loom открывает новую эру многопоточности, позволяя писать читаемый код с производительностью реактивных систем. Но и у реактивных библиотек, таких как Reactor или RxJava, остается своя ниша, особенно там, где требуется строгий контроль над потоками данных.

Главное — подходить к архитектуре с пониманием целей, ограничений и возможностей платформы Java. И не забывайте: даже реактивный код нужно профилировать, тестировать и оптимизировать.

https://max.ru/bookofgeek

https://max.ru/bookofgeek

Ещё больше свежей и актуальной литературы по Java
ты найдёшь в наших каналах:

https://max.ru/bookofgeek
https://t.me/bookofgeek

Глава 24.

Контейнеризация и производительность

https://max.ru/bookofgeek

О чём глава:

- Оптимизация Java в контейнерах.
- Настройки JVM при ограниченных ресурсах.
- Мониторинг в Kubernetes.

Контейнеризация изменила способ разработки, доставки и эксплуатации приложений. Вместе с преимуществами в виде изоляции, воспроизводимости и масштабируемости пришли и новые вызовы.

Java-приложения, будучи чувствительными к ресурсам, требуют особого подхода при запуске в контейнерах, таких как Docker и Kubernetes.

В этой главе мы рассмотрим, как правильно оптимизировать Java-приложения для запуска в контейнеризированной среде, какие настройки JVM следует учитывать и как осуществлять мониторинг производительности.

24.1. Оптимизация Java в контейнерах

Контейнер — это легковесная виртуализированная среда, в которой приложение работает изолированно от основной системы.

https://max.ru/bookofgeek

Java-приложения в контейнерах сталкиваются с несколькими особенностями:

- **Ограниченность ресурсов**. По умолчанию JVM может "видеть" все доступные ресурсы хост-машины, даже если контейнеру выделено меньше.
- **Изоляция процесса**. Взаимодействие с хостом затруднено, что усложняет отладку и профилирование.
- **Новая модель поставки**. Необходимость минимизировать размер образов и время запуска.

Лучшие практики оптимизации Java в контейнерах

- **Выбор базового образа.**

  Используйте образы с минимальным footprint'ом. Например:

  » *openjdk:17-slim* — оптимизированный образ без лишних пакетов.

  » *distroless* — образы только с необходимыми библиотеками, без шелла.

- **Минимизация размера образа.**

  Удаляйте временные файлы, используйте многоступенчатую сборку (multi-stage build) для разделения сборки и запуска.

  Пример *Dockerfile*:

```
FROM maven:3.9.3-eclipse-temurin-17 AS build
COPY. /app
WORKDIR /app
RUN mvn clean package -DskipTests

FROM openjdk:17-slim
COPY --from=build /app/target/my-app.jar /my-app.jar
ENTRYPOINT ["java", "-jar", "/my-app.jar"]
```

- **Анализ накладных расходов.**

  Измеряйте стартовое время, потребление памяти и поведение GC отдельно для контейнера. Это поможет выявить "горячие точки".

https://max.ru/bookofgeek

- **Слежение за версиями JDK.**

  Современные JDK (11+) лучше интегрированы с контейнерами. Поддержка *cgroups* позволяет JVM автоматически учитывать ограничения памяти и CPU.

- **Настройка метрик и логов.**

  Интеграция с системами мониторинга (Prometheus, Grafana) и логирования (ELK, Loki) должна быть учтена на этапе проектирования контейнера.

24.2. Настройки JVM при ограниченных ресурсах

Когда приложение запускается в контейнере с ограничениями по памяти и CPU, JVM должна быть проинформирована об этих ограничениях. Ранние версии JVM игнорировали *cgroup*, но начиная с Java 10 ситуация изменилась.

Основные настройки JVM:

- **-XX:+UseContainerSupport (по умолчанию в Java 10+).**

  Включает поддержку контейнеров (учет cgroup-ограничений).

- **Ограничение памяти.**

  JVM может использовать только часть выделенной контейнеру памяти, поэтому имеет смысл явно задавать *-Xmx* и *-Xms*.

  Пример:

```
java -Xms512m -Xmx1024m -XX:MaxRAMPercentage=75.0 -jar app.jar
```

Здесь *MaxRAMPercentage* означает, что JVM может использовать до 75% доступной памяти контейнера.

https://max.ru/bookofgeek

- **Настройки GC.**

  Для контейнеров с малым количеством CPU или памяти, более предпочтительны:

  » **Serial GC** — если приложение однопоточное.
  » **G1 GC** — сбалансированный для разных нагрузок.
  » **ZGC или Shenandoah** — для приложений с низкими паузами (в Java 11+).

- **Число потоков компилятора JIT.**

  Контейнеру может быть выделено всего 1 или 2 CPU. В таких случаях JIT-компилятор должен использовать меньше потоков:

```
-XX:CICompilerCount=2
```

- **Отключение лишних сервисов.**

  Например:

```
-XX:+DisableAttachMechanism
-XX:+ExitOnOutOfMemoryError
```

- **Вывод информации о конфигурации контейнера.**

  Проверка видимости ресурсов:

```
java -XX:+PrintFlagsFinal -version | grep -iE 'InitialHeapSize|MaxHeapSize'
```

  Практический пример запуска:

```
java \
 -XX:+UseContainerSupport \
 -XX:MaxRAMPercentage=70.0 \
 -XX:InitialRAMPercentage=50.0 \
 -XX:+UseG1GC \
 -XX:MaxGCPauseMillis=200 \
 -Xlog:gc* \
 -jar app.jar
```


https://max.ru/bookofgeek

Этот запуск гарантирует, что приложение учитывает ограничения контейнера, а GC настроен под низкие задержки.

24.3. Мониторинг в Kubernetes

Контейнеризация редко используется в изоляции — в большинстве случаев приложения разворачиваются в Kubernetes. Мониторинг становится критически важным как для отладки, так и для масштабирования.

Основные компоненты мониторинга Java в Kubernetes

- **Prometheus + Grafana**.

  Для сбора и визуализации метрик. Используйте экспортеры:

  » *jmx_exporter* — подключается к JVM и преобразует метрики в формат Prometheus.

  » *micrometer* — абстрактный уровень для сбора метрик, интегрируется с Spring Boot.

  Пример настройки *jmx_exporter:*

```
java -javaagent:/opt/jmx_prometheus_javaagent.jar=12345:/config.yaml
-jar app.jar
```

- **Liveness и Readiness probes.**

  Kubernetes позволяет следить за "здоровьем" контейнера (yaml):

```
readinessProbe:
  httpGet:
    path: /health
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 10
```


https://max.ru/bookofgeek

- **Метрики контейнера.**

  Используются *kubelet, cAdvisor, metrics-server:*

  » Память, CPU, дисковая активность.

  » Частота перезапусков.

  » Утилизация пода.

- **Интеграция с OpenTelemetry.**

  Современные подходы объединяют метрики, логи и трейсинг. Вы можете видеть полный путь запроса, включая задержки GC, SQL-запросов и т.д.

- **Sidecar-контейнеры для мониторинга.**

  В Pod можно добавить sidecar, например с Promtail (для логов) или агентом Datadog/Influx.

Выводы

Контейнеризация — не просто способ упаковки приложения, но полноценная среда выполнения. Java-приложения, особенно долгоживущие и высоконагруженные, требуют осознанного подхода к запуску в контейнерах.

Важно:

- Правильно выбрать базовый образ.
- Настроить JVM с учетом *cgroup*-ограничений.
- Оптимизировать сборщик мусора.
- Следить за поведением приложения с помощью современных средств мониторинга.

https://max.ru/bookofgeek

Пренебрежение этими аспектами может привести к неочевидным сбоям, утечкам памяти, высоким задержкам и неэффективному использованию ресурсов. С другой стороны, грамотный подход к контейнеризации делает Java-приложение более масштабируемым, переносимым и управляемым.

https://max.ru/bookofgeek

Глава 25.

CI/CD и автоматизированный контроль производительности

https://max.ru/bookofgeek

О чём глава:

- JMH и нагрузочные тесты в CI.
- Метрики и алерты в CI/CD-пайплайнах.
- GitHub Actions, Jenkins, GitLab CI для контроля деградации.
- Интеграция нагрузочного тестирования.

Современная разработка программного обеспечения невозможна без непрерывной интеграции (Continuous Integration, CI) и непрерывного развертывания (Continuous Deployment, CD). В рамках этих практик особое место занимает автоматизированный контроль производительности, позволяющий не только обеспечивать функциональность приложения, но и гарантировать его быстродействие в условиях роста нагрузки и изменений в коде.

Производительность может деградировать от коммита к коммиту — иногда из-за неудачного алгоритма, иногда из-за неочевидной зависимости. Задача команды — вовремя заметить такие изменения.

25.1. JMH и нагрузочные тесты в CI

Почему стоит запускать JMH в CI?

JMH (Java Microbenchmark Harness) — это мощный инструмент для создания микробенчмарков, но его сила возрастает при автоматическом запуске.

https://max.ru/bookofgeek

Подключив JMH к пайплайну CI, можно:

- отслеживать производительность конкретных методов от версии к версии;
- выявлять регрессии до попадания кода в прод ;
- собирать историю метрик производительности.

Пример настройки JMH в Maven-проекте

```
<build>
    <plugins>
        <plugin>
            <groupId>org.openjdk.jmh</groupId>
            <artifactId>jmh-maven-plugin</artifactId>
            <version>1.37</version>
            <executions>
                <execution>
                    <goals>
                        <goal>benchmark</goal>
                    </goals>
                </execution>
            </executions>
        </plugin>
    </plugins>
</build>
```

Запуск JMH в GitHub Actions

```
jobs:
  benchmark:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Set up Java
        uses: actions/setup-java@v3
```


https://max.ru/bookofgeek

```
        with:
          java-version: '21'
          distribution: 'temurin'
      - name: Build and run benchmarks
        run: |
          mvn clean install
          java -jar target/benchmarks.jar > jmh-results.txt
      - name: Upload benchmark results
        uses: actions/upload-artifact@v3
        with:
          name: jmh-results
          path: jmh-results.txt
```

Парсинг и анализ результатов

Результаты JMH можно сравнивать с предыдущими запусками, например, храня их в артефактах или интегрируя с внешними системами (InfluxDB, Prometheus).

25.2. Метрики и алерты в CI/CD-пайплайнах

Какие метрики отслеживать?

- **Throughput** — количество операций в секунду.
- **Latency** — время выполнения операции (среднее, медиана, 95-й процентиль).
- **Memory allocation** — сколько памяти потребляется.
- **GC activity** — активность сборщика мусора.

Сравнение с эталонными значениями

https://max.ru/bookofgeek

Подход "fail the build if latency > X ms" позволяет предотвратить попадание деградаций в мастер-ветку.

Для этого удобно использовать shell-скрипты и регулярные выражения, чтобы извлекать значения из JMH-отчетов.

Пример bash-обработки результатов:

```
avg_latency=$(grep 'MyBenchmark' jmh-results.txt | awk '{print $4}')
if (( $(echo "$avg_latency > 5.0" | bc -l) )); then
  echo "Latency too high!"
  exit 1
fi
```

Визуализация метрик

Интеграция с такими инструментами, как Grafana + Prometheus, позволяет:

- отслеживать тренды производительности;
- быстро локализовать регрессии;
- настраивать алерты при превышении порогов.

25.3. GitHub Actions, Jenkins, GitLab CI для контроля деградации

GitHub Actions

Преимущества:

- Простота конфигурации (yaml).
- Поддержка кеширования зависимостей.
- Возможность загрузки артефактов (jmh-results.txt, flamegraph.svg и т.д.).

https://max.ru/bookofgeek

Особенности:

- Бесплатное количество минут ограничено.
- Визуализация требует дополнительных интеграций (например, с InfluxDB).

Jenkins

Преимущества:

- Максимальная гибкость конфигурации.
- Возможность настройки кастомных джобов для запуска JMH, Gatling и т.д.
- Поддержка Groovy-скриптов и параметрических билдов.

Недостатки:

- Требует установки и администрирования.
- Требует настройки плагинов (Performance Plugin, Plot Plugin и др.).

GitLab CI

Преимущества:

- Хорошая интеграция с GitLab репозиториями.
- Поддержка переменных окружения и пайплайнов.
- Возможность триггеров на Merge Request для запуска бенчмарков.

Пример *.gitlab-ci.yml:*

```
benchmark:
  stage: performance
  script:
    - mvn clean install
```


https://max.ru/bookofgeek

```
    - java -jar target/benchmarks.jar > jmh-results.txt
  artifacts:
    paths:
      - jmh-results.txt
```

25.4. Интеграция нагрузочного тестирования

Использование Gatling, k6, Apache JMeter

Нагрузочные тесты позволяют имитировать поведение реальных пользователей.

Запуск таких тестов в CI позволяет:

- выявить узкие места;
- контролировать латентность;
- отслеживать поведение GC и памяти под нагрузкой.

Пример Gatling в Jenkins

1. Установить Gatling CLI.
2. Написать сценарий.
3. Добавить шаг в Jenkins Pipeline:

```
stage('Gatling Performance') {
    steps {
        sh 'gatling.sh -s MySimulation'
    }
}
```

k6 в GitHub Actions:

https://max.ru/bookofgeek

```
- name: Run load test
  uses: grafana/k6-action@v0.2.0
  with:
    filename: test.js
```

Практические советы:

- Не запускайте JMH на перегруженных shared runners — производительность будет нестабильна.
- Фиксируйте конфигурацию окружения: количество ядер, частоту процессора, размер хипа.
- Автоматизируйте анализ результатов: пишите скрипты, которые сравнивают текущие результаты с baseline.
- Выводите ключевые показатели в readme, wiki или dashboard.
- Не ограничивайтесь JMH — используйте и нагрузочные тесты, и профилирование (async-profiler, JFR).
- Обновляйте baseline-метрики разумно — не занижайте пороги только для того, чтобы "все проходило".
- Следите за трендами, а не только за текущим значением.
- Интегрируйте с системами нотификаций — Slack, Teams, Telegram.

Выводы

Контроль производительности в CI/CD-пайплайне — это один из мощнейших способов обеспечить стабильную, отзывчивую и масштабируемую систему. Автоматизация бенчмарков, алертов и анализов позволяет команде быть уверенной, что новый код не "сломает" важные метрики.

https://max.ru/bookofgeek

Сегодня это уже не опциональный этап разработки, а необходимость для профессиональных Java-проектов.

Интегрируйте JMH, нагрузочные тесты, используйте GitHub Actions, Jenkins, GitLab CI — и вы получите не только качественный, но и быстрый продукт.

https://max.ru/bookofgeek

https://max.ru/bookofgeek

Глава 26.

Front-to-Back производительность

https://max.ru/bookofgeek

О чём глава:

- Эффективные форматы передачи данных (JSON, Protobuf, Avro).
- Оптимизация сериализации/десериализации.
- Латентность REST и gRPC.
- Практические рекомендации для снижения задержек и увеличения пропускной способности коммуникационного слоя.

В современном мире распределенных систем и микросервисной архитектуры, производительность не ограничивается только бэкендом или фронтендом.

> Одним из важнейших факторов, влияющих на общую скорость работы приложения, является производительность обмена данными между компонентами.
>
> Эта область часто называется *front-to-back performance* — производительность на всем пути данных: от клиента до сервера и обратно.

https://max.ru/bookofgeek

26.1. Эффективные форматы передачи данных

Формат данных определяет, как информация сериализуется и передается по сети.

Выбор формата критически влияет на:

- размер сообщения;
- скорость сериализации/десериализации;
- совместимость между сервисами;
- читаемость и отладку.

JSON

JSON (JavaScript Object Notation) — текстовый, человекочитаемый формат, который стал стандартом де-факто для обмена данными в веб-приложениях.

Преимущества:

- Простота и читаемость.
- Поддержка во всех языках программирования.
- Подходит для взаимодействия с браузерами.

Недостатки:

- Объем: JSON имеет избыточную структуру (например, дублирование ключей).
- Скорость: парсинг и генерация JSON медленнее бинарных форматов.
- Нет строгой схемы: возможны ошибки в структуре данных.

https://max.ru/bookofgeek

JSON отлично подходит для веб-клиентов, но для высоконагруженных систем часто становится узким местом.

Protobuf

Protocol Buffers (Protobuf) — бинарный формат от Google, ориентированный на компактность и скорость.

Преимущества:

- Очень компактный формат.
- Быстрая сериализация и десериализация.
- Строгая схема, что помогает избежать ошибок.

Недостатки:

- Требуется предварительная генерация кода из .proto-файлов.
- Не человекочитаемый.
- Менее гибкий в отладке.

Protobuf — отличный выбор для межсервисного общения, особенно в микросервисной архитектуре с gRPC.

Avro

Apache Avro — бинарный формат, ориентированный на Big Data и совместимость версий.

Преимущества:

- Содержит схему внутри сообщения — удобен для хранения.
- Поддерживает эволюцию схем.

https://max.ru/bookofgeek

- Эффективен для потоковой обработки (Kafka, Hadoop).

Недостатки:

- Сложнее в интеграции по сравнению с JSON.
- Требует больше усилий на настройку.

Avro — подходящий выбор для систем, где важна совместимость и долговечность хранения.

26.2. Оптимизация сериализации/десериализации

Процессы **сериализации** (преобразования объекта в поток байтов) и **десериализации** (обратного преобразования) являются важными этапами в цепочке обработки запроса. Их эффективность напрямую влияет на пропускную способность и латентность.

Общие рекомендации

1. Избегайте ненужных преобразований — если вы сериализуете объект в JSON, а потом снова парсите в объект, делайте это один раз, а не в каждом слое.
2. Профилируйте сериализацию — используйте JMH и инструменты профилирования (например, YourKit, JFR), чтобы замерить реальные затраты.
3. Не сериализуйте все подряд — ограничьте поля, которые включаются в сериализуемые объекты, исключите служебные и временные поля.
4. Используйте библиотеки с высокой производительностью:
   - Jackson (JSON);
   - Gson (JSON, медленнее Jackson);
   - Protobuf и gRPC (binary);

https://max.ru/bookofgeek

» Kryo (универсальная бинарная сериализация).

Jackson (JSON)

Jackson — одна из самых быстрых и популярных библиотек JSON.

Основные приемы оптимизации:

- использование аннотации *@JsonIgnore* для исключения лишних полей;
- предварительная настройка *ObjectMapper* с отключением ненужных проверок;
- использование *JsonView* или DTO для минимизации передаваемых данных;
- кеширование сериализованных результатов при повторяющихся ответах.

Protobuf

В случае Protobuf оптимизация чаще всего сводится к:

- минимизации вложенности сообщений;
- выбору правильных типов данных (например, *int32* против *int64*);
- избеганию повторяющихся вложенных структур (в пользу flat-схем).

JMH и сериализация

Для понимания реальной стоимости сериализации можно использовать JMH.

Пример сравнения Jackson и Protobuf по времени сериализации:

```
@Benchmark
public byte[] serializeJackson(MyObject obj) throws Exception {
    return objectMapper.writeValueAsBytes(obj);
}
```


https://max.ru/bookofgeek

```
@Benchmark
public byte[] serializeProtobuf(MyProtoObject obj) {
    return obj.toByteArray();
}
```

Разница может составлять десятки миллисекунд на тысячах объектов, что критично при большой нагрузке.

26.3. Латентность REST и gRPC

REST

REST — архитектурный стиль, использующий HTTP-протокол и часто JSON для передачи данных.

Плюсы:

- Простота и читабельность.
- Поддержка браузерами и инструментами (Postman).
- Хорошая документация (Swagger/OpenAPI).

Минусы:

- Высокая латентность из-за HTTP/1.1 (хедеры, размер, TCP-коннекты).
- Нет строгой схемы.
- Больше накладных расходов.

Для API с низкой частотой вызовов и ориентированных на человекочитаемость — REST вполне приемлем.

https://max.ru/bookofgeek

gRPC

gRPC — фреймворк от Google, основанный на Protobuf и HTTP/2.

Преимущества:

- Низкая латентность.
- Поддержка bidirectional streaming.
- Сжатие и мультиплексирование.
- Строгая схема.

Недостатки:

- Сложнее отлаживать.
- Требует генерации кода и поддержки схем.

gRPC особенно полезен при:

- высокочастотных запросах между сервисами;
- необходимости стриминга;
- критичных к задержке приложениях.

Таблица 26.1. Сравнение REST и gRPC

Характеристика	REST (JSON)	gRPC (Protobuf)
Формат	JSON (текст)	Protobuf (бинарный)
Протокол	HTTP/1.1	HTTP/2
Читаемость	Человекочитаемый	Машиночитаемый
Латентность	Выше	Ниже

https://max.ru/bookofgeek

Документируемость	Простая (Swagger)	Требует *.proto*
Стриминг	Ограниченный	Полноценный
Поддержка браузеров	Отличная	Ограниченная

26.4. Практические рекомендации для снижения задержек и увеличения пропускной способности коммуникационного слоя

1. Выбирайте формат под задачу:
   - » JSON — для внешних API и браузеров;
   - » Protobuf — для микросервисов и внутреннего взаимодействия;
   - » Avro — для хранения и потоковой обработки.
2. Измеряйте и сравнивайте:
   - » используйте JMH и профилировщики, чтобы понять, где узкие места;
   - » профилируйте сериализацию и сетевые задержки.
3. Минимизируйте размер сообщений:
   - » не передавайте ненужные поля;
   - » используйте сжатие (например, gzip или Brotli).
4. Оптимизируйте клиент и сервер, используйте:
   - » кеширование ответов;
   - » асинхронную обработку запросов;
   - » connection pooling.
5. Тестируйте производительность end-to-end:
   - » используйте инструменты вроде Gatling, k6, Jmeter;

https://max.ru/bookofgeek

» имейте эталонные показатели и отслеживайте регрессии.

Выводы

Производительность на пути от клиента до сервера — это результат множества решений: выбор формата данных, оптимизация сериализации, правильный протокол и архитектура API. Не существует универсального решения: только тщательное тестирование и профилирование позволят понять, что работает лучше всего для вашего случая.

Современные приложения требуют баланса между скоростью, гибкостью и читаемостью. Важно не просто выбрать технологию, а правильно ее настроить и использовать с учетом требований проекта. Только так можно добиться высокой производительности без ущерба для стабильности и удобства разработки.

https://max.ru/bookofgeek

Глава 27.

Мобильные и десктопные Java-приложения

https://max.ru/bookofgeek

О чём глава:

- Особенности JVM на Android.
- Оптимизация JavaFX/Swing-приложений.

27.1. Особенности JVM на Android

Java традиционно является основным языком разработки для Android, и, несмотря на растущую популярность Kotlin, значительная часть Android-приложений по-прежнему написана на Java или содержит в себе Java-компоненты.

Однако среда выполнения Java-программ на Android значительно отличается от стандартной JVM. Android использует собственную виртуальную машину — сначала это была Dalvik, затем ART (Android Runtime).

Dalvik и ART

- **Dalvik** была ориентирована на выполнение байт-кода в специальном формате .dex (Dalvik Executable). Программы компилировались в байт-код не JVM, а в .dex-файлы, которые оптимизированы под выполнение на устройствах с ограниченными ресурсами.

https://max.ru/bookofgeek

- **ART** заменил Dalvik, предоставляя Ahead-of-Time (AOT) компиляцию: при установке приложения байт-код преобразуется в машинный код, что ускоряет запуск и снижает нагрузку на CPU при выполнении.

Ограничения и особенности

- Нет полноценной поддержки JVM-специфичных библиотек.
- Ограниченная производительность по сравнению с серверной JVM.
- Контроль за памятью критичен: частые GC могут привести к «фризам».
- Использование Reflection или Heavy IO может вызывать задержки, особенно на слабых устройствах.

Практики оптимизации Android-приложений на Java

- Минимизировать создание объектов во время UI-обновлений.
- Использовать *SparseArray* вместо *HashMap* при работе с *integer*-ключами.
- Использовать *HandlerThread* или *Executor* для фоновых задач, не блокируя UI-поток.
- Использовать профилировщик Android Studio (CPU, Memory, Network).
- Оптимизировать лейауты, избегать глубоких иерархий.
- Применять кеширование изображений и ресурсов (например, через Glide или Picasso).

27.2. Оптимизация JavaFX/Swing-приложений

JavaFX и **Swing** — основные технологии для создания десктопных Java-приложений. Хотя Swing является более старой и менее активно развиваемой библиотекой, она по-прежнему используется в некоторых корпоративных решениях. JavaFX более современная и визуально привлекательная, особенно с поддержкой CSS-стилей, FXML и WebView.

https://max.ru/bookofgeek

Производительность JavaFX

- JavaFX активно использует аппаратное ускорение: отрисовка UI выполняется с помощью GPU.
- Поддерживает многопоточность, но важно помнить, что весь UI должен обновляться только из JavaFX Application Thread.

Практики оптимизации JavaFX

- Использовать *ObservableList*, *Bindings*, *Properties* для реактивных UI.
- Отложенная инициализация: подгружать тяжелые части UI только при необходимости.
- Использовать *Canvas* и низкоуровневую графику при необходимости высокой производительности.
- Кешировать ресурсы (иконки, изображения, стили).
- Минимизировать количество слушателей событий и активно отписываться от неиспользуемых.
- Использовать *Service* и *Task* для асинхронной загрузки данных.

Swing: советы по производительности

- Не блокировать EDT (Event Dispatch Thread): все долгие операции — в отдельные потоки.
- Использовать *SwingWorker* для фона.
- Не перегружать UI-компоненты: избегать вложенных панелей без необходимости.
- Не использовать слишком много прозрачности и сложной графики — это замедляет отрисовку.

Инструменты для профилирования десктопных Java-приложений

https://max.ru/bookofgeek

- VisualVM и JProfiler — для анализа CPU, памяти, потоков.
- Java Mission Control — особенно полезен при использовании OpenJDK или Oracle JDK.
- JavaFX Scene Builder — визуальное проектирование и структурирование UI.

Общие подходы

- Всегда использовать профилирование для нахождения узких мест.
- Строить архитектуру приложения с учетом возможностей платформы: слабые устройства, ограниченные ресурсы, разные разрешения экранов.
- Кешировать данные и использовать lazy loading.
- Учитывать влияние GC на отзывчивость интерфейса.
- Использовать современные подходы к архитектуре (MVVM, MVP) для упрощения поддержки и расширения.

Выводы

Оптимизация мобильных и десктопных приложений требует учета ограничений среды исполнения.

Для Android это особенности ART и ограничения мобильных устройств, для десктопа — сложность рендеринга и отзывчивость интерфейса. Однако современные инструменты, профилировщики и фреймворки позволяют достигать высокой производительности при соблюдении архитектурной дисциплины и следовании best practices. Важно помнить, что хороший пользовательский опыт напрямую зависит от скорости реакции интерфейса, и даже миллисекунды могут иметь значение.

https://max.ru/bookofgeek

https://max.ru/bookofgeek

Заключение

https://max.ru/bookofgeek

В этой книге мы рассмотрели различные методики тестирования и контроля написания кода Java, а также основные подходы и техники оптимизации производительности Java-приложений, начиная с основ и заканчивая продвинутыми методами, которые используются в реальных проектах.

Все эти темы: от правильного использования многопоточности до профилирования и оптимизации работы с памятью и I/O помогают создать высокопроизводительные, масштабируемые и стабильные системы.

Итоговые рекомендации

> **Оптимизация** — это не разовая задача, а непрерывный процесс, который требует внимательного подхода на каждом этапе разработки и эксплуатации.

Вот несколько рекомендаций, которые помогут вам максимально эффективно оптимизировать Java-приложения:

1. **Профилируйте и анализируйте код с самого начала**. Регулярно используйте инструменты профилирования и мониторинга, такие как **JProfiler**, **JVisualVM**, **YourKit**, чтобы заранее выявить узкие места и предотвратить проблемы производительности.

2. **Оптимизация многопоточности**. Используйте правильные подходы для управления потоками и синхронизации, такие как **ExecutorService**, **CompletableFuture** и **ForkJoinPool**. Понимание многопоточности и асинхронных операций помогает повысить производительность и масштабируемость приложений.

https://max.ru/bookofgeek

3. **Управление памятью**. Работайте с памятью эффективно: профилируйте использование кучи, избегайте утечек памяти и минимизируйте создание лишних объектов. Инструменты для анализа памяти, такие как **Heap Dumps** и **JVM GC logs**, помогут вам отслеживать и устранять проблемы.

4. **Используйте современные возможности Java**. Язык и его экосистема постоянно эволюционируют. Использование новых функций, таких как **Streams API**, **JMH** для бенчмаркинга и неблокирующего ввода-вывода (NIO), помогает значительно ускорить разработку и повысить производительность.

5. **Тестирование и контроль качества**. Регулярно проводите юнит-тестирование, интеграционное тестирование и нагрузочное тестирование. Автоматизируйте процессы, чтобы своевременно выявлять проблемы производительности на всех этапах разработки и эксплуатации.

6. **Не забывайте об оптимизации запросов к базе данных**. Оптимизация работы с базой данных, эффективное использование пулов соединений и индексация запросов — ключевые элементы, которые значительно повышают производительность приложений, работающих с большими объемами данных.

7. **Будьте осторожны с излишней оптимизацией**. Избыточная оптимизация может привести к усложнению кода и даже ухудшению производительности. Важно не стремиться оптимизировать все части системы, а сосредоточиться на действительно узких местах.

Ресурсы для дальнейшего изучения

Для тех, кто хочет углубить свои знания в области оптимизации Java, следующие ресурсы могут быть полезными:

1. **Книги**:

   » *Java Performance: The Definitive Guide* (Scott Oaks)

https://max.ru/bookofgeek

- *Effective Java* (Joshua Bloch) — книга, которая даёт полезные советы по улучшению производительности и качества кода.
- *Java Concurrency in Practice* (Brian Goetz) — углублённое руководство по многопоточности в Java.

2. **Онлайн-курсы**:
   - Coursera, Udemy и Pluralsight предлагают курсы по оптимизации Java и многопоточности, включая темы, связанные с производительностью, профилированием и использованием современных технологий Java.

3. **Блоги и статьи**:
   - **Baeldung** — предлагает множество статей по оптимизации производительности Java, включая профилирование, многопоточность и использование новых возможностей языка.
   - **OpenJDK Mailing Lists** — обсуждения и анонсы новых версий OpenJDK, которые часто включают улучшения в производительности.

4. **Инструменты для профилирования и анализа**:
   - **JProfiler**, **YourKit** — профилировщики для анализа производительности Java-приложений.
   - **JVisualVM** и **JConsole** — бесплатные инструменты для мониторинга и профилирования Java-приложений.
   - **JMH** — инструмент для микробенчмаркинга, который помогает точно измерить производительность различных частей вашего кода.

Будущее оптимизации Java

Java продолжает развиваться, и, как следствие, появляются новые возможности для оптимизации производительности.

В будущем можно ожидать улучшения в следующих областях:

1. **Улучшения в JIT-компиляции**. Современные версии JVM всё больше фокусируются на улучшении производительности JIT-компиляции,

https://max.ru/bookofgeek

включая поддержку **GraalVM**, который является экспериментальной виртуальной машиной с возможностями Just-In-Time и Ahead-Of-Time компиляции.

2. **Оптимизация многопоточности и параллелизма**. В Java 17 и позже продолжат развиваться возможности для многопоточности и параллельных вычислений. Будет расширяться поддержка таких технологий, как **Project Loom**, что обещает упростить работу с лёгкими потоками, позволяя разработчикам писать высокопроизводительные многозадачные приложения.

3. **Поддержка асинхронности и реактивного программирования**. С каждым новым релизом Java появляются новые возможности для асинхронного программирования, такие как улучшения в **CompletableFuture**, **Project Reactor** и **Quarkus** для реактивных приложений.

4. **Реализация новых алгоритмов и структур данных**. Java-комьюнити активно работает над улучшением стандартных библиотек. Возможно, появятся новые структуры данных, которые помогут ускорить выполнение операций и сделать работу с памятью более эффективной.

5. **Инструменты для анализа и мониторинга**. С развитием Cloud-Native приложений и микросервисной архитектуры растёт важность мониторинга и профилирования в распределённых системах. В будущем ожидается развитие инструментов, таких как **Prometheus** и **Grafana**, для глубокой интеграции с Java-приложениями.

Оптимизация Java-приложений — это многогранная и сложная задача, которая требует всестороннего подхода. Важно помнить, что оптимизация должна быть продуманной и целенаправленной, а не поспешной. Успех заключается в правильной идентификации узких мест, применении правильных инструментов и подходов, а также в постоянном анализе и тестировании производительности на всех этапах разработки и эксплуатации.

https://max.ru/bookofgeek

Будьте всегда готовы к новым вызовам и технологиям, которые появятся с каждым новым релизом Java.

https://max.ru/bookofgeek

Полезные утилиты и инструменты

https://max.ru/bookofgeek

В небольших приложениях к книге мы собрали несколько полезных утилит и инструментов для работы с Java-приложениями, а также ссылки на дополнительные материалы и глоссарий терминов, которые помогут вам более глубоко понять и использовать методы оптимизации, тестирования и контроля кода.

Полезные утилиты и инструменты

- **JProfiler** — инструмент для профилирования Java-приложений, который позволяет анализировать использование памяти, потоков и CPU, а также обнаруживать утечки памяти и другие проблемы производительности.

  Сайт: *https://www.ej-technologies.com/products/jprofiler/overview.html*

- **YourKit** — ещё один мощный профилировщик для Java, который предоставляет подробные отчёты о производительности приложения и помогает найти узкие места, такие как утечки памяти и неэффективное использование потоков.

  Сайт: *https://www.yourkit.com/*

- **JVisualVM** — бесплатный инструмент от Oracle для мониторинга и профилирования Java-приложений, который поставляется с JDK и позволяет анализировать поведение приложений в реальном времени.

  Сайт: *https://visualvm.github.io/*

https://max.ru/bookofgeek

- **JMH (Java Microbenchmarking Harness)** — фреймворк для точного измерения производительности отдельных частей Java-кода. Он используется для создания высокоточечных бенчмарков.

  Сайт: https://openjdk.java.net/projects/code-tools/jmh/

- **JConsole** — ещё один инструмент от Oracle для мониторинга JVM в реальном времени, который помогает отслеживать использование ресурсов и производительность вашего приложения.

  Сайт: *https://docs.oracle.com/javase/8/docs/technotes/guides/management/jconsole.html*

- **HikariCP** — высокопроизводительный пул соединений для JDBC, который помогает снизить время отклика при работе с базами данных.

  Сайт: *https://github.com/brettwooldridge/HikariCP*

- **Prometheus** — система мониторинга и оповещения с открытым исходным кодом, которая используется для сбора метрик с ваших приложений и их визуализации с помощью **Grafana**.

  Сайт: *https://prometheus.io/*

- **Grafana** — мощный инструмент для визуализации метрик и мониторинга в реальном времени. Он используется в связке с Prometheus для создания красивых панелей и отчётов.

  Сайт: *https://grafana.com/*

- **GraalVM** — универсальная виртуальная машина, которая поддерживает не только Java, но и другие языки программирования, такие как JavaScript, Python, Ruby и C++. Она предлагает улучшения в производительности с возможностью компиляции кода в нативные бинарные файлы.

  Сайт: *https://www.graalvm.org/*

- **Checkstyle** — инструмент для анализа качества кода и его соответствия стандартам и best practices.

  Сайт: *https://checkstyle.sourceforge.io/*

https://max.ru/bookofgeek

- **FindBugs** — инструмент для статического анализа кода, который помогает находить потенциальные ошибки и проблемы в Java-коде, такие как утечки памяти, неправильное использование коллекций и прочее.

  Сайт: *https://findbugs.sourceforge.io/*

- **SonarQube** — платформа для анализа и мониторинга качества кода, которая позволяет отслеживать ошибки, баги, уязвимости и анти-паттерны в приложении.

  Сайт: *https://www.sonarqube.org/*

https://max.ru/bookofgeek

Ссылки на дополнительные материалы

https://max.ru/bookofgeek

Ссылки на дополнительные материалы

- **Baeldung** — один из крупнейших ресурсов по Java, который регулярно публикует статьи по оптимизации, многопоточности и использованию новых возможностей языка.

  Сайт: *https://www.baeldung.com/*

- **OpenJDK** — репозиторий с исходным кодом и документацией для Java. Это основной источник для изучения JDK и JVM.

  Сайт: *https://openjdk.java.net/*

- **Oracle Java Documentation** — официальная документация для Java, которая включает в себя множество материалов по JDK, JVM, многопоточности и оптимизации.

  Сайт: *https://docs.oracle.com/en/java/*

- **Java Performance Blog** — блог, посвящённый анализу производительности Java-приложений.

  Сайт: *https://www.javaperformancetuning.com/*

- **Project Loom** — исследовательский проект в OpenJDK, направленный на упрощение работы с многопоточностью в Java.

  Сайт: *https://openjdk.java.net/projects/loom/*

https://max.ru/bookofgeek

- **Project Panama** — проект, который облегчает взаимодействие Java с кодом, написанным на C, C++ и других языках.

  Сайт: *https://openjdk.java.net/projects/panama/*

- **Project GraalVM** — проект, создающий многоязычную виртуальную машину для Java, который улучшает производительность и добавляет новые возможности для работы с другими языками программирования.

  Сайт: *https://www.graalvm.org/*

https://max.ru/bookofgeek

https://max.ru/bookofgeek

Глоссарий терминов

https://max.ru/bookofgeek

Глоссарий терминов

1. **JVM (Java Virtual Machine)** — виртуальная машина, на которой исполняются Java-программы. Отвечает за управление памятью, выполнение кода и взаимодействие с операционной системой.

2. **JIT-компиляция (Just-In-Time compilation)** — технология, при которой байт-код Java компилируется в машинный код прямо во время выполнения программы, что ускоряет ее работу.

3. **GC (Garbage Collection)** — автоматическое управление памятью, при котором объекты, которые больше не используются, удаляются для освобождения памяти.

4. **Heap** — область памяти, выделенная для объектов во время выполнения программы. Java-код работает в динамической памяти, а все объекты создаются в куче.

5. **Thread Dump** — снимок всех потоков, их состояния и стека вызовов на момент получения дампа, используется для диагностики блокировок и мёртвых блокировок в многопоточных приложениях.

6. **Heap Dump** — снимок содержимого кучи на момент выполнения программы. Используется для анализа утечек памяти и использования объектов.

https://max.ru/bookofgeek

7. **CompletableFuture** — класс в Java для работы с асинхронными операциями, который позволяет легко комбинировать и управлять многими параллельными задачами.

8. **ForkJoinPool** — пул потоков, оптимизированный для выполнения рекурсивных задач, который позволяет эффективно использовать многопроцессорные системы.

9. **ORM (Object-Relational Mapping)** — технология, которая упрощает взаимодействие между объектно-ориентированными приложениями и реляционными базами данных.

10. **NIO (Non-blocking I/O)** — неблокирующий ввод-вывод, технология для асинхронной обработки ввода-вывода, которая помогает повысить производительность при работе с файлами и сетевыми соединениями.

11. **Streams API** — библиотека в Java для работы с потоками данных, которая позволяет обрабатывать коллекции и другие источники данных с использованием функциональных подходов.

12. **JMH (Java Microbenchmarking Harness)** — фреймворк для точного измерения производительности Java-программ. Особенно полезен для оценки производительности небольших фрагментов кода.

13. **HikariCP** — высокопроизводительный пул соединений для работы с базами данных в Java-приложениях.

14. **Virtual Threads** — технология легковесных потоков в Java, реализованная в рамках проекта Loom. Позволяет создавать миллионы потоков без значительных накладных расходов на ресурсы.

15. **Project Loom** — проект OpenJDK по добавлению виртуальных потоков и упрощению написания высокопроизводительных многопоточных программ.

16. **Micrometer** — библиотека для сбора метрик в Java-приложениях, совместимая с различными системами мониторинга, такими как Prometheus, New Relic, Datadog.

https://max.ru/bookofgeek

17. **Prometheus** — система мониторинга и сбора метрик с поддержкой языка запросов PromQL. Часто используется для мониторинга контейнеризированных приложений.

18. **Grafana** — инструмент для визуализации метрик, логов и данных из различных источников, включая Prometheus.

19. **Ehcache** — кэш-библиотека для Java-приложений, обеспечивающая локальное кэширование данных.

20. **Caffeine** — современная высокопроизводительная библиотека для локального кэширования в Java, предлагающая оптимизацию Hit/Miss Rate.

21. **Redis** — быстрое in-memory хранилище данных, часто используемое для распределенного кэширования и обмена данными между приложениями.

22. **Hit Rate** — доля запросов к кэшу, которые успешно находят нужные данные (чем выше, тем эффективнее кэш).

23. **Miss Rate** — доля запросов к кэшу, которые не находят нужные данные (чем ниже, тем лучше).

24. **Protobuf (Protocol Buffers)** — эффективный формат сериализации структурированных данных, разработанный Google. Быстрее и компактнее, чем JSON.

25. **Avro** — формат сериализации данных от Apache, часто используемый с большими данными (Big Data).

26. **gRPC** — фреймворк удалённого вызова процедур (RPC) с использованием Protobuf для сериализации данных.

27. **Apache Spark** — распределенная платформа обработки больших данных для работы с большими объемами данных в памяти и на диске.

28. **Apache Flink** — фреймворк для распределенной потоковой обработки данных в реальном времени.

https://max.ru/bookofgeek

29. **Kubernetes** — система оркестрации контейнеров, автоматизирующая развертывание, масштабирование и управление контейнеризированными приложениями.

30. **GitHub Actions** — платформа CI/CD от GitHub для автоматизации рабочих процессов и проверок кода.

31. **GitLab CI** — интегрированная система CI/CD в GitLab для построения, тестирования и развертывания приложений.

32. **Jenkins** — популярный сервер автоматизации для CI/CD процессов.

33. **Observability** — комплексная способность системы обеспечивать мониторинг состояния, производительности и правильности работы через метрики, логи и трассировки.

34. **JIT-оптимизация** — оптимизация кода на этапе выполнения в JIT-компиляторе для ускорения выполнения часто используемых методов.

35. **HotSpot** — основная JVM от Oracle, обеспечивающая высокопроизводительное выполнение Java-программ за счёт JIT-компиляции и оптимизаций.

36. **Container Awareness** — способность приложений и JVM адаптироваться к ограниченным ресурсам контейнеров (CPU/memory limits).

https://max.ru/bookofgeek

https://max.ru/bookofgeek

Список использованных источников информации

https://max.ru/bookofgeek

Список использованных источников информации:

1. Oracle Java Documentation — Java Collections Framework (https://docs.oracle.com/javase/8/docs/technotes/guides/collections/overview.html)
2. Big-O Cheat Sheet (https://www.bigocheatsheet.com/)
3. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein — Introduction to Algorithms, 3rd Edition (MIT Press)
4. Robert Sedgewick, Kevin Wayne — Algorithms (4th Edition)
5. Steven S. Skiena — The Algorithm Design Manual
6. GeeksforGeeks — Algorithms and Data Structures (https://www.geeksforgeeks.org/)
7. Baeldung — Java Collections and Algorithms (https://www.baeldung.com/)
8. JMH — документация (https://openjdk.org/projects/code-tools/jmh/)
9. Java Concurrency Utilities (java.util.concurrent) — документация Oracle (https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/package-summary.htm)
10. Brian Goetz — Java Concurrency in Practice
11. Doug Lea — Concurrent Programming in Java: Design Principles and Patterns

https://max.ru/bookofgeek

12. Maurice Naftalin, Philip Wadler — Java Generics and Collections

13. Java Memory Model (JMM) (https://docs.oracle.com/javase/specs/)

14. Baeldung — Concurrency in Java
(https://www.baeldung.com/java-concurrency)

15. GeeksforGeeks — Multithreading in Java
(https://www.geeksforgeeks.org/multithreading-in-java/)

16. IBM Developer — Java concurrency and multithreading
(https://developer.ibm.com/languages/java/)

17. Nicolas J. Joubert — Java Performance Companion

18. Java Performance: The Definitive Guide

19. JEP 380: Unix-Domain Socket Channels (https://openjdk.org/jeps/380)

20. Baeldung — Java I/O и NIO (https://www.baeldung.com/java-io)

21. GeeksforGeeks — Java I/O (https://www.geeksforgeeks.org/java-io-tutorial/)

22. Netty — библиотека (https://netty.io/)

23. Apache Commons IO — утилита
(https://commons.apache.org/proper/commons-io/)

24. JMH (Java Microbenchmark Harness) — библиотека
(https://openjdk.org/projects/code-tools/jmh/)

25. Kent Beck — Test-Driven Development: By Example

26. Gerard Meszaros — xUnit Test Patterns: Refactoring Test Code

27. Lisa Crispin, Janet Gregory — Agile Testing: A Practical Guide for Testers and Agile Teams

28. Paul Hamill — Unit Test Frameworks

https://max.ru/bookofgeek

29. JUnit документация (https://junit.org/junit5/docs/current/user-guide/)

30. Mockito документация (https://site.mockito.org/)

31. SonarQube документация (https://docs.sonarqube.org/)

32. Baeldung — Best Practices for Unit Testing in Java (https://www.baeldung.com/java-unit-testing-best-practices)

33. Martin Fowler — Test Pyramid, Mocks Aren't Stubs (https://martinfowler.com/articles/mocksArentStubs.html)

34. ThoughtWorks Technology Radar (https://www.thoughtworks.com/radar)

35. Jacoco — документация (https://www.jacoco.org/)

36. ArchUnit — документация (https://www.archunit.org/)

37. Checkstyle, PMD, SpotBugs (https://blog.theodo.ma/checkstyle-pmd-and-findbugs-why-and-how/)

38. Ian Molyneaux — The Art of Application Performance Testing

39. Mark Tomlinson, Scott Barber — Performance Testing Guidance for Web Applications

40. Apache JMeter Documentation — документация (https://jmeter.apache.org/usermanual/index.html)

41. k6 документация (https://grafana.com/docs/k6/latest/)

42. Gatling документация (https://docs.gatling.io/)

43. Locust документация (https://docs.locust.io/en/stable/)

44. Chaos Monkey (Netflix) документация (https://netflix.github.io/chaosmonkey/)

45. Blazemeter Blog — Types of Performance Testing

https://max.ru/bookofgeek

(https://www.blazemeter.com/blog/types-performance-testing)

46. Grafana Labs Blog — Introduction to Performance Testing with Grafana k6 (https://grafana.com/workshops/introduction-to-performance-testing-with-grafana-k6/)

47. Microsoft Learn — Tutorial: Run a load test to identify performance bottlenecks in a web app

(https://learn.microsoft.com/en-us/azure/load-testing/tutorial-identify-bottlenecks-azure-portal)

48. Prometheus документация

(https://prometheus.io/docs/introduction/overview/)

49. Grafana документация (https://grafana.com/docs/)

https://max.ru/bookofgeek

Тестирование, контроль и оптимизация кода

JAVA

Это практическое руководство охватывает весь цикл разработки высокопроизводительного кода: от понимания внутренней работы **JVM** до настройки сборщиков мусора, анализа **JIT-компиляции**, применения **JMH**, работы с многопоточностью и построения эффективных **CI/CD пайплайнов** и многое другое.

Вы узнаете:

- какие инструменты и подходы позволяют выявлять узкие места;
- как замерять реальные метрики;
- оптимизировать многопоточные и реактивные приложения;
- правильно кешировать и анализировать байт-код;
- использовать современные сборщики мусора и эффективно оптимизировать мобильные и десктопные Java-приложения.

Отдельные главы посвящены виртуальным потокам, модулярности, **Observability** и работе в контейнерах и с большими данными **(Spark, Flink)**.

Прочитав книгу, Вы получите:

- целостное понимание производительности Java-приложений;
- знания о том, как проводить тестирование под нагрузкой и отслеживать деградации в CI/CD;
- десятки примеров, шаблонов и советов для ускорения приложений как на сервере, так и на десктопе или Android;
- рекомендации по выбору GC, построению observability-стека и оптимизации всего пути.

https://max.ru/bookofgeek